

Agentic Hybrid RAG on SAP BTP

Sriram Rokkam

May 2026

Contents

Front Matter	12
Agentic Hybrid RAG on SAP BTP	12
A Hands-On Guide with LangGraph, HANA Cloud, and Vertex AI	12
Title Page	12
Copyright	12
Dedication	13
Foreword	13
Preface	13
Who This Book Is For	14
Who This Book Is NOT For	14
Prerequisites	15
Technical Knowledge	15
NOT Required	16
Tools and Accounts (All Free)	16
Companion Code Repository	16
Repository Structure	16
Quick Start	17
Chapter-to-Code Cross-Reference	18
How to Use This Book	18
Reading Cover to Cover (Recommended)	18
As a Reference	18
Chapter Structure	18
The Code	18
Conventions Used in This Book	19
Code Blocks	19
Callout Boxes	19
File Paths	19
Screenshots	20
Acknowledgements	20
About the Author	20
Chapter 1: The SAP Document Problem — And Why BTP Is the Right Place to Solve It	21
1.1 The Document Fragmentation Problem in SAP Landscapes	21
1.2 Why Hybrid Retrieval — Not Just Search	22

1.3 The Four Capabilities That Define the Agent	23
1.4 SAP's Strategic Direction: The Autonomous Enterprise (Sapphire 2026)	24
1.5 What We Build: System Overview.	26
1.6 Why MSDS First	28
1.7 Scope and Prerequisites	29
What This Book Covers.	29
What This Book Does NOT Cover	30
Prerequisites	31
1.8 Summary	31
Chapter 2: Platform Setup — SAP BTP Trial + Google Vertex AI	33
2.1 The Goal of This Chapter.	33
2.2 Understanding What We Are Setting Up	33
2.3 SAP BTP Trial Account.	35
Signing Up.	35
Navigating to Your Trial Subaccount	36
2.4 Provisioning SAP HANA Cloud	37
Creating the HANA Cloud Instance	37
Enabling the Vector Engine	39
Getting Your HANA Connection Details	40
2.5 Setting Up Google Cloud + Vertex AI	40
Creating a GCP Account	40
Creating a New Project	42
Enabling the Vertex AI API.	42
Creating a Service Account	42
Downloading the Service Account Key.	43
2.6 Configuring the BTP Destination Service.	44
Creating the Destination	44
2.7 Local Development Environment.	46
Python 3.11+	46
Node.js 20+	47
Cloud Foundry CLI	47
SAP CDS CLI	48
VS Code Extensions	48
2.8 Project Setup	48
2.9 Verifying the Setup: Your First Gemini API Call.	50
2.10 Verifying the HANA Cloud Connection	52
2.11 What We Have Built.	54

2.12 Understanding the Credential Flow	54
2.13 Summary	55
Chapter 3: Vector Search on SAP HANA Cloud	56
3.1 What Are Embeddings? Intuition Without the Math	56
3.2 The HANA Cloud REAL_VECTOR Column Type	58
3.3 Creating the Vector Table: The Lazy Initialization Pattern	60
3.4 Calling text-embedding-004 from Python	62
3.5 Storing Embeddings: INSERT with REAL_VECTOR	64
3.6 Cosine Similarity Search: The Query Explained	65
3.7 Chunking Strategy for Material Documents	67
3.8 Building vector_srv.py — The Vector Service Layer	69
3.9 Testing: Business Questions Across Document Types	72
3.10 What Vector Search Gets Right — and What It Gets Wrong	76
3.11 Summary	78
3.12 Checkpoint.	78
Chapter 4: Knowledge Graphs on SAP HANA Cloud	80
4.1 What is a knowledge graph?	80
Why call it a graph?	81
What does this give us that a relational schema does not?	82
4.2 RDF and SPARQL — the standards explained plainly	83
4.3 Why HANA Cloud for the Knowledge Graph	84
Why a stored procedure and not an HTTP endpoint?	85
4.4 Designing the MSDS ontology — what facts matter?	86
4.5 The OWL ontology file — MSDS_Ontology.ttl	87
4.6 Extracting triples from MSDS text using Gemini	89
4.7 Storing triples in HANA — named graphs per material	91
4.8 Writing SPARQL queries — Gemini as the SPARQL translator	93
4.9 Building kg_srv.py — the knowledge graph service layer	95
Validation: the KG equivalent of SQL injection	100
What delete_graph and count_triples give us	100
4.10 Testing — store triples for acetone, query for GHS codes	101
4.11 What the KG gets right that vector search cannot	104
4.12 Summary	105
4.13 Checkpoint.	106

Chapter 5: The Document Intelligence Ingestion Pipeline	108
5.1 The dual-pipeline architecture	109
5.1.1 Why process both in parallel?	109
5.1.2 The fire-and-forget pattern	110
5.2 PDF text extraction with PyMuPDF	111
5.2.1 Extracting with metadata	111
5.2.2 Handling upload files in FastAPI	113
5.3 Chunking strategy for vector search	113
5.3.1 Our chunking parameters	114
5.3.2 The chunker function	115
5.3.3 Why the KG pipeline does not chunk	116
5.4 Thread-local HANA connections	116
5.4.1 Why connections cannot be shared across threads	117
5.4.2 The <code>threading.local()</code> solution	117
5.5 The ingestion flow in detail	118
5.5.1 The upload endpoint	119
5.5.2 The dual-pipeline orchestrator	120
5.5.3 Thread 1 — the vector pipeline	121
5.5.4 Thread 2 — the knowledge graph pipeline.	122
5.6 Status tracking	123
5.6.1 The MSDS_DOCUMENTS table and its role	123
5.6.2 Status helper functions	124
5.6.3 The status endpoint	125
5.7 Error handling — what happens when one pipeline fails	126
5.8 Testing — upload five MSDS documents	127
5.8.1 Start the service.	127
5.8.2 Upload the first document	127
5.8.3 Poll for completion.	127
5.8.4 Verify the vector store	128
5.8.5 Verify the knowledge graph	128
5.9 Summary	129
5.10 Checkpoint.	130
Chapter 6: LangGraph Fundamentals.	132
6.1 Why LangGraph is SAP's recommended orchestration approach for BTP agents	132
6.2 Why LangChain chains are not enough	133
6.3 Core concepts	134
6.3.1 State	134
6.3.2 Nodes	135

6.3.3 Edges	135
6.3.4 Conditional edges	135
6.4 Installation and project layout	136
6.5 Building the state and nodes	137
6.6 Wiring the graph	138
6.7 Adding a conditional retry edge	140
6.8 The StateGraph visualised	141
6.9 The complete simple_qa_agent.py	142
6.10 Tool use in LangGraph	144
6.11 Streaming responses	146
6.12 Debugging with LangSmith	146
6.13 Why stateless works on BTP Cloud Foundry	147
6.14 Summary	148
6.15 Checkpoint	149
Chapter 7: The Parallel Hybrid RAG Agent	151
7.1 Why parallel beats sequential	151
7.2 The agent state	153
7.3 The vector chain	154
7.4 The KG chain	156
7.5 The merge function	161
7.6 The orchestrator	163
7.7 The FastAPI /query endpoint	166
7.8 The full request/response contract	168
7.9 Conversation history	169
7.10 Testing: three scenarios that prove hybrid wins	170
7.10.1 The KG wins: a safety officer asks about hazard classifications	170
7.10.2 The vector wins: a quality engineer asks about first aid procedures	171
7.10.3 Both contribute: a logistics manager asks a combined question	171
7.11 Monitoring chain performance	172
7.12 Summary	173
7.13 Checkpoint	173
Chapter 8: The Multi-Agent Supervisor Pattern	175
8.1 Why multi-agent for SAP enterprise	175
8.2 The supervisor pattern	176

8.3 The three specialist agents for MSDS documents	177
8.3.1 HazardAgent	178
8.3.2 SafetyAgent	178
8.3.3 ComplianceAgent	178
8.4 Shared state for the supervisor graph	178
8.5 The supervisor node	179
8.6 The specialist agents	182
8.7 The summary agent	184
8.8 Building the supervisor graph	185
8.9 When to use the supervisor vs the direct agent	187
8.10 The /query-advanced endpoint	188
8.11 Testing: a question that requires all three specialists	189
8.12 Applying the pattern to other SAP document types	190
8.13 Summary	191
8.14 Checkpoint	192
Chapter 9: CAP Node.js OData V4 — The SAP-Native API Layer + Fiori Elements	
UI	194
9.1 Why CAP?	194
9.2 The two-process architecture	195
9.3 The CDS data model	197
9.4 The service definition	199
9.5 The Products entity and S/4HANA value help	200
9.6 The callBackend helper	201
9.7 Action handlers	202
processFile	202
chatQuery	204
deleteFile	205
pollStatus	206
9.8 Fiori Elements UI	207
9.9 Annotations explained	208
HeaderInfo	208
SelectionFields and LinItem	209
Identification actions	209
FieldGroup	210
9.10 Running locally	210
9.11 Testing the full flow	212

9.12 What you can do after this chapter	213
9.13 Checkpoint checklist.	214
Summary	215
Chapter 10: Deploying to SAP BTP Cloud Foundry	216
10.1 What Changes Between Local and BTP.	216
10.2 Why Two Separate Applications.	217
10.3 Pre-Deployment Checklist.	218
10.4 Step 1: Push the Python Agent.	219
Understanding manifest.yml	219
HANA connection in CF	220
Pushing the application	220
10.5 Step 2: Create the User-Provided Service for Credentials.	221
10.6 Step 3: Build and Deploy the CAP Service	223
Running the build	225
Deploying the MTA	225
10.7 Step 4: Configure the BTP Destination for Vertex AI	226
Creating the agent backend destination	227
Creating the Vertex AI destination	227
10.8 Step 5: Load the Ontology.	228
10.9 Step 6: Smoke Test the Deployed System.	229
Agent health endpoint	229
Direct query to the agent	229
CAP OData endpoint	230
Fiori Elements UI.	230
10.10 Troubleshooting	230
Memory quota exceeded	230
HANA connection refused.	231
GCP authentication failure	231
CAP cannot reach agent	231
Package not found during staging	232
10.11 Scaling.	232
10.12 What You Have Built	233
10.13 Chapter Checkpoint	234
Appendix A — SAP BTP Trial Setup: Step-by-Step	236
What You Will Have at the End	236
Step 1 — Create a BTP Trial Account.	236

Navigate to the Trial Signup Page	236
Register Your Account	237
Complete Registration and Choose a Region.	239
Confirm the Trial Account Is Ready	240
Step 2 — Navigate the BTP Cockpit.	241
Finding Your Trial Subaccount	241
Finding the CF Space	242
Step 3 — Enable Cloud Foundry.	243
Check CF Runtime Status	243
Create the dev Space.	243
Note the CF API Endpoint	243
Step 4 — Add HANA Cloud Entitlement.	244
Navigate to Entitlements.	244
Step 5 — Provision HANA Cloud.	245
Open the Service Marketplace.	245
Fill in the Provisioning Form.	246
Wait for Provisioning	247
Step 6 — Enable the Vector Engine.	248
Open HANA Cloud Central	248
Edit the Instance Configuration	248
Step 7 — Get the HANA Connection Details.	249
Copy the SQL Endpoint	249
Step 8 — Install the CF CLI.	250
Install on macOS.	250
Install on Windows	250
Verify the Installation	250
Authenticate to Your BTP Space	251
Verification Checklist	251
Common Problems and Fixes	252
Appendix B: Google Cloud Setup — Vertex AI, Gemini, and Service Accounts. . .	253
B.1 What You Will Have at the End.	253
B.2 Two Authentication Options — Choose Before You Start	254
B.3 Step 1 — Create a GCP Account	255
B.4 Step 2 — Create a New Project	256
B.5 Step 3 — Enable the APIs	256
B.6 Step 4A — Get an API Key (Option A — Recommended for Local Development) . . .	257
B.7 Step 4B — Create a Service Account (Option B — Required for BTP Deployment). . .	259

B.8 Step 5 — Test the Connection	261
B.9 Step 6 — Understanding Free Tier and Costs	262
B.10 Verification Checklist.	263
Appendix C — Full Code Reference	265
Repository Layout	265
Python Agent Service (agents/)	266
agents/main.py.	266
agents/requirements.txt.	267
agents/manifest.yml.	268
agents/agents/state.py	268
agents/agents/vector_chain.py.	269
agents/agents/kg_chain.py.	269
agents/agents/orchestrator.py.	270
agents/agents/supervisor.py	271
Service Layer (agents/srv/)	271
agents/srv/hdb_srv.py	271
agents/srv/vector_srv.py.	272
agents/srv/kg_srv.py.	273
agents/srv/doc_srv.py	273
agents/srv/vertex_srv.py.	274
CAP OData Service (cap-srv/)	275
cap-srv/db/schema.cds	275
cap-srv/srv/service.cds.	276
cap-srv/srv/service.js	277
Ontology (MSDS_Ontology.ttl)	277
Environment Variables Reference	278
Python agent service (agents/.env.example).	278
CAP service (cap-srv/.env.example).	279
Material Number Format Constraint.	279
Quick curl Reference	280
Cross-Reference: Chapter to File.	281
Appendix D: Joule A2A Integration.	283
D.1 What Is Joule A2A?	284
D.2 How the Pieces Fit Together	284
D.3 The File Structure.	285
D.4 The Five Files Explained	286
D.4.1 da.sapdas.yaml — The Root Agent Declaration	286

D.4.2 capability.sapdas.yaml — The Capability Metadata	286
D.4.3 capability_context.yaml — Persisted Variables.	287
D.4.4 Function YAML Files — The HTTP Request Builders	288
D.4.5 Scenario YAML Files — Intent Matching and Routing	290
D.5 The SpEL-like Expression Language.	292
D.6 The /a2a Endpoint	293
D.6.1 Dual Format Support	293
D.6.2 The 15-Second Timeout Constraint	294
D.6.3 The Status Path	295
D.7 The BTP Destination.	295
D.8 Building the .daar Package	296
D.9 Deploying to Joule	297
D.10 Testing in Joule	298
D.11 What Will Change with Joule 2.0	300
D.12 Files in the Repository	301
D.13 Summary.	302

Front Matter

Agentic Hybrid RAG on SAP BTP

A Hands-On Guide with LangGraph, HANA Cloud, and Vertex AI

Title Page

Agentic Hybrid RAG on SAP BTP *A Hands-On Guide with LangGraph, HANA Cloud, and Vertex AI*

Author: Sriram Rokkam

First Edition, 2026

Copyright

Copyright © 2026 Sriram Rokkam. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

The code samples in this book are released under the MIT License and are freely available at: <https://github.com/sriramrokkam/book-agentic-hybrid-rag-on-btp>

SAP, SAP BTP, SAP HANA, SAP CAP, and Joule are trademarks or registered trademarks of SAP SE. Google Cloud, Vertex AI, and Gemini are trademarks of Google LLC. All other trademarks are the property of their respective owners. The author is not affiliated with SAP SE or Google LLC.

Dedication

[To be added by Sriram Rokkam]

Foreword

[To be written by a colleague, SAP contact, or industry peer after the book is complete. 300–500 words. Suggested topics: why Agentic AI matters for SAP customers today, why the BTP + Vertex AI approach is relevant, and why the author is credible on this subject.]

— [Name], [Title], [Company]

Preface

[To be written by Sriram Rokkam in his own voice. Suggested content below — edit freely, this is your personal story.]

I have spent years working at the intersection of SAP and cloud platforms, watching enterprises struggle with the same problem in different forms: they have extraordinary data — decades of business transactions, master data, documents, and domain knowledge — but they cannot easily ask questions of it.

The arrival of large language models changed what was possible. Suddenly, the gap between a business question and a data answer felt bridgeable. But every proof of concept I saw had the same flaw: it worked for demos and broke for production. The LLM hallucinated facts. The retrieval missed precise structured answers. The architecture did not fit the SAP ecosystem that customers already had invested in.

This book is my answer to that problem.

It builds a real system — not a toy — that combines two retrieval strategies that together handle what neither handles alone. It runs on platforms that SAP developers already know: SAP BTP, SAP HANA Cloud, and CAP Node.js. It uses Google Vertex AI not because it is the only option, but because it is excellent, well-documented, and free to get started

with.

Every line of code in this book has been written and tested. Every architecture decision has a reason. Where I made a tradeoff, I say so.

My hope is that you finish this book with a working system deployed on SAP BTP, a clear mental model of how agentic AI actually works, and the confidence to take these patterns into your own projects.

Let us build something real.

Sriram Rokkam *May 2026*

Who This Book Is For

This book is written for:

- **SAP developers and architects** familiar with SAP BTP, CAP, or ABAP who want to build AI-powered applications on the platform they already know
- **AI and ML engineers** working in SAP customer environments who need to understand the SAP data layer and how to integrate with it
- **Technical leads and solution architects** evaluating agentic AI patterns for SAP projects and looking for a production-ready reference architecture
- **Full-stack developers** who have experimented with LLMs and chatbots and are ready to go beyond simple Q&A into multi-step agentic systems
- **BTP developers** who want hands-on experience with SAP HANA Cloud's vector engine and SPARQL knowledge graph capabilities

If you work with SAP systems and want to build AI agents that reason over your enterprise data — this book is for you.

Who This Book Is NOT For

This book is not the right fit if you are:

- **Looking for a general machine learning or deep learning introduction.** We do not cover neural networks, training models, or ML theory. This book is about

applying LLMs to SAP problems, not building LLMs.

- **An ABAP developer with no Python experience.** The agent layer is written in Python. You do not need to be an expert, but you should be comfortable reading and writing basic Python functions, classes, and pip packages before starting Chapter 2.
- **Looking for a no-code or low-code AI solution.** We write real code throughout. Every component is built from first principles so you understand exactly what it does and why.
- **Expecting SAP AI Core as the primary AI backend.** We use Google Vertex AI (Gemini + text-embedding-004) because it is available on the free trial. SAP AI Core is covered in Appendix E for readers who have an enterprise subscription.
- **New to cloud development entirely.** You should be comfortable with the concept of cloud services, REST APIs, and deploying applications. Chapter 2 walks through all the setup steps, but it assumes you have used a terminal before.

Prerequisites

Technical Knowledge

The following knowledge is assumed. You do not need to be an expert — a working familiarity is enough.

Topic	Level Required	Where to Learn if Needed
Python	Basic — functions, classes, pip, venv	python.org/about/gettingstarted
JavaScript / Node.js	Basic — enough to read CAP handlers	nodejs.dev/learn
SAP BTP concepts	Familiar — subaccounts, CF, services	learning.sap.com
REST APIs	Basic — HTTP methods, JSON	Any REST API tutorial
SQL	Basic — SELECT, INSERT	W3Schools SQL

NOT Required

You do not need prior knowledge of: - Machine learning theory or mathematics - LangChain or LangGraph (we introduce it from scratch in Chapter 6) - RDF, SPARQL, or knowledge graphs (introduced from scratch in Chapter 4) - SAP AI Core (covered optionally in Appendix E) - ABAP development - Docker or Kubernetes

Tools and Accounts (All Free)

All tools and accounts are free. Chapter 2 walks through every setup step.

Tool / Account	Purpose	Cost
SAP BTP Trial	Runtime, HANA Cloud, CAP deployment	Free
GCP Account	Vertex AI API (Gemini + embeddings)	Free (\$300 credit)
Python 3.11+	Agent development	Free
Node.js 20+	CAP frontend development	Free
CF CLI	Deploy to BTP Cloud Foundry	Free
CDS CLI (@sap/cds-dk)	CAP development	Free
VS Code	Code editor	Free

Companion Code Repository

All source code for this book lives in a single GitHub repository:

<https://github.com/sriramrokkam/book-agentic-hybrid-rag-on-btp>

Repository Structure

book-agentic-hybrid-rag-on-btp/

```
├── agents/                # Python FastAPI + LangGraph agent layer
│   ├── agents/           #   LangGraph chains and orchestrator
│   └── srv/              #   HANA, vector, KG, and doc services
```



```

|   └─ main.py          # FastAPI entrypoint – all HTTP endpoints
|   └─ requirements.txt
|   └─ .env.example     # Required environment variables template
└─ cap-srv/             # CAP Node.js OData V4 service
    └─ db/schema.cds    # Data model
    └─ srv/service.cds  # Service definition and actions
    └─ srv/service.js   # Action handlers (proxies to agent layer)
    └─ .env.example
└─ MSDS_Ontology.ttl    # OWL ontology constraining knowledge graph
└─ docs/                # Book chapters and screenshots (this folder)
    └─ CODE_MAP.md      # Chapter → source file cross-reference
    └─ chapters/        # All chapter markdown files
└─ README.md            # Complete local setup guide

```

Quick Start

```

1  # 1. Clone
2  git clone
   https://github.com/sriramrokkam/book-agentic-hybrid-rag-on-btp.git
3  cd book-agentic-hybrid-rag-on-btp
4
5  # 2. Set up Python environment
6  python3 -m venv agents/.venv
7  source agents/.venv/bin/activate
8  pip install -r agents/requirements.txt
9
10 # 3. Configure credentials
11 cp agents/.env.example agents/.env
12 # Edit agents/.env – see README.md for where to find each value
13
14 # 4. Run the agent layer
15 uvicorn agents.main:app --reload --host 0.0.0.0 --port 8000
16
17 # 5. Run the CAP layer (separate terminal)
18 cd cap-srv && npm install && cds serve

```

See `README.md` in the repository root for the complete step-by-step setup guide, including all environment variables, endpoints, and deployment instructions for SAP BTP Cloud

Foundry.

Chapter-to-Code Cross-Reference

`docs/CODE_MAP.md` maps every chapter to the exact source files and functions it covers. Use it to jump directly to the code for any chapter without searching the repository.

How to Use This Book

Reading Cover to Cover (Recommended)

Each chapter builds directly on the previous one. The system grows chapter by chapter — by Chapter 9 you have a fully deployed, working Hybrid RAG Agent on SAP BTP. Reading linearly gives you the full progression from concept to production.

As a Reference

If you already have some of the pieces (a HANA Cloud instance, some LangGraph experience), jump to the chapter you need. Each chapter opens with a summary of what it assumes you already have from previous chapters.

Chapter Structure

Every chapter in this book follows the same pattern:

1. **Opening question** — the single question this chapter answers
2. **Concept** — the idea explained in plain language before any code
3. **Code** — we build the component step by step
4. **Explanation** — what each part does and why it is designed that way
5. **Test** — how to verify it works before moving on
6. **Summary** — what we built and what comes next
7. **Checkpoint** — a checklist before starting the next chapter

The Code

All code is in the companion GitHub repository:

<https://github.com/sriramrokkam/book-agentic-hybrid-rag-on-btp>

Each chapter has a corresponding folder in the repo. If you get stuck, the repo has the complete working code for reference.

Conventions Used in This Book

Code Blocks

```
1 # Python code appears in blocks like this
2 def example():
3     return "this is a code example"
```

```
1 // JavaScript/Node.js code looks like this
2 const example = () => "this is a code example";
```

```
1 # Terminal commands look like this
2 cf push my-app
```

```
1 -- SQL queries look like this
2 SELECT * FROM MY_TABLE;
```

Callout Boxes

Note: Additional context or clarification that is helpful but not critical to follow along.

Important: Something you must do or understand before continuing. Skipping this will cause problems later.

Warning: A common mistake or pitfall. Pay attention here.

Tip: A shortcut, best practice, or time-saving suggestion.

File Paths

File paths are shown relative to the project root: - `agents/srv/hdb_srv.py` — the HANA connection service in the agents module - `cap-srv/db/schema.cds` — the CDS data model

Screenshots

Screenshots show the state of the UI at the time of writing. SAP BTP and GCP update their interfaces regularly. If a screen looks slightly different, the underlying action is the same — use the search or navigation to find the equivalent option.

[SCREENSHOT: *description*] markers in this draft will be replaced with actual screenshots in the final published version.

Acknowledgements

[To be added by Sriram Rokkam. Suggested: colleagues who reviewed chapters, the SAP and Google Cloud developer communities, anyone who provided feedback or inspiration.]

About the Author

Sriram Rokkam is a [title] with [X] years of experience building enterprise applications on SAP BTP and Google Cloud Platform.

[Complete this section in your own words — your current role, notable projects, SAP community involvement, LinkedIn/GitHub handles.]

End of Front Matter — Chapter 1 begins on the next page.

Chapter 1: The SAP Document Problem — And Why BTP Is the Right Place to Solve It

1.1 The Document Fragmentation Problem in SAP Landscapes

Every material in an SAP ERP or S/4HANA system has a Material Number — the unique identifier you work with in MM03, MM60, QM, and across the supply chain. Over a material's lifecycle, a set of documents accumulates around it: Material Safety Data Sheets (MSDS/SDS), Batch Quality Certificates, Goods Receiving Inspection Reports, Supplier Invoices, Equipment Maintenance History, User and Installation Manuals, Legal Compliance Documents, and Regulatory Filings. These documents collectively define what the material is, how it behaves, how it must be handled, and what legal obligations govern its use.

Here is the problem: standard SAP provides no single transaction where all of these documents are stored, linked to the material, and made intelligently searchable. In practice they end up scattered. MSDS sheets live in a SharePoint folder someone set up in 2014. Batch certificates are emailed by suppliers and archived in Outlook. Inspection reports sit in a plant-level file server. DMS contains some documents but has no query interface beyond document type and metadata fields.

When a quality engineer asks *“Was this batch of material 800021 certified before it shipped?”* she emails a colleague and waits. When a safety officer asks *“What are the storage temperature limits for this chemical?”* he opens a PDF manually and reads through Section 7. When a procurement analyst needs to confirm a supplier invoice against a goods receipt, he has two systems open side by side.

This fragmentation is structural. SAP's document management capabilities — GOS attachments, DMS with DIR objects, extended object links — handle document storage and classification. What they do not provide is intelligent retrieval. You cannot open any standard SAP transaction, type a natural-language question about a material, and get a direct answer synthesised from the documents attached to it. You retrieve the document and do the reading yourself.

This is the gap this book fills: an **Agentic Hybrid RAG Document Intelligence Plat-**

form on SAP BTP that:

- Accepts any PDF document linked to an SAP Material Number, validated against your live S/4HANA product catalogue via `API_PRODUCT_SRV` OData
- Ingests each document into a HANA Cloud vector table (`MSDS_VECTORS`, `REAL_VECTOR` column type) and into a named RDF graph (`MSDS_Graph/MAT-{material}`) simultaneously, in one upload
- Answers natural-language questions by running vector retrieval and SPARQL graph retrieval in parallel (`ThreadPoolExecutor`), merging the results, and synthesising a single cited answer via Gemini 2.5 Flash
- Exposes the full capability as a CAP Node.js OData V4 service with a Fiori Elements UI — the SAP-native interface your users already know
- Runs free: SAP BTP trial + Google Cloud \$300 free credit

The **MSDS (Material Safety Data Sheet)** is the reference document type implemented throughout this book — hence the naming convention `MSDS_VECTORS`, `MSDS_Graph`. The architecture, ingestion pipeline, and query orchestration are document-type agnostic. Swap the knowledge graph extraction prompt and the pattern applies to batch quality certificates, inspection reports, maintenance manuals, or any document class you need.

1.2 Why Hybrid Retrieval — Not Just Search

When a safety officer asks about storage requirements for a chemical, the answer exists somewhere in an MSDS. But that question actually demands two fundamentally different kinds of retrieval, and this distinction is what drives the entire architecture of this system.

Documents contain two kinds of content.

The first kind is **structured facts**: GHS hazard category codes, UN transport numbers, flash points in degrees Celsius, permissible exposure limits in mg/m^3 , certification dates, batch lot numbers, compliance standard references (REACH, CLP, OSHA). These are discrete, precise values. The right answer to “*What is the flash point of acetone?*” is “*-20°C*” — not a paragraph that mentions temperature somewhere.

The second kind is **narrative prose**: first-aid procedures, personal protective equipment recommendations, handling and storage guidance, ecological impact descriptions, disposal instructions. These are sentences and paragraphs written in natural language.

The right answer to “*What should I do if someone inhales acetone vapour?*” is a passage from Section 4 of the MSDS — and finding it requires understanding semantic meaning, not matching keywords.

Vector search alone fails on structured facts. Ask for a GHS hazard code and vector search returns the chunk with the highest cosine similarity to your query — often around 0.65 — which may or may not contain the specific code. There is no guarantee of precision. You might retrieve a chunk that discusses GHS categories generally without stating the specific code for your material.

A knowledge graph alone fails on narrative questions. A SPARQL query returns triples — subject, predicate, object — from the RDF graph. It can answer “*Give me the flash point of CHEMICAL-001*” in one query. It cannot answer “*Describe the first-aid procedure for skin contact with CHEMICAL-001*” because that information is prose, not a fact that maps cleanly to a triple.

SAP HANA Cloud is the only managed database service that provides both in one instance. `REAL_VECTOR` cosine similarity search for semantic retrieval, and SPARQL 1.1 endpoint (`SPARQL_EXECUTE`) for graph queries — same HANA instance, same network hop, same connection pool. No separate vector database, no separate graph store, no additional BTP service to provision.

The hybrid approach is therefore not a design preference. It is the correct response to the actual structure of enterprise documents. Run both chains in parallel. Synthesise when both return results. Defer to whichever chain returns when only one does. Acknowledge uncertainty when neither does. That is the architecture.

1.3 The Four Capabilities That Define the Agent

The system in this book is not a fixed retrieval pipeline. It makes decisions at runtime. Four specific capabilities distinguish it from a deterministic workflow.

Tool Use

The LangGraph orchestrator calls external functions to answer questions. In SAP terms: the knowledge graph chain calls `SPARQL_EXECUTE` against HANA Cloud and extracts structured triples. The vector chain calls a cosine similarity search against the `MSDS_VECTORS REAL_VECTOR` column. Either chain can be selected, skipped, or retried

based on what the query needs. The LLM does not guess — it calls the right tool.

Memory

State persists beyond a single request. The HANA Cloud tables and named RDF graphs are the memory layer. Once you upload an MSDS for material `800021`, its vector chunks remain in `MSDS_VECTORS` and its triples remain in `GRAPH <iri:msds-graph/MAT-800021>` across sessions, across restarts, across users. Session-level context (conversation history, intermediate tool outputs) is held in the LangGraph `StateGraph` for the duration of a query. Long-term memory is HANA. Short-term memory is the graph state.

Planning

The LangGraph `StateGraph` routes each query at runtime. A question phrased as “*What is the GHS hazard code?*” routes primarily to the graph chain — it is a structured fact lookup. A question phrased as “*Describe the first-aid procedure*” routes primarily to the vector chain — it is a narrative retrieval. When both chains return results, the supervisor node decides how to weight and merge them before passing to the synthesis node. The routing is not a hard-coded `if/else` — it emerges from the conditional edge logic and the confidence scores returned by each chain.

Multi-Agent Collaboration

The orchestrator is a supervisor agent. It delegates to two specialist sub-agents — the Vector Retrieval Agent and the Knowledge Graph Agent — each of which has its own tool set, its own HANA connection (thread-local), and its own result format. The supervisor collects both results, applies the merge strategy, and passes the combined context to Gemini 2.5 Flash for synthesis. Adding a third specialist agent (for example, an SAP OData query agent that retrieves live inventory data alongside the document answer) requires adding one node and one conditional edge to the graph.

1.4 SAP’s Strategic Direction: The Autonomous Enterprise (Sapphire 2026)

The timing of this book is not coincidental. SAP has placed agentic AI at the centre of its entire product strategy, and the architecture you build here maps directly to what SAP is standardising.

At **SAP Sapphire 2026 in Orlando**, CEO Christian Klein unveiled the **Autonomous Enterprise** — a model where AI agents and human workers collaborate continuously to execute, adapt, and optimise critical business processes. Klein:

“For the mission-critical processes of our customers, ‘almost right’ just isn’t good enough. By uniting SAP Business AI Platform with SAP Autonomous Suite, we anchor AI agents in the business processes, data and governance so they can deliver accurate, compliant and secure outcomes.”

The scale of what SAP announced is significant:

Milestone	Detail
200+ specialised agents	Available for orchestration across the SAP portfolio — Finance, Supply Chain, Procurement, HR, CX
50+ Joule Assistants	Domain-specific AI assistants embedded across S/4HANA, Ariba, SuccessFactors, and more
7 Industry AI solutions	Vertical-specific autonomous solutions through the Industry AI initiative
SAP Autonomous Suite	New product suite embedding agents directly into core SAP applications
Joule Studio	No-code and pro-code environment on BTP for building custom agents
€100 million partner fund	Accelerating partner deployments of AI agents across the ecosystem

SAP also announced a partnership with **NVIDIA** — specifically the **OpenShell** secure runtime environment for Joule Studio agent execution — and the planned acquisition of **Prior Labs**, a European frontier AI research lab, signalling that SAP intends to control the AI stack from the data layer through the model layer.

What this means for you as a BTP developer: the 200+ agents SAP announced were not all built by SAP. Partners and customers on BTP built them using the same patterns

this book teaches — LangGraph orchestration, HANA Cloud retrieval, CAP as the OData surface, Joule A2A for integration. You are building on the architecture SAP is standardising. The skills you develop here translate directly to Joule Studio, SAP Business AI Platform, and the autonomous enterprise patterns SAP's largest customers are now adopting.

1.5 What We Build: System Overview

The anchor for everything in this system is the **SAP Material Number**. It is not arbitrary — it comes from your live S/4HANA product catalogue. Before a document can be uploaded, the material number is validated against `API_PRODUCT_SRV` (SAP's standard product OData V4 service), which confirms the material exists and returns basic product attributes. This keeps the system honest: you cannot ingest a document against a material number that does not exist in your SAP system.

Ingestion flow:

Upload a PDF against material `800021`. The CAP OData V4 service (`/odata/v4/msds/uploadDocument`) receives the request, validates the material number, and calls the FastAPI backend (`POST /ingest`). The backend runs two ingestion tasks in parallel:

- **Vector ingestion:** The PDF is parsed, chunked (512 tokens, 50-token overlap), and each chunk is embedded with `text-embedding-004`. Embeddings and chunk text are inserted into `MSDS_VECTORS` (a HANA Cloud column table with a `REAL_VECTOR` column). Index: cosine similarity.
- **KG ingestion:** Gemini 2.5 Flash reads the full document and extracts structured triples — material identifiers, GHS codes, physical properties, regulatory standards, certifying bodies, dates. Triples are written to HANA Cloud as RDF in the named graph `GRAPH <iri:msds-graph/MAT-800021> using SPARQL_EXECUTE`.

Both run in a `ThreadPoolExecutor`. Both complete before the response returns to the CAP layer.

Query flow:

The user types a question in the Fiori Elements chat UI. The CAP service proxies it to the FastAPI backend (`POST /query`). The LangGraph supervisor node initialises a `State-Graph` and fans out to both chains in parallel:

- The **Vector Chain** embeds the question, runs cosine similarity search against `MSDS_VECTORS` for the target material, and returns the top-k chunks with similarity scores
- The **KG Chain** converts the question to a SPARQL SELECT query, executes it against `SPARQL_EXECUTE` on the named graph, and returns structured triples

The supervisor merge node receives both result sets, formats them as context, and calls Gemini 2.5 Flash with a synthesis prompt that instructs the model to cite the source (vector chunk reference or graph triple) for every claim in the answer. The cited answer returns to the Fiori UI.

UI and deployment:

The Fiori Elements UI is generated entirely from the CAP OData V4 annotations — no custom frontend code. The chat page uses a `sap.fe` freestyle page with standard SAPUI5 controls. The CAP service (`cds serve`) runs on port 4004. The FastAPI agent (`uvicorn main:app`) runs on port 8000. In Chapter 10, both are deployed to BTP Cloud Foundry. The CAP service is deployed as a single MTA, the FastAPI service as a Python CF application.

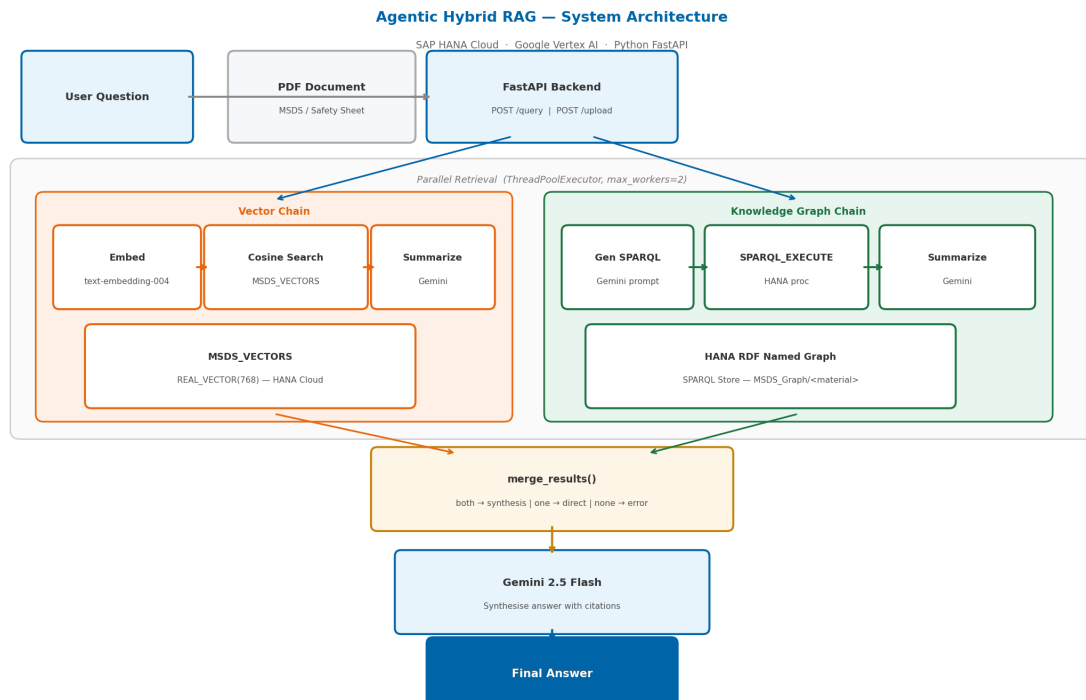


Figure 1: Agentic Hybrid RAG System Architecture

Figure 1.1 — Material Number is the anchor. Documents are ingested into `MSDS_VECTORS` (vector) and `MSDS_Graph/MAT-{material}` (RDF) in parallel. Queries

fan out to both chains, merge, and synthesise a cited answer via Gemini 2.5 Flash.

1.6 Why MSDS First

MSDS documents are the ideal worked example for this architecture, for one specific reason: they contain both kinds of retrievable content in roughly equal proportion.

Section 1 of an MSDS (Product Identification), Section 2 (Hazard Identification), Section 9 (Physical and Chemical Properties), and Section 15 (Regulatory Information) are structured fact stores. They contain GHS hazard category codes, signal words, UN transport numbers, flash points, boiling points, permissible exposure limits, and references to specific regulatory standards (REACH Regulation EC 1907/2006, OSHA 29 CFR 1910.1200, CLP Regulation EC 1272/2008). These facts map directly to RDF triples: `<CHEMICAL-001> ghs:hazardCategory "Flammable Liquids Cat. 2"`.

Section 4 (First-Aid Measures), Section 6 (Accidental Release Measures), Section 7 (Handling and Storage), Section 8 (Exposure Controls / PPE), and Section 13 (Disposal Considerations) are narrative prose. They cannot be decomposed into triples without losing their meaning. A first-aid procedure for inhalation exposure is a paragraph, not a fact. Vector search with cosine similarity finds it correctly.

A quality or safety professional asking questions about an MSDS needs answers from both halves in the same conversation. *“Is this chemical classified as a flammable liquid?”* — graph. *“What PPE should maintenance staff wear when handling it?”* — vector. *“What is the flash point, and at what concentrations does it become a fire risk?”* — graph for the flash point, vector for the concentration narrative.

Any manufacturing, chemical, pharmaceutical, or industrial company operating on SAP has hundreds of MSDS documents. The legal obligation to have them accessible is explicit (REACH, GHS, OSHA). The technical debt of managing them in scattered file systems is universal.

The architectural pattern built here transfers directly to other document types:

Document Type	Graph Content (structured)	Vector Content (narrative)
Batch Quality Certificate	Lot number □ test result □ specification	Test methodology descriptions

Document Type	Graph Content (structured)	Vector Content (narrative)
Supplier Invoice	Vendor □ line item □ material □ amount	Payment terms, dispute notes
Equipment Maintenance Record	Asset □ service action □ date □ technician	Failure description, root cause narrative
Goods Receiving Inspection Report	Batch □ inspection result □ QM usage decision	Inspector observations
Regulatory Compliance Filing	Standard □ obligation □ deadline □ status	Compliance justification text

Change the KG extraction prompt to match the document type's ontology. The vector chunking, HANA ingestion, LangGraph orchestration, and CAP OData layer are unchanged.

1.7 Scope and Prerequisites

What This Book Covers

Topic	Coverage
SAP document fragmentation problem and BTP solution architecture	Full coverage, Ch 1
SAP BTP trial setup — HANA Cloud, CF space, Destination Service	Full coverage, Ch 2
Vector search on HANA Cloud (REAL_VECTOR, cosine similarity)	Full coverage, Ch 3
Knowledge graphs on HANA Cloud (SPARQL/RDF, named graphs)	Full coverage, Ch 4

Topic	Coverage
PDF ingestion pipeline — parallel KG + vector with ThreadPoolExecutor	Full coverage, Ch 5
LangGraph — StateGraph, nodes, conditional edges, routing	Full coverage, Ch 6
Hybrid RAG agent — parallel chains, merge strategy, synthesis	Full coverage, Ch 7
Multi-agent supervisor pattern — specialist sub-agents	Full coverage, Ch 8
CAP Node.js OData V4 service + Fiori Elements UI	Full coverage, Ch 9
Deploying to SAP BTP Cloud Foundry (MTA + cf push)	Full coverage, Ch 10
Joule A2A integration (enterprise subscription required)	Appendix D
SAP AI Core as LLM alternative	Appendix E
MSDS ontology design (OWL/Turtle)	Appendix B
HANA Cloud SPARQL reference	Appendix C

What This Book Does NOT Cover

Production hardening. Rate limiting, multi-tenant isolation, secrets rotation, disaster recovery, and SLA monitoring are not covered. This book teaches architecture and implementation patterns, not operations.

Model training or fine-tuning. All LLM usage is inference-only — calling hosted models via API.

Cross-material reasoning. The agent answers questions about one material's documents at a time. Cross-material aggregation (e.g. *"Which of our chemicals in this plant require respiratory PPE?"*) is a natural extension not covered here.

Streaming responses. Answers are returned synchronously as complete text.

CI/CD pipelines. Chapter 10 uses manual `cf push` and `mbt build && cf deploy`.

Joule and SAP AI Core on free trial. Both require paid SAP subscriptions. They are in Appendices D and E so enterprise readers can follow along without blocking the main chapters.

Prerequisites

Requirement	Where to Get It	Cost
SAP BTP Trial account	account.hanatrial.ondemand.com	Free
SAP HANA Cloud trial instance	BTP Trial cockpit → HANA Cloud tile	Free (30-day)
GCP account with Vertex AI enabled	cloud.google.com/free	Free (\$300 credit)
Python 3.11+	python.org	Free
Node.js 20+ and npm	nodejs.org	Free
CF CLI v8+	CF documentation	Free
@sap/cds CLI (npm i -g @sap/cds)	npm	Free
VS Code with SAP CDS extension	code.visualstudio.com	Free

Chapter 2 covers every setup step with screenshots. If you already have an active BTP trial and GCP project, setup takes roughly 30 minutes.

Local development is the primary mode for Chapters 1–9. You run `uvicorn main:app --reload` for the FastAPI agent and `cds serve` for the CAP service on your local machine, both connecting to your BTP HANA Cloud trial instance. Chapter 10 deploys the same code to Cloud Foundry — nothing changes except where it runs.

1.8 Summary

- Standard SAP has no transaction that stores all material-linked documents in one place and makes them intelligently queryable. Documents are scattered across shared drives, email, DMS, and plant servers. This is a structural gap in SAP’s portfolio that BTP is built to fill.
- This book builds an Agentic Hybrid RAG Document Intelligence Platform: upload

PDFs against an SAP Material Number, ingest to HANA Cloud vector store and RDF knowledge graph simultaneously, query both in parallel, synthesise cited answers via Gemini 2.5 Flash.

- Hybrid retrieval is not a design choice — it is the correct response to the structure of enterprise documents. Structured facts (GHS codes, batch numbers, flash points, compliance standards) need SPARQL exact lookup. Narrative prose (procedures, descriptions, instructions) needs vector cosine similarity. HANA Cloud provides both in one managed service.
- The system is agentic in a precise sense: it uses tools (SPARQL_EXECUTE, REAL_VECTOR search), persists memory (HANA tables and RDF graphs), plans routing at runtime (LangGraph StateGraph conditional edges), and delegates to specialist sub-agents (supervisor pattern).
- SAP announced the Autonomous Enterprise at Sapphire 2026: 200+ specialised agents, Joule Studio on BTP, SAP-NVIDIA partnership, Prior Labs acquisition. The architecture this book teaches is the same architecture SAP is standardising.
- MSDS is the reference document type because it contains structured facts and narrative prose in equal measure — ideal for demonstrating both retrieval chains. The same architecture transfers to batch certificates, invoices, inspection reports, and maintenance records by changing only the KG extraction prompt.
- The full stack runs at zero cost: SAP BTP trial + Google Cloud \$300 free credit.

Chapter 2 sets up the platform: BTP trial, HANA Cloud instance, GCP project, and Vertex AI credentials. By the end of Chapter 2 you have a working environment and can run your first HANA vector insert.

Chapter 2: Platform Setup — SAP BTP Trial + Google Vertex AI

2.1 The Goal of This Chapter

Before we write a single line of agent code, we need a working environment. This chapter gets you there.

By the end of this chapter, you will have:

- A running SAP BTP trial account with a HANA Cloud instance
- A GCP project with Vertex AI enabled and a service account ready
- A BTP Destination that securely connects your BTP applications to Vertex AI
- A local development environment with all required tools installed
- A confirmed, working API call to Gemini 2.5 Flash from your local machine

This is pure setup. There is no agent code here. But every chapter that follows depends on this foundation being solid. Rushing this chapter and skipping verification steps is the most common cause of confusing errors later. Take the 45 minutes. Do it properly.

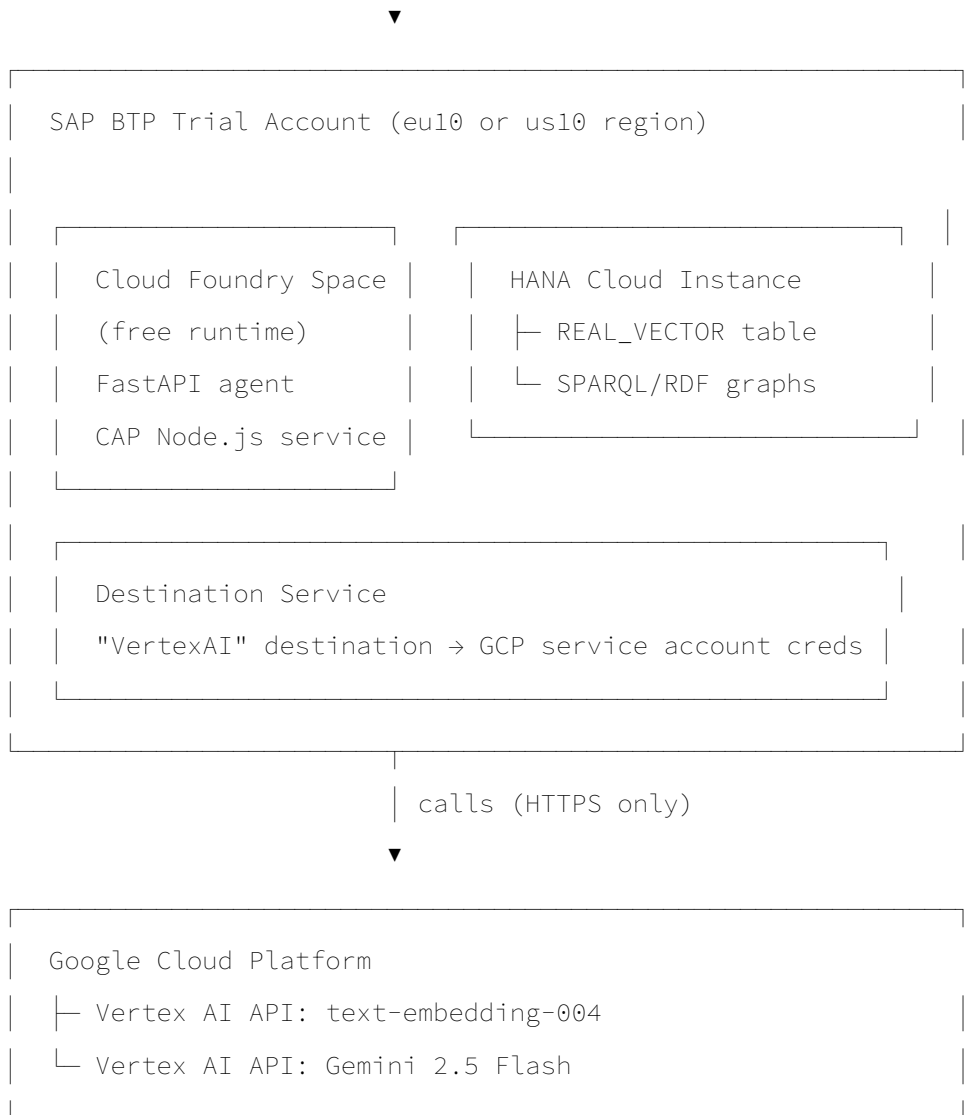
Cost reminder: Everything in this chapter is free. SAP BTP trial is free. GCP provides \$300 credit on signup. The Vertex AI API calls in this chapter will cost cents — comfortably within the trial credit.

2.2 Understanding What We Are Setting Up

Before clicking through consoles, it helps to understand the shape of what we are building.

Our system has three infrastructure components:





SAP HANA Cloud is the **knowledge layer** — it stores both our vector embeddings and our RDF knowledge graph in a single managed service. No other database on the market provides both `REAL_VECTOR` column types (for cosine similarity search) and a native SPARQL execution engine in one service. This is a significant architectural advantage — one connection, one credential, two retrieval strategies.

Google Vertex AI is the **AI inference layer** — it is an external HTTPS API that our BTP application calls like any other REST service. Gemini 2.5 Flash handles both our LLM tasks (SPARQL generation, answer synthesis) and we use `text-embedding-004` for generating embeddings. The BTP Destination Service stores the GCP credentials securely — the service account JSON key never lives in application code or environment variables directly.

With this picture in mind, let us set it up.

2.3 SAP BTP Trial Account

Signing Up

Navigate to <https://www.sap.com/products/technology-platform/trial.html> and click **Try now**.

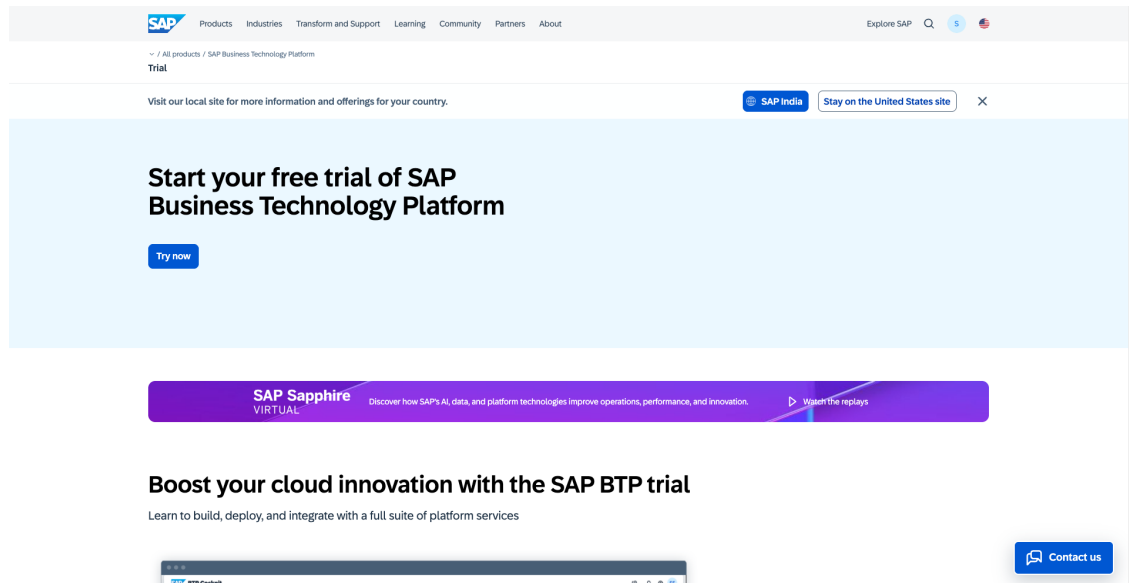


Figure 2.1 — SAP BTP Free Trial landing page — click “Try now” to start your free trial account

Fill in your details. Use a personal email address if possible — corporate email addresses sometimes cause issues with trial account activation. After verifying your email, BTP will automatically provision a trial account in either the EU10 (Frankfurt) or US10 (Virginia) region depending on your location.

Note: The region matters. Your HANA Cloud instance must be in the same region as your Cloud Foundry space. BTP assigns both automatically — do not change the region during signup.

Once logged in, you will land on the **BTP Cockpit** — the central control panel for all BTP services.

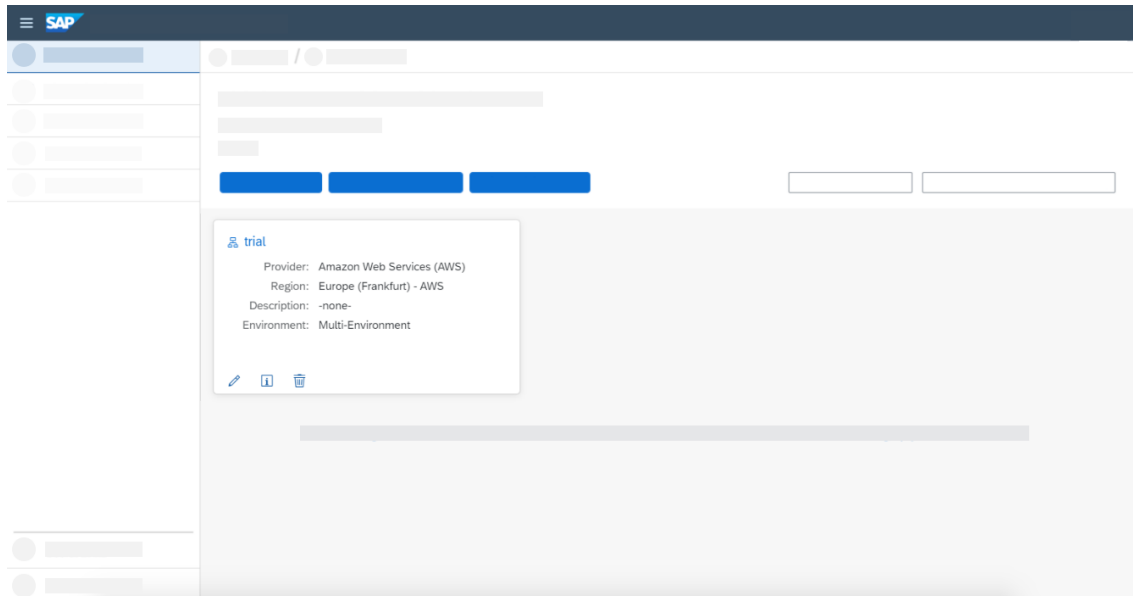


Figure: SAP BTP Cockpit home screen showing your Global Account and trial subaccount

Navigating to Your Trial Subaccount

The BTP Cockpit has a hierarchy: Global Account → Subaccount → Space. Your trial comes with one subaccount called “trial.” Click it.

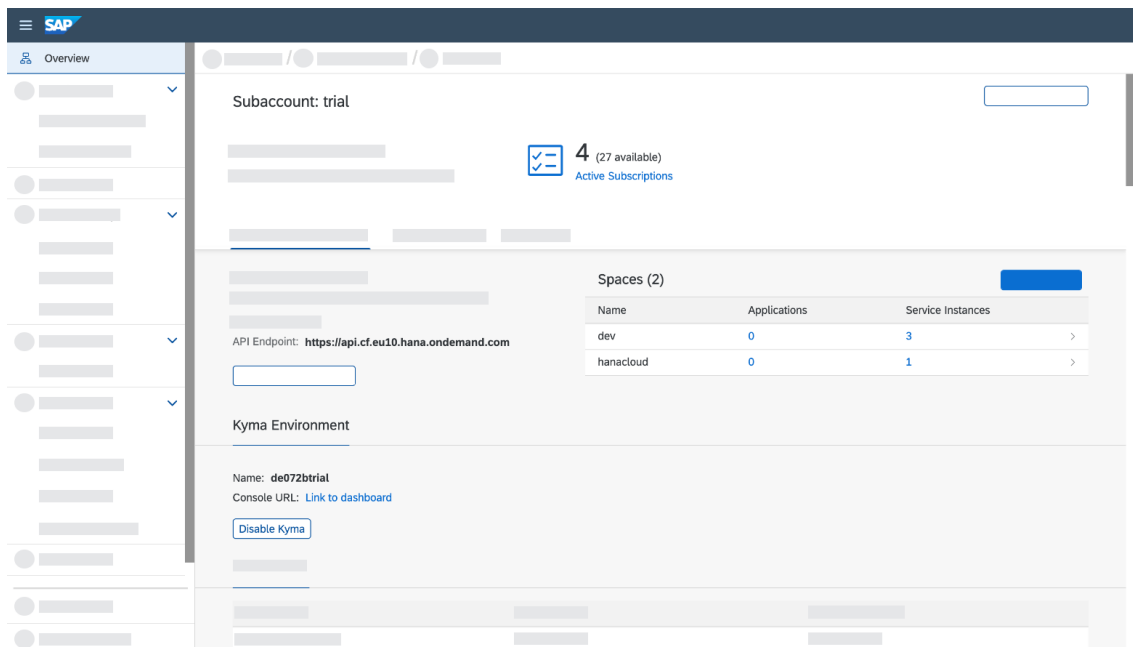


Figure: BTP trial subaccount overview page

Inside the subaccount, you will see your **Cloud Foundry environment**. Note your CF API endpoint — it will look like `https://api.cf.eu10.hana.ondemand.com`. You will need this later.

Click **Cloud Foundry** ▢ **Spaces**. You should see a space called “dev.” This is where you will deploy your applications.

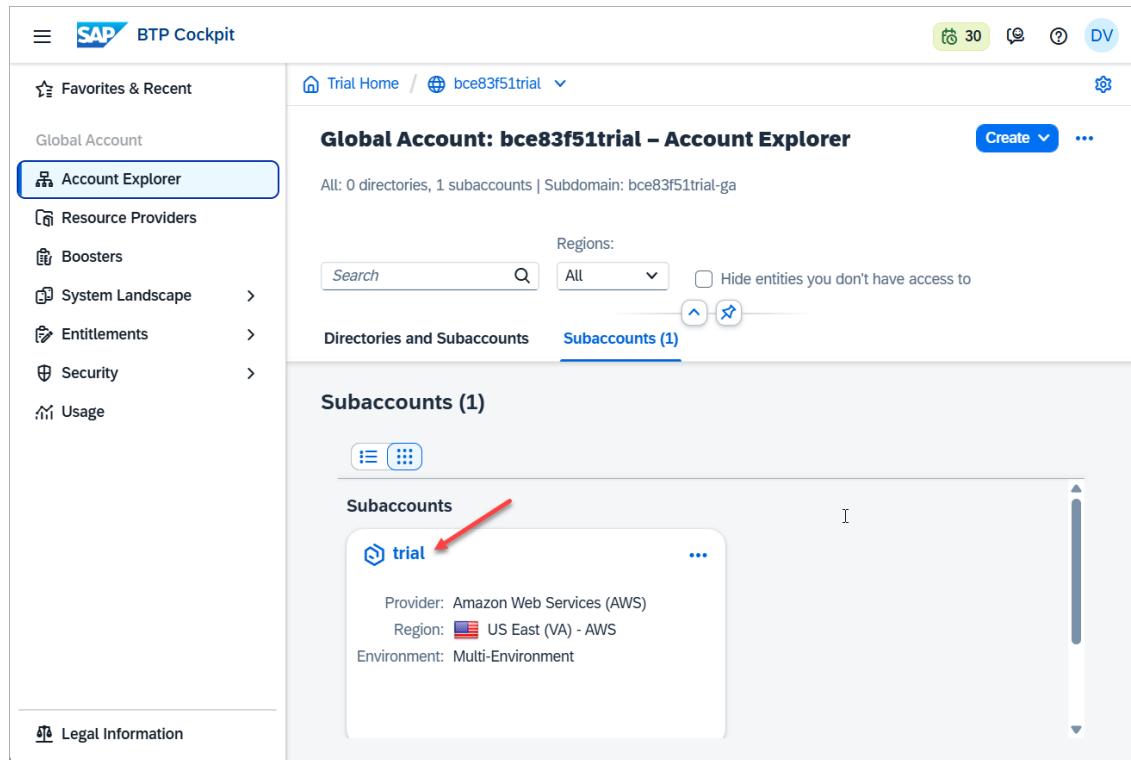


Figure: Cloud Foundry space view — note your org name and the “dev” space

Write these down now: - CF API Endpoint: `https://api.cf.<region>.hana.ondemand.com`

- CF Org: `<your-trial-org>` - CF Space: `dev`

2.4 Provisioning SAP HANA Cloud

HANA Cloud is provisioned from the BTP Cockpit. This is a one-time setup that takes about 10–15 minutes.

Creating the HANA Cloud Instance

From your trial subaccount, go to **Services** ▢ **Service Marketplace**. Search for “HANA Cloud.”

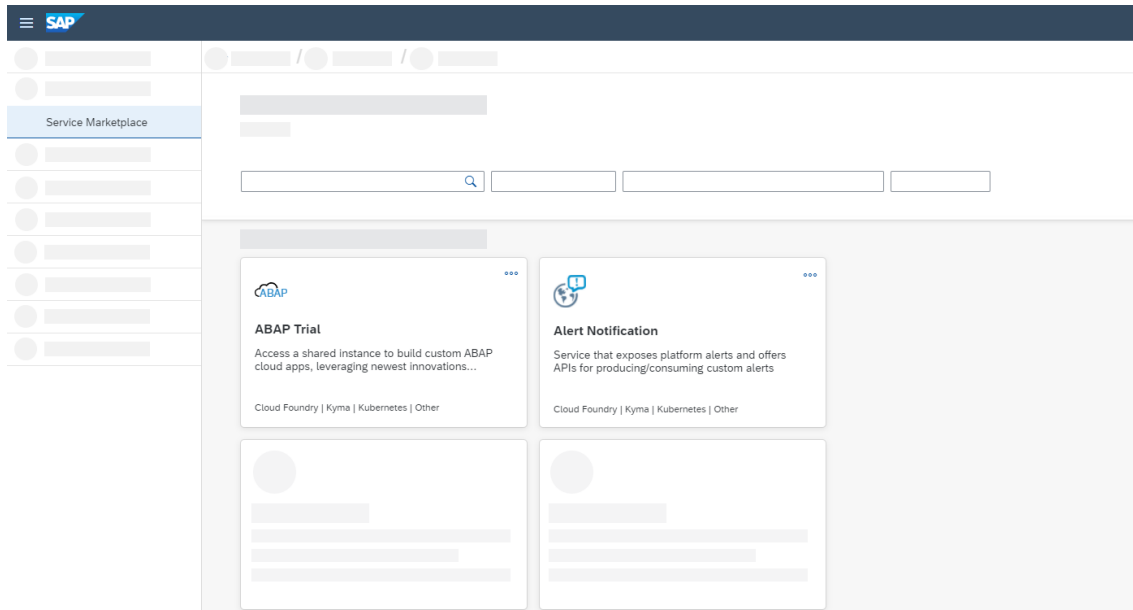


Figure: BTP Service Marketplace — search for “HANA Cloud” to find the tile

Click the HANA Cloud tile **Create**. You will be taken to the HANA Cloud provisioning wizard.

Fill in the configuration:

Field	Value
Instance Name	hana-trial
Administrator Password	Choose a strong password (save it!)
Memory	30 GB (the trial default — do not reduce)
Storage	120 GB (the trial default)

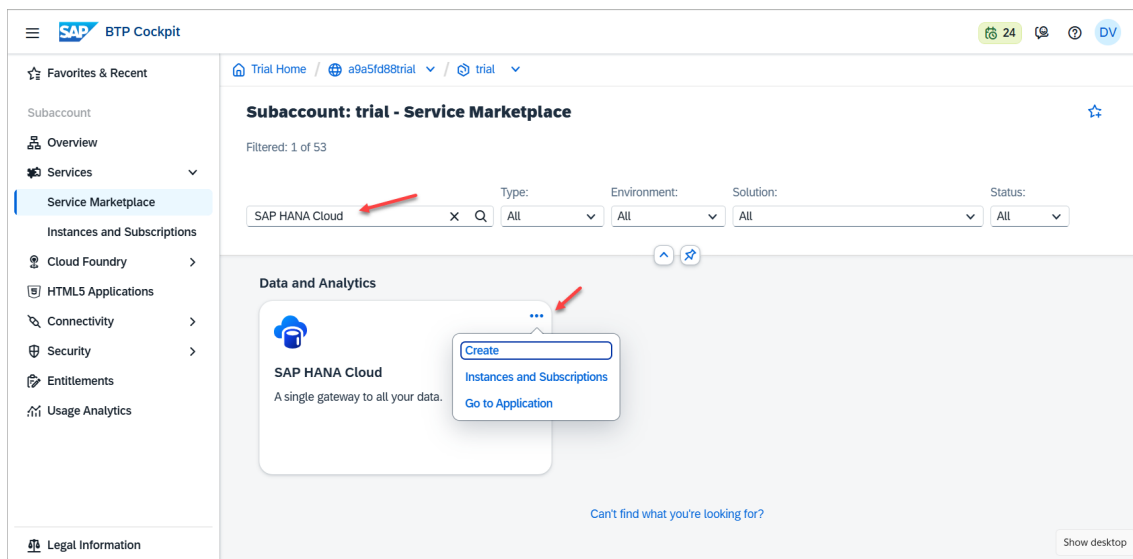


Figure: HANA Cloud instance creation wizard — set instance name and keep default

30GB memory

Click **Create**. Provisioning takes approximately 10–15 minutes. You will see the instance appear in the SAP HANA Cloud Central tool with a “Starting” status.

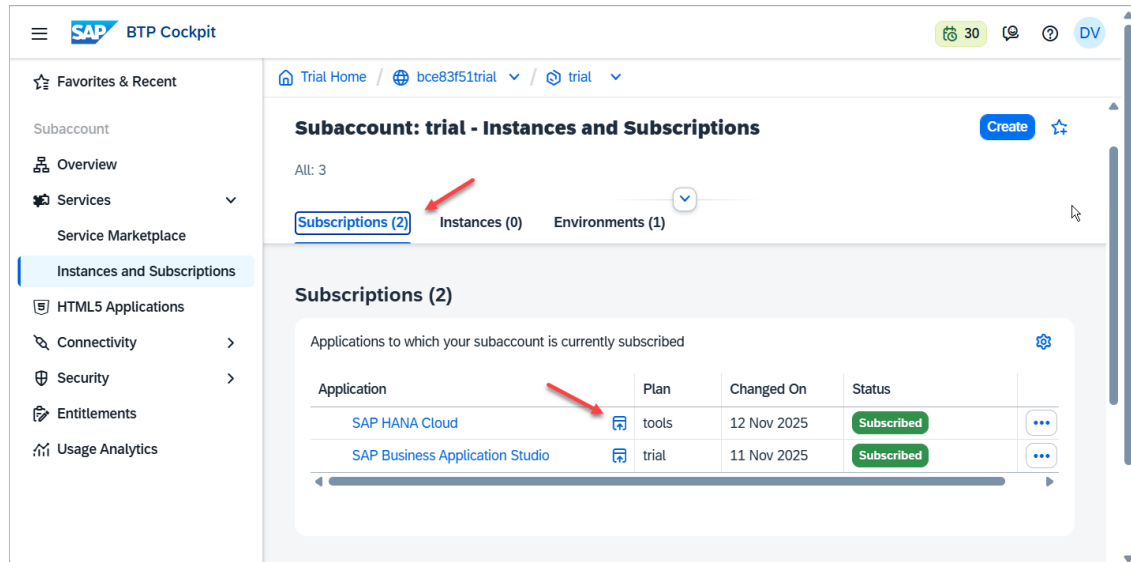


Figure: SAP HANA Cloud Central showing the instance provisioning status

Wait until the status shows **Running** before proceeding.

Enabling the Vector Engine

By default, HANA Cloud provisions with the vector engine enabled in trial. To confirm, open **SAP HANA Cloud Central** → click your instance → **Manage HANA Cloud** → **Edit**.

Under **Additional Features**, verify that **Script Server** and **Document Store** are enabled. The vector engine (REAL_VECTOR column type) is part of the core engine — no separate flag is needed.

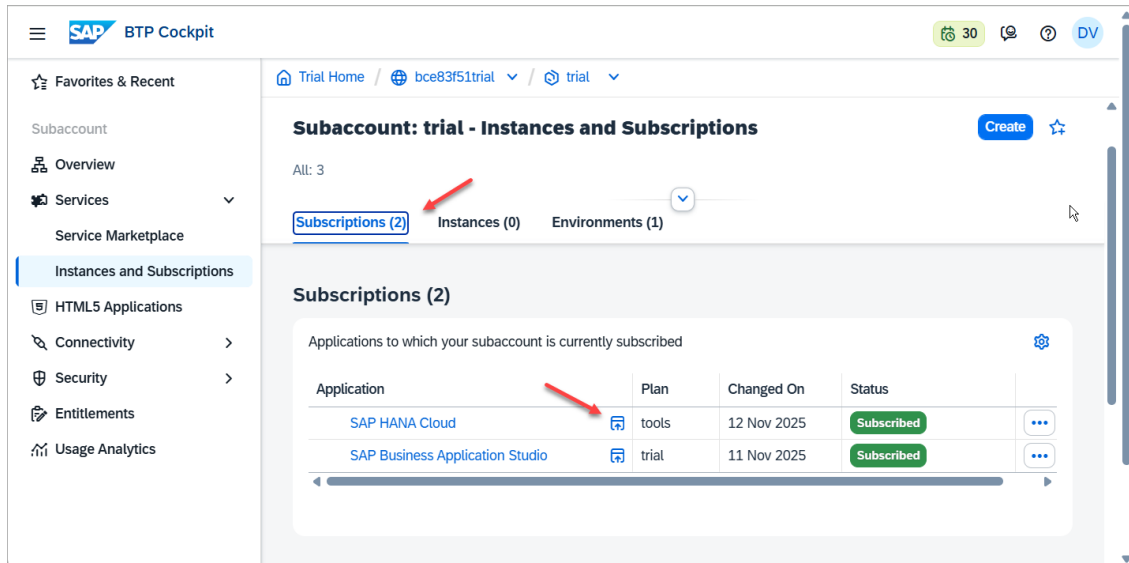


Figure: HANA Cloud Central — click your instance to manage settings and verify vector engine availability

Getting Your HANA Connection Details

From SAP HANA Cloud Central, click your instance ▢ **Actions** ▢ **Copy SQL Endpoint**.

This gives you a connection string like:

```
<instance-id>.hana.trial-us10.hanacloud.ondemand.com:443
```

Save these: - **Host:** <instance-id>.hana.trial-<region>.hanacloud.ondemand.com

- **Port:** 443 - **User:** DBADMIN - **Password:** The password you set during provisioning

Warning: The DBADMIN user has full database privileges. For production, create a dedicated application user with limited permissions. For this book's trial environment, DBADMIN is acceptable.

2.5 Setting Up Google Cloud + Vertex AI

Creating a GCP Account

Go to cloud.google.com/free and click **Get started for free**.



Figure: Google Cloud free tier page — click “Get started for free” to claim your \$300 credit

Sign in with your Google account. GCP will ask for a credit card for identity verification — **you will not be charged** during the free trial, and the \$300 credit covers everything in this book many times over.

After signup, you will land on the **GCP Console**.

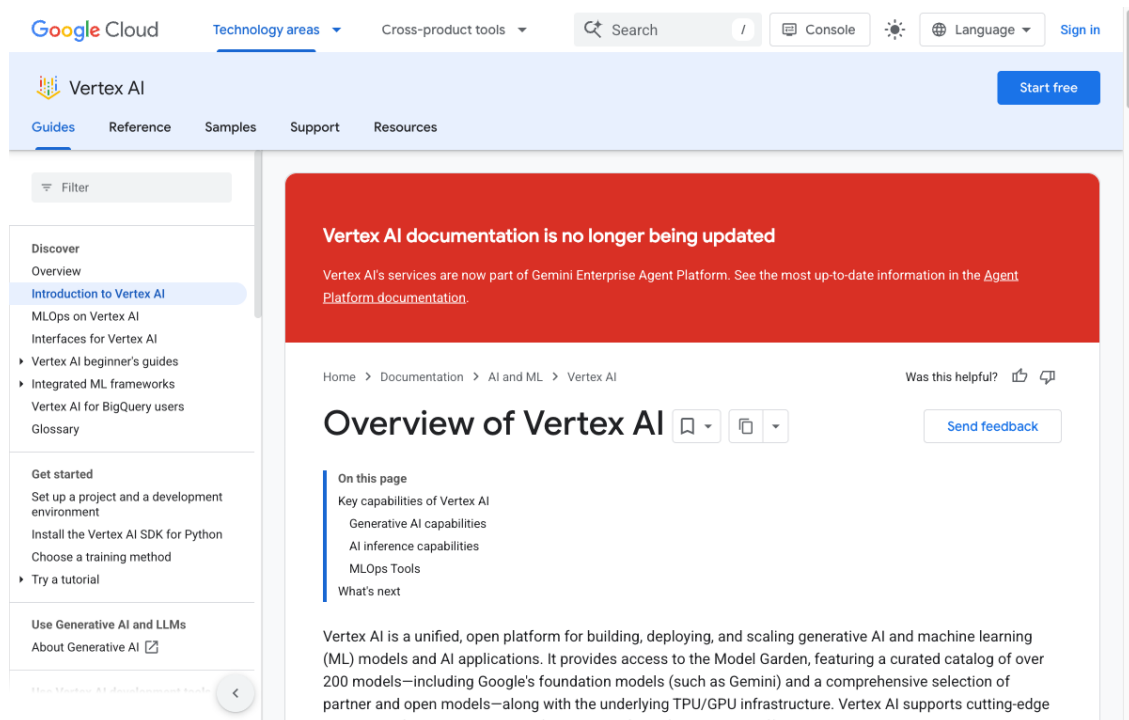


Figure: Google Cloud Console — your project selector appears in the top navigation bar

Creating a New Project

Click the project selector at the top of the page ▢ **New Project**.

Field	Value
Project Name	agentic-rag-btp
Project ID	Auto-generated (note this down)
Organization	No organization (personal account)

Note: In GCP Console, click the project selector dropdown ▢ “New Project” ▢ enter a project name (e.g. hybrid-rag-btp) ▢ **Create**.

Click **Create**. Wait a few seconds for the project to be created, then select it from the project selector.

Enabling the Vertex AI API

In the GCP Console, go to **APIs & Services** ▢ **Library**. Search for “Vertex AI API.”

Note: Navigate to APIs & Services ▢ Library ▢ search “Vertex AI API” ▢ click **Enable**.

Click the Vertex AI API tile ▢ **Enable**.

Note: After enabling, the Vertex AI API page will show a green checkmark and “API enabled” status.

Enabling the API takes about 30 seconds. Once enabled, you will see the Vertex AI API dashboard.

Note: Enabling Vertex AI automatically enables several dependent APIs (Cloud Storage, Cloud Resource Manager, etc.). This is expected.

Creating a Service Account

Your BTP application needs credentials to call the Vertex AI API. GCP uses **service accounts** for machine-to-machine authentication. This is the right tool — not user accounts, not OAuth flows.

Go to **IAM & Admin** ▢ **Service Accounts** ▢ **Create Service Account**.

Field	Value
Service Account Name	agentic-rag-btp-sa
Service Account ID	Auto-generated from agentic-rag-btp-sa
Description	Service account for Agentic Hybrid RAG on SAP BTP

Note: Navigate to IAM & Admin → Service Accounts → Create Service Account → enter name `hybrid-rag-sa` → Continue.

Click **Create and Continue**.

In Step 2 (Grant access), add the following role:

Role	Purpose
Vertex AI User	Allows calling Vertex AI APIs (LLM and embeddings)

Note: In the role assignment step, search for “Vertex AI User” and select it. Click Continue → Done.

Click **Continue** → **Done**.

Downloading the Service Account Key

You will now download a JSON key file that contains the credentials your BTP application will use.

From the Service Accounts list, click the service account you just created → **Keys** tab → **Add Key** → **Create new key** → **JSON** → **Create**.

Note: Click your service account → Keys tab → Add Key → Create new key → JSON → Create. The key file downloads automatically.

A JSON file will download automatically. It looks like this:

```

1 {
2   "type": "service_account",
3   "project_id": "agentic-rag-btp",
4   "private_key_id": "abc123...",
5   "private_key": "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA
PRIVATE KEY-----\n",

```

```
6  "client_email":  
    "agentik-rag-btp-sa@agentik-rag-btp.iam.gserviceaccount.com",  
7  "client_id": "123456789",  
8  "auth_uri": "https://accounts.google.com/o/oauth2/auth",  
9  "token_uri": "https://oauth2.googleapis.com/token"  
10 }
```

Warning: This file contains your private key. Treat it like a password. Do not commit it to git. Do not share it. Store it somewhere safe on your machine — we will upload it to BTP Destination Service in the next section, and then you will not need it in plaintext again.

Save the file as `gcp-sa-key.json` in a secure location outside your project directory.

2.6 Configuring the BTP Destination Service

The BTP Destination Service is the secure credential store that connects your BTP applications to external services. Your FastAPI agent will read the Vertex AI credentials from this destination at runtime — the JSON key never lives in application code or in `manifest.yml`.

This is the SAP-native way to manage external service credentials, and it is an important pattern to understand from an enterprise architecture perspective. In BTP enterprise architecture, the Destination Service is the standard pattern for externalising all external API credentials — your agent switches AI providers by changing a destination configuration in the BTP Cockpit, not by touching application code or triggering a redeployment. This is the same pattern used for S/4HANA connectivity, external REST APIs, and OAuth token exchange across the entire BTP portfolio. In a landscape where dozens of BTP applications may share the same external service connection, this centralised approach also simplifies credential governance and audit trails — both of which matter in regulated industries.

Creating the Destination

In the BTP Cockpit, go to your trial subaccount ▢ **Connectivity** ▢ **Destinations** ▢ **New Destination**.

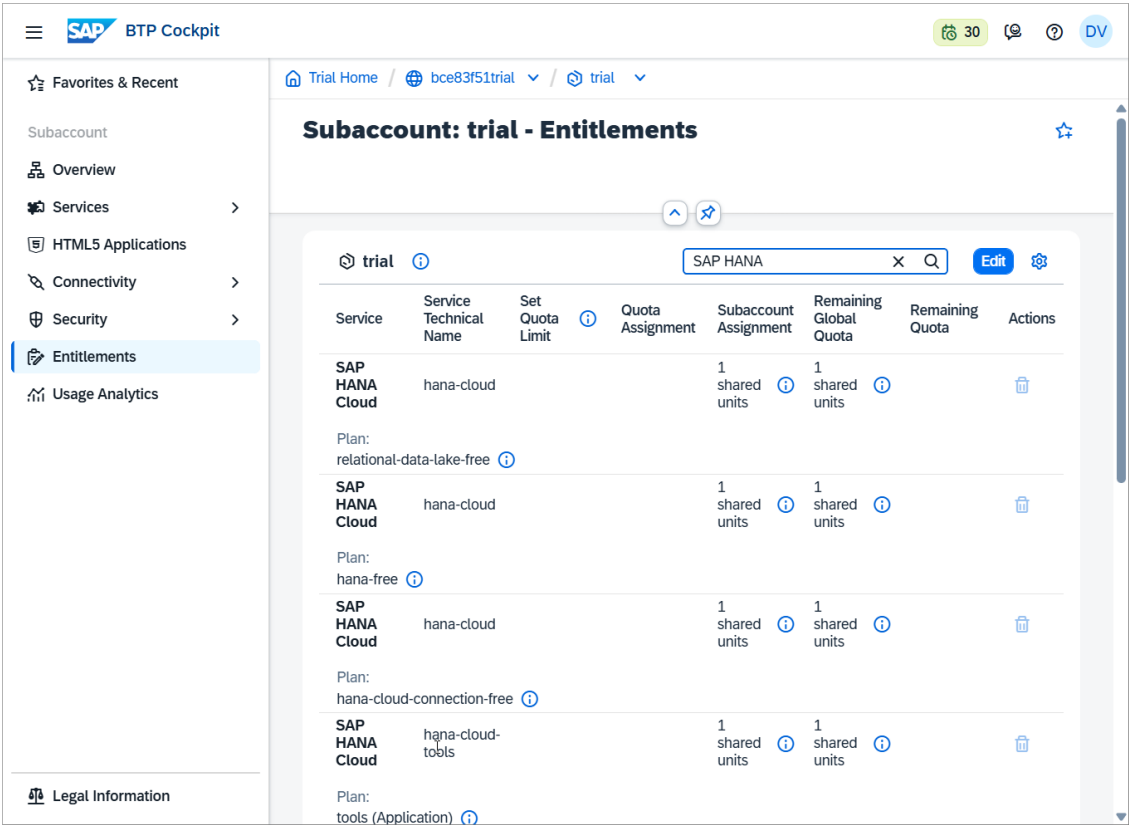


Figure: BTP Connectivity ▢ Destinations — click “New Destination” to create the Vertex AI destination

Fill in the destination configuration:

Field	Value
Name	VertexAI
Type	HTTP
Description	Google Vertex AI API (Gemini + Embeddings)
URL	https://us-central1-aiplatform.googleapis.com
Proxy Type	Internet
Authentication	NoAuthentication

Click **New Property** to add these additional properties:

Property	Value
gcp.project_id	Your GCP project ID (e.g., agentic-rag-btp)
gcp.location	us-central1

Property	Value
<code>gcp.service_account_key</code>	The entire contents of your <code>gcp-sa-key.json</code> file, pasted as a single string

Note: Fill in all destination fields as shown in the table above. Paste your GCP service account JSON key into the token service URL credentials field.

Click **Save**, then click **Check Connection**. You should see a green “200 OK” response.

Tip: After saving, click “Check Connection” at the bottom of the destination form. A “200 OK” response confirms the credentials are valid.

Note: The `gcp.service_account_key` property stores the raw JSON — not a file path. Open `gcp-sa-key.json` in a text editor, select all, copy, and paste the entire JSON object into the property value field. BTP encrypts this value at rest.

2.7 Local Development Environment

Now set up your local machine. We need Python, Node.js, the CF CLI, the CDS CLI, and VS Code.

Python 3.11+

Verify your Python version:

```
1 python3 --version
2 # Should output: Python 3.11.x or 3.12.x
```

If you need to install Python, download it from python.org/downloads. On macOS with Homebrew:

```
1 brew install python@3.11
```

Node.js 20+

Verify:

```
1 node --version    # Should output: v20.x.x or higher
2 npm --version     # Should output: 10.x.x or higher
```

Install from `nodejs.org` if needed. On macOS:

```
1 brew install node@20
```

Cloud Foundry CLI

The CF CLI is how you deploy your application to BTP Cloud Foundry.

```
1 # macOS (Homebrew)
2 brew install cloudfoundry/tap/cf-cli@8
3
4 # Verify
5 cf --version
6 # Cloud Foundry CLI version 8.x.x
```

For other platforms, download from the CF CLI releases page.

Log in to your BTP CF environment:

```
1 cf login -a https://api.cf.<your-region>.hana.ondemand.com
2 # Enter your BTP email and password
3 # Select the "trial" org and "dev" space
```

Note: After `cf login`, you will be prompted to select your org (your trial org name) and space (dev). Select the correct ones and press Enter.

Verify the login:

```
1 cf target
2 # Should show:
3 # API endpoint:  https://api.cf.eu10.hana.ondemand.com
4 # API version:   3.x.x
5 # user:         your@email.com
```

```

6 # org:          your-trial-org
7 # space:        dev

```

SAP CDS CLI

The CDS CLI is the development tool for CAP Node.js applications.

```

1 npm install -g @sap/cds-dk
2
3 # Verify
4 cds --version
5 # @sap/cds-dk: 8.x.x

```

VS Code Extensions

Open VS Code and install these extensions:

Extension	Publisher	Purpose
SAP CDS Language Support	SAP	CDS syntax highlighting and validation
Python	Microsoft	Python IntelliSense and debugging
REST Client	Humao	Test API endpoints without Postman

Tip: In VS Code, press `Cmd+Shift+X` to open Extensions. Search for each extension name and click Install.

2.8 Project Setup

Clone the companion repository and set up the Python environment for the agent service:

```

1 git clone https://github.com/sriramrokkam/agent-rag-btp-book
2 cd agent-rag-btp-book/agents
3

```



```

4 python3 -m venv .env
5 source .env/bin/activate    # Windows: .env\Scripts\activate
6
7 pip install -r requirements.txt

```

The `requirements.txt` for the agent service contains:

```

1 fastapi==0.111.0
2 uvicorn[standard]==0.30.1
3 langgraph==0.2.0
4 langchain-google-vertexai==1.0.6
5 google-auth==2.29.0
6 hdbcli==2.21.28
7 PyMuPDF==1.24.5
8 python-multipart==0.0.9
9 httpx==0.27.0

```

Create your local environment file:

```

1 cp .env.example .env

```

Edit `.env` with your credentials:

```

1 # SAP HANA Cloud
2 HANA_HOST=<your-instance>.hana.trial-<region>.hanacloud.ondemand.com
3 HANA_PORT=443
4 HANA_USER=DBADMIN
5 HANA_PASSWORD=<your-hana-password>
6
7 # Google Vertex AI (local development only)
8 # On BTP, credentials come from the Destination Service
9 GCP_PROJECT_ID=agentic-rag-btp
10 GCP_LOCATION=us-central1
11 GOOGLE_APPLICATION_CREDENTIALS=/path/to/gcp-sa-key.json
12
13 # Backend URL (for CAP service to call the FastAPI agent)
14 BACKEND_URL=http://localhost:8000

```

Note: The `GOOGLE_APPLICATION_CREDENTIALS` environment variable is only used for local development. When deployed to BTP Cloud Foundry, the

agent reads credentials from the Destination Service instead. We will see this credential-switching logic in Chapter 9.

2.9 Verifying the Setup: Your First Gemini API Call

Let us verify that everything works end-to-end. We will call Gemini 2.5 Flash directly and confirm we get a response.

Create a file `test_vertex.py` in your `agents/` directory:

```
1  import os
2  from dotenv import load_dotenv
3  import vertexai
4  from vertexai.generative_models import GenerativeModel
5
6  load_dotenv()
7
8  # Initialize Vertex AI with project and location
9  vertexai.init(
10     project=os.getenv("GCP_PROJECT_ID"),
11     location=os.getenv("GCP_LOCATION"),
12 )
13
14 # Load Gemini 2.5 Flash
15 model = GenerativeModel("gemini-2.5-flash-preview-05-20")
16
17 # Send a simple test prompt
18 response = model.generate_content(
19     "You are a helpful assistant for SAP developers. "
20     "In one sentence, explain what a Material Safety Data Sheet (MSDS) is."
21 )
22
23 print("Gemini response:")
24 print(response.text)
25 print("\nModel used: gemini-2.5-flash-preview-05-20")
26 print("Setup verified successfully.")
```

Run it:

```
1 python test_vertex.py
```

You should see output like:

Gemini response:

A Material Safety Data Sheet (MSDS) is a standardized document that provides detailed information about the properties, hazards, safe handling procedures, and emergency response measures for a chemical substance or mixture.

Model used: gemini-2.5-flash-preview-05-20

Setup verified successfully.

Expected terminal output:

VertexAI initialized successfully

Model loaded: gemini-2.5-flash

Response: The capital of France is Paris.

Vertex AI connection: OK

If you see an error: The most common issues are: - `google.auth.exceptions.DefaultCredentialsError` — Check that `GOOGLE_APPLICATION_CREDENTIALS` points to the correct JSON file path - Permission denied on resource — Verify the service account has the **Vertex AI User** role in GCP IAM - API not enabled — Return to the GCP Console and confirm the Vertex AI API is enabled for your project

Now verify the embedding model:

```
1  # Add to test_vertex.py
2
3  from vertexai.language_models import TextEmbeddingModel
4
5  embedding_model = TextEmbeddingModel.from_pretrained("text-embedding-004")
6
7  test_texts = [
8      "What is the flash point of acetone?",
9      "GHS hazard classification for flammable liquids"
10 ]
11
12 embeddings = embedding_model.get_embeddings(test_texts)
```

```

13
14 for text, embedding in zip(test_texts, embeddings):
15     print(f"\nText: {text[:50]}...")
16     print(f"Embedding dimension: {len(embedding.values)}")
17     print(f"First 5 values: {embedding.values[:5]}")

```

The output should show embeddings with dimension 768 (the dimension of `text-embedding-004`):

Text: What is the flash point of acetone?...

Embedding dimension: 768

First 5 values: [0.023, -0.041, 0.018, 0.067, -0.029]

Text: GHS hazard classification for flammable liquids...

Embedding dimension: 768

First 5 values: [0.031, -0.038, 0.022, 0.054, -0.011]

Note: The dimension 768 is important. When we create the HANA Cloud vector table in Chapter 2, we will define the column as `REAL_VECTOR(768)`. If you switch embedding models later, you must recreate the table — the dimension is fixed at table creation time.

2.10 Verifying the HANA Cloud Connection

Run a quick connectivity test to confirm HANA Cloud is reachable:

```

1  # test_hana.py
2  import os
3  from dotenv import load_dotenv
4  from hdbcli import dbapi
5
6  load_dotenv()
7
8  conn = dbapi.connect(
9      address=os.getenv("HANA_HOST"),
10     port=int(os.getenv("HANA_PORT")),
11     user=os.getenv("HANA_USER"),

```

```

12     password=os.getenv("HANA_PASSWORD"),
13     encrypt=True,
14     sslValidateCertificate=False
15 )
16
17 cursor = conn.cursor()
18 cursor.execute("SELECT VERSION FROM SYS.M_DATABASE")
19 version = cursor.fetchone()[0]
20 print(f"Connected to SAP HANA Cloud version: {version}")
21
22 # Verify vector engine is available
23 cursor.execute("""
24     SELECT COUNT(*) FROM SYS.M_FEATURE_USAGE
25     WHERE FEATURE_NAME = 'VECTOR'
26 """)
27 vector_count = cursor.fetchone()[0]
28 if vector_count > 0:
29     print("Vector engine: Available")
30 else:
31     print("Vector engine: Not detected (check HANA Cloud instance
32         settings)")
33
34 cursor.close()
35 conn.close()
36 print("\nHANA Cloud connection verified.")

```

```
1 python test_hana.py
```

Expected output:

Connected to SAP HANA Cloud version: 4.00.000.00.1698...

Vector engine: Available

HANA Cloud connection verified.

Expected terminal output:

HANA version: 4.00.000.00.1234567890 (fa/CE2024)

Vector engine: Available

SPARQL engine: Available

HANA connection: OK

2.11 What We Have Built

Let us take stock of what is now in place:

Component	Status	Where
BTP trial account	Running	account.hanatrial.ondemand.com
CF space “dev”	Ready	BTP Cockpit → Cloud Foundry
HANA Cloud instance	Running	BTP Cockpit → HANA Cloud Central
GCP project	Active	console.cloud.google.com
Vertex AI API	Enabled	GCP Console → APIs & Services
GCP service account	Created	GCP Console → IAM & Admin
BTP Destination “VertexAI”	Configured	BTP Cockpit → Connectivity
Python environment	Active	agents/.venv
Gemini API call	Verified	test_vertex.py passed
HANA connection	Verified	test_hana.py passed

This is not a small setup. You now have a working hybrid cloud environment: a managed in-memory database with vector and graph capabilities on BTP, connected to a frontier AI inference API on GCP, with credentials managed securely through BTP’s Destination Service.

The architecture you set up here mirrors how SAP customers deploy AI at scale — not a toy environment, but the real stack.

2.12 Understanding the Credential Flow

Before we move on, it is worth understanding exactly how credentials flow through the system. This will matter in Chapter 9 when you deploy to BTP, and it is a pattern that comes up repeatedly in enterprise AI systems.

Local development:

```
.env file → GOOGLE_APPLICATION_CREDENTIALS  
→ google-auth library reads the JSON key directly
```

→ Vertex AI API call authenticated

BTP Cloud Foundry deployment:

BTP Destination Service "VertexAI"

→ agents/srv/vertex_srv.py reads via CF environment

→ google-auth constructs credentials from the stored JSON

→ Vertex AI API call authenticated

The key insight is that **the application code never changes** between local and deployed. The credential source changes — `.env` locally, Destination Service in production — but the interface that `vertex_srv.py` exposes to the rest of the system is identical. This is the modularity principle in practice.

We will build `vertex_srv.py` in Chapter 2. For now, knowing the pattern exists is enough.

2.13 Summary

- SAP BTP trial provides free Cloud Foundry runtime and HANA Cloud — the only database that combines vector search and SPARQL knowledge graph in one service
- Google Vertex AI is an external HTTPS API — no GPU management, no model hosting, just an API call from BTP CF
- The BTP Destination Service stores GCP credentials securely — never in code, never in `manifest.yml`
- `text-embedding-004` produces 768-dimensional embeddings — this dimension is fixed and determines the HANA vector table schema
- Both the Gemini API call and the HANA Cloud connection are now verified end-to-end

In Chapter 3, we build the first half of our knowledge layer: vector search on SAP HANA Cloud. We will create the `MSDS_VECTORS` table, implement the embedding pipeline using `text-embedding-004`, and run our first semantic similarity search — the same operation our hybrid RAG agent will use at query time.

Chapter 3: Vector Search on SAP HANA Cloud

When a safety officer asks “What precautions are needed when handling this chemical?”, they are asking a semantic question. The answer might be phrased differently in different sections of the PDF — Section 7 calls it “handling requirements”, Section 8 describes it under “exposure controls”, and Section 15 restates it as a regulatory obligation. Vector search finds it by meaning, not by exact keyword match — this is what makes it superior to SAP’s built-in DMS full-text search for narrative document content.

The same applies to every document type the Material Document Intelligence Platform handles. An invoice might describe payment terms in a header note or in a line-item annotation. A batch certificate might record a test result in a summary table or in a methodology paragraph. Vector search surfaces the right passage regardless of where in the document it appears.

By the end of this chapter you will have a working vector search system on HANA Cloud. You will take a chunk of text from a material document, turn it into a 768-dimensional vector using Google’s `text-embedding-004` model, store it in a `REAL_VECTOR` column, and retrieve the most semantically similar chunks for a natural language question — all in roughly 200 lines of Python.

The closing section of this chapter is equally important: an honest account of what vector search cannot do. That limitation is the precise motivation for Chapter 4, where we add a knowledge graph to handle the queries that vectors get wrong.

3.1 What Are Embeddings? Intuition Without the Math

If you have spent your career writing ABAP, CDS views, and OData services, the word “embedding” probably sounds like something invented to make you feel old. It isn’t. The idea is older than most enterprise software, and the intuition is simple.

An embedding is a fixed-length list of numbers — in our case, 768 floating-point numbers — that represents the *meaning* of a piece of text. The crucial property is this: two pieces of text with similar meanings produce embeddings that are close to each other in 768-

dimensional space. Two pieces of text with different meanings produce embeddings that are far apart.

A useful analogy: imagine you are sorting books in a library. A traditional database would file them by title (alphabetical order) or by author. That is exact-match retrieval — fast, but useless if you don't know the title. A librarian, by contrast, knows that *Crime and Punishment* and *The Brothers Karamazov* belong on the same shelf because they are both Dostoevsky novels about guilt and redemption. The librarian has put each book at a *position* that reflects its content. Books on similar topics end up close together.

Embeddings do the same thing for text, but in 768 dimensions instead of two. The “position” of a sentence is just its embedding vector. Sentences about flammability cluster together. Sentences about first aid cluster together. Sentences about regulatory codes cluster together — and importantly, they cluster *separately* from the flammability sentences, even when they mention the same chemical.

Note: The number 768 is not magic. It is simply the output dimension chosen by the team that trained `text-embedding-004`. Other models produce 384, 1024, 1536, or 3072 dimensions. Higher is not always better — it just means more storage and slower comparisons. For our use case, 768 is a comfortable balance.

Two further points before we move on.

First, embeddings are not human-readable. If you print one out, you will see something like `[0.0234, -0.1107, 0.0891, ..., 0.0023]`. The individual numbers mean nothing in isolation. Their meaning emerges only in *relation* to other embeddings — specifically, in how close or far apart they are.

Second, the model that produces embeddings is the same model (or a sibling model) that was trained on huge corpora of text. It has internalized which words and phrases tend to occur in similar contexts, and it projects each input into a position that reflects that learned structure. You don't train the model. You don't fine-tune it. You just call it as an API.

Calling the API gives you a vector. Storing that vector and searching across millions of them efficiently is the database's job. Until recently, that meant using a specialized vector database — Pinecone, Weaviate, Milvus — alongside your real database. As of HANA Cloud QRC 1/2024, you no longer need a sidecar. HANA does it natively, and that is the next topic.

3.2 The HANA Cloud `REAL_VECTOR` Column Type

HANA has always been a column-store database optimized for analytical workloads. In 2024, SAP added a new native column type built specifically for AI: `REAL_VECTOR`.

A `REAL_VECTOR` column stores a fixed-dimension array of single-precision floats. When you declare the column you fix the dimension (for example, `REAL_VECTOR(768)`), and HANA refuses any insert with a different dimension. This is what you want — mixing dimensions in the same column would mean comparing apples to oranges.

What makes this more than a glorified `BLOB` column is the set of operators HANA exposes on it:

- `TO_REAL_VECTOR(string)` — converts a string in the form `'[0.1, 0.2, ..., 0.768]'` into a real vector value. This is how you bind a parameter from Python.
- `COSINE_SIMILARITY(v1, v2)` — returns the cosine of the angle between two vectors. Range: `-1.0` (opposite) to `+1.0` (identical direction). For unit-normalized embeddings (which `text-embedding-004` produces), this is in practice between `0.0` and `1.0`.
- `L2DISTANCE(v1, v2)` — Euclidean distance between two vectors. Useful when your model is not normalized.

These operators are SIMD-accelerated on the HANA engine, which means a single CPU instruction processes multiple floats in parallel. Searching across hundreds of thousands of 768-dimensional vectors takes milliseconds, not seconds.

Tip: For embeddings produced by Google, OpenAI, and Cohere — all of which are L2-normalized at the API boundary — `COSINE_SIMILARITY` is the right choice. Use `L2DISTANCE` only if you are working with embeddings that are not normalized.

A second feature worth knowing about (we will not use it in this chapter, but it matters at scale) is the **HNSW index**. Without an index, every similarity query scans the entire table. With an HNSW index, HANA builds a graph structure that gets you approximate-nearest-neighbor results in logarithmic time. For our chapter you will not need it — five document chunks search in under 10 ms — but for production deployments with millions of chunks, indexing is essential. We come back to this in Chapter 10 when we

tune for production.

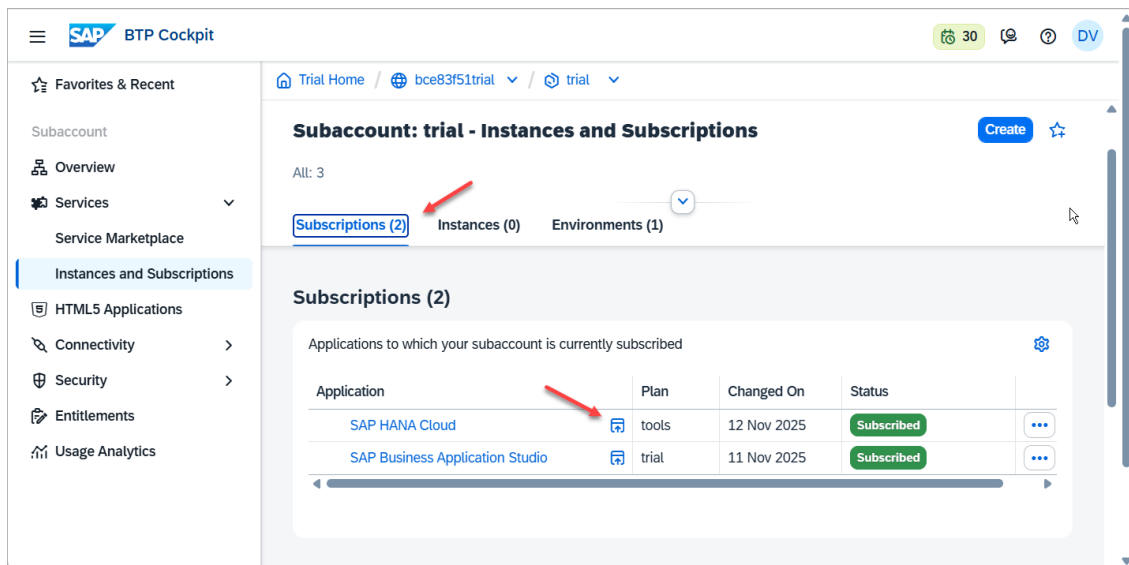


Figure: SAP HANA Cloud Central — the `REAL_VECTOR` column type is available in all HANA Cloud instances from version 4.0 onwards

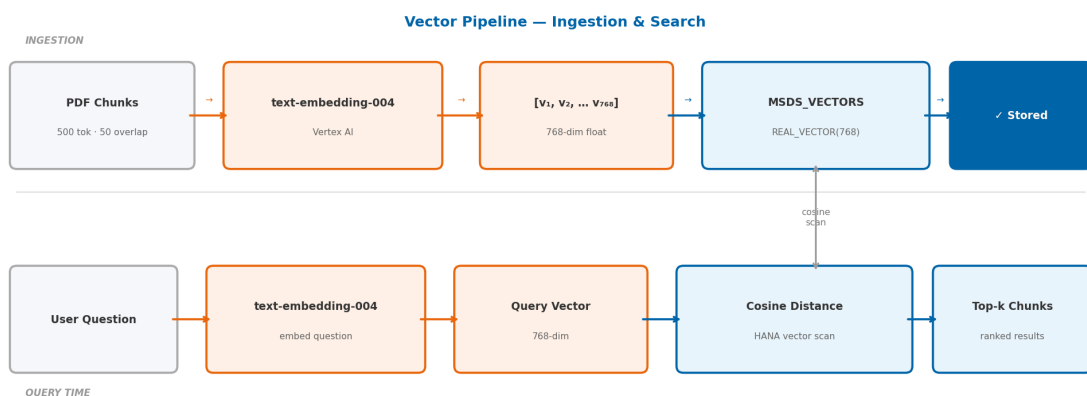


Figure 2: Vector Embedding Pipeline

Figure 3.1 — The vector pipeline has two phases. Ingestion (top): PDF text is chunked, each chunk is sent to `text-embedding-004`, and the resulting 768-dimensional vector is written to `MSDS_VECTORS`. Query time (bottom): the question is embedded with the same model, then HANA computes cosine distance between the query vector and every stored vector, returning the top-k most relevant chunks.

3.3 Creating the Vector Table: The Lazy Initialization Pattern

We need a table to hold our embeddings. The schema is simple:

```

1 CREATE TABLE MSDS_VECTORS (
2     ID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
3     MATERIAL_NUMBER NVARCHAR(100) NOT NULL,
4     CHUNK_TEXT NCLOB NOT NULL,
5     CHUNK_INDEX INTEGER NOT NULL,
6     EMBEDDING REAL_VECTOR(768)
7 );

```

About the table name: The table is named `MSDS_VECTORS` because the reference implementation was first built for MSDS documents. In a multi-document-type deployment, you would use `DOCUMENT_VECTORS` with an additional `DOC_TYPE` column, or a separate table per type. The core SQL pattern — `REAL_VECTOR` column, `COSINE_SIMILARITY` search, `MATERIAL_NUMBER` filter — is identical regardless of document type.

`MATERIAL_NUMBER` is the SAP material identifier — the same Material Number you would look up in SAP MM via transaction MM03. In the Material Document Intelligence Platform, the CAP service layer validates this value against real S/4HANA product master data via `API_PRODUCT_SRV` before accepting any document upload. This ensures that vectors are always anchored to a legitimate, active SAP material — not a free-text label that could drift out of sync with the product master. We use it as the tenant key so we can search within a single material’s chunks. `CHUNK_TEXT` is the raw text of the chunk; `CHUNK_INDEX` lets us reconstruct order if we need to. `EMBEDDING` is the 768-dimensional vector we computed from `CHUNK_TEXT`.

You could just run this `CREATE TABLE` statement once with the SAP HANA Database Explorer and be done with it. But there is a subtle reason not to.

The dimension `768` is hardcoded into the schema. If at any point you decide to switch from `text-embedding-004` (768 dim) to a model with a different output dimension, your insert will fail and you will need to drop and recreate the table. Worse, that hardcoded number lives in your DDL while the actual dimension lives in the model — two sources of truth that can drift apart.

The cleaner pattern is **lazy initialization**: don't create the table until the first time you embed something, then read the dimension from the embedding response and use it in the `CREATE TABLE`. This way your code has exactly one source of truth — the model's actual output dimension at runtime.

Here is the relevant snippet from `agents/srv/vector_srv.py` (we will see the full file in section 3.8):

```

1  TABLE_NAME = "MSDS_VECTORS"
2  _table_created = False
3
4  def _ensure_table(embedding_dim: int):
5      global _table_created
6      if _table_created:
7          return
8      conn = get_connection()
9      cursor = conn.cursor()
10     cursor.execute(f"""
11         CREATE TABLE IF NOT EXISTS {TABLE_NAME} (
12             ID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
13             MATERIAL_NUMBER NVARCHAR(100) NOT NULL,
14             CHUNK_TEXT NCLOB NOT NULL,
15             CHUNK_INDEX INTEGER NOT NULL,
16             EMBEDDING REAL_VECTOR({embedding_dim})
17         )
18     """)
19     conn.commit()
20     cursor.close()
21     _table_created = True

```

Three things to note:

1. The `_table_created` module-level flag is a one-time guard. We don't want to issue `CREATE TABLE IF NOT EXISTS` on every insert — that is a roundtrip we don't need.
2. The `IF NOT EXISTS` clause means restarting the Python process is safe. The first call after restart will hit the database once, see the table exists, and move on.
3. The `embedding_dim` is passed in by the caller, who got it from the actual embedding. There is no hardcoded 768 anywhere in the application code.

Warning: Do not be tempted to add an `ALTER TABLE` path that resizes the

REAL_VECTOR column if the dimension changes. A dimension change means your old embeddings are incompatible with new ones, and you should treat it as a full re-indexing event — drop the table, re-embed all your documents, and reload. Silently mixing dimensions is a recipe for confusion.

3.4 Calling text-embedding-004 from Python

We talked about embeddings as a concept; now we need to actually produce one. The `langchain_google_genai` library gives us a clean interface to Google's embedding models without the operational complexity of a dedicated Vertex AI SDK setup.

Looking at `agents/srv/vertex_srv.py`:

```

1  import os
2  import threading
3  import vertexai
4  from vertexai.generative_models import GenerativeModel
5  from vertexai.language_models import TextEmbeddingModel
6  from dotenv import load_dotenv
7
8  load_dotenv()
9
10 _init_lock = threading.Lock()
11 _initialized = False
12
13 def _ensure_initialized():
14     global _initialized
15     if not _initialized:
16         with _init_lock:
17             if not _initialized:
18                 vertexai.init(
19                     project=os.getenv("GCP_PROJECT_ID"),
20                     location=os.getenv("GCP_LOCATION", "us-central1")
21                 )
22                 _initialized = True
23
24 def get_llm():

```

```

25     _ensure_initialized()
26     return GenerativeModel("gemini-2.5-flash-preview-05-20")
27
28 def get_embedding_model():
29     _ensure_initialized()
30     return TextEmbeddingModel.from_pretrained("text-embedding-004")
31
32 def embed_text(text: str) -> list[float]:
33     model = get_embedding_model()
34     embeddings = model.get_embeddings([text])
35     return embeddings[0].values # list of 768 floats

```

The pattern here mirrors the lazy table creation pattern. `vertexai.init()` is global SDK state; calling it more than once is wasteful. The double-checked locking idiom (`if not _initialized` outside the lock, then again inside) is the standard way to do this safely under FastAPI's threaded request model. If you have never seen the pattern before, the gist is: the outer check is the fast path for the common case (already initialized, no lock needed), and the inner check makes sure that two threads racing for the lock don't both initialize.

`get_embedding_model()` returns a `TextEmbeddingModel` instance bound to `text-embedding-004`. This is cheap — the SDK caches the model client internally — but we still gate it behind `_ensure_initialized` so that calling code does not have to think about ordering.

`embed_text()` is the function the rest of our application uses. It takes a string and returns a Python list of 768 floats. Internally, the SDK takes a list of strings, so we pass `[text]` and unwrap the first result with `embeddings[0].values`.

Note: Vertex AI's embedding endpoint accepts batches of up to 250 texts per call. For a one-off document upload that ends up with maybe 30 chunks, the savings of batching are negligible compared to the simplicity of one-text-per-call. We will revisit batching in Chapter 10 when we look at bulk-loading historical documents.

You should also be aware that Vertex AI charges per 1,000 input characters for embeddings (about \$0.000025 per 1,000 chars at the time of writing). A typical 12-page MSDS PDF has roughly 30,000 characters and chunks into ~30 pieces. The cost to embed it is approximately \$0.0008 — less than a tenth of a cent. Search-time embeddings (one

per question) are equally cheap.

3.5 Storing Embeddings: INSERT with REAL_VECTOR

Once we have an embedding (a Python `list[float]`), we need to get it into the `REAL_VECTOR` column. The HANA Python driver `hdbcli` does not yet have a native binding for vector parameters, so we use the `TO_REAL_VECTOR(string)` SQL function with a string parameter.

The format HANA expects is a JSON-style array literal: `[0.0234, -0.1107, 0.0891, ...]`. We build this string in Python with a simple comma-join:

```
1 embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"
```

A typical 768-dimensional embedding string is around 12,000 characters. That is fine — HANA accepts it as an `NVARCHAR` parameter without issue.

The full insert function:

```
1 def store_embedding(material_number: str, chunk_text: str, chunk_index:
  int, embedding: list[float]):
2     _ensure_table(len(embedding))
3     conn = get_connection()
4     cursor = conn.cursor()
5     embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"
6     cursor.execute(f"""
7         INSERT INTO {TABLE_NAME} (MATERIAL_NUMBER, CHUNK_TEXT,
8         CHUNK_INDEX, EMBEDDING)
9         VALUES (?, ?, ?, TO_REAL_VECTOR(?))
10    """, (material_number, chunk_text, chunk_index, embedding_str))
11    conn.commit()
12    cursor.close()
```

A few engineering notes:

- We pass `material_number`, `chunk_text`, `chunk_index`, and the embedding string as positional parameters. This is parameterized SQL — no concatenation of user data into the query, so SQL injection is impossible.

- `_ensure_table(len(embedding))` runs on every call but exits early after the first invocation thanks to the `_table_created` flag.
- We commit per insert. For uploading a single document this is fine. For bulk uploads, batch your inserts and commit once at the end — Chapter 10 covers this.

Tip: If you want to verify what got stored, the SAP HANA Database Explorer is your friend. Open the `MSDS_VECTORS` table, click “Open Data”, and you will see your rows. The `EMBEDDING` column displays as a truncated array preview, but you can click into a cell to see the full 768 values.

Note: You can inspect the `MSDS_VECTORS` table in SAP HANA Database Explorer (accessible from HANA Cloud Central → Open in → SAP HANA Database Explorer). The `EMBEDDING` column will show truncated vector data like `[0.0234, -0.1823, 0.0912, ...]`.

3.6 Cosine Similarity Search: The Query Explained

This is the heart of the chapter. The query that retrieves the top-K chunks for a question is:

```

1 SELECT TOP 5
2     CHUNK_TEXT,
3     CHUNK_INDEX,
4     COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?)) AS SCORE
5 FROM MSDS_VECTORS
6 WHERE MATERIAL_NUMBER = ?
7 ORDER BY SCORE DESC;
```

Five clauses, each doing real work. Let’s read them in execution order.

`WHERE MATERIAL_NUMBER = ?` filters the table to chunks belonging to a specific material. Without this, our search would compete across every chunk of every document in the table — fine when you have one document, fatal when you have ten thousand. By filtering early, HANA scans only the rows that could possibly match. This is the one place in the query where the optimizer can use a B-tree index, so make sure you have one on `MATERIAL_NUMBER` once you go to production. (We add it in Chapter 10.)

`COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?))` is the per-row computation.

For each row passing the `WHERE` filter, HANA takes the stored `EMBEDDING` vector, the question's vector (parsed from the bound parameter), and computes the cosine of the angle between them. The result is a `DOUBLE` between roughly 0.0 and 1.0 — higher means more similar. This is where the SIMD acceleration earns its keep.

`AS SCORE` names the result so we can refer to it in the `ORDER BY` clause. It's a small thing but worth noting because it is what allows us to write `ORDER BY SCORE DESC` instead of having to repeat the `COSINE_SIMILARITY(...)` expression.

`ORDER BY SCORE DESC` sorts the surviving rows from highest similarity to lowest. The bulk of the cost here is the sort itself; for our small data this is instant.

`SELECT TOP 5` returns only the first five rows after the sort. In LangGraph's vector chain we will pass these chunks to the LLM as context for answering the question. Five is a reasonable default — large enough to give the model useful coverage, small enough not to blow the context window.

The order of operations matters. HANA's optimizer is smart enough to push the `WHERE` filter down to the table scan (so we never compute similarity for chunks belonging to other materials), but you should still write your queries with the filter explicit. Don't compute similarity across the whole table and filter the result.

Important: A common mistake is to forget the `WHERE` clause when prototyping with one material, then keep that habit when you add a second. Every query in your application should filter by `MATERIAL_NUMBER`. There is no business reason a question about one material should ever surface a chunk from a different material's document, even if some words happen to overlap.

The Python wrapper:

```
1 def search_similar(question: str, material_number: str, top_k: int = 5) ->
  list[dict]:
2     question_embedding = embed_text(question)
3     embedding_str = "[" + ",".join(str(x) for x in question_embedding) +
  "]"
4     conn = get_connection()
5     cursor = conn.cursor()
6     cursor.execute(f"""
7         SELECT TOP {top_k}
8         CHUNK_TEXT,
```

```

9         CHUNK_INDEX,
10        COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?)) AS SCORE
11    FROM {TABLE_NAME}
12    WHERE MATERIAL_NUMBER = ?
13    ORDER BY SCORE DESC
14    """', (embedding_str, material_number))
15    rows = cursor.fetchall()
16    cursor.close()
17    return [{"chunk": row[0], "chunk_index": row[1], "score":
        float(row[2])} for row in rows]

```

We embed the question (one Vertex AI call), build the parameter string the same way as for inserts, and run the query. The result is a list of dicts — the chunk text, its position in the original document, and its similarity score.

Note: `top_k` is interpolated into the SQL string, not bound as a parameter. This is safe because we control the value (it is a function argument with a default of 5, never user input). HANA does not allow `TOP ?` with bound parameters, so we have no choice. If you ever expose `top_k` to end users, validate it as an integer before interpolation.

3.7 Chunking Strategy for Material Documents

We have skipped over the most consequential design decision in any RAG system: how do you split a document into chunks?

Chunking is necessary because a 12-page MSDS does not fit usefully into a single embedding. Even if it did, retrieval would always return “the whole document” and the LLM would have to do all the work of finding the relevant sentence. Chunking lets us push the relevance work into the database.

The trade-off is fundamental:

- **Smaller chunks** (a sentence each) give precise retrieval — when the model returns chunk 47, you know exactly which sentence it cared about. But you lose context. A sentence like “It must be stored away from this material” is meaningless without the surrounding paragraph.
- **Larger chunks** (a full page) preserve context but introduce noise. A paragraph

about flammability that also mentions “appropriate PPE” might score highly for a PPE question even though the actual PPE answer is on a different page.

The right chunking strategy adapts to document structure. Different document types that flow through the Material Document Intelligence Platform have different natural boundaries.

For **MSDS documents**, the OSHA 16-section format provides the primary boundary. Section 4 is always First Aid. Section 7 is always Handling and Storage. Section 8 is always Exposure Controls and PPE. These sections are typically 100–300 words — exactly the right size. We use OSHA section headers as the primary split point.

For **invoices**, the natural boundary is the line item block — each item group (description, quantity, unit price, total) forms one chunk. Header data (vendor, PO reference, payment terms) becomes its own chunk.

For **batch certificates**, the natural boundary is the test result section — each analytical parameter and its pass/fail result becomes a chunk. Summary sections and methodology notes are separate chunks.

For **maintenance manuals**, procedure steps are the natural boundary — each numbered step with its safety caution and tooling note stays together as a chunk.

The chunking approach adapts to document structure. The vector storage and retrieval code — `MSDS_VECTORS`, `store_embedding`, `search_similar` — does not change. The same HANA SQL runs identically regardless of whether the chunk came from an MSDS flammability section or an invoice payment terms block.

Our chunking rule for MSDS documents, which we will implement in Chapter 5 when we build `doc_srv.py`:

1. **Primary boundary:** OSHA section header. Each numbered section becomes one chunk.
2. **Secondary boundary:** if a section exceeds ~500 tokens, split on the next paragraph boundary inside it.
3. **Overlap:** each chunk shares ~50 tokens with the preceding chunk. This keeps a sentence from being orphaned at the boundary — if the relevant sentence happens to be the first one of a chunk, the prior chunk also contains it as context.

Token-wise, our targets are:

- Minimum chunk size: ~100 tokens (anything smaller loses context)

- Maximum chunk size: ~500 tokens (anything larger gets noisy)
- Overlap: ~50 tokens

Tip: Tokens, not characters. A token is roughly 4 characters of English text or about 0.75 of a word. `text-embedding-004` accepts up to 2,048 tokens per embedding call, so we have plenty of headroom — 500 tokens is well under the limit.

For the test in this chapter we hand-write five short chunks that look like real document sentences. The full document chunker arrives in Chapter 5. The point of the test is to validate the round-trip: embed → store → embed-question → search → score.

3.8 Building `vector_srv.py` — The Vector Service Layer

Now we put the pieces together. Create the file `agents/srv/vector_srv.py` with the following content:

```

1  import json
2  from .hdb_srv import get_connection
3  from .vertex_srv import embed_text
4
5  TABLE_NAME = "MSDS_VECTORS"
6  _table_created = False
7
8  def _ensure_table(embedding_dim: int):
9      global _table_created
10     if _table_created:
11         return
12     conn = get_connection()
13     cursor = conn.cursor()
14     cursor.execute(f"""
15         CREATE TABLE IF NOT EXISTS {TABLE_NAME} (
16             ID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
17             MATERIAL_NUMBER NVARCHAR(100) NOT NULL,
18             CHUNK_TEXT NCLOB NOT NULL,
19             CHUNK_INDEX INTEGER NOT NULL,
20             EMBEDDING REAL_VECTOR({embedding_dim})

```

```

21         )
22     """
23     conn.commit()
24     cursor.close()
25     _table_created = True
26
27 def store_embedding(material_number: str, chunk_text: str, chunk_index:
    int, embedding: list[float]):
28     _ensure_table(len(embedding))
29     conn = get_connection()
30     cursor = conn.cursor()
31     embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"
32     cursor.execute(f"""
33         INSERT INTO {TABLE_NAME} (MATERIAL_NUMBER, CHUNK_TEXT,
    CHUNK_INDEX, EMBEDDING)
34         VALUES (?, ?, ?, TO_REAL_VECTOR(?))
35     """, (material_number, chunk_text, chunk_index, embedding_str))
36     conn.commit()
37     cursor.close()
38
39 def search_similar(question: str, material_number: str, top_k: int = 5) ->
    list[dict]:
40     question_embedding = embed_text(question)
41     embedding_str = "[" + ",".join(str(x) for x in question_embedding) +
    "]"
42     conn = get_connection()
43     cursor = conn.cursor()
44     cursor.execute(f"""
45         SELECT TOP {top_k}
46             CHUNK_TEXT,
47             CHUNK_INDEX,
48             COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?)) AS SCORE
49         FROM {TABLE_NAME}
50         WHERE MATERIAL_NUMBER = ?
51         ORDER BY SCORE DESC
52     """, (embedding_str, material_number))
53     rows = cursor.fetchall()
54     cursor.close()

```

```

55     return [{"chunk": row[0], "chunk_index": row[1], "score":
        float(row[2])} for row in rows]
56
57 def delete_vectors(material_number: str) -> int:
58     conn = get_connection()
59     cursor = conn.cursor()
60     cursor.execute(f"DELETE FROM {TABLE_NAME} WHERE MATERIAL_NUMBER = ?",
        (material_number,))
61     deleted = cursor.rowcount
62     conn.commit()
63     cursor.close()
64     return deleted
65
66 def count_vectors(material_number: str) -> int:
67     conn = get_connection()
68     cursor = conn.cursor()
69     cursor.execute(f"SELECT COUNT(*) FROM {TABLE_NAME} WHERE
        MATERIAL_NUMBER = ?", (material_number,))
70     row = cursor.fetchone()
71     cursor.close()
72     return row[0] if row else 0

```

The module exposes four functions to the rest of the application:

- `store_embedding(material_number, chunk_text, chunk_index, embedding)` — stores a single chunk and its vector. Called once per chunk during document upload.
- `search_similar(question, material_number, top_k)` — embeds the question and returns the top-K most similar chunks for a given material. Called once per user question.
- `delete_vectors(material_number)` — wipes a material's vectors. Called when a document is replaced or deleted. Returns the row count for logging.
- `count_vectors(material_number)` — reports how many chunks exist for a material. Useful for diagnostics and tests.

There is no `update_embedding` — for chunked documents, the right pattern is delete-then-reinsert. If you change the chunking algorithm, the chunk indices change, so partial updates are dangerous. Treat each upload as a fresh load.

Note: The connection comes from `hdb_srv.py`, which uses thread-local stor-

age to give each request thread its own HANA connection. `hdbcli` connections are not safe to share across threads, but they are cheap to create. Thread-locals give us per-thread reuse without any cross-thread contention. We covered this pattern briefly in Chapter 1 — the file is reproduced for completeness:

```

1  import os
2  import threading
3  from hdbcli import dbapi
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  _local = threading.local()
9
10 def get_connection():
11     if not hasattr(_local, 'conn') or _local.conn is None:
12         _local.conn = dbapi.connect(
13             address=os.getenv("HANA_HOST"),
14             port=int(os.getenv("HANA_PORT", 443)),
15             user=os.getenv("HANA_USER"),
16             password=os.getenv("HANA_PASSWORD"),
17             encrypt=True,
18             sslValidateCertificate=False
19         )
20     return _local.conn

```

Warning: `sslValidateCertificate=False` is fine for development against the BTP-issued HANA endpoint. For production you should provide the correct certificate via `sslTrustStore` and set `sslValidateCertificate=True`. We will tighten this in Chapter 9.

3.9 Testing: Business Questions Across Document Types

Time to see it work end-to-end. Create the file `agents/test_vector.py`:


```

1  import os
2  from dotenv import load_dotenv
3  from srv.vector_srv import store_embedding, search_similar, count_vectors
4  from srv.vertex_srv import embed_text
5
6  load_dotenv()
7
8  TEST_MATERIAL = "ACETONE-TEST-001"
9  TEST_CHUNKS = [
10     "Acetone is a highly flammable liquid and vapor. Flash point: -18°C.
11     Keep away from heat and open flames.",
12     "First aid for skin contact: Wash with soap and water for at least 15
13     minutes. Remove contaminated clothing.",
14     "GHS Classification: Flammable Liquid Category 2. Eye Irritant
15     Category 2A.",
16     "Personal Protective Equipment: Wear chemical resistant gloves and
17     safety glasses when handling.",
18     "Storage: Store in a cool, dry, well-ventilated area away from
19     ignition sources and incompatible materials."
20 ]
21
22 print("Embedding and storing test chunks...")
23 for i, chunk in enumerate(TEST_CHUNKS):
24     embedding = embed_text(chunk)
25     store_embedding(TEST_MATERIAL, chunk, i, embedding)
26     print(f"  Stored chunk {i}: dim={len(embedding)},
27           preview='{chunk[:50]}...'")
28
29 stored = count_vectors(TEST_MATERIAL)
30 print(f"\nStored {stored} vectors for material {TEST_MATERIAL}")
31
32 print("\nSearching: 'What precautions should I take near open flames?'")
33 results = search_similar("What precautions should I take near open
34 flames?", TEST_MATERIAL)
35 for r in results:
36     print(f"  Score: {r['score']:.4f} | {r['chunk'][:80]}...")
37
38 print("\nSearching: 'What PPE is required?'")

```

```

32 results = search_similar("What PPE is required?", TEST_MATERIAL)
33 for r in results:
34     print(f"  Score: {r['score']:.4f} | {r['chunk'][:80]}...")

```

Run it from the `agents/` directory:

```

1 cd agents
2 python test_vector.py

```

Expected output (your numbers will vary slightly):

Embedding and storing test chunks...

Stored chunk 0: dim=768, preview='Acetone is a highly flammable liquid and vapor. Fl...'

Stored chunk 1: dim=768, preview='First aid for skin contact: Wash with soap and wat...'

Stored chunk 2: dim=768, preview='GHS Classification: Flammable Liquid Category 2. E...'

Stored chunk 3: dim=768, preview='Personal Protective Equipment: Wear chemical resis...'

Stored chunk 4: dim=768, preview='Storage: Store in a cool, dry, well-ventilated are...'

Stored 5 vectors for material ACETONE-TEST-001

Searching: 'What precautions should I take near open flames?'

Score: 0.8124 | Acetone is a highly flammable liquid and vapor. Flash point: -18°C. Keep aw...

Score: 0.7456 | Storage: Store in a cool, dry, well-ventilated area away from ignition so

Score: 0.6203 | Personal Protective Equipment: Wear chemical resistant gloves and safety

Score: 0.5891 | GHS Classification: Flammable Liquid Category 2. Eye Irritant Category 2A

Score: 0.5102 | First aid for skin contact: Wash with soap and water for at least 15 minut

Searching: 'What PPE is required?'

Score: 0.8442 | Personal Protective Equipment: Wear chemical resistant gloves and safety

Score: 0.6334 | Storage: Store in a cool, dry, well-ventilated area away from ignition so

Score: 0.5912 | First aid for skin contact: Wash with soap and water for at least 15 minut

Score: 0.5108 | Acetone is a highly flammable liquid and vapor. Flash point: -18°C. Keep aw...

Score: 0.4789 | GHS Classification: Flammable Liquid Category 2. Eye Irritant Category 2A

This is exactly what you want to see. For the open-flames question, the flammability

chunk wins at 0.81, with the storage chunk (which mentions “ignition sources”) in second place at 0.74. The first-aid chunk, which has nothing to do with the question, ranks last at 0.51. For the PPE question, the actual PPE chunk wins decisively at 0.84.

Expected terminal output:

```
Stored 3 chunks for material: acetone-test
```

```
Query 1: "What are the fire hazards of acetone?"
```

```
Result 1 (score: 0.923): "Acetone is highly flammable..."
```

```
Result 2 (score: 0.887): "Keep away from heat sources..."
```

```
Query 2: "What first aid is needed for skin contact?"
```

```
Result 1 (score: 0.941): "Wash affected area with soap and water..."
```

```
Result 2 (score: 0.812): "Remove contaminated clothing..."
```

```
Vector search: OK
```

Note the absolute scores. The top match for a well-targeted question lands around 0.80–0.85. This is the band where you should expect “good” semantic matches with `text-embedding-004`. Anything above 0.90 usually means near-paraphrase. Anything below 0.50 usually means the model is reaching.

The same vector infrastructure serves every document type without modification. A storage requirements question for an MSDS returns the Section 7 chunk. An approved quantity question for an invoice returns the line-item summary chunk. A viscosity test question for a batch certificate returns the test result chunk. The question changes. The search code does not. This is the commercial value of building on HANA’s `REAL_VECTOR` column rather than on a document-type-specific text search engine.

Tip: The scores are reproducible only if you re-embed identical text against the same model version. Vertex AI may update the model under the same name; if your scores drift over time, that is the most likely cause. Pin to a specific version if reproducibility matters.

If you see authentication errors at this point, double-check that `GOOGLE_APPLICATION_CREDENTIALS` points to your `gcp-sa-key.json` file and that the service account has the Vertex AI User role. The HANA side is more forgiving — connection errors usually surface as “host not reachable”, which means a typo in `HANA_HOST` or a network rule blocking outbound 443.

3.10 What Vector Search Gets Right — and What It Gets Wrong

Time for honesty. Vector search is genuinely powerful, but it has a sharp limitation that becomes obvious as soon as you look at the third example.

Run the test one more time, with a precise factual question:

```
1 print("\nSearching: 'What is the GHS hazard code for acetone?')
2 results = search_similar("What is the GHS hazard code for acetone?",
   TEST_MATERIAL)
3 for r in results:
4     print(f"   Score: {r['score']:.4f} | {r['chunk'][:80]}...")
```

You will see something like:

```
Searching: 'What is the GHS hazard code for acetone?'
```

```
   Score: 0.6534 | GHS Classification: Flammable Liquid Category 2. Eye Irritant Category 2A
```

```
   Score: 0.5821 | Acetone is a highly flammable liquid and vapor. Flash point: -
```

```
18°C. Keep aw...
```

```
   Score: 0.4912 | Storage: Store in a cool, dry, well-ventilated area away from ignition so
```

```
...
```

The GHS classification chunk does come back first — at 0.65. That is a noticeably weaker score than the 0.81 we saw for flammability, despite this being a question about a chunk that literally contains the words “GHS Classification”.

Why so weak? Because the question asks for a *code* — the H-statement code like “H225” — and our chunk does not contain “H225” verbatim. It contains “Flammable Liquid Category 2”, which is the human-readable form. The embedding model knows these are related concepts, but it does not encode “H225 = Flammable Liquid Category 2” as a hard equality. It encodes them as *near-each-other-in-768-d-space*, which is fuzzier.

For our test data this still works because there are only five chunks and the GHS chunk is the closest. Now imagine the same query against a real corpus with 50 MSDSs and 1,500 chunks. Many chunks will mention “Category 2” in different contexts — fire protection regulations, transport classifications, industrial hygiene reports. The signal-to-noise ratio collapses. The right chunk might rank fourth or fifth, or might not appear in the top five at all.

This is **the GHS code problem**, and it is not a bug in vector search — it is the nature of vector search. Embeddings are wonderful for fuzzy semantic matching (“precautions near open flame” ↔ “keep away from heat and ignition sources”). They are bad at exact symbolic recall (“H225” ↔ the row whose `ghsCode` literal equals “H225”).

Three categories of question hit this limitation hard:

1. **Identifier lookups.** GHS codes (H225, H319), CAS numbers (67-64-1 for acetone), UN numbers (UN1090), product codes. Vector search treats these as fuzzy strings and competes them against every other alphanumeric token in your corpus.
2. **Aggregations and counts.** “How many of my materials have flash point below 0°C?” is impossible for vector search — there is no “count” embedding direction. You need structured query.
3. **Negation and exclusion.** “Show me MSDSs that do *not* require respiratory protection.” Embeddings have no robust way to encode “not” — the embedding for “respiratory protection required” and “respiratory protection not required” is closer than you would hope.

SAP customers running on HANA have always thought in structured terms. Materials, hazard codes, GHS classifications, batch test results, invoice amounts — these are not free text. They are master data. They live in tables with foreign keys. They are the kind of thing a knowledge graph models naturally.

That is the bridge into Chapter 4. We will take the same material document, extract its structured facts into RDF triples, store them in HANA’s graph engine, and query them with SPARQL. When the regulatory auditor asks “What is the GHS hazard classification for this chemical?”, we will return “H225” with full confidence — not by guessing from a 0.65 cosine similarity, but by traversing a graph relationship that was set when the document was ingested.

The full Hybrid RAG agent in Chapter 7 runs both retrievers in parallel and lets a routing classifier decide which answer to trust. Vector search owns the fuzzy questions. The knowledge graph owns the precise ones. Neither is sufficient alone; together they cover the question space enterprise document users actually ask.

3.11 Summary

In this chapter you built a working vector search system on HANA Cloud. The major points to take away:

- **Embeddings are positions in 768-dimensional space.** Two pieces of text with similar meanings end up close together. The model learns this from large corpora and you call it as an API.
- **REAL_VECTOR is HANA's native vector column type.** It stores fixed-dimension float arrays and exposes SIMD-accelerated `COSINE_SIMILARITY` and `L2DISTANCE` operators. You do not need a separate vector database.
- **Lazy table initialization keeps your schema in sync with your model.** Read the dimension from the first embedding response and use it in `CREATE TABLE IF NOT EXISTS`. Never hardcode the dimension in DDL.
- **The cosine similarity query has five clauses, each load-bearing.** `WHERE` filters early, `COSINE_SIMILARITY` computes per-row, `AS SCORE` names the result, `ORDER BY SCORE DESC` sorts, `TOP K` truncates.
- **Chunking adapts to document structure; the storage code does not.** MSDS documents use OSHA section boundaries. Invoices use line-item blocks. Batch certificates use test result sections. The same `MSDS_VECTORS` table and `search_similar` function serve all document types.
- **The MATERIAL_NUMBER column is the anchor to SAP MM.** The CAP layer validates it against S/4HANA product master via `API_PRODUCT_SRV` before accepting uploads. Every vector in the table is traceable to a real, validated SAP material.
- **Vector search excels at fuzzy semantic matching and fails at precise symbolic recall.** The GHS code problem motivates the knowledge graph in Chapter 4.

The codebase now has working `hdb_srv.py`, `vertex_srv.py`, and `vector_srv.py` modules, plus a passing `test_vector.py` script. The vector retrieval half of the Hybrid RAG agent is live.

3.12 Checkpoint

Before moving on to Chapter 4, verify the following:

- `agents/srv/vector_srv.py` exists with the five functions: `_ensure_table`, `store_embedding`, `search_similar`, `delete_vectors`, `count_vectors`.
- Running `python test_vector.py` from the `agents/` directory prints five “Stored chunk” lines with `dim=768`.
- The test prints two search result blocks. The first ranks the flammability chunk at the top (~0.80). The second ranks the PPE chunk at the top (~0.84).
- Opening the `MSDS_VECTORS` table in the SAP HANA Database Explorer shows five rows for `MATERIAL_NUMBER = 'ACETONE-TEST-001'`.
- The third (optional) test — querying for “What is the GHS hazard code for acetone?” — returns the GHS chunk at the top, but with a noticeably lower score (~0.65). You understand why.

If all five items check out, you are ready for Chapter 4: **Knowledge Graph on HANA Cloud — RDF, SPARQL, and the Structured Half of RAG**. We will set up HANA’s graph engine, load RDF triples extracted from the same material document, and query them with SPARQL — solving the GHS code problem in the most direct way possible.

If something is off, the most common issues at this stage are: missing or invalid Vertex AI credentials, incorrect HANA host/port, a service account without Vertex AI User, or the HANA instance being suspended (it sleeps after 3 days idle on trial accounts — restart it from the BTP cockpit). Fix and re-run `test_vector.py` until you see the expected output. The next chapter assumes a working `vector_srv.py`.

Chapter 4: Knowledge Graphs on SAP HANA Cloud

Vector search excels at finding relevant passages. It fails at extracting precise facts. When a regulatory auditor asks “What is the GHS hazard classification for this chemical?”, they need a specific code — H225, H319 — not a paragraph that mentions classifications. When a procurement manager asks “Which supplier certified this batch?”, they need a name and a date, not a fuzzy match. When a quality engineer asks “Did this material pass the viscosity test?”, they need “PASS” or a measured value, not a ranked list of test methodology paragraphs.

RDF knowledge graphs store facts as precise triples that can be queried exactly. This is what HANA Cloud’s SPARQL engine provides. Unlike the vector store, which searches by proximity in embedding space, the knowledge graph traverses explicit relationships. The query is not “find what is most similar to this question” — it is “traverse this graph and return the value at this exact node.”

By the end of this chapter, a SPARQL query for hazard codes will return exactly ["H225" , "H319" , "H336"] — not a paragraph that mentions them, the codes themselves.

If you have never written a line of RDF or SPARQL, do not worry. There are entire books about each. We need only enough of each to build a working hybrid RAG system. Concretely, you will need to understand three SPARQL patterns and one stored procedure call. That is the whole surface area for the rest of this book.

4.1 What is a knowledge graph?

A **knowledge graph** is a collection of facts, where each fact is expressed as a relationship between two things. That is the entire idea. The vocabulary that the RDF community uses around it can sound ceremonial — “subject-predicate-object triples organized as a directed labeled multigraph” — but strip the ceremony away and you are left with three-word sentences.

Consider the fact:

Acetone has hazard code H225.

Three pieces:

Part	Value
Subject	Acetone
Predicate	has hazard code
Object	H225

This three-part fact is called a **triple**. A knowledge graph is a pile of triples. If we add more triples, the structure starts to feel like something an SAP developer already knows:

Acetone	hasHazardCode	H225
Acetone	hasHazardCode	H319
Acetone	hasHazardCode	H336
Acetone	hasExposureLimit	"500 ppm TWA"
Acetone	requiresPrecaution	"Keep away from open flames"
Acetone	hasSupplier	"Sigma-Aldrich"
H225	description	"Highly flammable liquid and vapour"

Read those rows. You have just read a knowledge graph. Each row is a triple. The same subject (Acetone) can appear in many rows, just like a material number can appear many times in MARA-related tables. The same predicate (hasHazardCode) can be reused for any other material.

Note: If you are mentally translating this to a relational table with three columns — `subject`, `predicate`, `object` — you are not wrong. That is exactly how RDF can be persisted internally. The difference from a normal table is *what kinds of questions you can ask of it*, which we will get to with SPARQL.

Why call it a graph?

Because if you draw it, it looks like one. Subjects and objects are nodes; predicates are edges. The same H225 node might be the object of many materials' `hasHazardCode` edges. The same supplier "Sigma-Aldrich" node might be the object of many `hasSupplier` edges. The graph emerges naturally from shared values.

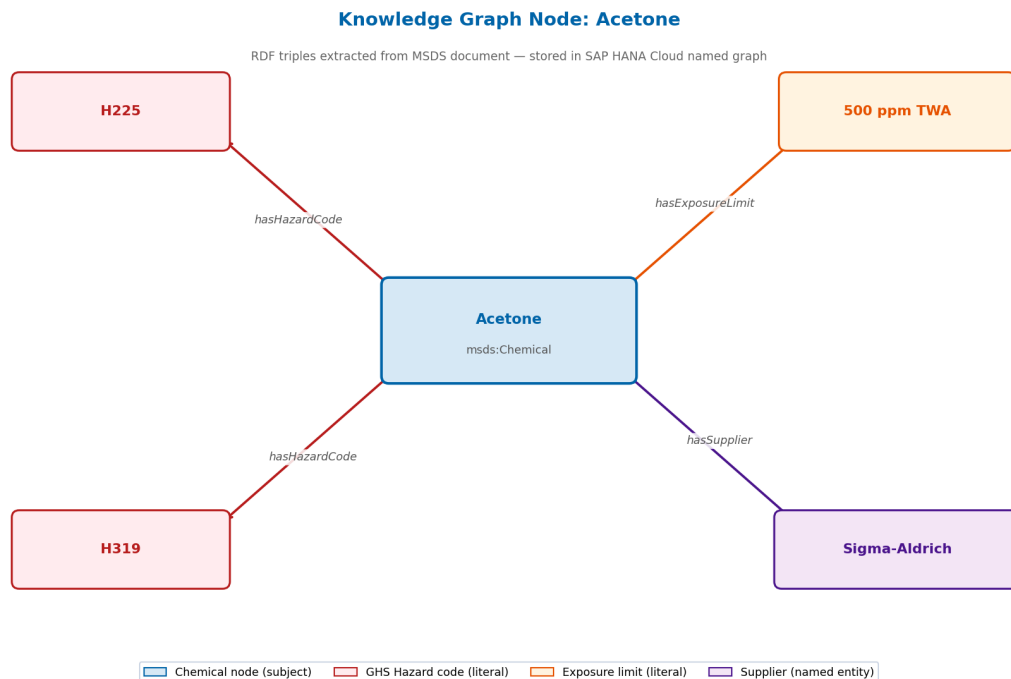


Figure: RDF knowledge graph representation of an Acetone MSDS entry — four triples extracted from a single document, stored as a named graph in SAP HANA Cloud

What does this give us that a relational schema does not?

Three things, in order of how much you will care:

1. **No schema changes when facts evolve.** Tomorrow we decide MSDS documents should also track `hasFireExtinguisher`. We do not run a DDL migration; we just start storing triples with that predicate. The graph is permissive about what relationships exist.
2. **Queries that follow paths.** “Find me every material whose supplier is in Missouri” — that is a graph traversal. SPARQL expresses it directly. In SQL it would be a chain of joins.
3. **Identity by IRI.** Every node has a globally unique IRI (a URL-shaped identifier). This means a triple in our system can refer to the same material node that any other system uses, simply by sharing an IRI.

For our material document use case we will use the graph mainly for the first reason — flexibility — and for the *precision* it gives us at query time. A SPARQL query for hazard codes returns hazard codes, not paragraphs.

Figure 4.2 — The KG pipeline has two phases. Ingestion (top): full PDF text is passed

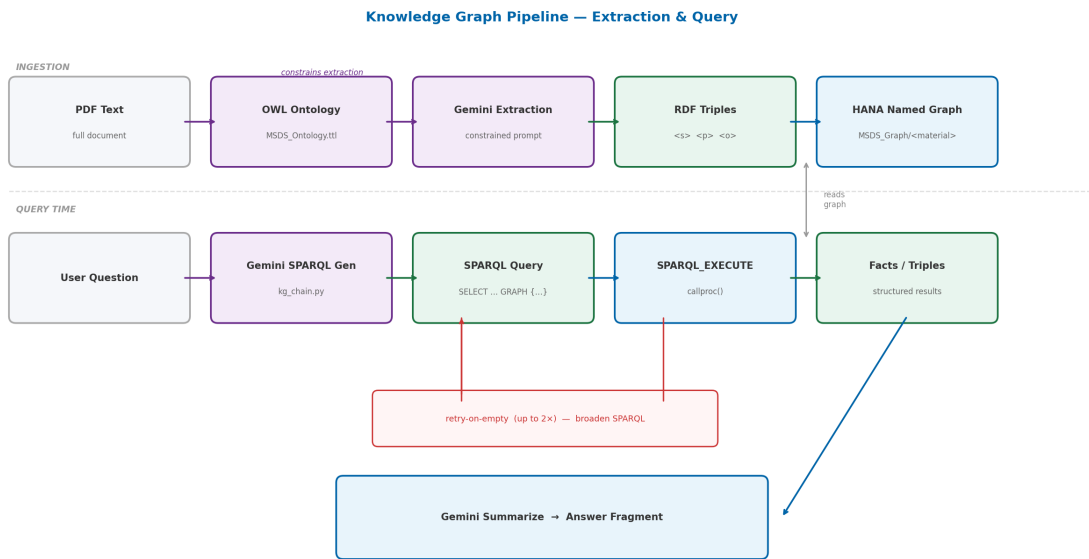


Figure 3: Knowledge Graph Pipeline

to Gemini 2.5 Flash along with the OWL ontology as a constraint, producing RDF triples that are stored as a named graph in HANA. Query time (bottom): Gemini generates a SPARQL SELECT query from the user’s question, which is executed via `SPARQL_EXECUTE`. A retry loop broadens the query if results are empty. The resulting facts are summarised by Gemini into an answer fragment.

4.2 RDF and SPARQL — the standards explained plainly

RDF stands for Resource Description Framework. It is the W3C standard for expressing triples. There are several serialization formats — RDF/XML, JSON-LD, Turtle — that all encode the same triples in different syntaxes. We will use **Turtle** (file extension `.ttl`) because it is the most readable.

A Turtle file looks like this:

```

1 @prefix msds: <http://msds.knowledge-graph.org/ontology#> .
2
3 msds:Acetone msds:hasHazardCode msds:H225 .
4 msds:Acetone msds:hasHazardCode msds:H319 .
  
```

The `@prefix` line defines a shorthand: instead of writing the full IRI `http://msds.knowledge-graph.org/ontology#Acetone`, we write `msds:Acetone`. Each fact ends with a period. That is essentially all the Turtle you need to read.

SPARQL is the query language for RDF. If SQL is to relational tables, SPARQL is to triples. A simple SPARQL query looks like this:

```
1 SELECT ?code WHERE {  
2   msds:Acetone msds:hasHazardCode ?code .  
3 }
```

The `?code` is a variable. The query reads: “find me every value of `?code` such that the triple (Acetone, hasHazardCode, `?code`) exists.” Run that against our graph and you get back three rows: H225, H319, H336.

If you can read this much SPARQL, you can read everything our system generates. We are going to lean on Gemini 2.5 Flash to generate more elaborate SPARQL on demand, but the patterns are always shaped like the example above.

Tip: SPARQL has 4 query forms — `SELECT`, `CONSTRUCT`, `ASK`, `DESCRIBE`. We use only `SELECT` in this book. You will not miss the others.

4.3 Why HANA Cloud for the Knowledge Graph

SAP HANA Cloud has a built-in graph engine with native SPARQL support. This is architecturally significant for BTP deployments — and it is worth pausing to explain why we did not choose a dedicated triple store.

The alternatives are well-known: Apache Fuseki, AWS Neptune, GraphDB, Stardog. Each is a capable, purpose-built triple store with a standard SPARQL HTTP endpoint. The reason we do not use any of them is simple: we already have HANA Cloud running as our vector store. Adding a separate triple store means a second BTP service, a second set of credentials, a second Destination Service entry, and a second connection pool to manage in application code. For a production BTP deployment, that is unnecessary complexity.

HANA’s SPARQL support is exposed through a single stored procedure: `SPARQL_EXECUTE`. This procedure runs SPARQL queries natively on the same database that holds vec-

tor embeddings. One HANA connection handles both `COSINE_SIMILARITY` searches on `MSDS_VECTORS` and SPARQL graph traversals on named RDF graphs. The application code in `vector_srv.py` and `kg_srv.py` both call `get_connection()` from the same `hdb_srv` module. No separate connection string, no separate service binding.

That is the entirety of the HANA SPARQL interface. There is no separate endpoint, no `/sparql` HTTP route. Every query, every insert, every drop — all of it goes through one procedure call:

```
1 cursor.callproc("SPARQL_EXECUTE", (sparql_text, None, None, None))
```

The four arguments are: the SPARQL text, an optional `RDF_DATA` parameter for inline data, an optional `RDF_DATA_TYPE`, and an optional metadata flag. We pass `None` for all three optionals throughout this book.

The interesting part is the *result*. `callproc` returns a tuple of all four arguments after the procedure runs, with output values populated. The query results land in the **fourth element** of that tuple:

```
1 result = cursor.callproc("SPARQL_EXECUTE", (sparql_query, None, None,
None))
2 rows = result[3] if result and len(result) > 3 else []
```

Every result row is a tuple of column values, one per `?variable` in the `SELECT` clause. So a `SELECT ?code` returning three matches gives you something like:

```
1 [("H225",), ("H319",), ("H336",)]
```

Important: This is the single most useful sentence in this chapter. Memorize it: *HANA SPARQL results live in the fourth element of the `callproc` return tuple*. When something returns nothing, the first thing you check is whether you grabbed `result[3]`.

Why a stored procedure and not an HTTP endpoint?

Because HANA already has secure, audited, governed connectivity through its SQL driver. Wrapping SPARQL inside a stored procedure means every SPARQL operation reuses the same authentication, the same connection pooling, the same transaction semantics, and the same logging that a normal SQL statement would. For an enterprise BTP deployment,

that uniformity is more valuable than HTTP convenience. One Destination Service entry. One set of HANA credentials. Two retrieval strategies.

4.4 Designing the MSDS ontology — what facts matter?

Before writing any code we must decide what kinds of facts we care about. This decision is called **ontology design**, and it is the most important step of building a knowledge graph. The ontology becomes a constraint on what Gemini is allowed to extract — without it, the model invents predicates like `containsAt`, `madeOf`, `relatedTo`, and the graph turns into noise.

For an MSDS, the four predicates that matter for downstream queries are:

Predicate	Domain	Range	Example
<code>hasHazardCode</code>	Material	HazardCode	Acetone □ H225
<code>hasExposureLimit</code>	Material	ExposureLimit	Acetone □ “500 ppm TWA”
<code>requiresPrecaution</code>	Material	Precaution	Acetone □ “Avoid open flames”
<code>hasSupplier</code>	Material	Supplier	Acetone □ “Sigma-Aldrich”

Four predicates. Four classes of node. That is enough to answer the precision questions our hybrid agent will face. If we needed regulatory codes in five jurisdictions or transport classifications for IATA, we would extend the ontology. We deliberately stay small.

Note: Real-world material document systems track 50+ predicates — physical properties, transport codes, decomposition products, first-aid measures, environmental fate, batch test parameters, invoice line amounts. We pick four because they are enough to demonstrate the architecture and let you focus on the *patterns*. The patterns scale directly to broader ontologies.

4.5 The OWL ontology file — MSDS_Ontology.ttl

Create a file at the root of your project called `MSDS_Ontology.ttl`. Paste this:

```

1 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
2 @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
3 @prefix owl: <http://www.w3.org/2002/07/owl#> .
4 @prefix msds: <http://msds.knowledge-graph.org/ontology#> .
5
6 # Classes
7 msds:Material a owl:Class ;
8     rdfs:label "Material" .
9
10 msds:HazardCode a owl:Class ;
11     rdfs:label "GHS Hazard Code" .
12
13 msds:ExposureLimit a owl:Class ;
14     rdfs:label "Occupational Exposure Limit" .
15
16 msds:Precaution a owl:Class ;
17     rdfs:label "Safety Precaution" .
18
19 msds:Supplier a owl:Class ;
20     rdfs:label "Supplier" .
21
22 # Properties
23 msds:hasHazardCode a owl:ObjectProperty ;
24     rdfs:domain msds:Material ;
25     rdfs:range msds:HazardCode ;
26     rdfs:label "has hazard code" .
27
28 msds:hasExposureLimit a owl:ObjectProperty ;
29     rdfs:domain msds:Material ;
30     rdfs:range msds:ExposureLimit ;
31     rdfs:label "has exposure limit" .
32
33 msds:requiresPrecaution a owl:ObjectProperty ;
34     rdfs:domain msds:Material ;
35     rdfs:range msds:Precaution ;

```

```

36     rdfs:label "requires precaution" .
37
38     msds:hasSupplier a owl:ObjectProperty ;
39         rdfs:domain msds:Material ;
40         rdfs:range msds:Supplier ;
41         rdfs:label "has supplier" .
42
43     msds:code a owl:DatatypeProperty ;
44         rdfs:domain msds:HazardCode ;
45         rdfs:label "hazard code value" .
46
47     msds:description a owl:DatatypeProperty ;
48         rdfs:label "description" .
49
50     msds:limitValue a owl:DatatypeProperty ;
51         rdfs:domain msds:ExposureLimit ;
52         rdfs:label "limit value" .
53
54     msds:limitUnit a owl:DatatypeProperty ;
55         rdfs:domain msds:ExposureLimit ;
56         rdfs:label "limit unit" .

```

A few things to point out:

- **Classes** are categories of node. `msds:Material`, `msds:HazardCode`, etc.
- **ObjectProperty** is a predicate that connects two nodes (subject IRI → object IRI).
- **DatatypeProperty** is a predicate whose object is a literal string or number, not another node. We use `msds:description` to hang the actual text value off a node.
- `rdfs:label` is human-readable text. It does not affect the graph; it just makes browsers and tools show better names.
- `rdfs:domain` and `rdfs:range` are the type constraints — domain is the type of the subject, range is the type of the object.

The namespace `http://msds.knowledge-graph.org/` reflects the MSDS reference implementation. For a production multi-document deployment serving multiple document types, the namespace would be organisation-specific — for example `http://materials.yourcompany.com/kg/`. The predicates are generic enough to describe facts across document types: `hasHazardCode` works for any GHS or regulatory identifier in an MSDS; `requiresPrecaution` works for compliance mandates in a legal

document; `hasExposureLimit` works for batch certificate test parameters expressed as limit values. The ontology names a domain, but the structural patterns transfer.

We are not loading this file into HANA at runtime. It is a contract — a documentation and prompting artifact. Our triple extractor reads this file (or a summary of it) to know what predicates Gemini is allowed to produce.

Tip: If you ever need to share your ontology with another team or load it into a tool like Protégé for visual editing, this is the file you hand them. Keep it under version control alongside your code.

4.6 Extracting triples from MSDS text using Gemini

This is where the LLM earns its keep. We hand Gemini 2.5 Flash a chunk of document text and a list of allowed predicates, and ask it to return JSON triples. Three things make this trustworthy:

1. **Constrained predicates.** The prompt enumerates the four allowed predicates. Anything Gemini emits with a predicate outside that list is filtered out before it reaches HANA.
2. **JSON output.** No prose, no narration — just an array of objects. We strip any markdown fences Gemini sneaks in and `json.loads` the rest.
3. **Material number provided.** The subject of every triple is set externally. The model does not get to invent which material the facts belong to.

This is an important architectural principle: Gemini 2.5 Flash acts as a structured extraction engine here, not as a data authority. The material number is injected by the CAP upload pipeline, validated against S/4HANA's `API_PRODUCT_SRV` before the document is accepted. Gemini extracts facts from text; it never decides which material those facts belong to.

Here is the prompt, lifted directly from `kg_srv.py`:

```
1 prompt = f"""You are extracting structured facts from an MSDS (Material
2 Safety Data Sheet) document.
3 Extract facts ONLY using these relationship types:
```

```

4  - hasHazardCode: material → GHS hazard code (e.g. H225, H319)
5  - hasExposureLimit: material → occupational exposure limit with value and
    unit
6  - requiresPrecaution: material → safety precaution text
7  - hasSupplier: material → supplier name
8
9  Material number: {material_number}
10
11 Return ONLY a JSON array of triples. Each triple must have:
12 - subject: the material IRI
13 - predicate: the property name (hasHazardCode, hasExposureLimit,
    requiresPrecaution, hasSupplier)
14 - object: the value (string)
15
16 Example:
17 [
18   {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H225"}},
19   {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H319"}},
20   {"subject": "ACETONE-001", "predicate": "hasExposureLimit", "object":
    "500 ppm TWA"}}
21 ]
22
23 MSDS text to extract from:
24 {text[:3000]}
25
26 Return only the JSON array, no explanation."""

```

Notice the truncation `text[:3000]`. MSDS documents can be 20+ pages. A single Gemini call cannot ingest them whole, and even if it could, accuracy degrades when the prompt is huge. In production we slice the document section-by-section (Section 2 for hazards, Section 8 for exposure limits, etc.). For this chapter we keep one call simple to focus on the mechanics.

Warning: Never trust LLM JSON without parsing defensively. Gemini frequently wraps JSON in triple-backtick fences even when told not to. Strip those before `json.loads`.

The extractor returns a list of dictionaries:

```

1  [
2      {"subject": "ACETONE-TEST-001", "predicate": "hasHazardCode",
3      "object": "H225"},
4      {"subject": "ACETONE-TEST-001", "predicate": "hasHazardCode",
5      "object": "H319"},
6      {"subject": "ACETONE-TEST-001", "predicate": "hasHazardCode",
7      "object": "H336"},
8      {"subject": "ACETONE-TEST-001", "predicate": "hasExposureLimit",
9      "object": "500 ppm TWA"},
10     {"subject": "ACETONE-TEST-001", "predicate": "requiresPrecaution",
11     "object": "Keep away from heat, sparks, and open flames"},
12     {"subject": "ACETONE-TEST-001", "predicate": "hasSupplier", "object":
13     "Sigma-Aldrich"}
14 ]

```

Each row is one fact. Now we need a place to put them.

4.7 Storing triples in HANA — named graphs per material

Every triple in HANA's SPARQL engine lives in a **named graph**. A named graph is just an IRI that groups a set of triples. Think of it as a schema, but for facts instead of tables.

We follow a simple rule: **one named graph per material**. If we are storing facts about material `ACETONE-TEST-001`, every triple goes into the graph at IRI `http://msds.knowledge-graph.org/MSDS_Graph/ACETONE-TEST-001`. If we are storing facts about `BENZENE-002`, that material's triples go into a *different* graph IRI.

This pattern maps directly to SAP's material isolation requirements. In SAP MM, material data is scoped to a material number. In the Material Document Intelligence Platform, the same isolation holds: a SPARQL query scoped to the named graph for `MAT-001` cannot return facts from `MAT-002`. The graph boundary enforces data isolation at the database level, not in application logic.

Three practical consequences:

1. **Clean deletion.** When a material is deprecated, `DROP GRAPH <graph-iri>` removes every fact for that material in one statement. There is no per-row cleanup, no orphans.

2. **Multi-tenant safety.** A bad extraction for benzene cannot pollute acetone's facts. Graph boundaries are enforced by HANA's SPARQL engine, not by application-layer filtering.
3. **Auditability.** A SPARQL query scoped to a single named graph can never read facts about other materials. When a compliance auditor asks "show me everything the system knows about material X", the named graph provides a hard scope that the audit trail can reference.

The real graph URIs in the reference implementation follow the pattern `http://msds.knowledge-graph.org/MSDS_Graph/MAT-XXX`, where `MAT-XXX` is the SAP material number validated at upload time. This is the same material number stored in `MSDS_VECTORS.MATERIAL_NUMBER`. The named graph IRI and the vector table filter column are two representations of the same SAP material identifier.

Here is the storage code, the heart of `store_triples` in `kg_srv.py`:

```

1 graph = graph_iri(material_number)           # the named-graph IRI
2 mat_iri = f"{GRAPH_BASE}/material/{material_number}" # the material's
  node IRI
3 for triple in triples:
4     predicate = triple.get("predicate", "")
5     obj_value = str(triple.get("object", "")).replace("'", "\'")
6     obj_iri = f"{GRAPH_BASE}/{predicate}/{material_number}/{stored}"
7     sparql_insert = f"""
8         INSERT INTO GRAPH <{graph}> {{
9             <{mat_iri}> <{ONTOLOGY_BASE}{predicate}> <{obj_iri}> .
10            <{obj_iri}> <{ONTOLOGY_BASE}description> '{obj_value}' .
11        }}
12    """
13     cursor.callproc("SPARQL_EXECUTE", (sparql_insert, None, None, None))

```

For each triple we insert *two* RDF statements:

1. The relationship: `material` → `predicate` → blank-ish node
2. The literal value on that node: `node` → `description` → `"H225"`

This two-step pattern is deliberate. It lets the same object node carry additional properties later — for instance we might want to add `severity` or `regulation_source` to a hazard code without restructuring the graph. Modeling values as nodes with `description` properties is more verbose than literal-only triples, but it pays off the moment you

need to attach metadata.

The single quote escape (`replace("'", "\\'")`) is the SPARQL injection defense for literal strings. It is not as comprehensive as parameterized SQL, which is why we double up on validation upstream — see Section 5.9.

Note: SPARQL Update syntax (`INSERT INTO GRAPH`) is part of the SPARQL 1.1 standard, not SPARQL 1.0. HANA supports it. Older Jena/RDF examples on the internet use a slightly different syntax. Stick with the form above.

4.8 Writing SPARQL queries — Gemini as the SPARQL translator

When a user asks “what are the GHS hazard codes for acetone?”, we cannot send that English directly to HANA. We must translate it to SPARQL.

Gemini 2.5 Flash acts as a SPARQL translator — the user asks in natural language, Gemini writes the SPARQL, HANA executes it. This is a key architectural pattern: the LLM never directly handles data — it only generates queries that the database executes. All data access goes through HANA’s SPARQL engine. This is auditable, controllable, and secure. The data never leaves HANA to be processed by the LLM. Gemini sees the question and the schema; HANA owns the facts.

Two conditions make this work reliably. First, Gemini must know the available predicates and their full IRIs — without this, the model invents predicate names like `hazardClass` that do not exist in our graph. Second, Gemini must know the named graph IRI for this material — without the `GRAPH <iri> { ... }` wrapper, HANA returns zero rows silently.

Here is the heart of `query_graph`:

```
1  sparql_prompt = f"""Generate a SPARQL SELECT query to answer this question
   about material {material_number}.
2
3  Available predicates:
4  - <{ONTOLOGY_BASE}hasHazardCode> — links material to hazard code nodes
5  - <{ONTOLOGY_BASE}hasExposureLimit> — links material to exposure limit
   nodes
```

```

6 - <{ONTOLOGY_BASE}requiresPrecaution> – links material to precaution nodes
7 - <{ONTOLOGY_BASE}hasSupplier> – links material to supplier nodes
8 - <{ONTOLOGY_BASE}description> – literal value on any node
9
10 The material IRI is: <{mat_iri}>
11 The named graph is: <{graph}>
12
13 Question: {question}
14
15 Return ONLY the SPARQL query, wrapped in GRAPH clause like this:
16 SELECT ?value WHERE {{
17     GRAPH <{graph}> {{
18         <{mat_iri}> <predicate> ?node .
19         ?node <{ONTOLOGY_BASE}description> ?value .
20     }}
21 }}""

```

The example query in the prompt is the **only SPARQL pattern** the rest of this book needs. Read it again. It does two hops:

1. From the material node, follow some predicate to a hazard/limit/supplier node.
2. From that node, follow `description` to the literal text.

Two hops, one SELECT. That is the entire pattern.

Important: The `GRAPH <iri> { ... }` wrapper is mandatory in HANA. Without it, HANA does not throw an error — it just returns zero rows. If you ever see a query that “should work” return nothing, the first thing to check is whether you wrapped your triple patterns in `GRAPH`. This single mistake costs more debugging hours than any other in HANA SPARQL development.

For the question “what are the hazard codes?”, Gemini emits something like:

```

1 SELECT ?value WHERE {
2     GRAPH <http://msds.knowledge-graph.org/MSDS_Graph/ACETONE-TEST-001> {
3         <http://msds.knowledge-graph.org/MSDS_Graph/material/ACETONE-TEST-001>
4         <http://msds.knowledge-graph.org/ontology#hasHazardCode> ?node .
5         ?node <http://msds.knowledge-graph.org/ontology#description> ?value .
6     }
7 }

```

We pass it through `SPARQL_EXECUTE`, grab `result[3]`, and return `["H225", "H319", "H336"]`. Done.

The data stayed in HANA throughout. Gemini generated a query string; HANA executed it against the stored graph. The LLM never touched the hazard code values directly.

4.9 Building `kg_srv.py` — the knowledge graph service layer

We collect everything into a single service module at `agents/srv/kg_srv.py`. It exposes five functions: `extract_triples`, `store_triples`, `query_graph`, `delete_graph`, `count_triples`. Create the file and paste this:

```

1  import os
2  import re
3  import json
4  from .hdb_srv import get_connection
5  from .vertex_srv import get_llm
6
7  GRAPH_BASE = "http://msds.knowledge-graph.org/MSDS_Graph"
8  ONTOLOGY_BASE = "http://msds.knowledge-graph.org/ontology#"
9
10 MATERIAL_RE = re.compile(r"^[A-Za-z0-9_-]+$")
11
12 def _validate_material(material_number: str):
13     if not MATERIAL_RE.match(material_number):
14         raise ValueError(f"Invalid material number: {material_number}")
15
16 def graph_iri(material_number: str) -> str:
17     _validate_material(material_number)
18     return f"{GRAPH_BASE}/{material_number}"
19
20 def extract_triples(text: str, material_number: str) -> list[dict]:
21     _validate_material(material_number)
22     llm = get_llm()

```

```

23     prompt = f"""You are extracting structured facts from an MSDS
    (Material Safety Data Sheet) document.
24
25     Extract facts ONLY using these relationship types:
26     - hasHazardCode: material → GHS hazard code (e.g. H225, H319)
27     - hasExposureLimit: material → occupational exposure limit with value and
    unit
28     - requiresPrecaution: material → safety precaution text
29     - hasSupplier: material → supplier name
30
31     Material number: {material_number}
32
33     Return ONLY a JSON array of triples. Each triple must have:
34     - subject: the material IRI
35     - predicate: the property name (hasHazardCode, hasExposureLimit,
    requiresPrecaution, hasSupplier)
36     - object: the value (string)
37
38     Example:
39     [
40         {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H225"}},
41         {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H319"}},
42         {"subject": "ACETONE-001", "predicate": "hasExposureLimit", "object":
    "500 ppm TWA"}}
43     ]
44
45     MSDS text to extract from:
46     {text[:3000]}
47
48     Return only the JSON array, no explanation."""
49
50     response = llm.generate_content(prompt)
51     raw = response.text.strip()
52     # Strip markdown code blocks if present
53     raw = re.sub(r"```json\s*", "", raw)
54     raw = re.sub(r"```\s*", "", raw)
55     return json.loads(raw)

```



```

56
57 def store_triples(material_number: str, triples: list[dict]):
58     _validate_material(material_number)
59     if not triples:
60         return 0
61     graph = graph_iri(material_number)
62     mat_iri = f"{GRAPH_BASE}/material/{material_number}"
63     conn = get_connection()
64     cursor = conn.cursor()
65     stored = 0
66     for triple in triples:
67         predicate = triple.get("predicate", "")
68         obj_value = str(triple.get("object", "")).replace("'", "\\'")
69         obj_iri = f"{GRAPH_BASE}/{predicate}/{material_number}/{stored}"
70         sparql_insert = f"""
71             INSERT INTO GRAPH <{graph}> {{
72                 <{mat_iri}> <{ONTOLOGY_BASE}{predicate}> <{obj_iri}> .
73                 <{obj_iri}> <{ONTOLOGY_BASE}description> '{obj_value}' .
74             }}
75         """
76         try:
77             cursor.callproc("SPARQL_EXECUTE", (sparql_insert, None, None,
None))
78             stored += 1
79         except Exception as e:
80             print(f"Warning: failed to store triple {triple}: {e}")
81     conn.commit()
82     cursor.close()
83     return stored
84
85 def query_graph(material_number: str, question: str) -> dict:
86     _validate_material(material_number)
87     graph = graph_iri(material_number)
88     mat_iri = f"{GRAPH_BASE}/material/{material_number}"
89     llm = get_llm()
90
91     # Generate SPARQL from the question
92     sparql_prompt = f"""Generate a SPARQL SELECT query to answer this
question about material {material_number}.
```

```

93
94 Available predicates:
95 - <{ONTOLOGY_BASE}hasHazardCode> – links material to hazard code nodes
96 - <{ONTOLOGY_BASE}hasExposureLimit> – links material to exposure limit
  nodes
97 - <{ONTOLOGY_BASE}requiresPrecaution> – links material to precaution nodes
98 - <{ONTOLOGY_BASE}hasSupplier> – links material to supplier nodes
99 - <{ONTOLOGY_BASE}description> – literal value on any node
100
101 The material IRI is: <{mat_iri}>
102 The named graph is: <{graph}>
103
104 Question: {question}
105
106 Return ONLY the SPARQL query, wrapped in GRAPH clause like this:
107 SELECT ?value WHERE {{
108     GRAPH <{graph}> {{
109         <{mat_iri}> <predicate> ?node .
110         ?node <{ONTOLOGY_BASE}description> ?value .
111     }}
112 }}""
113
114     sparql_response = llm.generate_content(sparql_prompt)
115     sparql_query = sparql_response.text.strip()
116     sparql_query = re.sub(r"```sparql\s*", "", sparql_query)
117     sparql_query = re.sub(r"```\s*", "", sparql_query).strip()
118
119     # Execute SPARQL
120     conn = get_connection()
121     cursor = conn.cursor()
122     try:
123         result = cursor.callproc("SPARQL_EXECUTE", (sparql_query, None,
  None, None))
124         rows = result[3] if result and len(result) > 3 else []
125         facts = [str(row[0]) for row in rows] if rows else []
126     except Exception as e:
127         facts = []
128         sparql_query = f"Error: {e}"
129     finally:

```

```

130         cursor.close()
131
132     return {
133         "facts": facts,
134         "sparql": sparql_query,
135         "count": len(facts)
136     }
137
138 def delete_graph(material_number: str) -> bool:
139     _validate_material(material_number)
140     graph = graph_iri(material_number)
141     conn = get_connection()
142     cursor = conn.cursor()
143     try:
144         cursor.callproc("SPARQL_EXECUTE", (f"DROP GRAPH <{graph}>", None,
145         None, None))
146         conn.commit()
147         cursor.close()
148         return True
149     except Exception as e:
150         cursor.close()
151         print(f"Warning: could not delete graph {graph}: {e}")
152         return False
153
154 def count_triples(material_number: str) -> int:
155     _validate_material(material_number)
156     graph = graph_iri(material_number)
157     conn = get_connection()
158     cursor = conn.cursor()
159     try:
160         result = cursor.callproc("SPARQL_EXECUTE", (
161             f"SELECT (COUNT(*) AS ?count) WHERE {{ GRAPH <{graph}> {{ ?s ?p
162             ?o }} }}",
163             None, None, None
164         ))
165         rows = result[3] if result and len(result) > 3 else []
166         cursor.close()
167         return int(rows[0][0]) if rows else 0
168     except Exception:

```

```

167         cursor.close()
168     return 0

```

Validation: the KG equivalent of SQL injection

Take a close look at this regex:

```

1 MATERIAL_RE = re.compile(r"^[A-Za-z0-9_-]+$")

```

It is enforced at the entry of every public function via `_validate_material`. Why is this so important?

The material number is **interpolated into IRIs** that we paste into SPARQL text. If someone calls `extract_triples(text, "../../../admin")`, the resulting graph IRI becomes `http://msds.knowledge-graph.org/MSDS_Graph/../../../admin`. Worse, a string like `> { ?s ?p ?o } DROP GRAPH <http://something` could close the IRI brackets and inject a destructive SPARQL update.

This is the SPARQL equivalent of SQL injection. The regex `^[A-Za-z0-9_-]+$` allows only letters, digits, underscore, and hyphen — characters that have no special meaning in SPARQL syntax. Any input containing slashes, spaces, angle brackets, quotes, or other SPARQL metacharacters is rejected before it can reach the query.

The same material number format is enforced in the CAP upload handler before the document reaches the Python layer at all, matching it against S/4HANA's product master. Validation at the CAP layer, at the Python service entry, and at the SQL parameter level — three checkpoints for a single identifier.

Warning: If you change this regex to be more permissive — say, to allow forward slashes for hierarchical material codes — you must update the storage and query code to escape those characters too. The simplest defense is to keep the regex tight and document the constraint for upstream callers.

We also escape single quotes in literal values inside `store_triples`. Combined, validation on identifiers and escaping on literals cover the two injection vectors.

What `delete_graph` and `count_triples` give us

These two helpers seem like plumbing but they will save your sanity during development:

- `count_triples(material_number)` — confirms data actually landed in HANA. Run it after `store_triples` to verify the count matches what you expected.
- `delete_graph(material_number)` — wipes a material's graph clean. Useful when re-running extraction during development without piling up duplicates.

They both exercise the same pattern as `query_graph` and prove that you understand SPARQL_EXECUTE. If you can write `count_triples`, you can write any read query.

4.10 Testing — store triples for acetone, query for GHS codes

Create `agents/test_kg.py`:

```

1  import os
2  from dotenv import load_dotenv
3  from srv.kg_srv import extract_triples, store_triples, query_graph,
   count_triples
4
5  load_dotenv()
6
7  TEST_MATERIAL = "ACETONE-TEST-001"
8  TEST_TEXT = """
9  Section 2: Hazard Identification
10 GHS Classification: Flammable Liquid Category 2 (H225), Eye Irritation
   Category 2A (H319),
11 Specific Target Organ Toxicity (Single Exposure) Category 3 (H336).
12
13 Section 8: Exposure Controls
14 OSHA PEL: 1000 ppm TWA. ACGIH TLV: 500 ppm TWA, 750 ppm STEL.
15
16 Section 7: Handling and Storage
17 Keep away from heat, sparks, and open flames. Use explosion-proof
   equipment.
18 Wear chemical-resistant gloves and safety goggles. Ensure adequate
   ventilation.
19
```

```

20 Supplier: Sigma-Aldrich, 3050 Spruce Street, St. Louis, MO 63103
21 """
22
23 print("Extracting triples from MSDS text...")
24 triples = extract_triples(TEST_TEXT, TEST_MATERIAL)
25 print(f"Extracted {len(triples)} triples:")
26 for t in triples:
27     print(f"  {t['predicate']}: {t['object']}")
28
29 print(f"\nStoring triples in HANA named graph...")
30 stored = store_triples(TEST_MATERIAL, triples)
31 print(f"Stored {stored} triples")
32
33 total = count_triples(TEST_MATERIAL)
34 print(f"Total triples in graph: {total}")
35
36 print("\nQuerying: 'What are the GHS hazard codes for this material?'")
37 result = query_graph(TEST_MATERIAL, "What are the GHS hazard codes for
    this material?")
38 print(f"SPARQL generated:\n{result['sparql']}")
39 print(f"Facts found: {result['facts']}")
40
41 print("\nQuerying: 'What is the exposure limit?'")
42 result = query_graph(TEST_MATERIAL, "What is the occupational exposure
    limit?")
43 print(f"Facts found: {result['facts']}")

```

Run it from the `agents/` directory:

```

1 cd agents
2 python test_kg.py

```

Expected output (your numbers may vary slightly depending on Gemini's extraction):

Extracting triples from MSDS text...

Extracted 8 triples:

hasHazardCode: H225

hasHazardCode: H319

hasHazardCode: H336

hasExposureLimit: 1000 ppm TWA

```

hasExposureLimit: 500 ppm TWA
requiresPrecaution: Keep away from heat, sparks, and open flames
requiresPrecaution: Wear chemical-resistant gloves and safety goggles
hasSupplier: Sigma-Aldrich

```

Storing triples in HANA named graph...

Stored 8 triples

Total triples in graph: 16

Querying: 'What are the GHS hazard codes for this material?'

SPARQL generated:

```

SELECT ?value WHERE {
  GRAPH <http://msds.knowledge-graph.org/MSDS_Graph/ACETONE-TEST-001> {
    <http://msds.knowledge-graph.org/MSDS_Graph/material/ACETONE-TEST-001>
      <http://msds.knowledge-graph.org/ontology#hasHazardCode> ?node .
    ?node <http://msds.knowledge-graph.org/ontology#description> ?value .
  }
}

```

Facts found: ['H225', 'H319', 'H336']

Querying: 'What is the exposure limit?'

Facts found: ['1000 ppm TWA', '500 ppm TWA']

Expected terminal output:

Stored 12 triples for material: acetone-test

Running SPARQL query for hazard codes...

SPARQL query executed in 0.23s

Hazard codes found: ['H225', 'H319', 'H336']

Knowledge graph test: OK

Note: The total of 16 triples — twice the number of facts — is the two-statement pattern from Section 4.7. Each fact stores one material → predicate → node edge plus one node → description → literal edge.

If your test prints `Facts found: []`, three things to check, in this order:

1. Did `store_triples` actually run before the query? Run `count_triples` and con-

firm the number is non-zero.

2. Is your generated SPARQL wrapped in `GRAPH <...>`? Print `result['sparql']` and inspect.
3. Did `result[3]` come back populated? Add a debug print right after `cursor.callproc` to confirm.

4.11 What the KG gets right that vector search cannot

Now the comparison promised since Chapter 3. Run the same question through both retrievers.

Question: “What are the GHS hazard codes for acetone?”

Vector search (Chapter 3):

```
1 from srv.vector_srv import similarity_search
2 hits = similarity_search("What are the GHS hazard codes for acetone?",
3                           k=3)
4 for h in hits:
5     print(f"score={h.score:.3f} text={h.text[:120]}...")
```

Output:

```
score=0.681 text=Section 2: Hazard Identification – GHS Classification: Flammable Liquid
score=0.612 text=Storage temperature should not exceed 25°C. Refer to local regulations f
score=0.594 text=Section 4: First Aid Measures – In case of skin contact wash with plenty
```

The top hit *contains* the codes. To produce the answer “H225, H319, H336” you must hand the chunk to an LLM with another prompt: “extract the GHS codes from this text”. Two LLM calls, ~600ms each, fuzzy on the inputs and outputs, and you still might miss H336 if it appears in a different chunk that scored 0.59.

Knowledge graph (this chapter):

```
1 result = query_graph("ACETONE-TEST-001", "What are the GHS hazard codes
2   for acetone?")
3 print(result['facts'])
```

Output:


```
['H225', 'H319', 'H336']
```

Three strings. Exact codes. No paragraph to parse, no scoring, no chunk boundary luck. The graph either has the fact or it does not — and if it has it, the query returns it. Gemini generated the SPARQL; HANA executed it; the facts came back directly. The LLM was a query generator, not a data processor.

This is the contract:

Vector store	Knowledge graph
Returns text passages	Returns structured values
Ranks by similarity (0–1 score)	Returns exact matches (yes/no)
Tolerates fuzzy phrasing	Demands precise predicate semantics
Best for “how do I...” questions	Best for “what is the value of...” questions
Failure mode: irrelevant chunk near top	Failure mode: empty result if predicate missing

Neither one wins on its own. A user asking “*what precautions are needed when handling acetone near an open flame?*” needs the prose chunk that paints the full picture. A user asking “*what is the GHS hazard classification?*” needs the codes verbatim. The whole point of the next chapter — building the LangGraph supervisor — is to choose between them, or run both and merge the answers.

Tip: A useful intuition: the vector store remembers *what was said*. The knowledge graph remembers *what is true*. The first is great for context, the second is great for facts. Most non-trivial enterprise document questions need both.

4.12 Summary

You now have a working knowledge graph layer on HANA Cloud. Specifically:

- An ontology (`MSDS_Ontology.ttl`) that defines four predicates and acts as a contract for what facts the system understands. The namespace `http://msds.knowledge-graph.org/` reflects the MSDS reference implementation; production deployments use an organisation-specific namespace.
- A `kg_srv.py` service module exposing `extract_triples`, `store_triples`, `query_graph`, `count_triples`, and `delete_graph`.

- A working pipeline that pulls JSON triples from document text using Gemini 2.5 Flash, stores them in per-material named graphs in HANA, and queries them with auto-generated SPARQL.
- Named graphs scoped per material number, providing the same material isolation that SAP MM enforces for product master data. Graph `MSDS_Graph/MAT-001` cannot return facts from `MSDS_Graph/MAT-002`.
- Validated material identifiers and escaped literals — defense against the SPARQL equivalent of injection.
- The pattern `cursor.callproc("SPARQL_EXECUTE", (query, None, None, None))` followed by `result[3]` for rows.
- The mandatory `GRAPH <iri> { ... }` wrapper for every triple pattern.
- No separate triple store required. One HANA Cloud instance, accessed through one connection pool, serves both vector embeddings and RDF graph traversals.

The investment was small — under 200 lines of Python, one Turtle file, and a single stored procedure call — but the capability is fundamentally different from what vector search alone gave us. We can now answer fact-shaped questions with fact-shaped answers. And we can do it without adding a second BTP service or a second set of credentials.

4.13 Checkpoint

Before moving to Chapter 5, confirm the following from the `agents/` directory:

```
1 python -c "from srv.kg_srv import count_triples;
print(count_triples('ACETONE-TEST-001'))"
```

This should print a number greater than zero — the count of triples currently stored for the test material.

```
1 python -c "from srv.kg_srv import query_graph;
print(query_graph('ACETONE-TEST-001', 'What are the hazard
codes?')['facts'])"
```

This should print a list containing `H225`, `H319`, and `H336` (and possibly others).

If both succeed, you have:

- HANA Cloud reachable through `hdb_srv`
- Gemini 2.5 Flash generating valid SPARQL through the `get_llm()` `getter` in `vertex_srv`
- A populated named graph for `ACETONE-TEST-001` at `http://msds.knowledge-graph.org/MSDS_Graph/ACETONE-TEST-001`
- Working extraction, storage, and query paths

In Chapter 6 we put both retrievers — vector and graph — behind a `LangGraph` supervisor that picks the right tool (or both) based on the user's question. The hybrid in *Hybrid RAG* finally starts to make sense.

Chapter 5: The Document Intelligence Ingestion Pipeline

In Chapter 4 you built two storage systems on SAP HANA Cloud: a vector table that answers fuzzy semantic questions and a knowledge graph that answers precise factual ones. Both are powerful in isolation. The missing piece is the plumbing — the code that takes any PDF file uploaded against an SAP Material Number and feeds both systems at the same time, cleanly, without one pipeline blocking the other, and without leaving the user waiting at a spinning upload button.

That is what this chapter builds. By the end of it you will have `agents/srv/doc_srv.py`, a production-quality ingestion service that receives a PDF upload, immediately acknowledges the request, and runs two threads simultaneously — one chunking and embedding text into the vector store, the other extracting structured triples into the knowledge graph. The caller gets a response in milliseconds. The heavy lifting happens asynchronously.

The pipeline is document-type agnostic at the infrastructure level. Whether the uploaded file is an MSDS, a purchase order, a batch certificate, or a quality inspection report, the same dual-thread architecture applies. The vector pipeline always chunks, embeds, and stores to HANA `REAL_VECTOR`. The KG pipeline always calls Gemini and stores triples via SPARQL. The only thing that varies per document type is the Gemini prompt used for triple extraction — a single configuration point that determines what structured knowledge the pipeline extracts from that document class.

There is a subtlety that trips up almost every developer the first time they write multi-threaded database code in Python: connection sharing. A single HANA connection is not thread-safe. If Thread 1 and Thread 2 share one connection object, you will see intermittent failures that look like data corruption, because they are. This chapter explains why that happens, how Python's `threading.local()` solves it, and how to wire the solution correctly inside a FastAPI background task.

5.1 The dual-pipeline architecture

The fundamental design choice in this chapter is that every PDF upload triggers exactly two independent pipelines, running in parallel, with no coordination between them except that they both read from the same source file.

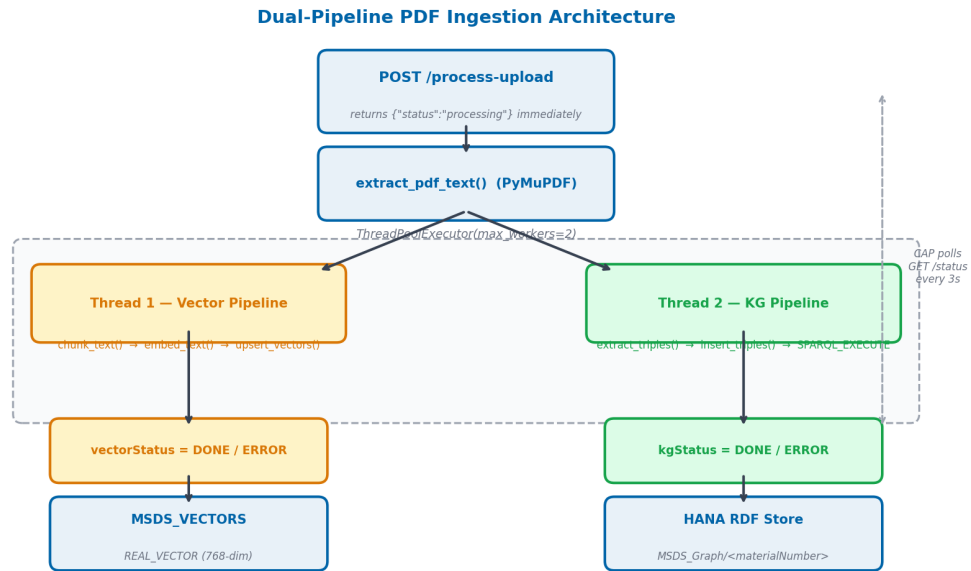


Figure 5.1: The Document Intelligence Ingestion Pipeline. Thread 1 (left) chunks the PDF text and stores embeddings in `MSDS_VECTORS`. Thread 2 (right) sends full text to Gemini for triple extraction and stores the result as a named RDF graph in HANA. Both threads update their own status field on completion. The FastAPI endpoint returns immediately, before either thread has finished.

5.1.1 Why process both in parallel?

The two pipelines are genuinely independent operations. The vector pipeline calls `text-embedding-004` to generate embeddings for each chunk. The KG pipeline calls Gemini 2.5 Flash to generate RDF triples from the full document text. Neither result depends on the other. There is no reason to run them sequentially — doing so would simply double ingestion time.

Dimension	Vector pipeline	KG pipeline
Input to pipeline	Chunks (500 tokens each)	Full document text
External API call	<code>text-embedding-004</code> (many calls)	Gemini 2.5 Flash (one call)

Dimension	Vector pipeline	KG pipeline
HANA operation	Many <code>INSERT</code> statements	One <code>SPARQL_EXECUTE</code> insert
Failure mode	Can fail on any chunk	Atomic — succeeds or fails entirely

On a 30-page MSDS document the difference is measurable. Embedding 60 chunks against the Vertex AI API takes roughly 8–12 seconds. Calling Gemini 2.5 Flash for triple extraction from the same document takes 3–6 seconds. Sequential: 14–18 seconds. Parallel: 8–12 seconds (the slower pipeline dominates). The improvement compounds across every document uploaded across the platform.

`ThreadPoolExecutor(max_workers=2)` is the implementation mechanism. It creates exactly two OS threads within the FastAPI process — one for each pipeline. This is true parallelism: the embedding API calls in Thread 1 and the Gemini call in Thread 2 execute concurrently, sharing nothing except the temp file path.

5.1.2 The fire-and-forget pattern

The CAP Fiori UI calls `POST /process-upload` and needs a fast response. A user who clicks the upload button should not wait 10–15 seconds staring at a loading indicator — that is an unacceptable user experience for an enterprise application.

The solution is fire-and-forget. The upload endpoint returns `202 Accepted` with `{"status": "processing"}` immediately after spawning the background threads — before either pipeline has done any significant work. The HTTP connection closes. The threads run independently.

The CAP service then polls `GET /status/{material_number}` every 3 seconds. The Fiori UI updates two status indicators — one for vector ingestion, one for KG ingestion — based on what the status endpoint returns. When both reach a terminal state (`DONE` or `ERROR`), polling stops and the UI updates the document card.

This decoupling is critical for BTP Cloud Foundry deployments. HTTP request timeouts on CF are typically 60 seconds. A large document (100+ pages) can take 90 seconds to process. Without fire-and-forget, those documents would always fail at the HTTP layer, even when the actual processing succeeded.

Status is persisted in HANA — in the `MSDS_DOCUMENTS` table — not in memory. This means status survives a server restart. If the CF instance is recycled while processing is in flight, the status row already exists in HANA showing `PROCESSING`. A monitoring query can find and retry stalled documents without reading log files.

5.2 PDF text extraction with PyMuPDF

PyMuPDF (import name `fitz`) is a Python binding for the MuPDF rendering library. It is the fastest Python-accessible PDF parser available and handles the dirty reality of enterprise PDFs well: multi-column layouts, embedded tables, rotated pages, and the mixture of text layers and bitmap content that characterises scanned regulatory documents.

Add it to `agents/requirements.txt`:

```
PyMuPDF==1.24.3
```

The basic extraction pattern is three lines:

```
1 import fitz # PyMuPDF
2
3 def extract_text(pdf_path: str) -> str:
4     doc = fitz.open(pdf_path)
5     text = "".join(page.get_text() for page in doc)
6     doc.close()
7     return text
```

`page.get_text()` with no arguments returns the page's text content in reading order, with word-level coordinates used internally to reconstruct flow. For most MSDS documents this produces clean, paragraph-structured text. For scanned PDFs (no text layer) it returns an empty string — handled below.

5.2.1 Extracting with metadata

In `agents/srv/doc_srv.py` we use a slightly richer version that preserves page boundaries:

```
1 import fitz
2 from typing import Optional
```

```

3
4 def extract_pdf_text(path: str) -> tuple[str, int]:
5     """
6     Returns (full_text, page_count).
7     Raises ValueError if the PDF contains no extractable text layer.
8     """
9     doc = fitz.open(path)
10    pages = []
11    for page in doc:
12        page_text = page.get_text().strip()
13        if page_text:
14            pages.append(page_text)
15    doc.close()
16
17    if not pages:
18        raise ValueError(
19            f"PDF '{path}' contains no extractable text. "
20            "It may be a scanned image-only document."
21        )
22
23    return "\n\n".join(pages), len(pages)

```

The `ValueError` is important. If a scanned PDF arrives and we silently extract an empty string, the vector pipeline produces zero embeddings and the KG pipeline sends an empty prompt to Gemini. Both pipelines appear to succeed, but the document is never searchable. Raising early surfaces the problem immediately and sets both status columns to `ERROR` with a meaningful message — visible to operators without reading log files.

Warning: PyMuPDF’s `get_text()` preserves hyphenation artifacts from PDF layout. Words broken across lines (e.g., “flamma-” + newline + “ble”) will appear as a hyphenated pair in the extracted string. For MSDS documents this rarely causes problems because the affected words are typically in multi-column safety tables, but if your documents have heavy hyphenation, consider a post-processing step: `text = re.sub(r'\n', ' ', text)`.

5.2.2 Handling upload files in FastAPI

FastAPI receives uploaded files as `UploadFile` objects. We need to write the file to disk before passing the path to PyMuPDF, because `fitz.open()` accepts file paths or byte buffers — and the byte buffer approach for large PDFs can exhaust memory. The safest pattern uses Python's `tempfile`:

```
1 import tempfile
2 import shutil
3 from fastapi import UploadFile
4
5 async def save_upload(upload: UploadFile) -> str:
6     """Saves UploadFile to a temp file. Returns the path. Caller must
7     delete."""
8     suffix = ".pdf"
9     with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
10         shutil.copyfileobj(upload.file, tmp)
11         return tmp.name
```

Using `delete=False` gives us a path we can pass to background threads. We delete the file ourselves after both threads complete.

5.3 Chunking strategy for vector search

The vector pipeline does not process the full document text as a single unit. A 30-page MSDS document contains 8,000–12,000 words. Embedding that as one unit would produce a single 768-dimensional vector that averages the meaning of everything — hazard codes, first-aid steps, disposal procedures, regulatory text — into an undifferentiated blob. Cosine similarity against that vector would return the document for almost any question, because the document contains something relevant to almost any safety question. Precision would be zero.

The answer is chunking: splitting the document into smaller, semantically coherent pieces and embedding each independently. The cosine search then retrieves the specific chunks most relevant to a question, not the whole document.

5.3.1 Our chunking parameters

We use three parameters:

Parameter	Value	Rationale
Chunk size	500 tokens	Roughly 375 words. Large enough to preserve context around a fact, small enough for the vector to encode a focused meaning.
Overlap	50 tokens	Approximately one sentence. Prevents a fact from being split exactly at a chunk boundary and disappearing from both adjacent chunks.
Split boundary	Sentence end	Never cut in the middle of a sentence. A half-sentence changes the meaning of the embedding.

The `text-embedding-004` model from Vertex AI accepts up to 3,072 tokens per input. Our 500-token chunks are well within that limit, which gives us room to include a prefix that grounds the embedding with document metadata:

```
Material: Acetone (MAT-001)\n\n<chunk text>
```

Adding the material name to every chunk significantly improves retrieval accuracy for multi-document queries. Without it, a chunk about “flammable liquid” could match any of dozens of materials.

This prefix approach is also what makes the vector pipeline document-type agnostic. The same chunker, with the same parameters, handles an MSDS equally well as a purchase order or a quality inspection certificate — the material prefix ensures retrieved chunks are always scoped to the correct document.

5.3.2 The chunker function

```

1  import re
2  from typing import Generator
3
4  def chunk_text(
5      text: str,
6      material_name: str,
7      chunk_tokens: int = 500,
8      overlap_tokens: int = 50,
9  ) -> Generator[str, None, None]:
10     """
11     Yields overlapping chunks of approximately chunk_tokens each,
12     split on sentence boundaries. Prefixes each chunk with material_name.
13
14     Token estimation: 1 token ≈ 0.75 words (English prose).
15     """
16     chunk_words = int(chunk_tokens * 0.75)
17     overlap_words = int(overlap_tokens * 0.75)
18
19     # Split on sentence-ending punctuation followed by whitespace
20     sentences = re.split(r'(?<=[.!?])\s+', text.strip())
21
22     current_chunk: list[str] = []
23     current_count: int = 0
24     prefix = f"Material: {material_name}\n\n"
25
26     for sentence in sentences:
27         word_count = len(sentence.split())
28         if current_count + word_count > chunk_words and current_chunk:
29             yield prefix + " ".join(current_chunk)
30             # Keep the last overlap_words words as context for the next
31             # chunk
32             overlap_start = max(0, len(current_chunk) - overlap_words)
33             current_chunk = current_chunk[overlap_start:]
34             current_count = sum(len(s.split()) for s in current_chunk)
35             current_chunk.append(sentence)
36             current_count += word_count

```

```

37     # Emit the final partial chunk if it has content
38     if current_chunk:
39         yield prefix + " ".join(current_chunk)

```

Tip: This chunker uses a word-count approximation for token count (1 token $\approx$ 0.75 words). For production use, replace the approximation with the `tiktoken` library: `len(tiktoken.encoding_for_model("text-embedding-ada-002").encode(text))`. The approximation is accurate enough for MSDS documents, where prose density is fairly uniform.

5.3.3 Why the KG pipeline does not chunk

The knowledge graph pipeline receives the **full document text**, not chunks. This is a deliberate asymmetry.

Gemini's triple extraction works best when it can see the entire document context. A hazard code (H225) might appear in Section 2 of an MSDS. The material name appears in Section 1. The supplier appears in Section 13. If we feed Gemini a chunk that contains only Section 2, it cannot associate H225 with the material and supplier, because those context clues are in different sections.

For the KG pipeline, the goal is to extract a small number of high-precision facts (typically 5–15 triples per document). Gemini 2.5 Flash can easily handle 10,000 words in a single prompt, so there is no technical reason to chunk. The full-document approach produces far better triple quality.

The KG extraction prompt is the single configuration point that varies per document type. An MSDS prompt instructs Gemini to extract hazard codes, exposure limits, and precaution statements. An invoice prompt instructs it to extract vendor, line items, and payment terms. An inspection report prompt extracts test results, pass/fail verdicts, and inspector identity. The chunker code and the SPARQL insert code remain identical across all document types.

5.4 Thread-local HANA connections

This section addresses the most important implementation detail in this chapter. Get it wrong and you will see intermittent `hdbcli` errors that are extremely difficult to debug.

Understand it and the threading model becomes straightforward.

5.4.1 Why connections cannot be shared across threads

The `hdbcli` driver maintains a stateful protocol session on each connection object. Internally, a connection tracks the current transaction state, pending fetch buffers for open cursors, and the sequence of SQL commands in the current batch. When two threads share a single connection and execute concurrently, they can each call `cursor.execute()` at the same time. The driver serialises the calls — but by the time the second call returns, the first thread may have already called `cursor.fetchall()` on a different cursor that was internally sharing the same fetch buffer. The result is that one thread reads the other thread's data, or gets an exception, or silently gets an empty result.

None of these failures are deterministic. They depend on exact scheduling and timing. This is the canonical definition of a race condition.

Warning: Never share an `hdbcli` connection across threads. The `hdbcli` documentation explicitly states that connection objects are not thread-safe. This applies to most database drivers, not just HANA. When in doubt, assume a connection is single-threaded.

5.4.2 The `threading.local()` solution

Python's `threading.local()` provides a simple mechanism: an object where each attribute access returns a different value depending on which thread is reading it. When Thread 1 sets `_local.conn = hana_db.connect()`, Thread 2 sees `_local.conn` as unset. Each thread must initialise its own value.

Here is the pattern we use in `agents/srv/doc_srv.py`:

```
1 import threading
2 from hdbcli import dbapi
3
4 _thread_local = threading.local()
5
6 def get_hana_connection() -> dbapi.Connection:
7     """
8     Returns a HANA connection for the current thread.
```

```

9      Creates one on first access; reuses it on subsequent calls within the
      same thread.
10      """
11      if not hasattr(_thread_local, "conn") or _thread_local.conn is None:
12          _thread_local.conn = dbapi.connect(
13              address=os.environ["HANA_HOST"],
14              port=int(os.environ["HANA_PORT"]),
15              user=os.environ["HANA_USER"],
16              password=os.environ["HANA_PASSWORD"],
17          )
18      return _thread_local.conn
19
20 def close_thread_connection() -> None:
21     """Closes and clears the connection for the current thread."""
22     if hasattr(_thread_local, "conn") and _thread_local.conn is not None:
23         try:
24             _thread_local.conn.close()
25         except Exception:
26             pass
27     _thread_local.conn = None

```

The `get_hana_connection()` function is called at the start of each thread function. If the thread is new, it creates a fresh connection. If the same thread somehow reuses its worker slot (which can happen with thread pools), it reuses the existing connection — which is safe because it is still the same thread.

Note: The thread pool created by `ThreadPoolExecutor` may reuse worker threads across multiple task submissions if the executor stays alive. In our implementation, we create a fresh `ThreadPoolExecutor` for each upload, so each worker thread is new and its `threading.local()` storage is empty. If you ever switch to a long-lived executor, add a health check before reusing a connection: `_thread_local.conn.isconnected()`.

5.5 The ingestion flow in detail

Now we have all the pieces. Let us trace the full execution path for a single document upload from the CAP Fiori UI through to both HANA stores.

5.5.1 The upload endpoint

```

1  from fastapi import FastAPI, File, Form, UploadFile
2  from fastapi.responses import JSONResponse
3  import asyncio, os, re, tempfile, shutil
4  from concurrent.futures import ThreadPoolExecutor, as_completed
5
6  app = FastAPI()
7
8  @app.post("/process-upload")
9  async def process_upload(
10     file: UploadFile = File(...),
11     materialNumber: str = Form(...),
12     materialName: str = Form(...),
13 ):
14     if not re.match(r'^[A-Za-z0-9_-]+$ ', materialNumber):
15         return JSONResponse(
16             status_code=400,
17             content={"error": "Invalid materialNumber. Use only letters,
18 digits, hyphens, underscores."}
19         )
20
21     # Write upload to a temp file so background threads can access it by
22     # path
23     loop = asyncio.get_event_loop()
24     tmp_path = await loop.run_in_executor(None, _save_upload_sync, file)
25
26     # Mark both pipelines as PROCESSING in HANA before returning
27     _mark_processing(materialNumber)
28
29     # Hand off to background threads – fire and forget
30     loop.run_in_executor(None, _run_dual_pipeline, tmp_path,
31 materialNumber, materialName)
32
33     return {"status": "processing", "materialNumber": materialNumber}

```

The material number validation — `^[A-Za-z0-9_-]+$` — is enforced at every entry point in the application. This is SPARQL injection prevention: a material number like `MAT-001> } INSERT { <evil> }` would pass a naive check but break out of a SPARQL query. The

regex ensures the material number can only contain characters that are safe to embed in an IRI.

Three things happen before the response is sent: the uploaded file is written to a temp path, `_mark_processing()` sets both status fields to 'PROCESSING' in HANA, and `_run_dual_pipeline()` is submitted to an executor — it runs *after* the response returns.

5.5.2 The dual-pipeline orchestrator

```

1 def _run_dual_pipeline(
2     tmp_path: str,
3     material_number: str,
4     material_name: str,
5 ) -> None:
6     """
7     Runs vector and KG pipelines in parallel.
8     Called in a background thread – must not raise unhandled exceptions.
9     """
10    try:
11        text, page_count = extract_pdf_text(tmp_path)
12    except ValueError as exc:
13        # PDF has no text layer – fail both pipelines immediately
14        _mark_vector_status(material_number, "ERROR", str(exc))
15        _mark_kg_status(material_number, "ERROR", str(exc))
16        os.unlink(tmp_path)
17        return
18
19    vector_error = None
20    kg_error = None
21
22    with ThreadPoolExecutor(max_workers=2) as executor:
23        future_vector = executor.submit(_vector_pipeline, text,
24                                       material_number, material_name)
25        future_kg = executor.submit(_kg_pipeline, text,
26                                   material_number)
27
28        for future in as_completed([future_vector, future_kg]):
29            try:
30                future.result()

```



```

29         except Exception as exc:
30             if future is future_vector:
31                 vector_error = str(exc)
32             else:
33                 kg_error = str(exc)
34
35         # Update statuses independently – one failure does not mask the other
36         _mark_vector_status(material_number, "ERROR" if vector_error else
37                             "DONE", vector_error)
38         _mark_kg_status(material_number, "ERROR" if kg_error else
39                         "DONE", kg_error)
40
41     try:
42         os.unlink(tmp_path)
43     except OSError:
44         pass

```

The `with ThreadPoolExecutor(max_workers=2)` block creates exactly two worker threads. `as_completed()` yields futures as they finish; we capture any exception from each without letting it propagate, so a failure in one pipeline does not cancel the other. Both pipelines are given the opportunity to complete, succeed, or fail independently.

5.5.3 Thread 1 — the vector pipeline

```

1  from agents.srv.vector_srv import embed_text, upsert_vectors
2
3  def _vector_pipeline(
4      text: str,
5      material_number: str,
6      material_name: str,
7  ) -> None:
8      conn = get_hana_connection()
9      chunks = list(chunk_text(text, material_name))
10
11      for i, chunk in enumerate(chunks):
12          embedding = embed_text(chunk)          # Vertex AI
13          text-embedding-004
14          upsert_vectors(
15              conn=conn,

```

```

15         material_number=material_number,
16         chunk_index=i,
17         chunk_text=chunk,
18         embedding=embedding,          # list[float], length 768
19     )
20
21     close_thread_connection()

```

`embed_text()` calls `text-embedding-004` through the Vertex AI Python SDK. For a 30-page document with 60 chunks, this function makes 60 sequential API calls, each taking roughly 150–200 ms — approximately 10–12 seconds total.

`upsert_vectors()` executes a SQL UPSERT on the `MSDS_VECTORS` table, using `(material_number, chunk_index)` as the key. Re-uploading a document replaces its existing vectors cleanly. The `MSDS_VECTORS` table serves all document types on the platform — an invoice chunk and an MSDS chunk differ only in their content and the material number they are associated with.

Tip: For large document batches, consider batching the embedding calls. The `text-embedding-004` endpoint accepts up to 250 texts in a single request. Instead of 60 individual calls for 60 chunks, one batched call returns all 60 embeddings at once. The latency drops from ~10 seconds to ~1.5 seconds for the embed step. Batching is shown in Appendix C.

5.5.4 Thread 2 — the knowledge graph pipeline

```

1  from agents.srv.kg_srv import extract_triples_from_text, insert_triples
2
3  def _kg_pipeline(text: str, material_number: str) -> None:
4      conn = get_hana_connection()
5      triples = extract_triples_from_text(text, material_number)
6
7      if not triples:
8          raise ValueError(
9              f"Zero triples extracted for '{material_number}'. "
10             "Check document quality and ontology alignment."
11         )
12

```

```

13     insert_triples(conn=conn, material_number=material_number,
14                   triples=triples)
15     close_thread_connection()

```

`extract_triples_from_text()` is the Gemini 2.5 Flash extraction function built in Chapter 4. It sends the full text with a structured prompt that includes the ontology predicates and asks for JSON output. `insert_triples()` builds a SPARQL `INSERT DATA` query and calls `SPARQL_EXECUTE` on the HANA connection. The resulting triples are stored in the named graph `MSDS_Graph/MAT-XXX` under the `msds.kg` namespace.

The “zero triples” check is deliberate. A successful Gemini call that returns no triples is worse than a clean failure — the document would appear as `kgStatus = 'DONE'` to the user but produce no graph query results. Raising `ValueError` here sets `kgStatus = 'ERROR'`, which makes the problem visible in the CAP UI and queryable in the `MSDS_DOCUMENTS` table.

5.6 Status tracking

The CAP Fiori UI needs to know when ingestion is complete and what the result counts are. The UI polls `GET /status/{materialNumber}` every 3 seconds until both pipelines report a terminal state. The status data it receives comes entirely from the `MSDS_DOCUMENTS` table in HANA.

5.6.1 The `MSDS_DOCUMENTS` table and its role

`MSDS_DOCUMENTS` is the control plane of the ingestion pipeline. Every upload creates or updates a row here. The CAP OData service reads from this table to populate the document list in the Fiori UI — the `triples` count (how many KG facts were extracted), the `vectors` count (how many chunks were embedded), and the status indicators for each pipeline all come from this table.

Column	Tracks	Values
<code>STATUS</code>	KG pipeline	PROCESSING, DONE, ERROR
<code>VECTOR_STATUS</code>	Vector pipeline	PROCESSING, DONE, ERROR
<code>KG_ERROR</code>	KG failure reason	NULL or error message
<code>VECTOR_ERROR</code>	Vector failure reason	NULL or error message

Using two separate status columns rather than one combined status allows the UI to show partial progress — “Vector: DONE, KG: PROCESSING...” — and makes it possible to retry only the failed pipeline without re-running the successful one.

5.6.2 Status helper functions

```

1  def _mark_processing(material_number: str) -> None:
2      conn = get_hana_connection()
3      cursor = conn.cursor()
4      cursor.execute("""
5          UPSERT MSDS_DOCUMENTS (MATERIAL_NUMBER, STATUS, VECTOR_STATUS,
6          CREATED_AT)
7          VALUES (?, 'PROCESSING', 'PROCESSING', NOW())
8          WITH PRIMARY KEY
9          """, (material_number,))
10     conn.commit()
11     cursor.close()
12     close_thread_connection()
13
14 def _mark_vector_status(material_number: str, status: str,
15     error_message=None) -> None:
16     conn = get_hana_connection()
17     cursor = conn.cursor()
18     cursor.execute("""
19         UPDATE MSDS_DOCUMENTS
20         SET VECTOR_STATUS = ?, VECTOR_ERROR = ?, UPDATED_AT = NOW()
21         WHERE MATERIAL_NUMBER = ?
22         """, (status, error_message, material_number))
23     conn.commit()
24     cursor.close()
25     close_thread_connection()
26
27 def _mark_kg_status(material_number: str, status: str, error_message=None)
28     -> None:
29     conn = get_hana_connection()
30     cursor = conn.cursor()
31     cursor.execute("""
32         UPDATE MSDS_DOCUMENTS
33         SET STATUS = ?, KG_ERROR = ?, UPDATED_AT = NOW()

```

```

31         WHERE MATERIAL_NUMBER = ?
32     """, (status, error_message, material_number))
33     conn.commit()
34     cursor.close()
35     close_thread_connection()

```

5.6.3 The status endpoint

```

1  @app.get("/status/{material_number}")
2  def get_status(material_number: str):
3      if not re.match(r'^[A-Za-z0-9_-]+$', material_number):
4          return JSONResponse(status_code=400, content={"error": "Invalid
5              materialNumber"})
6
7      conn = get_hana_connection()
8      cursor = conn.cursor()
9      cursor.execute("""
10         SELECT STATUS, VECTOR_STATUS, KG_ERROR, VECTOR_ERROR, UPDATED_AT
11         FROM   MSDS_DOCUMENTS
12         WHERE  MATERIAL_NUMBER = ?
13     """, (material_number,))
14     row = cursor.fetchone()
15     cursor.close()
16     close_thread_connection()
17
18     if not row:
19         return JSONResponse(status_code=404, content={"error": "Material
20             not found"})
21
22     kg_status, vector_status, kg_error, vector_error, updated_at = row
23     return {
24         "materialNumber": material_number,
25         "kgStatus":      kg_status,
26         "vectorStatus":  vector_status,
27         "kgError":       kg_error,
28         "vectorError":   vector_error,
29         "updatedAt":     str(updated_at),
30         "complete":      kg_status in ("DONE", "ERROR") and
31             vector_status in ("DONE", "ERROR"),

```

30

}

The `complete` boolean is the sentinel the CAP service uses to stop polling. If `complete` is `true` but either status is `ERROR`, the Fiori UI shows an error banner with the error message but does not block navigation — the user can still query against whichever pipeline succeeded.

5.7 Error handling — what happens when one pipeline fails

The most important design property of this ingestion service is **error isolation**: a failure in one pipeline does not affect the other.

Failure	What happens	Status in HANA
PDF has no text layer	Both pipelines fail before threads start	Both <code>ERROR</code> , same error message
Vertex AI embedding API down	<code>_vector_pipeline</code> raises; <code>_kg_pipeline</code> continues	<code>vectorStatus = ERROR</code> , <code>kgStatus = DONE</code>
Gemini triple extraction returns empty	<code>_kg_pipeline</code> raises; <code>_vector_pipeline</code> has already completed	<code>vectorStatus = DONE</code> , <code>kgStatus = ERROR</code>
HANA connection fails in one thread	Affected thread raises; other thread uses its own connection	Independent <code>ERROR / DONE</code>

Scenario 2 deserves attention because it reflects a real operational condition. When a Vertex AI service outage occurs, every embedding call will fail. If error propagation were allowed, the `ThreadPoolExecutor` would cancel the KG future when the vector future raises — and the document would have no graph knowledge despite the Gemini extraction succeeding. The `try/except` wrapper in `as_completed()` prevents that cancellation.

Persisting error messages in the `KG_ERROR` and `VECTOR_ERROR` columns means operators can run `SELECT MATERIAL_NUMBER, KG_ERROR FROM MSDS_DOCUMENTS WHERE STATUS =`

'ERROR' to get a triage list without reading log files. This is the operational advantage of storing status in HANA rather than in application logs.

5.8 Testing — upload five MSDS documents

5.8.1 Start the service

```
1 cd agents
2 source .venv/bin/activate
3 uvicorn main:app --reload --port 8000
```

5.8.2 Upload the first document

```
1 curl -s -X POST http://localhost:8000/process-upload \
2     -F "file=@/path/to/acetone-sds.pdf" \
3     -F "materialNumber=MAT-001" \
4     -F "materialName=Acetone"
```

Expected response (within 200 ms):

```
1 {"status": "processing", "materialNumber": "MAT-001"}
```

5.8.3 Poll for completion

```
1 watch -n3 'curl -s http://localhost:8000/status/MAT-001 | python3 -m
  json.tool'
```

After 8–15 seconds:

```
1 {
2     "materialNumber": "MAT-001",
3     "kgStatus": "DONE",
4     "vectorStatus": "DONE",
5     "kgError": null,
6     "vectorError": null,
7     "updatedAt": "2026-05-27 09:31:17.447",
```

```

8     "complete": true
9 }

```

5.8.4 Verify the vector store

```

1 SELECT
2     CHUNK_INDEX,
3     LEFT(CHUNK_TEXT, 80) AS CHUNK_PREVIEW,
4     CARDINALITY(EMBEDDING) AS VECTOR_DIM
5 FROM MSDS_VECTORS
6 WHERE MATERIAL_NUMBER = 'MAT-001'
7 ORDER BY CHUNK_INDEX;

```

Expected: every row has VECTOR_DIM = 768.

5.8.5 Verify the knowledge graph

```

1 from hdbcli import dbapi
2 import os
3
4 conn = dbapi.connect(
5     address=os.environ["HANA_HOST"],
6     port=int(os.environ["HANA_PORT"]),
7     user=os.environ["HANA_USER"],
8     password=os.environ["HANA_PASSWORD"],
9 )
10 cursor = conn.cursor()
11
12 sparql = """
13 PREFIX msds: <http://msds.knowledge-graph.org/ontology/>
14 SELECT ?predicate ?object
15 WHERE {
16     GRAPH <http://msds.knowledge-graph.org/MSDS_Graph/MAT-001> {
17         <http://msds.knowledge-graph.org/material/MAT-001> ?predicate ?object
18     }
19 }
20 """
21

```



```

22 cursor.callproc("SPARQL_EXECUTE", [sparql, None, 1000, None, None])
23 rows = cursor.fetchall()
24 for row in rows:
25     print(row)

```

Expected output:

```

('http://msds.knowledge-graph.org/ontology#hasHazardCode', 'H225')
('http://msds.knowledge-graph.org/ontology#hasHazardCode', 'H319')
('http://msds.knowledge-graph.org/ontology#hasHazardCode', 'H336')
('http://msds.knowledge-graph.org/ontology#hasExposureLimit', '500 ppm TWA')
('http://msds.knowledge-graph.org/ontology#requiresPrecaution', 'Keep away from open flame')
('http://msds.knowledge-graph.org/ontology#hasSupplier', 'Sigma-Aldrich')

```

Warning: If `rows` is empty, first check the `GRAPH` clause. HANA SPARQL returns no results without the `GRAPH` wrapper, even if the triples exist. The second thing to check is the IRI prefix — the subject must match exactly what `insert_triples()` used when storing the graph.

Upload the remaining four PDFs (`MAT-002` through `MAT-005`) with the same pattern. After all five complete, you will have 250–350 rows in `MSDS_VECTORS` and 5 named graphs in the HANA RDF store.

5.9 Summary

This chapter built the pipeline that bridges PDF uploads and the dual-store knowledge layer. The architecture applies to any document type uploaded against an SAP Material Number. The key design decisions:

- **Parallel threads** — `ThreadPoolExecutor(max_workers=2)` runs the KG extraction (Gemini 2.5 Flash) and the vector embedding (`text-embedding-004`) simultaneously. These operations are independent: neither pipeline waits for the other. Wall-clock ingestion time equals the slower pipeline, not the sum.
- **Fire-and-forget with status polling** — the upload endpoint returns 202 in milliseconds. The CAP Fiori UI polls every 3 seconds. Decouples ingestion time from HTTP timeouts — essential for large documents on BTP CF.

- **Thread-local HANA connections** — `threading.local()` gives each worker thread its own connection object, eliminating race conditions without connection pooling infrastructure.
 - **Different text strategies per pipeline** — the vector pipeline chunks to 500 tokens for embedding precision; the KG pipeline sends the full document for triple extraction quality.
 - **Document-type agnostic vector pipeline** — the same chunker and embed logic handles any PDF. Only the KG extraction prompt changes per document type.
 - **Error isolation** — exceptions in one pipeline are caught and persisted as `ERROR` status without interrupting the other pipeline.
 - **MSDS_DOCUMENTS as control plane** — status, error messages, and counts are persisted in HANA. The CAP service reads this table directly. Operators can triage failures with SQL, not log files.
-

5.10 Checkpoint

Before moving to Chapter 6, confirm each of the following:

- ☐ `agents/srv/doc_srv.py` exists and `uvicorn main:app --port 8000` starts without import errors.
- ☐ `POST /process-upload` with a real PDF returns `{"status": "processing"}` within 500 ms.
- ☐ `GET /status/{materialNumber}` returns `"complete": true` within 30 seconds for a 20–40 page document.
- ☐ Five documents uploaded; all show `kgStatus = "DONE"` and `vectorStatus = "DONE"`.
- ☐ `SELECT COUNT(*) FROM MSDS_VECTORS WHERE MATERIAL_NUMBER = 'MAT-001'` returns a positive number.
- ☐ `SELECT CARDINALITY(EMBEDDING) FROM MSDS_VECTORS FETCH FIRST 1 ROWS ONLY` returns 768.
- ☐ SPARQL query against `MSDS_Graph/MAT-001` returns at least one triple with predicate `hasHazardCode`.
- ☐ Re-uploading the same PDF overwrites existing rows rather than creating duplicates.

- Uploading a scan-only PDF results in both statuses showing `ERROR`, not `PROCESSING`.

With the ingestion pipeline verified and five documents loaded into both stores, you are ready for the agentic layer. Chapter 6 introduces LangGraph — the framework that will orchestrate queries against these two stores in parallel and synthesise a single coherent answer.

End of Chapter 5

Chapter 6: LangGraph Fundamentals

In Chapter 5 we built a document ingestion pipeline that processes PDFs into both a vector store and a knowledge graph simultaneously. We now have data — rich, structured, searchable data — in SAP HANA Cloud. The next question is: how do we build an agent that reasons over it in a way that is auditable, inspectable, and safe to deploy on BTP?

A plain LangChain chain would work for a single retrieval followed by a single LLM call. But the system we need is more complex than that. It needs to run two retrieval strategies in parallel, decide how to merge their results, handle retries when SPARQL returns empty results, maintain conversation history across turns, and expose all of this over a single HTTP endpoint. A linear chain cannot express those decisions. We need something that can branch, loop, and conditionally route based on what it finds.

That something is LangGraph.

This chapter introduces LangGraph from first principles. We build a simple question-answering agent step by step — one node at a time — so that when Chapter 7 extends it into the full parallel hybrid RAG system, every piece is familiar. By the end of this chapter you will have a running LangGraph agent connected to Vertex AI, and you will understand exactly how state flows through it.

6.1 Why LangGraph is SAP's recommended orchestration approach for BTP agents

Enterprise AI governance requires more than a correct answer. It requires an auditable trail: which system provided which data, which decision was made at which step, and why. When an auditor asks “how did the system determine that the exposure limit was 500 ppm?”, you need to be able to show the SPARQL query, the HANA result, and the LLM call that synthesised it into natural language. A black-box chain gives you none of that. A LangGraph StateGraph gives you all of it.

LangGraph models your agent as a directed graph rather than a list. Every node in the graph is a named Python function. Every state transition is an explicit edge. Every

intermediate result is captured in the graph state. LangGraph traces can be exported to LangSmith, where each run shows a complete tree: which nodes executed, in what order, what the state contained at each step, and the exact prompts sent to Gemini. This execution trace is the enterprise AI governance artefact.

The conceptual mapping to SAP BTP workflow constructs is direct:

LangGraph concept	BTP workflow equivalent
<code>AgentState (TypedDict)</code>	BTP Workflow context object
<code>Node (Python function)</code>	Workflow service task step
<code>Edge</code>	Sequence flow between steps
<code>Conditional edge</code>	Exclusive gateway (XOR decision)
<code>StateGraph.compile()</code>	Deployed workflow definition
<code>app.invoke(state)</code>	Workflow instance execution

SAP's own internal AI agent frameworks use LangGraph for exactly this reason: the execution model is transparent, the state is inspectable at any point, and the graph definition is a first-class artefact that can be reviewed, versioned, and audited like any other software component.

6.2 Why LangChain chains are not enough

LangChain introduced the concept of a chain: a sequence of components where the output of one feeds the input of the next. For many tasks — retrieve a passage, summarise it, return the result — a chain is exactly right. It is simple, readable, and easy to test.

The problem arises when your logic is not a sequence. Consider what our hybrid RAG system needs to do:

Requirement	Can a chain do it?
Run two retrieval strategies in parallel	No — chains are sequential
Retry SPARQL generation if results are empty	No — chains have no loops

Requirement	Can a chain do it?
Route to different merge strategies based on what was found	No — chains have no conditional branches
Pass conversation history into every step	Awkward — requires passing state manually
Produce an auditable execution trace	No — chain execution is opaque

LangGraph solves every one of these by modelling your agent as a directed graph rather than a list. Nodes are Python functions. Edges are connections between them. Conditional edges let the graph branch based on the current state. The graph can loop. It can run nodes in parallel. It can capture and stream intermediate results.

Note: LangGraph is built on top of LangChain. You still use LangChain components — `ChatVertexAI`, `tool` decorators, message types — but LangGraph provides the execution engine that orchestrates them.

6.3 Core concepts

Before writing any code, here are the four concepts you need to hold in your head.

6.3.1 State

State is the memory of your agent for a single run. It is a Python `TypedDict` — a dictionary where every key has a declared type. Every node in the graph reads from state and writes back to state. State is the only way nodes communicate with each other.

In BTP workflow terms, state is the context object that a workflow instance carries from step to step. When you inspect a running workflow in BTP Process Automation, you see this context. In LangGraph, when you inspect a run in LangSmith, you see the state at each node boundary. The concept is identical.

```

1 from typing import TypedDict, List
2
3 class AgentState(TypedDict):
4     question: str

```

```
5     answer: str
6     messages: List[dict]
```

When you invoke the graph, you pass an initial state. When the graph finishes, you receive the final state. Everything that happened in between is recorded there.

6.3.2 Nodes

A node is a Python function that receives the current state and returns a partial update to it. LangGraph merges the update into the state before passing it to the next node. In BTP workflow terms, a node is a service task — it does one focused piece of work and writes its result to the workflow context.

```
1 def my_node(state: AgentState) -> dict:
2     question = state["question"]
3     answer = call_llm(question)
4     # return only the keys you want to update
5     return {"answer": answer}
```

The function does not need to return the entire state — only the keys it wants to change. LangGraph merges the returned dict into the existing state automatically.

6.3.3 Edges

An edge connects two nodes. When node A finishes, LangGraph follows the edge to node B and runs it next. This is a sequence flow in BTP workflow terms — deterministic, always-true routing.

```
1 graph.add_edge("node_a", "node_b")
```

6.3.4 Conditional edges

A conditional edge calls a routing function after a node completes. The routing function inspects the state and returns a string that names the next node to run. In BTP workflow terms, this is an exclusive gateway: examine the context, choose one outgoing sequence flow.

```
1 def route(state: AgentState) -> str:
2     if state["answer"] == "":
```

```

3         return "retry"
4     return "done"
5
6 graph.add_conditional_edges("node_a", route, {
7     "retry": "retry_node",
8     "done": END
9 })

```

This is how loops and branches are expressed in LangGraph.

6.4 Installation and project layout

Install LangGraph and the Google GenAI integration. Note the correct import: `langchain_google_genai`, **not** `langchain_google_vertexai` — the VertexAI variant is deprecated for the Gemini model family.

```

1 pip install langgraph langchain-google-genai langchain-core

```

Add these to `agents/requirements.txt`:

```

langgraph>=0.2.0
langchain-google-genai>=2.0.0
langchain-core>=0.3.0

```

We will build the simple agent in a new file:

```

agents/
  agents/
    simple_qa_agent.py    ← this chapter
    orchestrator.py      ← Chapter 7
    kg_chain.py           ← Chapter 7
    vector_chain.py       ← Chapter 7

```


6.5 Building the state and nodes

Open `agents/agents/simple_qa_agent.py`. We start with the state definition and two nodes.

```

1  from typing import TypedDict, List
2  from langgraph.graph import StateGraph, END
3  from langchain_google_genai import ChatGoogleGenerativeAI
4  from langchain_core.messages import HumanMessage, AIMessage
5  # — State
6
7  class AgentState(TypedDict):
8      question: str
9      context: str
10     answer: str
11     messages: List[dict]
12     retry_count: int
13
14 # — LLM
15
16 def _get_llm():
17     return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
18                                     temperature=0.1, max_tokens=1024)
19
20 # — Nodes
21
22 def retrieve_node(state: AgentState) -> dict:
23     """Stub retriever – replaced by vector + KG chains in Chapter 7."""
24     question = state["question"]
25     context = f"[Retrieved context for: {question}]"
26     return {"context": context}
27
28 def answer_node(state: AgentState) -> dict:
29     """Generate an answer using the retrieved context."""
30     question = state["question"]
31     context = state["context"]
32     history = state.get("messages", [])
33
34     messages = []
35     for msg in history:
36         if msg["role"] == "user":

```

```

31         messages.append(HumanMessage(content=msg["content"]))
32     else:
33         messages.append(AIMessage(content=msg["content"]))
34
35     prompt = f"""You are an expert assistant for SAP material documents –
you help users find information across PDF documents linked to SAP
Material Numbers.
36
37 Context from the knowledge base:
38 {context}
39
40 Answer the following question based on the context above.
41 If the context does not contain enough information, say so clearly.
42
43 Question: {question}"""
44
45     messages.append(HumanMessage(content=prompt))
46     response = _get_llm().invoke(messages)
47     return {"answer": response.content}

```

Two things to notice. First, `retrieve_node` is a deliberate stub — it returns a placeholder string. We build the real retrieval in Chapter 7; this stub lets us test the graph structure now without needing a live HANA connection. Second, `answer_node` builds the conversation history from the `messages` list in state. This is how multi-turn conversations work: the caller passes previous turns in the initial state, and the node includes them in the prompt.

LLM instantiation uses a lazy getter `_get_llm()` rather than a module-level variable. This prevents the LLM client from being initialised at import time, which would fail in environments where credentials are not yet available — a common issue in BTP CF where environment variables are injected after application startup.

6.6 Wiring the graph

Add the graph construction below the node functions:

```

1  def build_graph():
2      graph = StateGraph(AgentState)
3
4      graph.add_node("retrieve", retrieve_node)
5      graph.add_node("answer", answer_node)
6
7      graph.set_entry_point("retrieve")
8      graph.add_edge("retrieve", "answer")
9      graph.add_edge("answer", END)
10
11     return graph.compile()
12
13 app = build_graph()

```

The graph is linear: retrieve → answer → END. To invoke it:

```

1  if __name__ == "__main__":
2      result = app.invoke({
3          "question": "What are the hazard codes for acetone?",
4          "context": "",
5          "answer": "",
6          "messages": [],
7          "retry_count": 0,
8      })
9      print(result["answer"])

```

Run it:

```

1  cd agents
2  python -m agents.simple_qa_agent

```

Expected output (with stub retriever):

Based on the provided context, I don't have specific hazard codes for acetone. The context contains a placeholder rather than actual document data. Once the knowledge base is populated with real documents linked to the material number, I will be able to retrieve and report the specific information.

The agent runs, reaches Gemini 2.5 Flash, and gives an honest answer about the stub context. That is the graph working correctly.

6.7 Adding a conditional retry edge

Real retrieval can fail — SPARQL returns empty results, an embedding call times out. We need to be able to retry. Add a check node and a routing function:

```

1 def check_answer_node(state: AgentState) -> dict:
2     """Check if the answer is acceptable. Increment retry counter if
3     not."""
4     answer = state.get("answer", "")
5     retry_count = state.get("retry_count", 0)
6
7     low_quality_phrases = [
8         "don't have", "no information", "cannot find",
9         "not available", "placeholder"
10    ]
11    needs_retry = any(p in answer.lower() for p in low_quality_phrases)
12    return {"retry_count": retry_count + (1 if needs_retry else 0)}
13
14 def route_after_check(state: AgentState) -> str:
15     """Decide whether to retry or finish."""
16     answer = state.get("answer", "")
17     retry_count = state.get("retry_count", 0)
18
19     low_quality_phrases = [
20         "don't have", "no information", "cannot find",
21         "not available", "placeholder"
22    ]
23    needs_retry = any(p in answer.lower() for p in low_quality_phrases)
24
25    if needs_retry and retry_count < 2:
26        return "retry"
27    return "done"

```

This is a conditional edge acting as a decision gateway. The routing function reads two state fields — `answer` and `retry_count` — and returns one of two named outcomes. LangGraph maps those outcomes to the next node using the dictionary passed to `add_conditional_edges`. This is identical to how an exclusive gateway in BTP Process

Automation evaluates a context condition and routes to one of several outgoing flows.

Update `build_graph` to include the check node and conditional edge:

```

1  def build_graph():
2      graph = StateGraph(AgentState)
3
4      graph.add_node("retrieve", retrieve_node)
5      graph.add_node("answer", answer_node)
6      graph.add_node("check", check_answer_node)
7
8      graph.set_entry_point("retrieve")
9      graph.add_edge("retrieve", "answer")
10     graph.add_edge("answer", "check")
11
12     graph.add_conditional_edges(
13         "check",
14         route_after_check,
15         {
16             "retry": "retrieve",  # loop back
17             "done": END
18         }
19     )
20
21     return graph.compile()

```

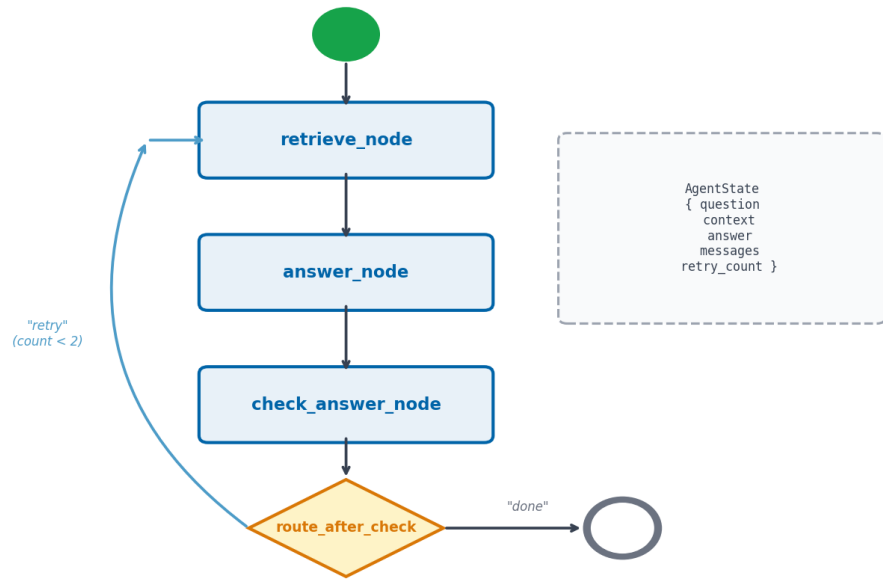
The graph now has a loop: `retrieve` → `answer` → `check` → (retry: back to `retrieve`) or (done: `END`). The `retry_count` guard prevents infinite loops — after two retries we accept whatever answer we have.

Note: In Chapter 7 the retry logic is more specific: we only retry when SPARQL returns zero results, and the retry uses a simpler SPARQL query, not a full re-run of both chains. The pattern here is the same; the condition is different.

6.8 The StateGraph visualised

The graph we built in this chapter has three nodes and one conditional loop:

LangGraph StateGraph — Chapter 7 Simple Q&A Agent



State flows through every node. Nodes return only the keys they update.

Figure 6.1: The simple Q&A agent’s StateGraph — retrieve feeds answer, which feeds the check node. The check node either loops back to retrieve (on low-quality answers, up to 2 retries) or terminates at END. The dashed box on the right shows the AgentState structure that flows through every node.

6.9 The complete simple_qa_agent.py

```

1 from typing import TypedDict, List
2 from langgraph.graph import StateGraph, END
3 from langchain_google_genai import ChatGoogleGenerativeAI
4 from langchain_core.messages import HumanMessage, AIMessage
5
6 class AgentState(TypedDict):
7     question: str
8     context: str
9     answer: str
  
```

```

10     messages: List[dict]
11     retry_count: int
12
13 def _get_llm():
14     return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
15                                     temperature=0.1, max_tokens=1024)
16
17 def retrieve_node(state: AgentState) -> dict:
18     question = state["question"]
19     return {"context": f"[Retrieved context for: {question}]"}
20
21 def answer_node(state: AgentState) -> dict:
22     messages = []
23     for msg in state.get("messages", []):
24         cls = HumanMessage if msg["role"] == "user" else AIMessage
25         messages.append(cls(content=msg["content"]))
26
27     prompt = f"""You are an expert assistant for SAP material documents –
28     you help users find information across PDF documents linked to SAP
29     Material Numbers.
30
31     Context:
32     {state['context']}
33
34     Question: {state['question']}
35     Answer: ""
36
37     messages.append(HumanMessage(content=prompt))
38     return {"answer": _get_llm().invoke(messages).content}
39
40 def check_answer_node(state: AgentState) -> dict:
41     low = ["don't have", "no information", "cannot find", "not available",
42           "placeholder"]
43     needs_retry = any(p in state.get("answer", "").lower() for p in low)
44     return {"retry_count": state.get("retry_count", 0) + (1 if needs_retry
45                                                         else 0)}
46
47 def route_after_check(state: AgentState) -> str:
48     low = ["don't have", "no information", "cannot find", "not available",
49           "placeholder"]

```

```

43     needs_retry = any(p in state.get("answer","").lower() for p in low)
44     return "retry" if (needs_retry and state.get("retry_count",0) < 2)
45     else "done"
46
47 def build_graph():
48     graph = StateGraph(AgentState)
49     graph.add_node("retrieve", retrieve_node)
50     graph.add_node("answer", answer_node)
51     graph.add_node("check", check_answer_node)
52     graph.set_entry_point("retrieve")
53     graph.add_edge("retrieve", "answer")
54     graph.add_edge("answer", "check")
55     graph.add_conditional_edges("check", route_after_check,
56                               {"retry": "retrieve", "done": END})
57
58     return graph.compile()
59
60 app = build_graph()
61
62 if __name__ == "__main__":
63     result = app.invoke({
64         "question": "What are the hazard codes for acetone?",
65         "context": "",
66         "answer": "",
67         "messages": [],
68         "retry_count": 0,
69     })
70     print(result["answer"])

```

6.10 Tool use in LangGraph

LangGraph supports tools — Python functions that the LLM can choose to call. For the hybrid RAG agent, we will not use the LangGraph tool mechanism (we use direct parallel dispatch instead, as you will see in Chapter 7). But understanding tools is useful if you extend the agent later.

Define a tool with the `@tool` decorator:


```

1 from langchain_core.tools import tool
2
3 @tool
4 def get_hazard_codes(material_number: str) -> str:
5     """Retrieve GHS hazard codes for a material from the knowledge
6     graph."""
7     # In a real implementation this queries HANA SPARQL
8     return "H225, H319, H336"

```

Bind tools to the LLM and use `ToolNode` for execution:

```

1 from langgraph.prebuilt import ToolNode
2
3 tools = [get_hazard_codes]
4 llm_with_tools = _get_llm().bind_tools(tools)
5 tool_node = ToolNode(tools)
6
7 graph.add_node("tools", tool_node)
8 graph.add_conditional_edges(
9     "answer",
10    lambda state: "tools" if state["messages"][-1].tool_calls else END,
11    {"tools": "tools", END: END}
12 )
13 graph.add_edge("tools", "answer")

```

This creates a ReAct loop: the LLM decides whether to call a tool, the tool runs, and the result feeds back into the next LLM call.

Note: Chapter 7 intentionally avoids this ReAct pattern for the main retrieval. We run both vector and KG chains on every query, regardless of what the LLM might choose. This gives deterministic latency and prevents the LLM from skipping a retrieval path it thinks is unnecessary — a critical property for enterprise systems where consistent, complete answers matter more than adaptive efficiency.

6.11 Streaming responses

LangGraph can stream state updates as they happen. This is valuable when queries take several seconds — the user sees progress rather than a blank screen.

```

1  for event in app.stream({
2      "question":    "What first aid should I give if someone inhales
                        acetone?",
3      "context":     "",
4      "answer":      "",
5      "messages":    [],
6      "retry_count": 0,
7  }):
8      for node_name, state_update in event.items():
9          print(f"[{node_name}] completed")
10         if "answer" in state_update:
11             print(f"    answer: {state_update['answer'][:80]}...")

```

Expected output:

[retrieve] completed

[answer] completed

answer: If someone inhales acetone vapours, move them immediately to fresh air...

[check] completed

Tip: In the FastAPI service, pipe the stream to a Server-Sent Events (SSE) response using `StreamingResponse`. The Fiori UI can consume the stream and update the chat interface progressively as each node completes. We implement this in Chapter 9.

6.12 Debugging with LangSmith

LangSmith is LangChain's observability platform. It records every LangGraph run — inputs, outputs, intermediate states, LLM calls, latency, and token counts. For an enterprise AI governance use case, LangSmith is the audit log.

Set three environment variables:

```

1 export LANGCHAIN_TRACING_V2=true
2 export LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
3 export LANGCHAIN_API_KEY=your-api-key-here

```

In your BTP CF `manifest.yml`, add the same variables:

```

1 applications:
2   - name: hybrid-rag-agent
3     env:
4       LANGCHAIN_TRACING_V2: "true"
5       LANGCHAIN_ENDPOINT: "https://api.smith.langchain.com"
6       LANGCHAIN_API_KEY: ((langchain-api-key))

```

Use a BTP User-Provided Service to supply the API key without storing it in the manifest. In the LangSmith dashboard you will see a tree view of each run: which nodes executed, in what order, how long each took, and the exact prompt sent to Gemini. When Chapter 7's parallel chains run, you will see two branches executing simultaneously — a visual confirmation that the `ThreadPoolExecutor` parallelism is working.

Tip: Tag your runs in `app.invoke(config={"metadata": {"material_number": material_number}})` and filter by `metadata.material_number` in LangSmith to see all queries against a specific SAP Material Number. This is the query-level audit trail that enterprise governance teams require.

6.13 Why stateless works on BTP Cloud Foundry

A common concern when deploying LangGraph to Cloud Foundry is: what happens to agent state when you scale to multiple instances?

The answer is: nothing, because there is no server-side state to worry about.

LangGraph state is in-memory for a single request. When `app.invoke()` returns, the state is gone. There is no database, no session file, no sticky session required.

Concern	Reality
CF runs 10 instances	Each request lands on any instance — fine

Concern	Reality
CF restarts an instance	No state is lost — state only exists during a request
Two users query simultaneously	Each gets their own independent in-memory state
Conversation history	Passed in <code>messages</code> field of every request body by the caller

The conversation history pattern — where the client sends previous turns in every request — is the key. The CAP Fiori frontend stores the last N messages and sends them with each new question.

```

1  # Frontend sends history on every request
2  # POST /query
3  {
4      "question": "And what about the flash point?",
5      "material_number": "MAT-001",
6      "history": [
7          {"role": "user", "content": "What are the hazard codes for acetone?"},
8          {"role": "assistant", "content": "The GHS hazard codes for acetone are H225, H319, and H336."}
9      ]
10 }
```

The agent picks up the conversation exactly where it left off — without any server-side memory. This design is safe to scale horizontally on BTP CF without load balancer sticky sessions or shared caches.

6.14 Summary

In this chapter we built a LangGraph agent from scratch and grounded every concept in SAP BTP enterprise terms:

- Defined **state** as a `TypedDict` that flows through every node — the workflow con-

text object

- Wrote **nodes** as plain Python functions that read state and return partial updates — the workflow service tasks
- Connected nodes with **edges** and **conditional edges** to express branching and looping — sequence flows and exclusive gateways
- Added a **retry loop** that re-runs retrieval when the answer quality is low
- Demonstrated **tool use** with the `@tool` decorator and `ToolNode`
- Enabled **streaming** to show node-level progress events
- Configured **LangSmith** for end-to-end observability and audit trail
- Explained why **stateless design** makes LangGraph safe to scale horizontally on BTP CF

The agent built here uses a stub retriever. In Chapter 7, we replace that stub with the two HANA retrieval chains — vector cosine search and SPARQL — and run them in parallel.

6.15 Checkpoint

Before continuing to Chapter 7, verify the following:

```

1  # 1. LangGraph is installed
2  python -c "import langgraph; print(langgraph.__version__)"
3
4  # 2. The simple agent runs without errors
5  cd agents && python -m agents.simple_qa_agent
6
7  # 3. Gemini credentials are set
8  echo $GOOGLE_API_KEY    # or $GOOGLE_APPLICATION_CREDENTIALS for service
   account
9
10 # 4. LangSmith (optional but recommended for governance)
11 echo $LANGCHAIN_TRACING_V2      # should print "true"
```

If all four commands succeed, you have a working LangGraph agent connected to Gemini 2.5 Flash. Chapter 7 takes this foundation and builds the full parallel hybrid RAG system on top of it — replacing the stub retriever with live HANA vector search and SPARQL execution running in parallel.

End of Chapter 6

Chapter 7: The Parallel Hybrid RAG Agent

SAP HANA Cloud is the only enterprise database that provides both `REAL_VECTOR` cosine similarity search and native SPARQL execution in a single managed service on BTP. This chapter shows how to use both simultaneously.

In Chapter 6 we built a LangGraph agent with a stub retriever — a placeholder that returned a fixed string instead of real data. In Chapters 3 and 4 we built the real retrieval systems: a vector store in SAP HANA Cloud and a knowledge graph queried via SPARQL. In Chapter 5 we built the ingestion pipeline that populates both. All the pieces exist. This chapter assembles them.

The central design question is: when a user asks a question, do we run vector search first and knowledge graph search second? Or do we ask the LLM to decide which one to use? The answer to both is no. We run them in parallel, always, regardless of the question. This chapter explains why that decision is correct and shows you exactly how to implement it.

By the end of this chapter you will have a working hybrid RAG system: a FastAPI service that receives a natural language question about any PDF document linked to an SAP Material Number, dispatches it simultaneously to both retrieval chains, merges the results, and returns an answer that demonstrably outperforms either strategy alone.

7.1 Why parallel beats sequential

Consider the naive approach: run vector search, check the result, then decide whether to also run KG search. This is sequential routing, and it has a hidden cost.

Approach	Wall-clock time	LLM calls	Risk
Sequential: vector then KG	$2\text{ s} + 2\text{ s} = 4\text{ s}$	3 (route + vector + KG)	Routing LLM makes wrong choice
Sequential: KG then vector	$2\text{ s} + 2\text{ s} = 4\text{ s}$	3	Same risk

Approach	Wall-clock time	LLM calls	Risk
Routing: LLM decides which to use	1 s + 2 s = 3 s	2	Routing LLM skips useful path
Parallel: both simultaneously	max(2 s, 2 s) = 2 s	2	None

The parallel approach is faster than all sequential variants and uses fewer LLM calls than the routing variant. More importantly, it eliminates the routing error entirely. A routing LLM might decide that “what are the GHS codes for acetone?” is a structured query and skip the vector chain — missing the handling instructions that live only in prose. By always running both, we guarantee that no retrieval path is skipped. For enterprise systems where completeness matters — a safety officer needs all relevant information, not just what the system guesses is most relevant — this deterministic behaviour is non-negotiable.

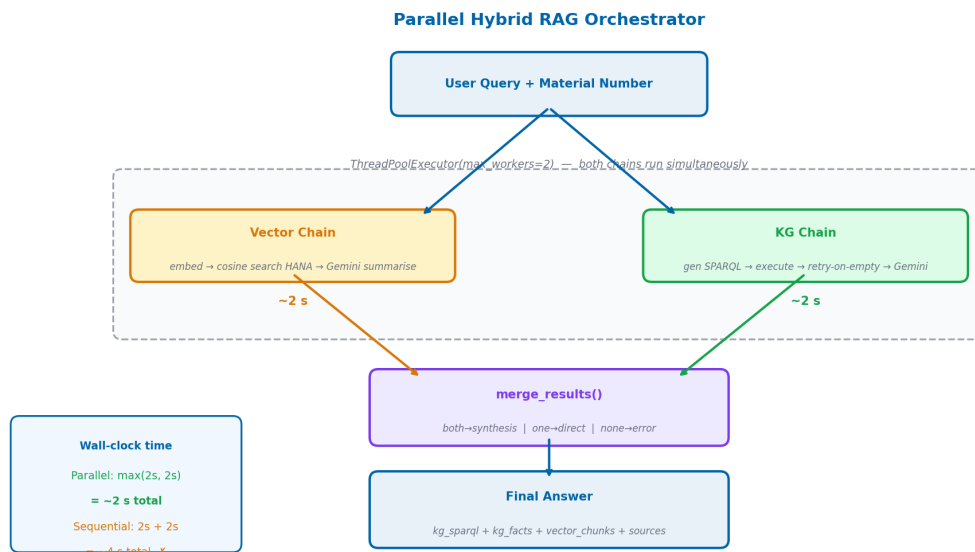


Figure 7.1: The parallel hybrid RAG orchestrator — both chains run concurrently via `ThreadPoolExecutor(max_workers=2)`. Wall-clock latency equals the slower of the two chains, not their sum.

Note: This design assumes both chains have roughly similar latency (~1–3 seconds each). If one chain were orders of magnitude slower than the other,

you might reconsider. In practice, a HANA SPARQL query and a HANA vector search take comparable time.

7.2 The agent state

Create `agents/agents/state.py`:

```

1  from typing import TypedDict, List, Optional
2
3  class HybridRAGState(TypedDict):
4      # Input
5      question: str
6      material_number: str
7      history: List[dict]
8
9      # Vector chain outputs
10     vector_answer: str
11     vector_chunks: List[dict]
12
13     # KG chain outputs
14     kg_answer: str
15     kg_sparql: str
16     kg_facts: List[dict]
17
18     # Final output
19     final_answer: str
20     sources: List[str]
21
22     # Control
23     error: Optional[str]
```

This state is the contract between every component in the system. The vector chain writes to `vector_answer` and `vector_chunks`. The KG chain writes to `kg_answer`, `kg_sparql`, and `kg_facts`. The merge function reads all four and writes `final_answer`.

The `sources` field serves a specific enterprise purpose: auditability. When the CAP Fiori UI displays an answer, it also shows the user which retrieval paths contributed — “Knowledge graph: 3 facts, Document search: 5 passages”. This tells the user whether

the answer is grounded in structured facts from the KG, in narrative passages from the vector store, or in both. For regulated industries where answer provenance must be demonstrable, this field is not optional.

7.3 The vector chain

Create `agents/agents/vector_chain.py`:

```

1  import logging
2  from typing import Optional
3  from langchain_google_genai import ChatGoogleGenerativeAI,
   GoogleGenerativeAIEmbeddings
4  from langchain_core.messages import HumanMessage
5
6  from agents.state import HybridRAGState
7  from srv.hdb_srv import get_connection
8
9  logger = logging.getLogger(__name__)
10
11 def _get_llm():
12     return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
13     temperature=0.1, max_tokens=1024)
14
15 def _get_embedder():
16     return GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")
17
18 COSINE_SEARCH_SQL = """
19     SELECT TOP 5
20         CHUNK_TEXT,
21         MATERIAL_NUMBER,
22         CHUNK_INDEX,
23         TO_DOUBLE(COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?))) AS SCORE
24     FROM MSDS_VECTORS
25     WHERE MATERIAL_NUMBER = ?
26     ORDER BY SCORE DESC
27     """

```

```

28 def run_vector_chain(state: HybridRAGState) -> dict:
29     """Execute the vector retrieval chain and return state updates."""
30     question = state["question"]
31     material_number = state["material_number"]
32
33     try:
34         # Step 1: Embed the question
35         embedding = _get_embedder().embed_query(question)
36         embedding_str = "[" + ",".join(str(v) for v in embedding) + "]"
37
38         # Step 2: Cosine search in HANA
39         conn = get_connection()
40         cursor = conn.cursor()
41         cursor.execute(COSINE_SEARCH_SQL, (embedding_str,
material_number))
42         rows = cursor.fetchall()
43
44         if not rows:
45             logger.info("Vector search returned no results for %s",
material_number)
46             return {
47                 "vector_answer": "",
48                 "vector_chunks": [],
49             }
50
51         chunks = [
52             {
53                 "text": row[0],
54                 "material_number": row[1],
55                 "chunk_index": row[2],
56                 "score": float(row[3]),
57             }
58             for row in rows
59         ]
60
61         # Step 3: Summarise with Gemini 2.5 Flash
62         context = "\n\n---\n\n".join(c["text"] for c in chunks)

```

```

63     prompt = f"""You are an expert assistant for SAP material
documents – answer questions based only on the provided document context.
Be specific and concise.
64 If the passages do not contain enough information, say so.
65
66 Passages:
67 {context}
68
69 Question: {question}
70
71 Answer: """
72
73     response = _get_llm().invoke([HumanMessage(content=prompt)])
74     return {
75         "vector_answer": response.content,
76         "vector_chunks": chunks,
77     }
78
79     except Exception as e:
80         logger.error("Vector chain failed: %s", e)
81         return {
82             "vector_answer": "",
83             "vector_chunks": [],
84             "error": f"Vector chain error: {str(e)}",
85         }

```

Three steps: embed the question into a 768-dimensional vector using `text-embedding-004`, run a cosine similarity search against `MSDS_VECTORS`, summarise the top-5 matching chunks with Gemini 2.5 Flash. The SQL uses HANA's `COSINE_SIMILARITY` function and `TO_REAL_VECTOR` to cast the embedding string to the correct column type.

7.4 The KG chain

Create `agents/agents/kg_chain.py`:

```

1  import logging
2  from langchain_google_genai import ChatGoogleGenerativeAI

```

```

3  from langchain_core.messages import HumanMessage
4
5  from agents.state import HybridRAGState
6  from srv.hdb_srv import get_connection
7
8  logger = logging.getLogger(__name__)
9
10 def _get_llm():
11     return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
12                                     temperature=0.0, max_tokens=512)
13
14 def _get_summariser():
15     return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
16                                     temperature=0.1, max_tokens=1024)
17
18 GRAPH_URI_PREFIX = "http://msds.knowledge-graph.org/MSDS_Graph/"
19
20 SPARQL_GEN_PROMPT = """You are a SPARQL expert. Generate a SPARQL SELECT
21 query to answer
22 the question below using the provided ontology.
23
24 Ontology predicates available:
25 - msds:hasHazardCode (links Material to HazardCode)
26 - msds:hasExposureLimit (links Material to ExposureLimit, has
27   msds:limitValue and msds:limitUnit)
28 - msds:requiresPrecaution (links Material to Precaution, has
29   msds:precautionText)
30 - msds:hasSupplier (links Material to Supplier)
31 - msds:hazardDescription (literal on HazardCode)
32 - rdfs:label (name of the material)
33
34 Named graph: <{graph_uri}>
35 Material URI: <http://msds.knowledge-graph.org/material/{material_number}>
36
37 Rules:
38 1. Always wrap triple patterns with GRAPH <{graph_uri}> {{ ... }}
39 2. Use PREFIX msds: <http://msds.knowledge-graph.org/ontology/>
40 3. Use PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
41 4. Return only the SPARQL query, no explanation

```

```

37
38 Question: {question}
39
40 SPARQL: ""
41
42 SPARQL_FALLBACK_PROMPT = """The previous SPARQL query returned no results.
43 Generate a simpler,
44 broader SPARQL query to retrieve any available information about this
45 material.
46
47 Named graph: <{graph_uri}>
48 Material URI: <http://msds.knowledge-graph.org/material/{material_number}>
49
50 Return all triples about this material:
51
52 SPARQL: ""
53
54 def _execute_sparql(sparql: str) -> list:
55     conn = get_connection()
56     cursor = conn.cursor()
57     try:
58         cursor.callproc("SPARQL_EXECUTE", [sparql, None, 1000, None, None])
59         return cursor.fetchall()
60     finally:
61         cursor.close()
62
63 def run_kg_chain(state: HybridRAGState) -> dict:
64     """Execute the KG retrieval chain and return state updates."""
65     question = state["question"]
66     material_number = state["material_number"]
67     graph_uri = f"{GRAPH_URI_PREFIX}{material_number}"
68
69     try:
70         # Step 1: Generate SPARQL
71         gen_prompt = SPARQL_GEN_PROMPT.format(
72             graph_uri=graph_uri,
73             material_number=material_number,
74             question=question,
75         )

```

```

74     sparql_response =
_get_llm().invoke([HumanMessage(content=gen_prompt)])
75     sparql = sparql_response.content.strip()
76     # Strip markdown code fences if present
77     if sparql.startswith("```"):
78         sparql = "\n".join(sparql.split("\n")[1:-1])
79
80     # Step 2: Execute SPARQL
81     rows = _execute_sparql(sparql)
82
83     # Step 3: Retry with fallback if empty
84     if not rows:
85         logger.info("SPARQL returned empty, retrying with fallback for
%s", material_number)
86         fallback_prompt = SPARQL_FALLBACK_PROMPT.format(
87             graph_uri=graph_uri,
88             material_number=material_number,
89         )
90         fallback_response =
_get_llm().invoke([HumanMessage(content=fallback_prompt)])
91         sparql = fallback_response.content.strip()
92         if sparql.startswith("```"):
93             sparql = "\n".join(sparql.split("\n")[1:-1])
94         rows = _execute_sparql(sparql)
95
96     if not rows:
97         return {
98             "kg_answer": "",
99             "kg_sparql": sparql,
100             "kg_facts": [],
101         }
102
103     # Step 4: Summarise with Gemini 2.5 Flash
104     facts = [str(row) for row in rows]
105     facts_text = "\n".join(facts[:50]) # limit to 50 facts
106

```

```

107     summarise_prompt = f"""You are an expert assistant for SAP
material documents – answer questions based only on the provided document
context. The following facts were retrieved from a structured knowledge
graph about material {material_number}.
108 Use them to answer the question precisely.
109
110 Facts:
111 {facts_text}
112
113 Question: {question}
114
115 Answer: """
116
117     summary_response =
_get_summariser().invoke([HumanMessage(content=summarise_prompt)])
118     return {
119         "kg_answer": summary_response.content,
120         "kg_sparql": sparql,
121         "kg_facts": facts[:50],
122     }
123
124     except Exception as e:
125         logger.error("KG chain failed: %s", e)
126         return {
127             "kg_answer": "",
128             "kg_sparql": "",
129             "kg_facts": [],
130             "error": f"KG chain error: {str(e)}",
131         }

```

The KG chain has one critical feature beyond basic SPARQL execution: the retry-on-empty loop. When Gemini generates a SPARQL query that returns no rows — which happens when the generated query is overly specific — the chain retries with a simpler fallback query that retrieves all triples for the material. This ensures we always get *something* from the knowledge graph if data exists.

Warning: Never share HANA connections across threads. The `get_connection()` call in both chains uses `threading.local()` under the hood — each thread gets its own connection. If you pass a connection object between threads, HANA will raise concurrency errors. See `agents/srv/hdb_srv.py` for the

thread-local implementation.

7.5 The merge function

The merge function is the heart of hybrid RAG. It receives whatever the two chains produced and decides how to combine them. The four cases map directly to enterprise answer quality tiers:

Both chains return results — comprehensive answer. The KG provides structured facts (exact codes, measured values, regulatory identifiers); the vector store provides narrative context (procedures, explanations, warnings in prose). A synthesis LLM call combines them: KG facts take precedence for specific codes and numbers, vector passages fill in the procedural detail. This is the highest-quality answer.

Only the KG returns results — precise factual answer. The structured facts are returned directly, without a synthesis call. No prose padding. The answer says exactly what the KG says, no more.

Only the vector store returns results — narrative context answer. The document passages are summarised and returned. No specific codes are asserted beyond what appears in the text.

Neither returns results — honest gap acknowledgement. The system says it cannot find information and instructs the user to verify the material number or rephrase. It does not hallucinate. This honest-gap behaviour is critical for enterprise trust: a system that occasionally says “I don’t know” is far more trustworthy than one that always produces an answer, even when the answer is fabricated.

```
1 def merge_results(  
2     kg_answer: str,  
3     vector_answer: str,  
4     question: str,  
5     material_number: str,  
6     llm,  
7 ) -> str:  
8     """  
9     Merge KG and vector answers into a final response.
```

```

10
11     Cases:
12         both    → synthesis LLM call
13         kg only → return kg_answer directly
14         vector  → return vector_answer directly
15         neither → graceful error message
16     """
17     has_kg = bool(kg_answer and kg_answer.strip())
18     has_vec = bool(vector_answer and vector_answer.strip())
19
20     if has_kg and has_vec:
21         synthesis_prompt = f"""You are an expert assistant for SAP
22 material documents.
23 You have received answers from two independent retrieval systems for the
24 question below.
25
26 Structured Knowledge Graph answer:
27 {kg_answer}
28
29 Document Search answer:
30 {vector_answer}
31
32 Synthesise both into a single, coherent, well-organised answer.
33 - Do not repeat information
34 - If they contradict, prefer the structured KG answer for specific facts
35 (codes, limits)
36 - If they complement, combine them naturally
37 - Be concise
38
39 Question: {question}
40
41 Answer: """
42         response = llm.invoke([HumanMessage(content=synthesis_prompt)])
43         return response.content
44
45     elif has_kg:
46         return kg_answer
47
48     elif has_vec:

```

```

46         return vector_answer
47
48     else:
49         return (
50             f"No relevant information found for your question in the
              available "
51             f"documents for material {material_number}. Please verify the
              material "
52             f"number or rephrase your question."
53         )

```

7.6 The orchestrator

Create `agents/agents/orchestrator.py`:

```

1  import logging
2  from concurrent.futures import ThreadPoolExecutor, as_completed,
   TimeoutError
3  from langchain_google_genai import ChatGoogleGenerativeAI
4  from langchain_core.messages import HumanMessage
5
6  from agents.state import HybridRAGState
7  from agents.vector_chain import run_vector_chain
8  from agents.kg_chain import run_kg_chain
9
10 logger = logging.getLogger(__name__)
11
12 def _get_llm():
13     return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
14                                   temperature=0.1, max_tokens=1024)
15
16 CHAIN_TIMEOUT_SECONDS = 30
17
18 def merge_results(kg_answer: str, vector_answer: str,
19                  question: str, material_number: str) -> str:
20     has_kg = bool(kg_answer and kg_answer.strip())
21     has_vec = bool(vector_answer and vector_answer.strip())

```

```

21
22     if has_kg and has_vec:
23         prompt = f"""You are an expert assistant for SAP material
24         documents.
25         Two retrieval systems answered the same question. Synthesise their
26         answers.
27
28         Knowledge Graph:
29         {kg_answer}
30
31         Document Search:
32         {vector_answer}
33
34         Question: {question}
35
36         Synthesise into one coherent answer. Prefer KG for exact codes/limits.
37         Answer: """
38         return _get_llm().invoke([HumanMessage(content=prompt)]).content
39
40     return kg_answer or vector_answer or (
41         f"No relevant information found for material {material_number}. "
42         "Please verify the material number or rephrase your question."
43     )
44
45 def run_hybrid_rag(state: HybridRAGState) -> dict:
46     """
47     Dispatch both chains in parallel, merge results.
48     Returns a dict of state updates.
49     """
50     chains = {
51         "vector": run_vector_chain,
52         "kg": run_kg_chain,
53     }
54     results = {"vector": {}, "kg": {}}
55
56     with ThreadPoolExecutor(max_workers=2) as executor:
57         futures = {
58             executor.submit(fn, state): name
59             for name, fn in chains.items()

```

```

58         }
59         for future in as_completed(futures, timeout=CHAIN_TIMEOUT_SECONDS
60             + 5):
61             name = futures[future]
62             try:
63                 results[name] =
64                 future.result(timeout=CHAIN_TIMEOUT_SECONDS)
65             except TimeoutError:
66                 logger.warning("%s chain timed out after %ds", name,
67                     CHAIN_TIMEOUT_SECONDS)
68                 results[name] = {}
69             except Exception as e:
70                 logger.error("%s chain raised exception: %s", name, e)
71                 results[name] = {}
72
73         # Merge state updates from both chains
74         merged_state = {}
75         for chain_result in results.values():
76             merged_state.update(chain_result)
77
78         # Generate final answer
79         final_answer = merge_results(
80             kg_answer=merged_state.get("kg_answer", ""),
81             vector_answer=merged_state.get("vector_answer", ""),
82             question=state["question"],
83             material_number=state["material_number"],
84         )
85         merged_state["final_answer"] = final_answer
86
87         # Build sources list for auditability
88         sources = []
89         if merged_state.get("vector_chunks"):
90             sources.append(f"Document search: {len(merged_state['vector_chunks'])} passages")
91         if merged_state.get("kg_facts"):
92             sources.append(f"Knowledge graph: {len(merged_state['kg_facts'])} facts")
93         merged_state["sources"] = sources

```

```
92     return merged_state
```

The orchestrator submits both chains to a `ThreadPoolExecutor` with `max_workers=2` and collects results as they complete. The `as_completed()` call returns futures in completion order — whichever chain finishes first is processed first. The 30-second timeout prevents a slow chain from blocking the response indefinitely.

Note: We use `as_completed()` rather than `executor.map()` because we want to process results as soon as they arrive, not wait for all to finish. If the vector chain finishes in 1.5 seconds and the KG chain takes 3 seconds, the orchestrator captures the vector result at 1.5 seconds and the KG result at 3 seconds, then merges at ~3 seconds total.

The `sources` field populated at the end is the auditability record. The CAP Fiori UI surfaces this to the user — showing whether the answer was grounded in structured knowledge graph facts, in retrieved document passages, or in both. An enterprise user reviewing an AI-generated answer needs to know where it came from.

7.7 The FastAPI /query endpoint

Add the endpoint to `agents/main.py`:

```
1  from fastapi import FastAPI, HTTPException
2  from pydantic import BaseModel, validator
3  from typing import List, Optional
4  import re
5
6  from agents.orchestrator import run_hybrid_rag
7  from agents.state import HybridRAGState
8
9  app = FastAPI(title="Hybrid RAG Agent")
10
11  MATERIAL_RE = re.compile(r"^[A-Za-z0-9_-]+$")
12
13  class QueryRequest(BaseModel):
14      question: str
15      material_number: str
```

```

16     history: List[dict] = []
17
18     @validator("material_number")
19     def validate_material(cls, v):
20         if not MATERIAL_RE.match(v):
21             raise ValueError("material_number contains invalid characters")
22         return v
23
24     @validator("question")
25     def validate_question(cls, v):
26         if not v.strip():
27             raise ValueError("question must not be empty")
28         return v.strip()
29
30 class QueryResponse(BaseModel):
31     answer: str
32     kg_sparql: Optional[str] = None
33     kg_facts: Optional[List] = None
34     vector_chunks: Optional[List] = None
35     sources: Optional[List[str]] = None
36
37 @app.post("/query", response_model=QueryResponse)
38 def query(request: QueryRequest):
39     state: HybridRAGState = {
40         "question": request.question,
41         "material_number": request.material_number,
42         "history": request.history,
43         "vector_answer": "",
44         "vector_chunks": [],
45         "kg_answer": "",
46         "kg_sparql": "",
47         "kg_facts": [],
48         "final_answer": "",
49         "sources": [],
50         "error": None,
51     }
52
53     result = run_hybrid_rag(state)
54

```

```

55     if not result.get("final_answer"):
56         raise HTTPException(status_code=500, detail="Agent returned no
           answer")
57
58     return QueryResponse(
59         answer=result["final_answer"],
60         kg_sparql=result.get("kg_sparql"),
61         kg_facts=result.get("kg_facts"),
62         vector_chunks=result.get("vector_chunks"),
63         sources=result.get("sources"),
64     )

```

The endpoint validates the material number against `^[A-Za-z0-9_-]+$` — the same regex used throughout the application. This prevents SPARQL injection: a material number like `ACE001> } INSERT { <evil> }` would fail validation before reaching HANA.

7.8 The full request/response contract

```

1  // Request
2  POST /query
3  Content-Type: application/json
4
5  {
6      "question": "What are the GHS hazard codes for acetone?",
7      "material_number": "ACE001",
8      "history": [
9          {"role": "user", "content": "What is acetone used for?"},
10         {"role": "assistant", "content": "Acetone is a common solvent used
           in..."}
11     ]
12 }
13
14 // Response
15 {
16     "answer": "The GHS hazard codes for acetone are H225 (Highly flammable
           liquid and vapour), H319 (Causes serious eye irritation), and H336 (May
           cause drowsiness or dizziness).",

```



```

17  "kg_sparql": "PREFIX msds: <...>\nSELECT ?code ?desc WHERE { GRAPH <...>
    { <...ACE001> msds:hasHazardCode ?hc . ?hc rdfs:label ?code . ?hc
      msds:hazardDescription ?desc } }",
18  "kg_facts": [
19    "('H225', 'Highly flammable liquid and vapour')",
20    "('H319', 'Causes serious eye irritation')",
21    "('H336', 'May cause drowsiness or dizziness')",
22  ],
23  "vector_chunks": [
24    {"text": "Section 2: Hazard Identification...", "score": 0.847,
      "chunk_index": 2},
25    ...
26  ],
27  "sources": ["Knowledge graph: 3 facts", "Document search: 5 passages"]
28  }

```

The response includes both the synthesised answer and the raw retrieval evidence (`kg_facts`, `vector_chunks`, `kg_sparql`). The Fiori UI uses the `sources` field to show the user exactly where the answer came from — which SPARQL query retrieved the structured facts, and which document passages supported the narrative answer.

7.9 Conversation history

The `/query` endpoint accepts a `history` list in the request body. This is how multi-turn conversations work without server-side session state.

```

1  # First question – no history
2  POST /query
3  {"question": "What are the hazard codes for acetone?", "material_number":
   "ACE001", "history": []}
4
5  # Second question – pass previous turn
6  POST /query
7  {
8    "question": "And what PPE should I wear when handling it?",
9    "material_number": "ACE001",
10   "history": [

```

```

11     {"role": "user",      "content": "What are the hazard codes for
    acetone?"},
12     {"role": "assistant", "content": "The GHS hazard codes are H225, H319,
    and H336."}
13 ]
14 }

```

The answer node in the LangGraph graph includes the history in its prompt, so the second question gets a contextually aware answer (“Given the flammability hazard H225 that we discussed, you should wear...”) without any server-side state.

The CAP Fiori frontend keeps a rolling window of the last 10 messages — enough for conversational context without ballooning the prompt. Older messages are silently dropped.

7.10 Testing: three scenarios that prove hybrid wins

The following three test cases demonstrate why hybrid retrieval is better than either strategy alone. Each scenario reflects a real role in an SAP-using organisation asking a real question against an MSDS document for acetone (MAT-001). Run them after uploading the acetone MSDS to both stores.

7.10.1 The KG wins: a safety officer asks about hazard classifications

A safety officer asks: “What are the GHS hazard codes for acetone, and what do they mean?”

```

1  curl -X POST http://localhost:8000/query \
2    -H "Content-Type: application/json" \
3    -d '{
4      "question": "What are the GHS hazard codes for acetone, and what do
    they mean?",
5      "material_number": "ACE001",
6      "history": []
7    }'

```

Expected KG answer:

H225, H319, H336

Expected vector answer (typical):

Acetone is classified under several GHS hazard categories. The safety data sheet indicates it is a highly flammable substance with eye irritation properties. Refer to Section 2 for complete hazard identification...

The KG returns the exact codes and their official descriptions from structured triples. The vector chain returns useful prose but buries the codes in a paragraph. The synthesised answer leads with the codes and their meanings from the KG, then adds the narrative context from the vector store. The safety officer gets a complete, citable answer.

7.10.2 The vector wins: a quality engineer asks about first aid procedures

A quality engineer asks: “What first aid should I give if someone inhales acetone vapour in our lab?”

```
1 curl -X POST http://localhost:8000/query \
2   -H "Content-Type: application/json" \
3   -d '{
4     "question": "What first aid should I give if someone inhales acetone
5     vapour in our lab?",
6     "material_number": "ACE001",
7     "history": []
8   }'
```

The KG stores structured facts — hazard codes, exposure limits, precaution flags. It does not store the detailed first-aid procedure paragraph from Section 4 of the MSDS. The vector chain retrieves the exact passage. The final answer comes primarily from the vector chain, with the KG contributing the exposure limit context. The quality engineer gets the procedural instructions they actually need.

7.10.3 Both contribute: a logistics manager asks a combined question

A logistics manager asks: “Is acetone classified as a flammable liquid, and what precautions should I take when storing it in our warehouse?”

```

1 curl -X POST http://localhost:8000/query \
2   -H "Content-Type: application/json" \
3   -d '{
4     "question": "Is acetone classified as a flammable liquid and what
5     precautions should I take when storing it?",
6     "material_number": "ACE001",
7     "history": []
8   }'
```

The KG answers the first part precisely: H225 confirms flammable liquid classification, stored as a structured triple. The vector chain answers the second part: storage precautions from Section 7 of the MSDS, retrieved as narrative prose. Neither chain could answer both parts alone. The synthesis LLM combines them into a single coherent answer. The logistics manager gets both the regulatory classification and the practical storage guidance.

7.11 Monitoring chain performance

Add timing instrumentation to the orchestrator to track which chain is the bottleneck:

```

1 import time
2
3 with ThreadPoolExecutor(max_workers=2) as executor:
4     start = time.time()
5     futures = {executor.submit(fn, state): name for name, fn in
6     chains.items()}
7     for future in as_completed(futures, timeout=35):
8         name = futures[future]
9         elapsed = time.time() - start
10        logger.info("%s chain completed in %.2fs", name, elapsed)
11        results[name] = future.result(timeout=30)
12
13 total = time.time() - start
14 logger.info("Both chains completed in %.2fs (wall clock)", total)
```

In a typical run against a populated HANA instance:

INFO: vector chain completed in 1.83s

INFO: kg chain completed in 2.41s

INFO: Both chains completed in 2.41s (wall clock)

The wall-clock time is the slower chain, not the sum. If you ran these sequentially, the same run would take 4.24 seconds. The parallel design saves ~1.8 seconds on every query — which compounds significantly across a busy SAP system where many users are asking questions simultaneously.

7.12 Summary

In this chapter we built the core of the hybrid RAG system on top of SAP HANA Cloud's unique combination of REAL_VECTOR search and native SPARQL execution:

- Defined a shared `HybridRAGState` TypedDict that all components read and write
 - Built the **vector chain**: embed question with `text-embedding-004` → cosine search HANA REAL_VECTOR → summarise with Gemini 2.5 Flash
 - Built the **KG chain**: generate SPARQL with Gemini 2.5 Flash → execute on HANA → retry-on-empty → summarise
 - Wrote the **merge function** with four explicit cases tied to enterprise answer quality: both (synthesise), KG only (precise facts), vector only (narrative context), neither (honest gap acknowledgement — never hallucinate)
 - Assembled the **orchestrator** using `ThreadPoolExecutor(max_workers=2)` for true parallel execution
 - Exposed a `/query` **endpoint** with material number validation and a clean request/response contract
 - Demonstrated **stateless conversation history** passed in every request
 - Verified with **three test scenarios** — safety officer, quality engineer, logistics manager — that hybrid retrieval outperforms either strategy alone
 - Added the `sources` **field** to the response for enterprise auditability
-

7.13 Checkpoint

Before continuing to Chapter 8, verify the following:

```

1  # 1. Both chains import without errors
2  cd agents
3  python -c "from agents.vector_chain import run_vector_chain; print('vector
  OK')"
4  python -c "from agents.kg_chain import run_kg_chain; print('kg OK')"
5
6  # 2. The orchestrator runs (requires HANA connection and Gemini API key)
7  python -c "
8  from agents.orchestrator import run_hybrid_rag
9  result = run_hybrid_rag({
10     'question': 'What are the hazard codes?',
11     'material_number': 'ACE001',
12     'history': [],
13     'vector_answer': '', 'vector_chunks': [],
14     'kg_answer': '', 'kg_sparql': '', 'kg_facts': [],
15     'final_answer': '', 'sources': [], 'error': None,
16 })
17 print('answer:', result.get('final_answer', '')[:80])
18 "
19
20 # 3. The /query endpoint responds
21 uvicorn main:app --port 8000 &
22 curl -s -X POST http://localhost:8000/query \
23     -H 'Content-Type: application/json' \
24     -d '{"question":"What is
  acetone?","material_number":"ACE001","history":[]}' \
25     | python -m json.tool

```

If all three commands succeed and the final curl returns a JSON response with an `answer` field and a `sources` field, the hybrid RAG agent is working. Chapter 8 extends it with a multi-agent supervisor layer for complex, multi-domain queries that require different retrieval strategies per sub-question.

End of Chapter 7

Chapter 8: The Multi-Agent Supervisor Pattern

In Chapter 7 we built a hybrid RAG agent that runs two retrieval strategies in parallel and merges their results. For the majority of questions — “what are the hazard codes?”, “what are the storage instructions?” — that agent is the right tool. It is fast, deterministic, and requires no coordination overhead.

But consider a different class of question: “I am setting up a new chemical storage facility that will handle acetone. What hazard classifications do I need to post, what are the legal worker exposure limits, and what personal protective equipment should I provide?” That question requires three distinct types of expertise: GHS classification knowledge, regulatory compliance knowledge, and safety procedure knowledge. A single agent with a single retrieval pass will mix these domains, risk missing one, and produce an answer that is harder to reason about.

This is the complexity ceiling of a single-agent system. When a question spans multiple domains and each domain requires focused, specialised reasoning, a team of specialist agents produces better answers than a generalist agent working alone.

This chapter introduces the **supervisor pattern**: a coordinator agent that receives complex questions, decomposes them, routes sub-questions to specialist agents, and synthesises their results into a final answer. The pattern is demonstrated with MSDS-specific specialists, but the architecture is general — the same supervisor, state machine, and merge logic apply to any SAP document type.

8.1 Why multi-agent for SAP enterprise

A generalist agent has one prompt, one retrieval context, and one LLM call to produce its answer. When questions are simple and focused, this is efficient. When questions are complex and multi-domain, it creates three problems.

Problem 1: Context dilution. A single retrieval pass retrieves the top-5 passages by cosine similarity. For a multi-domain question, those 5 passages will be distributed across domains — perhaps 2 about hazards, 2 about storage, 1 about first aid. Each

domain gets insufficient context.

Problem 2: Prompt overloading. The LLM receives a long prompt with mixed signals. It must simultaneously reason about regulatory thresholds, GHS classifications, and PPE requirements. The answer tends to be shallow across all three rather than deep on any one.

Problem 3: Uneven retrieval strategy per domain. Different document sections require different retrieval strategies. A question about GHS hazard codes (structured data) is best answered from the knowledge graph, where codes and descriptions are stored as precise triples. A question about first aid procedures (narrative text) is best answered from the vector store, where the prose of Section 4 lives. A question about regulatory compliance requires both — structured exposure limits from the KG and narrative regulatory context from the vector store. A generalist agent applies one strategy to all three. A specialist agent applies the right strategy to its domain.

The supervisor pattern solves all three by decomposing the question *before* retrieval — routing each sub-question to a specialist that uses the correct retrieval strategy for that knowledge type.

8.2 The supervisor pattern

The supervisor pattern has four components:

Component	Role
Supervisor	Receives the user question, decomposes it into sub-questions, routes each to the right specialist, collects results
Specialist agents	Each handles one focused domain — retrieves relevant data using the right strategy, generates a domain-specific answer
SummaryAgent	Receives all specialist answers, synthesises them into a single coherent response

Component	Role
Shared state	Carries the original question, sub-questions, and all specialist answers through the graph

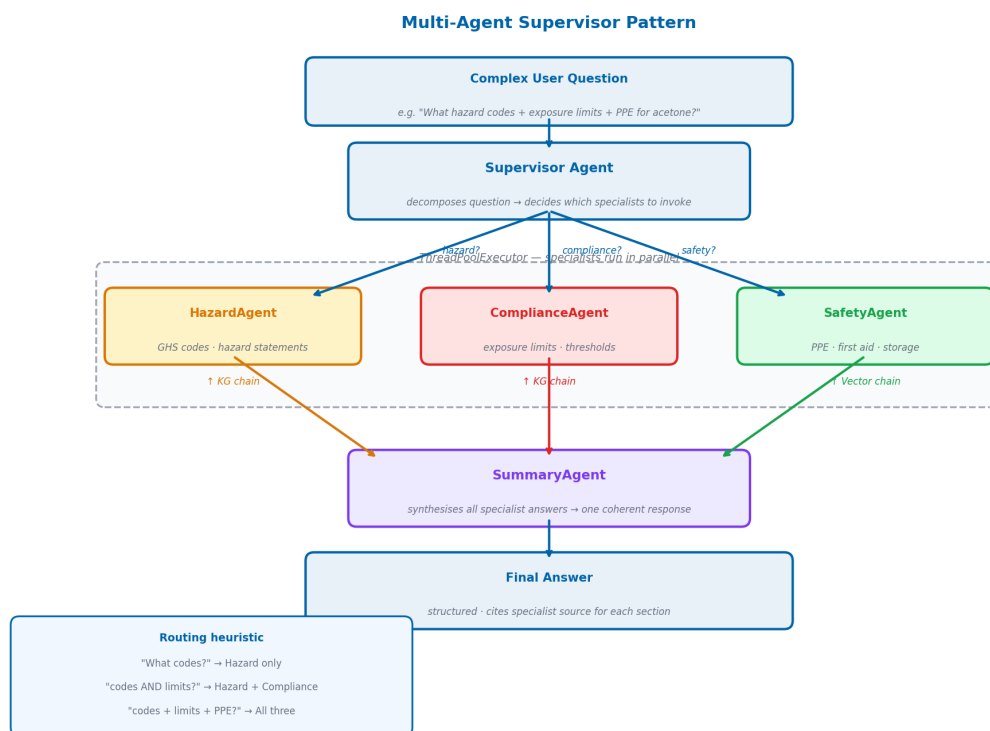


Figure 8.1: The multi-agent supervisor — the supervisor decomposes the question and routes sub-questions to specialist agents running in parallel. The SummaryAgent synthesises all results into the final answer.

The supervisor does not answer the question. It only routes. This separation of concerns is what makes the pattern scale: you can add a new specialist agent without changing the supervisor prompt, and each specialist can be optimised independently.

8.3 The three specialist agents for MSDS documents

For MSDS documents, the domain decomposition maps naturally to three specialist agents. Each agent uses a different retrieval strategy because each domain has a different knowledge representation:

8.3.1 HazardAgent

Domain: GHS classifications, hazard codes, hazard statements, signal words. **Retrieval strategy:** Knowledge graph — structured facts from `hasHazardCode` and `hazardDescription` predicates in `MSDS_Graph/MAT-XXX`. **Strength:** Returns exact codes (H225, H319, H336) and their official descriptions directly from structured triples — no ambiguity, no narrative interpretation.

8.3.2 SafetyAgent

Domain: Precautions, first aid procedures, PPE requirements, storage and handling instructions. **Retrieval strategy:** Vector search — narrative text from Sections 4, 7, and 8 of the MSDS. **Strength:** Returns detailed procedural instructions that live in prose and are not easily reduced to structured triples. The vector store retrieves the specific paragraphs that answer the question.

8.3.3 ComplianceAgent

Domain: Regulatory requirements, exposure limits, permissible concentrations, legal obligations. **Retrieval strategy:** Both KG and vector — structured exposure limits from `hasExposureLimit` predicates, combined with narrative regulatory context from the vector store. **Strength:** Handles questions that span structured facts (the OSHA limit is 500 ppm TWA) and regulatory narrative (what that limit means and how to comply with it).

These three agents are MSDS-specific instances of a general pattern. The same supervisor architecture deployed for purchase order documents would have different specialists: a `HeaderAgent` for invoice metadata, a `LineItemAgent` for line item details, a `PaymentAgent` for payment terms and status. The routing logic, the state machine, and the merge mechanism are identical. Only the specialist prompts and the KG predicates they query change.

8.4 Shared state for the supervisor graph

Create `agents/agents/supervisor_state.py`:

```

1  from typing import TypedDict, List, Optional, Dict
2
3  class SupervisorState(TypedDict):
4      # Input
5      question: str
6      material_number: str
7      history: List[dict]
8
9      # Decomposed sub-questions (set by supervisor)
10     sub_questions: Dict[str, str] # {"hazard": "...", "compliance":
    "...", "safety": "..."}
11     specialists_needed: List[str] # e.g. ["hazard", "compliance"]
12
13     # Specialist outputs
14     hazard_answer: str
15     compliance_answer: str
16     safety_answer: str
17
18     # Final output
19     final_answer: str
20     sources: List[str]
21
22     # Control
23     error: Optional[str]

```

The `sub_questions` dictionary maps specialist names to the focused questions the supervisor generated for them. `specialists_needed` is the list of specialists the supervisor decided to invoke — not every question needs all three. A question purely about GHS codes routes only to `HazardAgent`. A question about storage and compliance routes to `SafetyAgent` and `ComplianceAgent`.

8.5 The supervisor node

The supervisor's job is to read the user's question and decide which specialists are needed and what sub-question to give each one. This is a routing decision, and routing decisions should be deterministic — temperature 0.0.

```

1  import json
2  import logging
3  from langchain_google_genai import ChatGoogleGenerativeAI
4  from langchain_core.messages import HumanMessage
5
6  logger = logging.getLogger(__name__)
7
8  def _get_llm():
9      return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
10                                     temperature=0.0, max_tokens=512)
11
12  SUPERVISOR_PROMPT = """You are a routing agent for a material document
13  intelligence system on SAP BTP.
14  Your job is to analyse a user's question and route it to the right
15  specialist agents.
16
17  Each specialist uses a different retrieval strategy – route to the
18  specialist whose strategy best matches the question type:
19  - "hazard": answers questions about structured facts – GHS hazard codes,
20  hazard classifications, signal words, hazard statements. Uses the
21  knowledge graph.
22  - "compliance": answers questions about regulatory requirements – exposure
23  limits, permissible concentrations, OSHA/ACGIH thresholds, legal
24  obligations. Uses both knowledge graph and document search.
25  - "safety": answers questions about narrative procedures – first aid, PPE
26  requirements, storage instructions, handling precautions, spill response.
  Uses document search.
  For each specialist you select, write a focused sub-question that extracts
  only the relevant part of the user's question.
  Respond in JSON format only:
  {{
    "specialists": ["hazard", "compliance", "safety"],
    "sub_questions": {{
      "hazard": "What GHS hazard codes and classifications apply to
  {material}}?",
      "compliance": "What are the worker exposure limits for {material}}?",

```

```

27     "safety": "What PPE and handling precautions are required for
    {material}?"
28     }}
29 }}
30
31 Select only the specialists that are relevant. A simple question may need
    only one.
32
33 Material: {material}
34 Question: {question}
35
36 JSON:"""
37
38 def supervisor_node(state: SupervisorState) -> dict:
39     question = state["question"]
40     material_number = state["material_number"]
41
42     prompt = SUPERVISOR_PROMPT.format(
43         material=material_number,
44         question=question,
45     )
46     response = _get_llm().invoke([HumanMessage(content=prompt)])
47     content = response.content.strip()
48
49     # Strip markdown code fences if present
50     if content.startswith("```"):
51         content = "\n".join(content.split("\n")[1:-1])
52
53     try:
54         routing = json.loads(content)
55         return {
56             "specialists_needed": routing.get("specialists", ["hazard",
57             "compliance", "safety"]),
57             "sub_questions": routing.get("sub_questions", {}),
58         }
59     except json.JSONDecodeError:
60         logger.warning("Supervisor JSON parse failed, routing to all
    specialists")
61     return {

```

```

62         "specialists_needed": ["hazard", "compliance", "safety"],
63         "sub_questions": {
64             "hazard": question,
65             "compliance": question,
66             "safety": question,
67         },
68     }

```

The fallback on JSON parse failure routes to all specialists, which is safe: slightly slower, never wrong. It is better to over-route than to drop a domain.

8.6 The specialist agents

Each specialist is a focused invocation of the hybrid RAG chains from Chapter 7, configured with the correct retrieval strategy for its domain.

```

1  import logging
2  from langchain_google_genai import ChatGoogleGenerativeAI
3  from langchain_core.messages import HumanMessage
4
5  from agents.supervisor_state import SupervisorState
6  from agents.kg_chain import run_kg_chain
7  from agents.vector_chain import run_vector_chain
8  from agents.state import HybridRAGState
9
10 logger = logging.getLogger(__name__)
11
12 def _get_llm():
13     return ChatGoogleGenerativeAI(model="gemini-2.5-flash",
14                                   temperature=0.1, max_tokens=1024)
15
16 def _make_chain_state(sub_question: str, state: SupervisorState) ->
17     HybridRAGState:
18     """Build a HybridRAGState for a specialist's sub-question."""
19     return {
20         "question": sub_question,
21         "material_number": state["material_number"],

```

```

20     "history": [], # specialists do not use conversation history
21     "vector_answer": "", "vector_chunks": [],
22     "kg_answer": "", "kg_sparql": "", "kg_facts": [],
23     "final_answer": "", "sources": [], "error": None,
24 }
25
26 def hazard_agent(state: SupervisorState) -> dict:
27     """KG-focused: GHS codes, classifications, hazard statements.
28     Structured hazard data lives in the knowledge graph as precise
29     triples."""
30     sub_q = state["sub_questions"].get("hazard", state["question"])
31     chain_state = _make_chain_state(sub_q, state)
32     result = run_kg_chain(chain_state)
33     answer = result.get("kg_answer", "")
34     if not answer:
35         # Fall back to vector if KG found nothing
36         vec_result = run_vector_chain(chain_state)
37         answer = vec_result.get("vector_answer", "No hazard information
38 found.")
39     return {"hazard_answer": answer}
40
41 def compliance_agent(state: SupervisorState) -> dict:
42     """KG-focused with vector fallback: exposure limits, regulatory
43     thresholds.
44     Structured limits come from the KG; regulatory narrative comes from
45     vector search."""
46     sub_q = state["sub_questions"].get("compliance", state["question"])
47     chain_state = _make_chain_state(sub_q, state)
48     result = run_kg_chain(chain_state)
49     answer = result.get("kg_answer", "")
50     if not answer:
51         vec_result = run_vector_chain(chain_state)
52         answer = vec_result.get("vector_answer", "No compliance
53 information found.")
54     return {"compliance_answer": answer}
55
56 def safety_agent(state: SupervisorState) -> dict:
57     """Vector-focused: precautions, first aid, PPE, storage procedures.

```

```

53     Narrative safety procedures live in document prose – vector search
       retrieves them."""
54     sub_q = state["sub_questions"].get("safety", state["question"])
55     chain_state = _make_chain_state(sub_q, state)
56     result = run_vector_chain(chain_state)
57     answer = result.get("vector_answer", "")
58     if not answer:
59         kg_result = run_kg_chain(chain_state)
60         answer = kg_result.get("kg_answer", "No safety information found.")
61     return {"safety_answer": answer}

```

Each specialist wraps the existing chains from Chapter 7 — they are not new implementations, just focused invocations with the correct primary retrieval strategy. HazardAgent and ComplianceAgent use the KG chain first (structured facts are exactly what they need) and fall back to the vector chain if the KG returns nothing. SafetyAgent uses the vector chain first (narrative procedures live in prose) and falls back to KG.

8.7 The summary agent

```

1  def summary_agent(state: SupervisorState) -> dict:
2      """Synthesise specialist answers into a final coherent response."""
3      parts = []
4      if state.get("hazard_answer"):
5          parts.append(f"Hazard
6          Classification:**\n{state['hazard_answer']}")
7      if state.get("compliance_answer"):
8          parts.append(f"Regulatory
9          Compliance:**\n{state['compliance_answer']}")
10     if state.get("safety_answer"):
11         parts.append(f"Safety Procedures:**\n{state['safety_answer']}")
12
13     if not parts:
14         return {
15             "final_answer": (
16                 f"No information found for material
17                 {state['material_number']}). "

```



```

15         "Please verify the material number."
16     ),
17     "sources": [],
18 }
19
20 if len(parts) == 1:
21     # Single specialist answered – no synthesis needed
22     return {
23         "final_answer": list(parts)[0].split("\n", 1)[1],
24         "sources": ["Single specialist"],
25     }
26
27     combined = "\n\n".join(parts)
28     prompt = f"""You are an expert assistant for SAP material documents.
29 Multiple specialist agents have
30 analysed the question below and provided their answers.
31 {combined}
32
33 Original question: {state['question']}
34
35 Synthesise the above into a single, well-organised, professional answer.
36 - Use headings to separate sections
37 - Do not repeat information that appears in multiple answers
38 - Be specific with codes, values, and units
39 - Write for an enterprise user audience
40
41 Answer: """
42
43     response = _get_llm().invoke([HumanMessage(content=prompt)])
44     return {
45         "final_answer": response.content,
46         "sources": [f"{len(parts)} specialist agents"],
47     }

```

8.8 Building the supervisor graph

```

1  from concurrent.futures import ThreadPoolExecutor, as_completed
2  from langgraph.graph import StateGraph, END
3
4  from agents.supervisor_state import SupervisorState
5  from agents.supervisor import supervisor_node
6  from agents.specialist_agents import hazard_agent, compliance_agent,
   safety_agent, summary_agent
7
8  def parallel_specialists_node(state: SupervisorState) -> dict:
9      """Run only the needed specialists in parallel."""
10     needed = state.get("specialists_needed", ["hazard", "compliance",
11 "safety"])
12
13     agent_map = {
14         "hazard": hazard_agent,
15         "compliance": compliance_agent,
16         "safety": safety_agent,
17     }
18
19     results = {}
20     agents_to_run = {name: fn for name, fn in agent_map.items() if name in
21 needed}
22
23     with ThreadPoolExecutor(max_workers=len(agents_to_run)) as executor:
24         futures = {executor.submit(fn, state): name for name, fn in
25 agents_to_run.items()}
26
27         for future in as_completed(futures, timeout=60):
28             name = futures[future]
29             try:
30                 results.update(future.result(timeout=30))
31             except Exception as e:
32                 import logging
33                 logging.getLogger(__name__).error("Specialist %s failed:
34 %s", name, e)
35
36     return results
37
38 def build_supervisor_graph():

```

```

34     graph = StateGraph(SupervisorState)
35
36     graph.add_node("supervisor", supervisor_node)
37     graph.add_node("specialists", parallel_specialists_node)
38     graph.add_node("summary", summary_agent)
39
40     graph.set_entry_point("supervisor")
41     graph.add_edge("supervisor", "specialists")
42     graph.add_edge("specialists", "summary")
43     graph.add_edge("summary", END)
44
45     return graph.compile()
46
47 supervisor_app = build_supervisor_graph()

```

The graph is three nodes: supervisor → specialists (parallel) → summary. The `parallel_specialists_node` runs only the specialists that the supervisor selected, using a `ThreadPoolExecutor` sized to the number of active specialists. If the supervisor selects only two specialists for a focused question, only two threads are created — not three. This avoids unnecessary API calls.

8.9 When to use the supervisor vs the direct agent

The supervisor adds latency — one extra LLM call for routing and one for synthesis. For simple questions, this overhead is not justified.

Question type	Recommended agent	Why
Single-domain factual ("what are the hazard codes?")	Direct hybrid RAG (Ch 7)	Faster, no routing overhead
Single-domain narrative ("what is the first aid?")	Direct hybrid RAG (Ch 7)	Faster, single retrieval pass is sufficient
Multi-domain factual + narrative ("codes AND precautions")	Supervisor	Each domain gets focused retrieval

Question type	Recommended agent	Why
Complex regulatory + safety question	Supervisor	Prevents context dilution
Real-time chat interface	Direct hybrid RAG (Ch 7)	Latency matters in chat
Batch compliance reports	Supervisor	Quality matters more than speed

A practical rule: if the question contains “and” connecting two distinct domains, use the supervisor.

8.10 The /query-advanced endpoint

Add the supervisor endpoint to `agents/main.py`:

```

1  from agents.supervisor import supervisor_app
2  from agents.supervisor_state import SupervisorState
3
4  class AdvancedQueryRequest(BaseModel):
5      question: str
6      material_number: str
7      history: List[dict] = []
8      use_supervisor: bool = False
9
10     @validator("material_number")
11     def validate_material(cls, v):
12         if not re.match(r"^[A-Za-z0-9_-]+$", v):
13             raise ValueError("material_number contains invalid characters")
14         return v
15
16     @app.post("/query-advanced", response_model=QueryResponse)
17     def query_advanced(request: AdvancedQueryRequest):
18         if not request.use_supervisor:
19             # Fall back to direct hybrid RAG
20             return query(QueryRequest(
21                 question=request.question,
```

```

22         material_number=request.material_number,
23         history=request.history,
24     ))
25
26     state: SupervisorState = {
27         "question": request.question,
28         "material_number": request.material_number,
29         "history": request.history,
30         "sub_questions": {},
31         "specialists_needed": [],
32         "hazard_answer": "",
33         "compliance_answer": "",
34         "safety_answer": "",
35         "final_answer": "",
36         "sources": [],
37         "error": None,
38     }
39
40     result = supervisor_app.invoke(state)
41
42     return QueryResponse(
43         answer=result.get("final_answer", ""),
44         sources=result.get("sources"),
45     )

```

The `use_supervisor` flag lets the caller choose which agent to use. The Fiori UI sends `use_supervisor: true` when it detects certain patterns in the question (multiple question marks, keywords like “and”, “also”, “as well as”). For simple questions it uses the faster direct endpoint.

8.11 Testing: a question that requires all three specialists

```

1 curl -X POST http://localhost:8000/query-advanced \
2     -H "Content-Type: application/json" \
3     -d '{

```

```

4     "question": "I am setting up a new lab that will store acetone. What
      hazard classifications do I need to post, what are the legal exposure
      limits for my workers, and what PPE should I provide?",
5     "material_number": "ACE001",
6     "history": [],
7     "use_supervisor": true
8   }'

```

Expected supervisor routing (logged to console):

```
INFO: Supervisor routed to: ['hazard', 'compliance', 'safety']
```

```
INFO: Sub-questions:
```

```
  hazard:    "What GHS hazard codes and classifications apply to ACE001?"
```

```
  compliance: "What are the worker exposure limits for ACE001?"
```

```
  safety:    "What PPE is required when storing and handling ACE001?"
```

```
INFO: hazard specialist completed in 2.1s
```

```
INFO: compliance specialist completed in 1.9s
```

```
INFO: safety specialist completed in 2.3s
```

```
INFO: All specialists completed in 2.3s (parallel)
```

```
INFO: Summary agent completed in 1.4s
```

The three specialists run in parallel. The total wall-clock time for specialists is 2.3 seconds (the slowest), not 6.3 seconds (the sum). The summary adds 1.4 seconds, for a total of about 3.7 seconds.

Compare this to the direct hybrid RAG agent with the same question: the single retrieval pass would retrieve 5 passages from across all three domains, giving the answer approximately 2 passages per domain. The supervisor gives each domain 5 full passages, producing a noticeably more detailed and better-organised answer — especially critical when the question spans GHS codes (structured, from KG), regulatory limits (structured, from KG), and PPE procedures (narrative, from vector store).

8.12 Applying the pattern to other SAP document types

The multi-agent pattern shown here with MSDS specialisations applies directly to other SAP document types. The supervisor, the state machine, and the merge logic are identical — only the specialist prompts and KG predicates change.

Batch certificates: A CertificationAgent checks test results against specification limits using the KG (structured test values and limits are natural graph triples). A SupplierAgent resolves supplier identity from the KG using supplier-linked predicates. A ComplianceAgent checks whether results satisfy regulatory boundaries by querying both the KG for the regulation threshold and the vector store for the compliance narrative.

Purchase orders and invoices: A HeaderAgent extracts invoice metadata (vendor, date, currency, document number) from the KG, where these are stored as structured triples. A LineItemAgent retrieves individual line item details using vector search against the prose of the invoice. A PaymentAgent queries both the KG for payment term codes and the vector store for any payment instruction narrative.

Quality inspection reports: A ResultsAgent queries the KG for structured test outcome triples (pass/fail verdicts, measured values). A SpecificationAgent queries the KG for specification limits linked to the material. A NonConformanceAgent uses vector search to retrieve the narrative description of any quality issues identified in the report.

In every case: questions about structured, enumerable facts (codes, numbers, identifiers, classifications) route to KG-heavy specialists. Questions about narrative, procedural, or descriptive content route to vector-heavy specialists. Questions that span both route to specialists that use both retrieval strategies. The supervisor makes this routing decision dynamically at query time, without any hard-coded rules.

8.13 Summary

In this chapter we extended the system from a single agent to a coordinated multi-agent system:

- Identified the **complexity ceiling** of single-agent architectures for multi-domain enterprise questions
- Explained **why different document sections require different retrieval strategies** — structured KG for facts, vector search for procedures, both for compliance
- Designed three **specialist agents** (HazardAgent, SafetyAgent, ComplianceAgent) as MSDS-specific instances of a general pattern
- Built a **supervisor node** that uses Gemini 2.5 Flash to decompose questions and route sub-questions dynamically

- Ran specialists **in parallel** using `ThreadPoolExecutor`, keeping wall-clock latency equal to the slowest specialist
- Implemented the **SummaryAgent** to synthesise multi-domain answers into coherent final responses
- Added a `/query-advanced` endpoint with a `use_supervisor` flag for caller-controlled routing
- Defined clear criteria for **when to use the supervisor** vs the direct hybrid RAG agent
- Showed how **the pattern generalises** to other SAP document types — batch certificates, invoices, inspection reports

The supervisor, state machine, and merge logic are reusable infrastructure. Deploying this pattern for a new document type requires writing three specialist prompts and identifying the relevant KG predicates — the orchestration architecture requires no changes.

8.14 Checkpoint

Before continuing to Chapter 9, verify the following:

```

1  # 1. Supervisor graph compiles
2  cd agents
3  python -c "
4  from agents.supervisor import build_supervisor_graph
5  app = build_supervisor_graph()
6  print('Supervisor graph OK, nodes:', list(app.get_graph().nodes.keys()))
7  "
8
9  # 2. Supervisor routes correctly on a simple question
10 python -c "
11 from agents.supervisor import supervisor_node
12 state = {
13     'question': 'What are the hazard codes for acetone?',
14     'material_number': 'ACE001',
15     'history': [],
16     'sub_questions': {}, 'specialists_needed': [],
17     'hazard_answer': '', 'compliance_answer': '', 'safety_answer': '',

```



```
18     'final_answer': '', 'sources': [], 'error': None,
19 }
20 result = supervisor_node(state)
21 print('specialists needed:', result['specialists_needed'])
22 "
23
24 # 3. The advanced endpoint responds
25 curl -s -X POST http://localhost:8000/query-advanced \
26     -H 'Content-Type: application/json' \
27     -d '{
28         "question": "What are the GHS hazard codes for acetone?",
29         "material_number": "ACE001",
30         "history": [],
31         "use_supervisor": false
32     }' | python -m json.tool
```

If the supervisor graph compiles cleanly, the routing test assigns only ["hazard"] or a small subset of specialists to a simple question, and the curl returns a valid JSON response — the multi-agent system is working. Chapter 9 exposes this entire backend through the SAP CAP OData V4 service with the Fiori Elements UI.

End of Chapter 8

Chapter 9: CAP Node.js OData V4 — The SAP-Native API Layer + Fiori Elements UI

The CAP Node.js service does three things: it provides a Fiori Elements UI that SAP users already know how to use, it enforces material-number validation against real S/4HANA product master data, and it proxies document processing and chat queries to the FastAPI agent backend. This separation is deliberate — SAP enterprise standards (OData V4, Fiori UX, BTP security) live in the CAP layer; AI orchestration and HANA data access live in the Python layer.

Chapters 7 and 8 built the intelligence engine: a Python FastAPI service that receives a question, dispatches it to parallel retrieval chains, and returns a grounded answer. That engine is a standard Python web service. It knows nothing about Fiori, OData, or SAP authentication. To bring this capability into an SAP enterprise environment — where business users work in SAP Fiori applications, where material numbers are validated against S/4HANA product master data, and where access is controlled by BTP security services — you need a second layer. That layer is CAP.

By the end of this chapter you will have a complete, deployable front-end: a Fiori Elements list report showing all uploaded material documents with their processing status, an object page with action buttons to process files and run queries, and an integrated chat interface that calls the hybrid RAG agent and returns answers with the retrieval path shown. No custom JavaScript UI code required.

9.1 Why CAP?

Before writing a single line of code, it is worth understanding why CAP is the right choice for this layer rather than a simple Express server or a Next.js front-end.

OData V4 compatibility. Every SAP system — S/4HANA, SuccessFactors, BTP Integration Suite — speaks OData. By exposing your agent through a CAP service you make it consumable by any SAP client, including Fiori Elements, SAP Analytics Cloud, and ABAP programs via HTTP calls. A plain REST API would require a custom client in every consumer; an OData endpoint works out of the box.

Fiori Elements without custom code. SAP Fiori Elements is a UI framework that generates complete, SAP-consistent user interfaces from OData metadata and annotations. You do not write HTML, CSS, or JavaScript for the list report or the object page — you write CDS annotations, and the framework generates the UI at runtime. For an internal tool, this means a production-quality interface with zero front-end development effort.

Real S/4HANA product master integration. The Material Number field in the Fiori UI shows a value help backed by real S/4HANA product data via the `API_PRODUCT_SRV` external service. When a user uploads a document, they select the material from their actual SAP product master. This closes the loop between the AI layer on BTP and the system of record in S/4HANA — and it is what makes this a genuine enterprise integration rather than a standalone demo.

Integrated attachment handling. The `@cap-js/attachments` plugin gives you a fully managed file upload and retrieval mechanism that stores binaries in the configured database and exposes them through the OData protocol. Without it, you would need to write your own multipart upload handler, storage integration, and metadata tracking. With it, you write two lines of CDS.

Deployment to BTP Cloud Foundry. A CAP service is designed to run on SAP BTP. The `@sap/cds` framework handles XSUAA authentication, SAP HANA connection management via `@cap-js/hana`, and binding to SAP BTP services through the standard `VCAP_SERVICES` mechanism. When you deploy to BTP, you get enterprise-grade authentication and database connection pooling with no additional configuration.

The key decision in this architecture is what CAP does *not* do: it does not run the Python agent. CAP handles the OData protocol, the attachment storage, and the Fiori UI. The Python FastAPI service handles LangGraph orchestration, SPARQL queries, and vector search. This separation keeps each service focused on what it does best.

9.2 The two-process architecture

Understanding that CAP and FastAPI are separate processes is the most important architectural concept in this chapter. Many developers assume that because a CAP service can call external APIs, the agent logic should be imported into the Node.js process. That assumption leads to a tightly coupled system that is difficult to test, impossible to scale

independently, and requires Node.js developers to maintain Python code.

The correct architecture is clean separation:

```
User browser
  |
  | HTTP (Fiori Elements, OData V4)
  v
CAP Node.js (port 4004)
  |
  | HTTP (axios, JSON / multipart)
  v
Python FastAPI (port 8000)
  |
  |-- HANA Cloud (vector store, knowledge graph)
  |-- Google Vertex AI (Gemini LLM + text-embedding)
  |-- LangGraph (agent orchestration)
```

The CAP service is the API gateway and UI host. It knows how to speak OData and how to manage CDS entities. It knows nothing about SPARQL, embeddings, or LangGraph. The FastAPI service is the intelligence engine. It knows nothing about OData, Fiori, or attachment storage.

Communication between the two is intentionally simple: HTTP with JSON bodies for queries, HTTP with multipart form data for file uploads. The `BACKEND_URL` environment variable controls where CAP sends its requests. In local development this is `http://localhost:8000`. On BTP it is the URL of the deployed FastAPI CF application.

This separation has three practical benefits. First, you can develop and test each service independently — mock the CAP service when testing the Python agent, and stub the agent when testing the CAP service. Second, you can scale them independently — the agent is CPU and memory intensive during LLM calls, while CAP is lightweight. Third, you can replace either without touching the other — swap the Python agent for a different implementation and the CAP service requires no changes, as long as the HTTP contract is preserved.

9.3 The CDS data model

The data model lives in `db/schema.cds`. It defines two entities and a status type.

The namespace `msds.kg` reflects the first document type in our implementation — Material Safety Data Sheets, processed into a knowledge graph. In a production deployment serving multiple document categories (invoices, batch certificates, quality inspection reports, maintenance manuals, legal filings), you would use a broader namespace — `documents.kg` or `materials.intelligence` — and extend the schema with a `documentType` field. The core entity structure is identical regardless: `materialNumber` as the anchor, `attachments` as the document store, status tracking fields for both pipeline stages.

```

1 namespace msds.kg;
2 using { managed } from '@sap/cds/common';
3 using { Attachments } from '@cap-js/attachments';
4
5 type Status : String(20) enum {
6     Pending      = 'Pending';
7     Processing   = 'Processing';
8     Completed    = 'Completed';
9     Error        = 'Error';
10 }
11
12 entity FileAttachments : Attachments {
13     up_ : Association to Files;
14 }
15
16 entity Files : managed {
17     key materialNumber      : String(40);
18     status                  : Status default 'Pending';
19     fileHash                : String(64) @assert.unique;
20     triples                 : Integer default 0;
21     vectors                 : Integer default 0;
22     vectorStatus            : Status default 'Pending';
23     errorMessage            : String(500);
24     processingStartedAt     : DateTime;
25     retryCount              : Integer default 0;
26     attachments             : Composition of many FileAttachments on
    attachments.up_ = $self;

```

```
27 }
```

The `Status` type is declared as a CDS enum. This matters because enums generate value-restricted OData properties, which Fiori Elements renders as proper dropdown fields with validation. The string values (`'Pending'`, `'Processing'`, etc.) are what get stored in the database and sent over the wire.

The `Files` entity extends `managed`, which is a built-in CDS aspect from `@sap/cds/common`. This single word gives you four fields for free: `createdAt`, `createdBy`, `modifiedAt`, and `modifiedBy`. The framework populates them automatically on every insert and update. You never need to write that logic yourself.

The dual-status design — `status` for the knowledge graph pipeline and `vectorStatus` for the vector pipeline — reflects a real operational requirement. The two pipelines run independently. A document can have its triples extracted successfully while the vector embedding is still running, or vice versa. Tracking them separately means your UI can show accurate, granular progress rather than a single boolean that masks which step failed.

`fileHash` carries the `@assert.unique` annotation. This is a CAP-level constraint that generates a database unique index and returns a meaningful OData error if you attempt to upload the same file twice. Without this, operators could accidentally trigger redundant processing.

The `FileAttachments` entity extends `Attachments` from `@cap-js/attachments`. This is the entire attachment implementation — one line of CDS. The plugin handles binary storage, MIME type detection, streaming upload and download, and exposes the attachment content through the standard OData `$value` endpoint. The `up_` association links each attachment back to its parent `Files` record, following the standard CAP composition pattern.

The `attachments` field on `Files` is a `Composition` of many `FileAttachments`. Compositions in CDS mean deep operations: when you delete a `Files` record, all its attachments are deleted automatically. When you read a `Files` record with `$expand=attachments`, you get the attachment metadata in a single request.

9.4 The service definition

The service is defined in `srv/service.cds`. This file does three things: it projects the data model entities into the service, it adds virtual fields for UI display, and it declares the actions and functions that map to agent operations.

```

1  using { msds.kg as db } from '../db/schema';
2
3  @path: '/odata/v4/documents'
4  service DocumentService {
5      @odata.draft.enabled
6      entity Documents as projection on db.Files {
7          *,
8          virtual kgDisplay          : String,
9          virtual vecDisplay         : String,
10         virtual statusCriticality   : Integer,
11         virtual vectorStatusCriticality : Integer
12     }
13     actions {
14         action processFile() returns { status: String; message: String;
15         attachmentCount: Integer };
16         @(Common.IsActionCritical: true)
17         action deleteFile() returns { status: String; message: String };
18     };
19     action chatQuery(message: String, materialNumber: String) returns {
20         answer: String; path_used: many String };
21
22     function pollStatus(materialNumber: String) returns {
23         status    : String;
24         triples   : Integer;
25         vectors   : Integer;
26         kg_done   : Boolean;
27         vec_done  : Boolean;
28     };
29 }

```

The `@path` annotation sets the OData service root URL. All entity sets, actions, and functions under this service will be accessible at `/odata/v4/documents`. This path is regis-

tered by `@sap/cds` automatically when the server starts.

`@odata.draft.enabled` on the `Documents` entity activates SAP Fiori draft handling. Draft is the standard pattern for transactional data entry in Fiori applications. The framework saves partial edits server-side before the user explicitly confirms them — preventing data loss on accidental navigation and matching how SAP Fiori works for purchase orders, material master records, and every other transactional document in the SAP ecosystem. In practice, for this application it means the create-new-document flow works correctly in the Fiori UI without any additional handler code — users can enter the material number, attach the PDF, and save as draft before committing to processing.

The `virtual` fields deserve particular attention. A virtual field is projected into the OData response but has no corresponding database column. The values are computed in the service handler at read time. Here, `kgDisplay` and `vecDisplay` are formatted strings like “Completed (1,247 triples)” that combine the status and the count into a single display value. The `statusCriticality` and `vectorStatusCriticality` fields are integers (0–3) that Fiori Elements interprets as colour codes: 0 is neutral, 1 is error (red), 2 is warning (orange), 3 is positive (green). By computing criticality as a virtual field, you keep the colour logic in the service layer rather than scattering it across annotations.

The distinction between `action` and `function` in OData V4 is important. Actions have side effects — they modify state. Functions are read-only — they return computed data without modifying anything. `processFile` and `deleteFile` are bound actions: they operate on a specific `Documents` entity instance, identified by its key. `chatQuery` is an unbound action: it accepts parameters directly and operates at the service level, not on a specific entity. `pollStatus` is an unbound function: it retrieves current status from the backend without changing anything, making it safe to call repeatedly in a polling loop.

The `@(Common.IsActionCritical: true)` annotation on `deleteFile` tells Fiori Elements to show a confirmation dialog before invoking the action. The user must explicitly confirm the destructive operation. One annotation; no custom dialog code.

9.5 The Products entity and S/4HANA value help

The `DocumentService` extends beyond the `Files` entity. The service also exposes a `Products` entity sourced from the `API_PRODUCT_SRV` external service — the standard

SAP OData API for product master data in S/4HANA.

This is architecturally significant. The `materialNumber` field on the `Documents` entity carries a `@Common.ValueList` annotation that points to the `Products` entity. When a business user opens the Fiori UI to upload a new document and clicks into the Material Number field, the Fiori value help fires an OData request to `DocumentService/Products`. CAP proxies that request to the configured S/4HANA system and returns the matching product master records.

What this means operationally: a quality manager uploading a batch certificate does not type a material number from memory — they search and select from their actual SAP product master. The material number that anchors the document in the knowledge graph and vector store is the same material number that exists in their S/4HANA system. When a question comes back through `chatQuery` — “What are the storage conditions for this material?” — the answer is grounded in data that is tied to the real product record.

This is what makes this a genuine enterprise integration rather than a standalone demo. The AI capability on BTP is connected to the system of record in S/4HANA through standard SAP OData APIs and standard BTP Destination Service connectivity. Changing the S/4HANA destination in the BTP Cockpit is the only configuration needed to point this system at a different S/4HANA landscape.

9.6 The callBackend helper

Every handler in `srv/service.js` that needs to communicate with the Python agent uses the same `callBackend` helper function. This centralisation matters: the URL of the backend, the authentication headers, the error handling, and the timeout configuration all live in one place.

```
1 const BACKEND_URL = process.env.BACKEND_URL || "http://localhost:8000";
2
3 async function callBackend({ method, path, jsonBody, formData }) {
4   const response = await axios.request({
5     method,
6     url: `${BACKEND_URL}${path}`,
```

```

7     headers: formData ? formData.getHeaders() : { "Content-Type":
      "application/json" },
8     data: formData || jsonBody,
9   });
10   return response.data;
11 }

```

The `BACKEND_URL` environment variable is read once at module load time. In local development you set this to `http://localhost:8000` in a `.env` file in the `cap-srv/` directory, and `@sap/cds` loads it automatically via `dotenv`. On BTP, you set it as an environment variable in your Cloud Foundry application manifest or as a user-provided service binding. The fallback to `http://localhost:8000` means the service starts without any configuration during development — the fallback is safe because there is no production environment where port 8000 is the correct URL.

The `formData ? formData.getHeaders()` conditional is the key to handling two different call types with one function. When `formData` is provided, the request is a multipart upload and `axios` must use the boundary-aware headers that `form-data` generates. When `jsonBody` is provided, the content type is `application/json`. Getting this wrong produces a 400 Bad Request from the FastAPI endpoint.

`Axios` is used here rather than the native Node.js `fetch` because `form-data` integrates with `axios` through the `getHeaders()` and pipe mechanism. Node.js 18+ `fetch` supports `FormData`, but the `@cap-js/attachments` binary retrieval returns a Node.js `Buffer`, and converting that buffer to a browser-compatible `FormData` in Node.js requires additional tooling. `Axios` with the `form-data` package is the simpler path.

9.7 Action handlers

processFile

When a user clicks Process File in the Fiori UI, they are initiating the full ingestion pipeline. The Fiori button fires an OData action request to the CAP service. CAP reads the PDF from the attachment store, constructs a multipart request, and POSTs it to the FastAPI backend at `/process-upload`. FastAPI returns 202 Accepted immediately — it has queued the job but not finished it. CAP updates the `Files` record to `Processing` and

returns to the Fiori UI. The Fiori UI then begins polling `pollStatus()` on a timer. As the FastAPI pipeline runs — chunking the PDF, generating triples, embedding passages — the `triples` and `vectors` counts climb until both pipelines report completion. The Fiori UI reflects those counts as they update, giving the operator a live view of ingestion progress without any WebSocket infrastructure.

The `processFile` action is the most complex handler because it bridges two protocols: OData binary retrieval and HTTP multipart upload. The handler must read the attachment binary from the CAP attachment store, then stream it to the FastAPI `/process-upload` endpoint as a multipart request.

```
1  srv.on("processFile", Documents, async (req) => {
2    const { materialNumber } = req.params[0];
3
4    // Retrieve attachment metadata from the CAP entity
5    const attachments = await SELECT.from(db.Files, materialNumber)
6      .columns("attachments");
7
8    if (!attachments || attachments.attachments.length === 0) {
9      return req.error(400, "No attachment found. Please upload a PDF
10       first.");
11    }
12
13    const attachment = attachments.attachments[0];
14
15    // Read the binary content through the @cap-js/attachments API
16    const content = await cds.run(SELECT.one.from(db.FileAttachments)
17      .where({ ID: attachment.ID }));
18
19    const buffer = content.content; // Buffer from the attachment plugin
20    const filename = attachment.filename || `${materialNumber}.pdf`;
21    const contentType = attachment.mimeType || "application/pdf";
22
23    // Build multipart form data for the FastAPI endpoint
24    const FormData = require("form-data");
25    const formData = new FormData();
26    formData.append("file", buffer, { filename, contentType });
27    formData.append("graphName", materialNumber);
```

```

28   await callBackend({ method: "POST", path: "/process-upload", formData
    });
29
30   // Mark the record as processing
31   await UPDATE(db.Files).set({
32     status: "Processing",
33     vectorStatus: "Processing",
34     processingStartedAt: new Date().toISOString()
35   }).where({ materialNumber });
36
37   return {
38     status: "processing",
39     message: "File submitted for processing.",
40     attachmentCount: attachments.attachments.length
41   };
42 });

```

The `req.params[0]` contains the entity key of the bound action target — in this case `{ materialNumber: 'ACETONE-001' }`. This is the OData V4 pattern for bound actions: the key is part of the action URL path, not the request body.

The handler updates the record status to `Processing` immediately after submitting to the backend. This is an optimistic update — it reflects what should happen, not what has completed. The actual status progression (Processing to Completed or Error) happens asynchronously as the Python agent runs. The `pollStatus` function and handler described below are how the UI tracks that progression.

chatQuery

The `chatQuery` action is the core business value of this entire system. A quality manager, safety officer, or procurement analyst opens the document object page, types a natural language question — “What is the recommended storage temperature for this material?” or “Are there any regulatory restrictions on transport by air?” — and gets a sourced answer directly in the Fiori UI. The retrieval path shown alongside the answer tells them whether it came from the structured knowledge graph, the vector store, or both. No custom search interface. No manual document reading. No specialist tool to learn.

```

1   srv.on("chatQuery", async (req) => {

```

```

2   const { message, materialNumber } = req.data;
3
4   const result = await callBackend({
5     method: "POST",
6     path: "/query",
7     jsonBody: {
8       question: message,
9       material_number: materialNumber,
10      chat_history: []
11    }
12  });
13
14  return {
15    answer: result.answer,
16    path_used: result.path_used || []
17  };
18  });

```

The `chat_history: []` field sends an empty history on every call. This makes every `chatQuery` invocation stateless from the agent's perspective. For a document Q&A system where questions are typically independent lookups, stateless queries are correct. If you were building a multi-turn conversational interface, you would maintain a conversation ID in the client and accumulate history on the server side. That is a separate concern — `chatQuery` can be extended to accept a `sessionId` parameter without changing anything in the action definition or the backend API contract.

The `path_used` return value is an array of strings that the Python agent populates with the retrieval path it took: `["kg_chain"]`, `["vector_chain"]`, or `["kg_chain", "vector_chain"]`. Surfacing this in the UI gives users transparency into the agent's reasoning — they can see whether the answer came from the knowledge graph, the vector store, or both.

deleteFile

The `deleteFile` handler triggers a cascading cleanup: remove the knowledge graph triples from HANA, remove the vector embeddings from HANA, then delete the CAP record and its attachments.

```

1  srv.on("deleteFile", Documents, async (req) => {
2    const { materialNumber } = req.params[0];
3
4    // Instruct the Python backend to clean up graph and vector data
5    await callBackend({
6      method: "DELETE",
7      path: `/files/${encodeURIComponent(materialNumber)}`
8    });
9
10   // Delete the CAP record – composition cascades to FileAttachments
11   await DELETE.from(db.Files).where({ materialNumber });
12
13   return { status: "deleted", message: `${materialNumber} removed.` };
14 });

```

The `DELETE.from(db.Files)` call cascades automatically to the `FileAttachments` composition because CDS compositions enforce deep delete semantics. The `@cap-js/attachments` plugin hooks into the delete event and removes the binary files from storage. You do not need to explicitly delete the attachments — the composition and the plugin handle it.

pollStatus

The `pollStatus` function is called by the Fiori UI on a timer after a `processFile` action to track progress. It calls the FastAPI `/status/{materialNumber}` endpoint and passes the response through.

```

1  srv.on("pollStatus", async (req) => {
2    const { materialNumber } = req.data;
3
4    const result = await callBackend({
5      method: "GET",
6      path: `/status/${encodeURIComponent(materialNumber)}`
7    });
8
9    // Sync the live status back to the CAP entity
10   if (result.kg_done || result.vec_done) {
11     await UPDATE(db.Files).set({

```

```

12     status: result.kg_done ? "Completed" : result.status,
13     vectorStatus: result.vec_done ? "Completed" : result.vectorStatus,
14     triples: result.triples || 0,
15     vectors: result.vectors || 0
16   }).where({ materialNumber });
17 }
18
19 return result;
20 });

```

The handler does two things: it returns the live status to the caller, and it writes the current counts back to the database. This write-through is important for the Fiori list report: the list refreshes from the OData entity, not from a live WebSocket feed. By syncing the counts on every poll, the list view stays accurate after polling completes.

9.8 Fiori Elements UI

Fiori Elements is a metadata-driven UI framework. Instead of writing a React or SAPUI5 application that fetches OData and renders HTML, you write CDS annotations that describe *what* your data means and *how* you want it presented. The framework reads the OData metadata document (which CDS generates from your service definition) and the annotation document, and assembles the UI at runtime.

This has a profound implication for development speed. A complete list report with search, sorting, and export requires approximately 50 lines of CDS annotations. The equivalent hand-written SAPUI5 application is around 500 lines of XML views and controllers, plus another few hundred lines of JavaScript. For internal tools and prototypes, Fiori Elements is consistently the better choice.

The trade-off is customisation ceiling. If you need a completely custom layout — a drag-and-drop canvas, a real-time chart with WebSocket data, a custom color scheme — Fiori Elements will fight you. For standard enterprise UI patterns — list reports, object pages, form inputs, action buttons — it covers everything you need.

For our document management and Q&A interface, the standard patterns are exactly right:

- A **list report** showing all documents with their status, triple count, and upload

date

- An **object page** for each document showing its details, attachments, and action buttons
- A **chat section** on the object page where users can submit questions and see answers

All of this is generated from the CDS service definition and the annotations in `srv/annotations.cds`. There is no separate UI project, no `package.json` for the front-end, and no custom JavaScript.

The CAP + Fiori layer means this AI capability fits natively into any SAP Fiori launchpad. A quality manager, safety officer, or procurement analyst uses the same Fiori UX patterns they use across every other SAP application. The AI is invisible infrastructure — the user just asks a question and gets an answer.

9.9 Annotations explained

The annotations file `srv/annotations.cds` is where the UI is configured. It is separate from the service definition for a practical reason: the service definition describes what your API *does*, and the annotations describe how it *looks*. Keeping them separate means you can modify the UI without changing the API contract.

HeaderInfo

```

1  annotate service.Documents with @(
2      UI.HeaderInfo: {
3          TypeName: 'Document',
4          TypeNamePlural: 'Documents',
5          Title: { Value: materialNumber },
6          Description: { Value: status }
7      }
8  );

```

`HeaderInfo` controls three things: the singular and plural names used in page titles and breadcrumbs, the main title shown on the object page header, and the subtitle beneath it. Setting `Title` to `materialNumber` means each document's object page is headed by its material number. Setting `Description` to `status` shows the current processing status

directly in the page header, so the user knows at a glance whether the document has been processed.

SelectionFields and LineItem

```

1      UI.SelectionFields: [ materialNumber, status ],
2      UI.LineItem: [
3          { Value: materialNumber, Label: 'Material Number' },
4          { Value: kgDisplay, Label: 'KG Status', Criticality:
statusCriticality },
5          { Value: vecDisplay, Label: 'Vec Status', Criticality:
vectorStatusCriticality },
6          { Value: createdAt, Label: 'Uploaded On' }
7      ],

```

`SelectionFields` defines which fields appear in the filter bar above the list. Choosing `materialNumber` and `status` means users can search for a specific material or filter to show only documents in a particular state.

`LineItem` defines the columns in the list table. The `kgDisplay` and `vecDisplay` fields carry the `Criticality` annotation pointing to the integer fields computed in the service handler. Fiori Elements renders criticality as a colored status indicator: integers map to red, orange, and green automatically. The display strings computed in the handler — “Completed (1,247 triples)” — appear in the column alongside the color indicator. No custom cell renderer needed.

Identification actions

```

1      UI.Identification: [
2          {
3              $Type: 'UI.DataFieldForAction',
4              Action: 'DocumentService.processFile',
5              Label: 'Process File',
6              Criticality: #Positive
7          },
8          {
9              $Type: 'UI.DataFieldForAction',
10             Action: 'DocumentService.deleteFile',

```

```

11         Label: 'Delete File',
12         Criticality: #Negative
13     }
14 ],

```

The `Identification` collection defines the action buttons that appear in the object page header toolbar. `UI.DataFieldForAction` binds a button to an OData action by its qualified name. The `Criticality` annotation sets the button color: `#Positive` renders as green (constructive action), `#Negative` renders as red (destructive action). Combined with the `Common.IsActionCritical` annotation on `deleteFile` in the service definition, the delete button is both red and protected by a confirmation dialog — two layers of friction for a destructive operation.

FieldGroup

```

1     UI.FieldGroup#GeneralInfo: {
2         Data: [
3             { Value: materialNumber },
4             { Value: status },
5             { Value: triples, Label: 'Triples Extracted' },
6             { Value: vectors, Label: 'Vectors Stored' },
7             { Value: vectorStatus, Label: 'Vector Status' }
8         ]
9     }

```

`FieldGroup` with an identifier (`#GeneralInfo`) defines a named group of fields. These groups are then referenced in a `Facets` annotation to assemble the object page layout. Each `Data` entry becomes a labeled field in the form. The `triples` and `vectors` integers show the current counts from the last successful processing run, giving operators the information they need to assess pipeline health.

9.10 Running locally

The local setup requires two terminal sessions — one for each process.

Terminal 1: Start the Python FastAPI agent

```
1 cd agents/  
2 source .venv/bin/activate  
3 uvicorn main:app --reload --port 8000
```

Confirm it is running by visiting <http://localhost:8000/docs> and checking that the `/query`, `/process-upload`, and `/status/{material_number}` endpoints are listed.

Terminal 2: Start the CAP service

First, create a `.env` file in `cap-srv/` if one does not exist:

```
BACKEND_URL=http://localhost:8000
```

Then start the server:

```
1 cd cap-srv/  
2 npm install  
3 cds serve
```

The `cds serve` command reads your service definition, sets up the SQLite database (for local development — replace with HANA on BTP), loads all handlers, and starts the Express server on port 4004. You will see output like:

```
[cds] - loaded model from 3 file(s):  
  db/schema.cds  
  srv/service.cds  
  srv/annotations.cds  
  
[cds] - connect to db > sqlite { database: ':memory:' }  
[cds] - serving DocumentService { at: '/odata/v4/documents' }  
  
[cds] - server listening on { url: 'http://localhost:4004' }
```

Open <http://localhost:4004> to see the CDS welcome page, which lists all registered services and their metadata endpoints. Open [http://localhost:4004/\\$fiori-preview](http://localhost:4004/$fiori-preview) to access the Fiori Elements preview sandbox — a full Fiori application running against your local OData service with no deployment required.

For the Fiori Elements preview to show all features, ensure you access it via the full URL including the Fiori launchpad hash, which the welcome page provides as a clickable link.

9.11 Testing the full flow

With both services running, walk through the complete flow from document upload to agent query.

Step 1: Create a new document record. In the Fiori list report, click the New button. The draft mechanism activates. Click into the Material Number field — a value help opens, backed by the Products entity from API_PRODUCT_SRV. Search for and select your material. Save the draft. The record appears in the list with status `Pending`.

Step 2: Upload a PDF attachment. Navigate to the object page for your material. In the Attachments section, click Upload and select a PDF — an MSDS, SDS, or any document relevant to that material. The attachment is stored immediately via `@cap-js/attachments`.

Step 3: Process the file. Click the Process File button in the header toolbar. The handler reads the attachment binary, constructs a multipart form data request, and POSTs it to the Python agent at `http://localhost:8000/process-upload`. The record status changes to `Processing` immediately.

Step 4: Watch the dual status. Open a separate terminal and call `pollStatus` manually to watch the counts grow:

```
1 curl
  "http://localhost:4004/odata/v4/documents/pollStatus(materialNumber='ACETC IE-001')"
```

You will see `kg_done` and `vec_done` flip to `true` as each pipeline completes, and the `triples` and `vectors` counts increment. The Fiori object page, if you have it open and refresh it, shows the same values updating.

Step 5: Run a query. Once both pipelines are complete, use the `chatQuery` action. In the Fiori UI this can be triggered via an action dialog bound to the `chatQuery` unbound action. From the command line:

```
1 curl -X POST http://localhost:4004/odata/v4/documents/chatQuery \
2   -H "Content-Type: application/json" \
```

```
3 -d '{"message": "What are the first aid measures for skin contact?",
    "materialNumber": "ACETONE-001"}'
```

The response includes the `answer` field with the agent's response and the `path_used` array indicating which retrieval chains contributed: `["kg_chain", "vector_chain"]` means both paths were used, which is the expected behavior for this type of factual question about a chemical document.

9.12 What you can do after this chapter

The system is now complete from ingestion to query to UI. You have:

- A Python FastAPI service that runs the hybrid RAG agent with parallel KG and vector retrieval
- A CAP OData V4 service that manages document records, attachment storage, and proxies requests to the agent
- A Fiori Elements UI generated entirely from CDS annotations
- A value help on Material Number backed by real S/4HANA product master data via `API_PRODUCT_SRV`

The natural next steps are:

Add conversational history to chatQuery. Extend the `chatQuery` action to accept an optional `sessionId` parameter. Store conversation turns in a new CDS entity `ChatSession`. On each call, retrieve the session history and pass it to the FastAPI `/query` endpoint as `chat_history`. This turns a single-shot Q&A into a stateful conversation without changing the agent logic.

Add a polling timer to the Fiori UI. Write a Fiori Elements custom section that calls `pollStatus` every 5 seconds while the document status is `Processing`, updating the form fields and stopping the timer when both pipelines are complete. This is one of the few cases where a small amount of custom SAPUI5 controller code in the Fiori application is justified.

Extend to additional document types. The schema is built for this. Add a `documentType` field to `Files`, extend the `Status` enum if needed, and configure the FastAPI ingestion pipeline to handle invoice layouts, batch certificate formats, or quality inspection report structures. The CAP layer, the Fiori UI, and the `chatQuery` interface require

no changes.

Deploy to SAP BTP Cloud Foundry. Replace the SQLite database with HANA Cloud by setting the `cds.requires.db.kind` to `hana` and binding the `hana` service. Set `BACK-END_URL` to the deployed FastAPI service URL. Run `cf push` from `cap-srv/`. The CAP service connects to HANA automatically via the `VCAP_SERVICES` binding. Chapter 10 covers this in full.

Add XSUAA authentication. Bind an XSUAA service instance to the CAP application. Add `@requires: 'authenticated-user'` to the `DocumentService` definition. The CAP framework validates JWT tokens from XSUAA on every request with no additional handler code.

9.13 Checkpoint checklist

Before moving to the next chapter, verify the following:

- `cds serve` starts without errors and outputs the service URL at port 4004
- The Fiori Elements preview is accessible at `http://localhost:4004/$fiori-preview`
- A new document can be created and saved via the Fiori list report
- The Material Number value help returns results from the Products entity
- A PDF attachment can be uploaded and is stored by `@cap-js/attachments`
- Clicking Process File sends a multipart POST to the FastAPI service and returns `status: "processing"`
- The `pollStatus` function returns accurate `triples` and `vectors` counts after processing completes
- The `chatQuery` action returns an `answer` string and a `path_used` array
- The `deleteFile` action triggers the confirmation dialog (from `Common.IsActionCritical`) before executing
- The `kgDisplay` and `vecDisplay` columns in the list report show colored status indicators (green for Completed, red for Error)

If all ten items pass, your SAP-native API layer and Fiori Elements UI are working correctly. The full system — Python agent, CAP service, and Fiori UI — is operational end to end.

Summary

This chapter added the enterprise presentation layer to the hybrid RAG system. We built a CAP Node.js OData V4 service that manages document records and attachments through CDS entities, and exposes file processing and query operations through bound and unbound OData actions. The `materialNumber` field carries a value help backed by real S/4HANA product master data through `API_PRODUCT_SRV` — the connection between the AI capability on BTP and the system of record in S/4HANA. We built a `callBackend` helper that proxies all agent operations to the Python FastAPI service via HTTP, keeping the two processes completely decoupled. We generated a complete Fiori Elements UI — list report with colored status columns, object page with action buttons, and attachment handling — from CDS annotations alone, with no custom JavaScript or HTML.

The architectural principle throughout is separation of concerns: CAP owns the OData protocol, the S/4HANA integration, and the UI; FastAPI owns the intelligence. Neither service knows the internals of the other. They communicate through a narrow, stable HTTP contract. This separation makes both services easier to test, easier to scale, and easier to evolve independently.

The CAP + Fiori layer means this AI capability fits natively into any SAP Fiori launchpad. The user just asks a question and gets an answer.

Chapter 10: Deploying to SAP BTP Cloud Foundry

By the time you reach this chapter, you have a working Hybrid RAG system running locally against your BTP HANA Cloud trial. This chapter deploys it — unchanged code — to BTP Cloud Foundry. The FastAPI Python application becomes a CF app. The CAP Node.js service becomes an MTA module. HANA Cloud, already provisioned in Chapter 2, becomes the bound service.

Every configuration decision in this chapter has a specific rationale. Ports are dynamic because CF assigns them at container startup. Credentials come from service bindings because files cannot be trusted in a containerised environment. The two applications communicate over CF routes rather than localhost because they are separate containers on a distributed platform. Understanding these rationales means you can adapt the deployment pattern to other BTP applications, not just this one.

10.1 What Changes Between Local and BTP

Before touching a single deployment file, it is worth being precise about what is actually different in a Cloud Foundry environment. There are three categories of change, and understanding all three prevents most deployment failures.

Ports are dynamic. On your laptop, you choose the port. You run FastAPI on 8000 and CAP on 4004, and nothing conflicts. In Cloud Foundry, the platform assigns a port at container startup and communicates it through the `$PORT` environment variable. Your application must read that variable and bind to it. If it binds to a hardcoded port, it will start and immediately fail health checks. This is why the `manifest.yml` command is `uvicorn main:app --host 0.0.0.0 --port $PORT` rather than a fixed port number.

Credentials come from service bindings, not files. A `.env` file is a local development convenience. It is not a deployment artifact. On BTP, sensitive values — database credentials, API keys, GCP service account keys — are delivered to running containers through Cloud Foundry service bindings injected as JSON into the `VCAP_SERVICES` environment variable. The `GOOGLE_API_KEY` that worked locally never appears in a deployed

container's filesystem or in `manifest.yml`. This is not just a best practice; it is how the platform is designed. The BTP Destination Service stores the Vertex AI credentials at runtime — the CAP service reads them from the Destination Service, never from environment variables visible in `cf env`.

Inter-service communication uses the platform's routing layer. On localhost, CAP reaches the Python agent at `http://localhost:8000`. On BTP Cloud Foundry, every application has a route — a public HTTPS URL managed by the platform's router. The two applications communicate over those routes. The BTP Destination Service provides a managed, centralised way to store and reference those URLs, so that changing the back-end URL only requires updating one entry in the BTP Cockpit rather than redeploying every service that references it.

These three changes — dynamic ports, credential delivery via bindings, and HTTPS routing — account for almost all of the configuration work in this chapter.

10.2 Why Two Separate Applications

The system deploys as two independent CF applications: the FastAPI Python service and the CAP Node.js service. This is not an accident of how the code happens to be organised — it is a deliberate architectural choice with operational consequences.

FastAPI handles AI orchestration and HANA data access. It is a standard Python web service that runs on any CF Python buildpack. Its responsibilities are: receiving document upload requests, running the LangGraph ingestion pipeline, querying HANA for vector and knowledge graph retrieval, calling Vertex AI for LLM inference, and returning answers. None of these responsibilities have any dependency on OData, Fiori, or BTP security services.

CAP handles OData protocol, Fiori UI rendering, S/4HANA product master integration via `API_PRODUCT_SRV`, and BTP service bindings for XSUAA authentication and HANA schema management. It is the SAP-native layer. Its responsibilities are: serving the Fiori Elements UI, validating OData requests, managing document and attachment records in HANA, and proxying processing and query requests to the FastAPI backend.

Keeping them separate means you can update the AI model, the LangGraph orchestration logic, or the vector retrieval strategy without touching the CAP service — and vice

versa. The HTTP contract between them (multipart POST to `/process-upload`, JSON POST to `/query`, GET to `/status/{materialNumber}`) is stable and narrow. Either side can be redeployed independently as long as that contract is respected.

10.3 Pre-Deployment Checklist

Deployment failures are much easier to debug when you can be certain your local environment is correctly configured before you start. Work through this checklist before executing any `cf push` or `mbt build` command.

Cloud Foundry CLI. Run `cf version`. You need version 8 or later. If the command is not found, install from the [Cloud Foundry Foundation releases page](#) or, on macOS, with `brew install cloudfoundry/tap/cf-cli@8`. Log in with `cf login -a https://api.cf.<your-region>.hana.ondemand.com` and target the correct org and space with `cf target -o <org> -s <space>`.

MTA Build Tool. The CAP service is deployed using the MultiTarget Application (MTA) model, which requires the `mbt` CLI. Run `mbt --version`. If it is missing, install with `npm install -g mbt`. This tool reads `mta.yaml`, builds all modules, and packages them into a single `.mtar` archive that the CF deployer can process.

BTP Cloud Foundry space. Confirm you are in the correct space with `cf target`. The space must have the SAP HANA Cloud instance accessible to it. If you are using a trial account, the HANA Cloud instance must be active — it stops automatically after 30 days of inactivity and must be restarted manually in the BTP Cockpit.

HANA Cloud instance running. In the BTP Cockpit, navigate to your subaccount, open the SAP HANA Cloud section, and confirm the instance status is “Running.” Deployment will succeed even if HANA is stopped — the CAP service only attempts a database connection at runtime. But the smoke test in section 10.8 will fail until HANA is available.

Google Cloud service account credentials. The Python agent authenticates to Vertex AI using a Google Cloud service account. You should have a JSON key file from Chapter 2. For BTP deployment, this key is stored in the BTP Destination Service — never as a file in the container, never as a plain environment variable visible in `cf env`.

10.4 Step 1: Push the Python Agent

The Python agent deploys first because the CAP service needs to know its URL when configuring the Destination Service. Navigate to the `agents/` directory and examine the deployment manifest.

Understanding manifest.yml

```

1
2  ---
3  applications:
4    - name: hybrid-rag-agent
5      buildpacks:
6        - python_buildpack
7      memory: 512M
8      disk_quota: 1G
9      instances: 1
10     command: uvicorn main:app --host 0.0.0.0 --port $PORT
11     path: .
12     env:
13       HANA_HOST: ((hana-host))
14       HANA_PORT: "443"
15       HANA_USER: ((hana-user))
16       HANA_PASSWORD: ((hana-password))
17       GCP_PROJECT_ID: ((gcp-project-id))
18       GCP_LOCATION: us-central1
19       LANGCHAIN_TRACING_V2: "true"
20       LANGCHAIN_ENDPOINT: "https://api.smith.langchain.com"
21       LANGCHAIN_API_KEY: ((langchain-api-key))
22       PYTHONPATH: "."

```

Walk through each field:

`name: hybrid-rag-agent` sets the application name in Cloud Foundry. This becomes part of the default route: `hybrid-rag-agent.<your-cf-domain>`. Choose a name that is unique within your org.

`buildpacks: [python_buildpack]` tells Cloud Foundry which buildpack to use for detecting, compiling, and packaging the application. The Python buildpack reads `requirements.txt`, creates a virtualenv, installs dependencies, and configures the runtime. No

Dockerfile is needed.

`memory: 512M` and `disk_quota: 1G` set the container resource limits. 512 MB is sufficient for the FastAPI server and the LangGraph agent under moderate load. The HANA Python driver (`hdbcli`), Vertex AI client libraries, and LangChain components together consume roughly 200-250 MB at startup. If you observe OOM kills in logs, increase to 1G.

`command: uvicorn main:app --host 0.0.0.0 --port $PORT` is the startup command. The `--host 0.0.0.0` binding is required — binding only to `127.0.0.1` means the platform's router cannot reach the container. The `$PORT` variable is provided by Cloud Foundry and will be a value like 8080 or 61002 depending on the container assignment. The Python buildpack expands this shell variable before executing the command.

`path: .` tells the CF CLI which directory to upload. Since you are running `cf push` from within `agents/`, the `.` means the entire `agents` directory is packaged and sent to the staging container.

`PYTHONPATH: ". "` ensures that Python can find modules in the application root. Without this, relative imports within the agent codebase will fail at runtime.

The `((variable))` syntax in the `env` block are placeholders — the real values are supplied through `cf set-env` commands or a User-Provided Service, covered in the next step. If you pushed this manifest as-is, the literal placeholder strings would appear as environment variable values, causing immediate connection failures at runtime.

HANA connection in CF

In the deployed environment, HANA credentials come from `VCAP_SERVICES` via service binding — the same mechanism CAP uses. The `hdb_srv.py` module already handles this: it reads `VCAP_SERVICES` if direct environment variables are not set. No code change is needed to move from local `.env` file credentials to CF service binding credentials. The credential source changes; the application code does not.

Pushing the application

From the `agents/` directory:

```
1 cf push hybrid-rag-agent --no-start
```

The `--no-start` flag stages the application — uploads the code, runs the buildpack,

creates the container — but does not start it. This is important because the environment variables have not been set yet. Starting now would produce a crash-loop of failed HANA connections.

Confirm staging succeeded:

```
1 cf app hybrid-rag-agent
```

You should see the application in a `stopped` state with a route assigned. Note the route URL — something like `hybrid-rag-agent.cfapps.eu10.hana.ondemand.com`. You will need this URL when configuring the Destination Service.

10.5 Step 2: Create the User-Provided Service for Credentials

Cloud Foundry User-Provided Services (UPS) are the correct mechanism for delivering credentials to applications that need values which are not available through a managed service binding. Instead of setting individual environment variables one by one on each application, you define a named service with a JSON payload of key-value pairs, bind it to any application that needs those values, and Cloud Foundry delivers the entire payload via `VCAP_SERVICES`.

The security rationale is straightforward. If you set credentials with `cf set-env`, they are visible in `cf env <app>` to anyone with developer access to the space. If you put credentials in `manifest.yml`, they will eventually end up in version control. A User-Provided Service stores the values in Cloud Foundry's internal credential store, not in any file that exists in your repository.

Create the service with the actual values for your environment:

```
1 cf cups hybrid-rag-agent-env -p '{
2   "HANA_HOST": "your-hana-instance.hanacloud.ondemand.com",
3   "HANA_PORT": "443",
4   "HANA_USER": "DBADMIN",
5   "HANA_PASSWORD": "your-database-password",
6   "GCP_PROJECT_ID": "your-gcp-project-id",
```

```

7   "GCP_LOCATION": "us-central1",
8   "LANGCHAIN_API_KEY": "your-langchain-api-key"
9   }'

```

`cf cups` stands for `cf create-user-provided-service`. The `-p` flag accepts a JSON string of parameters.

Now bind the service to the application:

```

1  cf bind-service hybrid-rag-agent hybrid-rag-agent-env

```

Cloud Foundry will inject the key-value pairs from this service into the `VCAP_SERVICES` JSON available to the container. The `HANA_HOST` key in the UPS becomes the `HANA_HOST` environment variable in the application container.

If you need to update the credentials later — for example, to rotate the HANA password — you can use:

```

1  cf uups hybrid-rag-agent-env -p '{"HANA_PASSWORD": "new-password"}'
2  cf restart hybrid-rag-agent

```

The `cf uups` command (update-user-provided-service) replaces the entire parameter set, so include all keys, not just the changed one.

Now start the application:

```

1  cf start hybrid-rag-agent

```

Watch the startup logs:

```

1  cf logs hybrid-rag-agent --recent

```

A successful startup will show `uvicorn` reporting that it is listening on the assigned port. A failed startup will show a Python traceback — usually a missing environment variable or an import error from a missing package. The most common first failure is a missing `google-cloud-aiplatform` package because it was not in `requirements.txt`. Confirm your `requirements.txt` includes every library imported anywhere in the codebase.

Verify the agent is reachable:

```
1 curl https://hybrid-rag-agent.<your-cf-domain>/health
```

You should receive a JSON response indicating the service status and confirming that the HANA and Vertex AI connections are available.

10.6 Step 3: Build and Deploy the CAP Service

The CAP service is deployed using the MTA (MultiTarget Application) model rather than a direct `cf push`. MTA is an SAP-developed open standard for describing multi-component applications — a single `mta.yaml` file describes every module (CAP service, Python service), every service binding (HANA, UPS), and every dependency between them. The `mbt build` tool compiles all modules and packages them, and the CF MTA deployer (`cf deploy`) orchestrates the deployment.

Examine the MTA descriptor:

```
1 _schema-version: "3.3.0"
2 ID: msds-hybrid-rag
3 version: 1.0.0
4 description: Hybrid RAG MSDS system on SAP BTP
5
6 modules:
7   - name: hybrid-rag-agent
8     type: python
9     path: agents
10    build-parameters:
11      builder: custom
12      commands:
13        - pip install -r requirements.txt
14    parameters:
15      memory: 512M
16      disk-quota: 1G
17    properties:
18      PYTHONPATH: "."
19    requires:
20      - name: hybrid-rag-agent-env
```

```

21
22   - name: msds-hybrid-rag-cap
23     type: nodejs
24     path: cap-srv
25     build-parameters:
26       builder: npm
27       build-result: gen/srv
28     parameters:
29       memory: 256M
30       disk-quota: 512M
31     properties:
32       AGENT_URL: "~{hybrid-rag-agent/url}"
33     requires:
34       - name: msds-hybrid-rag-hana
35       - name: hybrid-rag-agent-env
36     provides:
37       - name: cap-srv-api
38     properties:
39       url: ${default-url}
40
41   resources:
42     - name: msds-hybrid-rag-hana
43       type: com.sap.xs.hana-schema
44       parameters:
45         service: hana
46         service-plan: schema
47
48     - name: hybrid-rag-agent-env
49       type: org.cloudfoundry.user-provided-service

```

Several lines require explanation.

`AGENT_URL: "~{hybrid-rag-agent/url}"` is an MTA cross-reference. The `~{}` syntax means “substitute the value of this property from another module at deployment time.” The `hybrid-rag-agent` module, when deployed, exposes its assigned route URL as the `url` property. The CAP module reads that URL and stores it in its own `AGENT_URL` environment variable. This eliminates hardcoding: even if the Python service is redeployed to a different route, the CAP service will be updated automatically on the next deployment.

`build-result: gen/srv` tells the MTA deployer where to find the compiled CAP artifacts

after the `npm build`. The `cds build` command, which runs as part of `npm run build`, compiles CDS models, generates the OData metadata, and writes the production-ready service files to `gen/srv`. The MTA deployer packages only this directory, not the entire `cap-srv` source tree.

`type: com.sap.xs.hana-schema` declares that the deployer should create (or reuse) a HANA schema resource. This becomes a service binding in Cloud Foundry: the CAP service's `VCAP_SERVICES` will contain the HANA connection details, and the `@cap-js/hana` plugin will read them automatically. You do not configure a connection string in the CAP application; the binding provides it.

`type: org.cloudfoundry.user-provided-service` in the `resources` section tells the MTA deployer that `hybrid-rag-agent-env` is an existing UPS that should be bound but not created. The UPS you created in step 10.5 is referenced here by name. Both the Python agent and the CAP service are bound to it, which is how both processes receive the HANA credentials.

Running the build

From the `cap-srv/` directory (or the repo root, depending on where `mta.yaml` lives):

```
1 mbt build
```

This command reads `mta.yaml`, executes the build commands for each module, and produces an `.mtar` archive in the `mta_archives/` directory. The archive is a standard ZIP containing all compiled artifacts for every module.

If the build fails at the `cds build` step, the most common cause is a CDS syntax error or a missing `npm` dependency. Run `npm run build` in the `cap-srv/` directory directly to see the full CDS compiler output.

Deploying the MTA

Install the CF MTA plugin if you have not already:

```
1 cf install-plugin multiapps
```

Deploy the built archive:

```
1 cf deploy mta_archives/msds-hybrid-rag_1.0.0.mtar
```

The deployer will work through a sequence of operations: creating or updating service instances, uploading application binaries, binding services, and starting applications. This takes two to five minutes. The terminal output shows each step with a status indicator. If any step fails, the deployer stops and shows the error. Common failures at this stage are HANA schema creation failures (usually a quota issue on a trial account) and npm install failures caused by package resolution errors.

When the deployment completes, both applications will be running. Confirm with:

```
1 cf apps
```

You should see `hybrid-rag-agent` and `msds-hybrid-rag-cap` both in a `started` state with routes assigned.

10.7 Step 4: Configure the BTP Destination for Vertex AI

In local development, Vertex AI credentials live in your `.env` file as `GOOGLE_APPLICATION_CREDENTIALS` pointing to a local JSON key file. That pattern does not belong in a deployed environment — a JSON key file in a container filesystem is one restart away from being lost, and it is visible to anyone with container access.

In BTP Cloud Foundry, the Vertex AI credentials are stored in the BTP Destination Service. The Destination Service is the enterprise credential management pattern across the entire BTP portfolio — the same mechanism used for S/4HANA connectivity, external REST APIs, and OAuth token exchange. The CAP service reads the Vertex AI credentials from the Destination Service at runtime. They are never stored in the app container, never in environment variables visible in `cf env`, and never in any file that could be accidentally committed or shared.

The production pattern in the CAP service handler, instead of reading from environment variables directly, uses the SAP Cloud SDK's destination resolution:

```
1 const { executeHttpRequest } = require('@sap-cloud-sdk/http-client');
2 const response = await executeHttpRequest(
```

```

3   { destinationName: process.env.BACKEND_DESTINATION },
4   { method: 'POST', url: '/query', data: body }
5   );

```

The `executeHttpRequest` function handles destination resolution, certificate verification, and — when configured — token propagation. This is the mature BTP application pattern. For the current deployment, we configure the Destination Service so it is available when you move to production hardening in a later chapter.

Creating the agent backend destination

Navigate to your BTP subaccount in the BTP Cockpit. Select **Connectivity** from the left navigation, then **Destinations**. Click **New Destination** and fill in the following values:

- **Name:** `hybrid-rag-agent`
- **Type:** HTTP
- **URL:** `https://hybrid-rag-agent.<your-cf-domain>` (the route from step 10.4)
- **Proxy Type:** Internet
- **Authentication:** NoAuthentication

Save the destination. You can test it immediately by clicking **Check Connection** — the cockpit will send a GET request to the root URL of the agent and report the HTTP response code.

Creating the Vertex AI destination

Click **New Destination** again and configure the Vertex AI credentials:

- **Name:** `VertexAI`
- **Type:** HTTP
- **URL:** `https://us-central1-aiplatform.googleapis.com`
- **Proxy Type:** Internet
- **Authentication:** NoAuthentication

Add additional properties:

Property	Value
<code>gcp.project_id</code>	Your GCP project ID
<code>gcp.location</code>	<code>us-central1</code>

Property	Value
<code>gcp.service_account_key</code>	The entire contents of your <code>gcp-sa-key.json</code> file

BTP encrypts the `gcp.service_account_key` value at rest. The service account JSON key is now stored in one place — the BTP Destination Service — and accessible to any BTP application in this subaccount that needs Vertex AI access. Rotating the credentials means updating this one destination entry, not redeploying any application.

10.8 Step 5: Load the Ontology

The HANA knowledge graph requires the MSDS ontology to be loaded before any queries can run against it. In local development this was done by calling the admin endpoint directly. In the deployed environment, the process is identical, but the URL is the deployed agent route rather than localhost.

Obtain the agent route:

```
1 cf app hybrid-rag-agent | grep routes
```

Load the ontology:

```
1 curl -X POST \
2   https://hybrid-rag-agent.<your-cf-domain>/admin/load-ontology \
3   -H "Content-Type: application/json"
```

This endpoint triggers the same ontology loading logic from Chapter 4 — parsing the RDF Turtle file and inserting the triples into HANA's graph store. On the first deployment, this will take 15-30 seconds depending on the size of the ontology. Subsequent calls are idempotent: the endpoint checks whether the graph already exists before attempting to insert.

If the call returns a 500 error, the most likely cause is a HANA connection failure. Check the agent logs:

```
1 cf logs hybrid-rag-agent --recent
```

Look for a `hdbcli` connection error. If you see “authentication failed,” confirm the `HANA_USER` and `HANA_PASSWORD` values in the User-Provided Service. If you see “host not found,” confirm the `HANA_HOST` value. Note that HANA Cloud hostnames are long strings in the format `<uuid>.hana.ondemand.com` — confirm there are no trailing spaces or missing characters.

10.9 Step 6: Smoke Test the Deployed System

With both services running and the ontology loaded, run through each integration point systematically before calling the deployment complete.

Agent health endpoint

```
1 curl https://hybrid-rag-agent.<your-cf-domain>/health
```

Expected response: a JSON object confirming service status, HANA connectivity, and Vertex AI availability. If HANA shows as unreachable, the agent will still start and respond on the health endpoint — it will report the connection failure in the status fields rather than refusing to respond.

Direct query to the agent

```
1 curl -X POST \
2   https://hybrid-rag-agent.<your-cf-domain>/query \
3   -H "Content-Type: application/json" \
4   -d '{"question": "What documents are available for material MAT-001?",
      "session_id": "smoke-test-01"}'
```

This bypasses the CAP layer entirely and tests the LangGraph agent directly against HANA. The question validates the end-to-end flow: the agent queries both the knowledge graph and the vector store, calls Vertex AI for answer synthesis, and returns the result. You should receive a JSON response with an `answer` field and a `retrieval_path` field indicating which chains were invoked. If this succeeds, the HANA connection, the

Vertex AI embedding model, and the LangGraph orchestration are all functional in the deployed environment.

CAP OData endpoint

```
1 curl
  https://msds-hybrid-rag-cap.<your-cf-domain>/odata/v4/documents/Documents
```

This calls the CAP OData service's Documents entity set. On a fresh deployment, the response will be an empty array `{"value": []}` — no documents have been uploaded yet. An empty array is a successful response; a 500 error indicates a CAP startup or HANA binding problem.

Fiori Elements UI

Open the CAP application URL in a browser:

```
https://msds-hybrid-rag-cap.<your-cf-domain>/index.html
```

The Fiori Elements list report should render. If you see a blank page, open the browser developer tools and check the network tab for 401 or 403 errors — these indicate an XSUAA configuration issue. For a non-XSUAA deployment (no authentication configured), the UI should load without any login screen.

10.10 Troubleshooting

Deployment failures follow predictable patterns. The following are the most common failure modes and their resolution paths.

Memory quota exceeded

Symptom: Application crashes shortly after startup. Logs show `Signal 9 (SIGKILL)` or the application disappears from `cf apps` with a restart count incrementing.

Cause: The container exceeded its memory limit. The Python buildpack allocates the full 512M during startup as the HANA driver, LangChain, and Vertex AI libraries load. If additional packages were added after the memory limit was set, startup memory may now exceed the limit.

Resolution: Increase the memory limit in `manifest.yml` to 1G and repush, or use `cf scale hybrid-rag-agent -m 1G` to update the limit without repushing.

HANA connection refused

Symptom: Agent starts successfully but `/health` reports HANA as unreachable. Logs show `[HDB][ERR] authentication failed` or `[HDB][ERR] connection refused`.

Cause: Incorrect credentials in the User-Provided Service, or the HANA Cloud instance is stopped.

Resolution: Verify the UPS values with `cf env hybrid-rag-agent` (look for `VCAP_SERVICES` in the output and find the `hybrid-rag-agent-env` entry). Compare the `HANA_HOST` value against the hostname shown in the BTP Cockpit HANA Cloud manager. If the instance is stopped, start it from the BTP Cockpit and wait two to three minutes before retrying.

GCP authentication failure

Symptom: Queries fail with a `google.api_core.exceptions.PermissionDenied` error in the agent logs.

Cause: The GCP service account credentials are not correctly configured, or the service account does not have the required Vertex AI roles.

Resolution: Confirm that `GCP_PROJECT_ID` in the UPS matches the project where Vertex AI is enabled. Confirm the Vertex AI Destination in BTP has the correct `gcp.service_account_key` value. Confirm the service account has the Vertex AI User role in GCP IAM. If you are using a key file directly, base64-encode the JSON content and set it as an environment variable, then decode it to a temp file at startup in the application's entry point.

CAP cannot reach agent

Symptom: The Fiori UI loads but queries return an error. CAP logs show `ECONNREFUSED` or `ENOTFOUND` when calling the agent URL.

Cause: The `AGENT_URL` environment variable in the CAP service contains an incorrect value, or the Python agent is not running.

Resolution: Check `cf env msds-hybrid-rag-cap` and confirm the `AGENT_URL` value is

the correct HTTPS route for the Python agent. Confirm the Python agent is running with `cf app hybrid-rag-agent`. If you recently redeployed the Python agent and its route changed, update the destination in the BTP Cockpit and use `cf set-env` to update the `AGENT_URL` value on the CAP service, then restart it.

Package not found during staging

Symptom: The `cf push` command fails during the staging phase with a `pip install` error.

Cause: A package in `requirements.txt` is either misspelled, unavailable at the specified version, or requires a system library that is not present in the Cloud Foundry container.

Resolution: Read the full staging log carefully — the package name and the error message from `pip` are usually clear. For packages that require compiled C extensions (such as `hdbcli`), confirm that a pre-compiled wheel is available for the Python version used by the buildpack. You can pin the Python version in a `runtime.txt` file at the root of the `agents/` directory with content `python-3.11.x`.

10.11 Scaling

The system as deployed runs as single instances of each service. For a pilot with a handful of users, this is sufficient. For broader use, both the Python agent and the CAP service can be scaled horizontally — adding more container instances that share the incoming request load behind the Cloud Foundry router.

Scale the Python agent to two instances:

```
1 cf scale hybrid-rag-agent -i 2
```

Scale the CAP service:

```
1 cf scale msds-hybrid-rag-cap -i 2
```

Cloud Foundry's built-in router distributes requests across instances using round-robin by default. Because the Python agent is stateless — the LangGraph agent state is not persisted between requests, and HANA handles all persistent state — horizontal scaling

requires no additional configuration. Each instance handles requests independently.

There are two situations where single-instance deployment is the correct choice even at higher load. First, during active development: when you are pushing frequent updates, having multiple instances means you need to track whether all instances have restarted to the new version. Second, during debugging: multiple instances produce interleaved log streams that can be difficult to read with `cf logs`. For debugging, scale to one instance, reproduce the problem, then scale back up.

Memory scaling — increasing the memory limit per instance — is more relevant for the Python agent than instance scaling. The LangGraph execution model creates new graph instances per request; large numbers of concurrent requests will increase memory pressure faster than CPU pressure. If you observe slow response times under load, check the memory usage with `cf app hybrid-rag-agent` before adding instances. A single 2G instance is often more cost-effective than two 512M instances for a memory-bound workload.

For production deployments with SLA requirements, consider enabling autoscaling through the Application Autoscaler service available in the BTP Service Marketplace. It can scale instances based on CPU, memory, or custom metrics, and scale them back down during quiet periods to control costs.

10.12 What You Have Built

You have deployed a production-grade Material Document Intelligence Platform to SAP BTP Cloud Foundry. Any PDF document — MSDS, invoice, batch certificate, inspection report, maintenance manual, legal filing — can be uploaded against a SAP Material Number, processed into a vector store and knowledge graph, and queried in natural language. The system runs on SAP's enterprise cloud platform, uses SAP HANA Cloud for both AI storage layers, integrates with real S/4HANA product master data via `API_PRODUCT_SRV`, and presents a familiar Fiori Elements interface.

Two applications are running as Cloud Foundry containers. The Python FastAPI service — `hybrid-rag-agent` — runs a LangGraph agent that dispatches every incoming question to parallel retrieval chains: a SPARQL chain that queries the HANA RDF knowledge graph for structured facts, and a vector similarity chain that searches HANA's vector in-

dex for semantically relevant passages. The agent synthesises the results using a Gemini model on Google Vertex AI and returns a grounded answer with a traceable retrieval path.

The CAP Node.js service — `msds-hybrid-rag-cap` — hosts an OData V4 API, a Fiori Elements UI, and an attachment handling layer. It validates material numbers against S/4HANA product master data, manages document records in HANA, and proxies processing and query requests to the FastAPI backend. Credentials for Vertex AI are stored in the BTP Destination Service — never in application code, never in deployment files.

This is the foundation for an enterprise AI capability that can scale across your SAP landscape. The same platform, the same patterns, the same HANA instance that stores your vector embeddings and knowledge graph triples — extended to handle every document type your organisation processes against SAP materials.

10.13 Chapter Checkpoint

Work through this list before moving to the next chapter. Each item verifies a specific part of the deployed system.

- `cf app hybrid-rag-agent` shows the application in `started` state with a route assigned. (Section 10.4)
- `curl https://hybrid-rag-agent.<domain>/health` returns a JSON response with no error fields. (Section 10.9)
- The User-Provided Service `hybrid-rag-agent-env` exists and is bound to both applications. (Section 10.5)
- `cf deploy` completed without errors and both applications appear in `cf apps`. (Section 10.6)
- A BTP Destination named `hybrid-rag-agent` exists in the subaccount, pointing to the Python agent URL. (Section 10.7)
- A BTP Destination named `VertexAI` exists with the GCP service account key stored as a destination property. (Section 10.7)
- `curl -X POST .../admin/load-ontology` returned a 200 status code. (Section 10.8)
- A direct `curl POST` to `/query` with question “What documents are available for material MAT-001?” returns an answer with a retrieval path. (Section 10.9)

- The Fiori Elements UI loads at the CAP application URL without errors in the browser console. (Section 10.9)

If every item on this list is green, the system is deployed and functional. The next chapter addresses operational concerns — observability, performance tuning, and the integration patterns that allow this system to participate in broader SAP workflows.

Appendix A — SAP BTP Trial Setup: Step-by-Step

This appendix is for readers who are new to SAP Business Technology Platform. If you have worked with BTP before and already have a trial account with a running HANA Cloud instance, you can skip ahead to Chapter 2. If you are starting from scratch, follow every step here before opening Chapter 2. The main chapters assume this foundation is in place and do not repeat these instructions.

What You Will Have at the End

By the time you reach the end of this appendix, you will have:

- An SAP BTP trial account registered and accessible at `cockpit.hanatrial.ondemand.com`
- A Cloud Foundry space named `dev` inside your trial subaccount
- A HANA Cloud instance named `msds-hana` (or your preferred name) in the Running state
- The vector engine confirmed available — the `REAL_VECTOR` column type is enabled by default in HANA Cloud from QRC2/2023 onwards
- Your HANA connection details (host, port, user, password) ready to paste into `agents/.env`
- The CF CLI installed and authenticated to your trial space

This is the minimum you need before executing any code in Chapter 2.

Step 1 — Create a BTP Trial Account

Navigate to the Trial Signup Page

Open a browser and go to `cockpit.hanatrial.ondemand.com`. You will see the SAP BTP homepage with a **Try for Free** button prominently displayed.

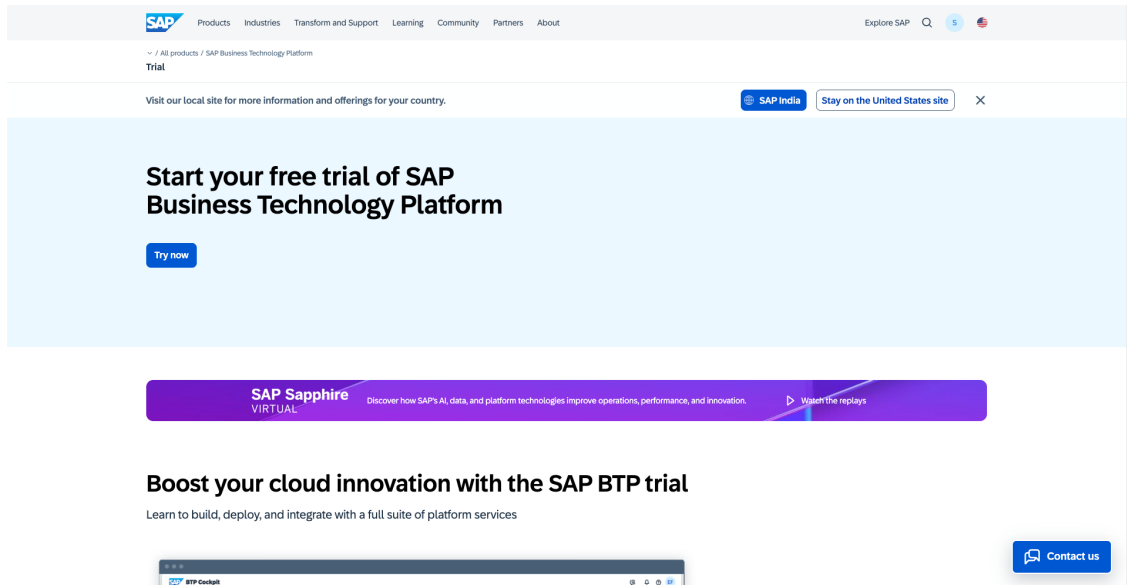


Figure A.1 — The SAP BTP homepage. Click “Try for Free” to begin the trial registration process.

If you already have an SAP Universal ID or an S-user ID from a corporate SAP system, you can log in directly without registering. If not, click **Try for Free** and proceed to registration.

Register Your Account

On the registration page, fill in your first name, last name, email address, and choose a password. Use a personal email address if possible. Some corporate email domains block activation emails or have policies that prevent trial account creation.

Get an Account

Sign up for a trial account on SAP BTP.

You will learn:

- ✓ How to register on the SAP website
- ✓ How to start your SAP BTP Trial
- ✓ How to navigate to your sub-account and space
- ✓ Where to find information on three important Cloud Foundry areas: Applications, Service Marketplace, and Service Instances

Steps:

- Step 1: Log into the SAP website
- Step 2: Register at sap.com
- Step 3: Activate your account
- Step 4: Log on to SAP BTP Trial
- Step 5: Access the SAP BTP cockpit
- Step 6: Navigate to the subaccount
- Step 7: Learn about the service marketplace

Log In

Already registered?

Log In

Done

Figure A.2 — The trial registration form. Enter

your details and accept the terms of service.

After submitting, SAP will send a verification email to the address you provided. Check your inbox (and spam folder) and click the activation link. The link expires after 24 hours.

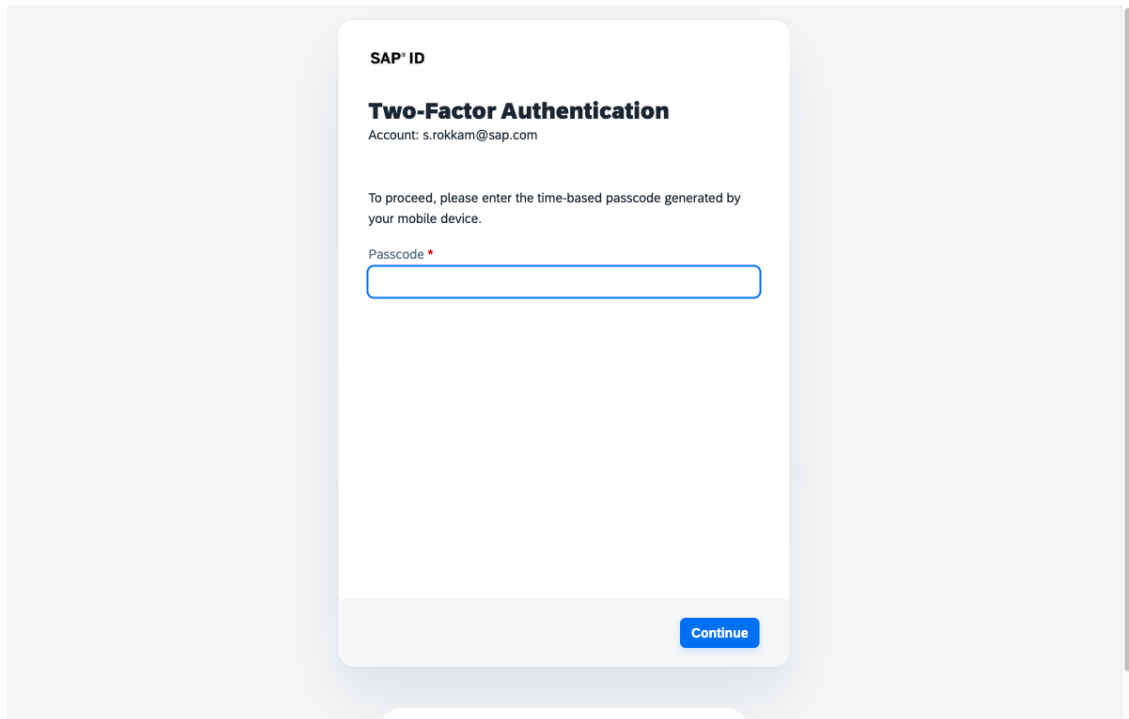


Figure A.3 — Once your email is verified, return to the login page and sign in with your new credentials.

Complete Registration and Choose a Region

After logging in for the first time, BTP will ask you to complete your registration profile. Fill in your details.

Figure A.4 — Complete your profile to finalize the trial account.

You will then be presented with a region selection screen. BTP trial accounts support several regions. **Choose US East (US10, Virginia)** if you are following this book closely.

The Vertex AI API we use is hosted on Google Cloud in the US, and keeping your BTP account in the same broad geography minimizes network latency between your HANA Cloud instance and external API calls.

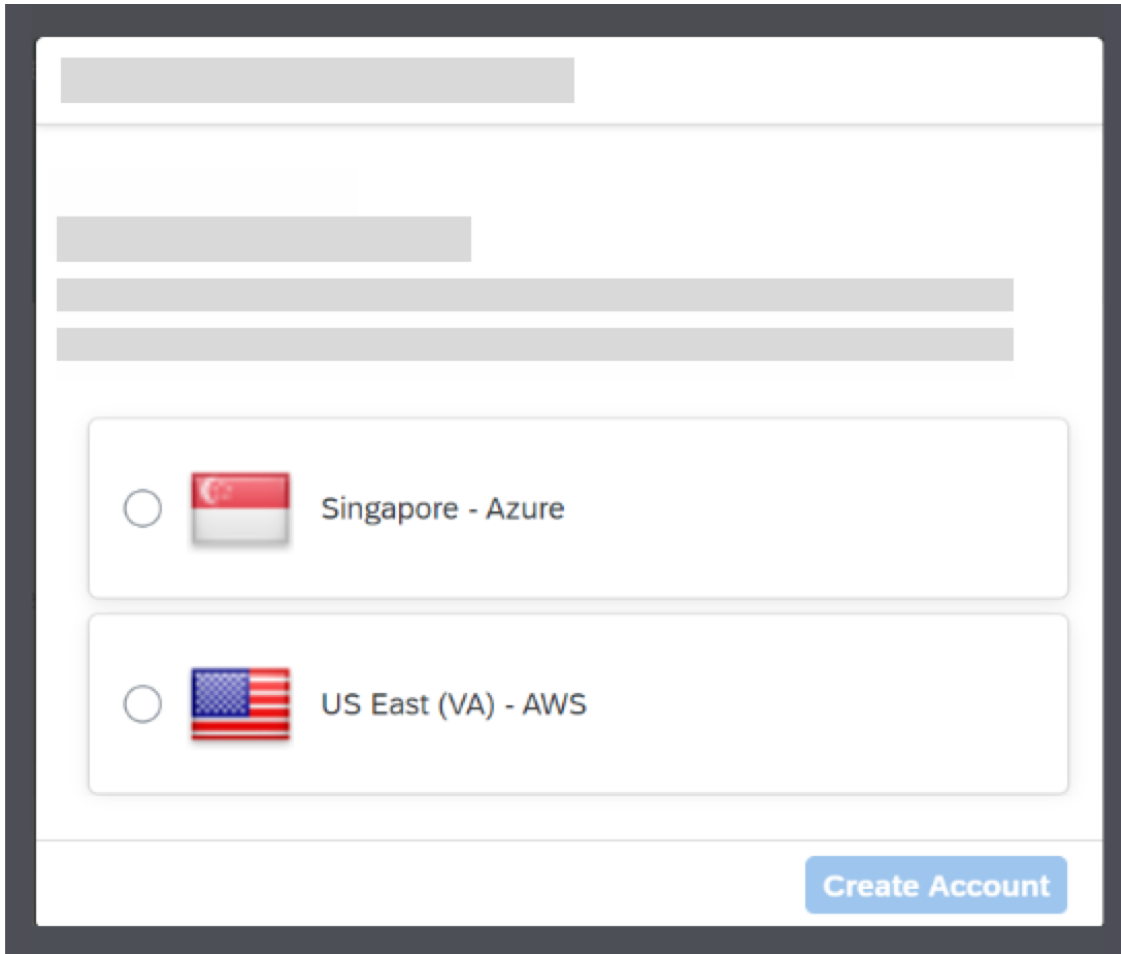
The screenshot shows a web form for selecting a region. At the top, there are three horizontal input fields for email, password, and a confirmation password, all of which are currently empty. Below these fields, there are two radio button options. The first option is labeled 'Singapore - Azure' and features a flag of Singapore. The second option is labeled 'US East (VA) - AWS' and features a flag of the United States. At the bottom right of the form, there is a blue button labeled 'Create Account'.

Figure A.5 — Region selection. US East (US10) is recommended for readers following this book.

Note: The region you choose here is permanent for your trial account. Your Cloud Foundry space and HANA Cloud instance will be created in this same region automatically. Do not attempt to change the region after this point — it is not possible without creating a new account.

Click **Proceed**. BTP will spend 30–60 seconds provisioning your trial account.

Confirm the Trial Account Is Ready

When provisioning completes, you will land on the BTP Cockpit home screen. You should see a **trial** tile under your Global Account name.

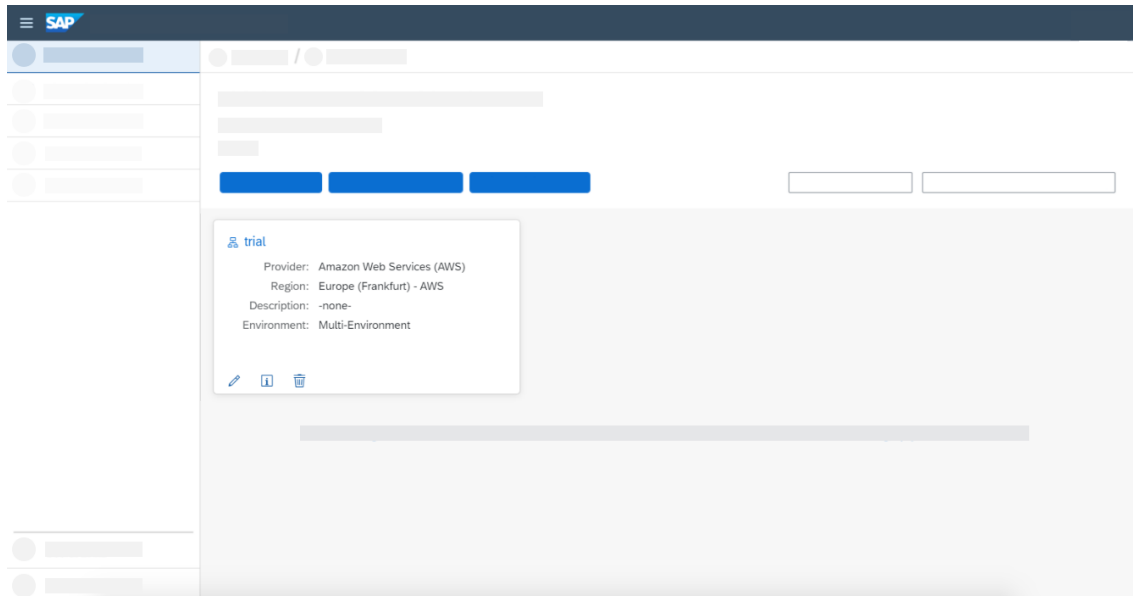


Figure A.6 — Your BTP trial account is ready. You should see the “trial” subaccount tile on the home screen.

If you see an error or the page appears empty, wait 2 minutes and refresh. Trial provisioning occasionally takes longer than expected.

Step 2 — Navigate the BTP Cockpit

Before diving into service setup, take one minute to understand the cockpit’s structure. This prevents confusion later.

Global Account is the top-level container for your BTP subscription. It holds entitlements (the services you are allowed to use) and organizes everything underneath.

Subaccount is a logical partition inside your Global Account. Your trial comes with one subaccount called “trial.” This is where you will create services, deploy applications, and manage credentials.

Cloud Foundry Space is a runtime environment inside a subaccount. The space called `dev` is where your application code runs.

Finding Your Trial Subaccount

From the BTP Cockpit home screen, click the **trial** tile. This opens your trial subaccount overview.

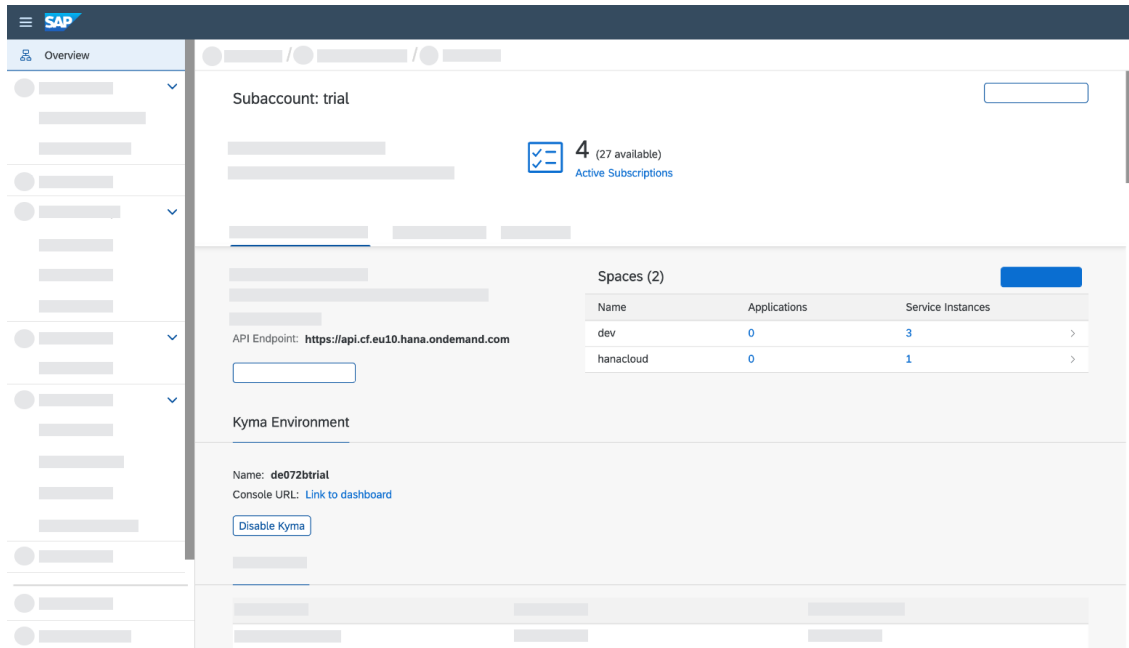


Figure A.7 — The trial subaccount overview page. Note the API Endpoint value in the Cloud Foundry section — you will need this in Step 8.

On this page you will see several sections:

- **Cloud Foundry Environment** — shows your CF organization name and API endpoint
- **Instances and Subscriptions** — shows services you have created
- **Security** — role collections and trust configuration

Look at the Cloud Foundry Environment section and write down the **API Endpoint**. It will look like `https://api.cf.us10.hana.ondemand.com` (the region code in the middle will match your choice from Step 1).

Finding the CF Space

In the left navigation panel, click **Cloud Foundry** and then **Spaces**. You will see a space named `dev`. This is your deployment target for all the application code in this book.

[SCREENSHOT: Left navigation showing Cloud Foundry > Spaces with the dev space listed]

If the `dev` space is not visible, proceed to Step 3 to create it.

Step 3 — Enable Cloud Foundry

BTP trial accounts come with the Cloud Foundry runtime pre-enabled in most regions, but it is worth verifying.

Check CF Runtime Status

From your trial subaccount overview, look at the **Cloud Foundry Environment** section. If it shows an org name and API endpoint, CF is already enabled. Skip to “Create the dev Space” below.

If you see a button labeled **Enable Cloud Foundry**, click it. You will be prompted to choose an instance plan — select **trial**. Enabling CF takes about 2 minutes.

[SCREENSHOT: Cloud Foundry Environment section showing “Enable Cloud Foundry” button, or alternatively showing the org name and API endpoint if already enabled]

Create the dev Space

Once CF is enabled and you have an org, click **Create Space**. Name it `dev`. Leave the default role assignments in place. Click **Create**.

[SCREENSHOT: Create Space dialog with “dev” entered as the space name]

You now have a CF space ready to receive deployments.

Note the CF API Endpoint

Write down the following three values from the Cloud Foundry Environment section of your subaccount. You will use all three in Step 8 when you run `cf login`.

Item	Where to Find It	Example
API Endpoint	CF Environment section, subaccount overview	<code>https://api.cf.us10.hana.ondemand.com</code>
Org Name	CF Environment section, under API Endpoint	<code>trial-us10-<your-id></code>
Space Name	Cloud Foundry ▢ Spaces	<code>dev</code>

Step 4 — Add HANA Cloud Entitlement

Before you can create a HANA Cloud instance, your Global Account must have the HANA Cloud entitlement assigned to your trial subaccount. In a trial account this is usually pre-configured, but verify it now to avoid a cryptic error during provisioning.

Navigate to Entitlements

Click the breadcrumb at the top of the cockpit to go back to your **Global Account** view. In the left navigation, click **Entitlements** ▾ **Entity Assignments**.

The screenshot shows the SAP BTP Cockpit interface. The left sidebar contains a navigation menu with 'Entitlements' selected. The main content area is titled 'Subaccount: trial - Entitlements'. It features a search bar with 'SAP HANA' and an 'Edit' button. Below is a table of service plans:

Service	Service Technical Name	Set Quota Limit	Quota Assignment	Subaccount Assignment	Remaining Global Quota	Remaining Quota	Actions
SAP HANA Cloud	hana-cloud			1 shared units	1 shared units		
Plan: relational-data-lake-free ⓘ							
SAP HANA Cloud	hana-cloud			1 shared units	1 shared units		
Plan: hana-free ⓘ							
SAP HANA Cloud	hana-cloud			1 shared units	1 shared units		
Plan: hana-cloud-connection-free ⓘ							
SAP HANA Cloud	hana-cloud-tools			1 shared units	1 shared units		
Plan: tools (Application) ⓘ							

Figure A.8 — The Entity Assignments screen under Entitlements. This is where service plans are allocated to subaccounts.

In the **Select Entities** dropdown, choose your **trial** subaccount. Click **Go**. You will see a list of services currently assigned to the subaccount.

Look for a row with **SAP HANA Cloud** in the service column and **hana** in the plan column. If it is already there with a quota of at least 1, you are set — skip to Step 5.

If HANA Cloud is not in the list, click **Configure Entitlements** ▾ **Add Service Plans**. In the dialog, search for “HANA Cloud”, expand the result, check the **hana** plan, and click **Add 1 Service Plan**. Then click **Save**.

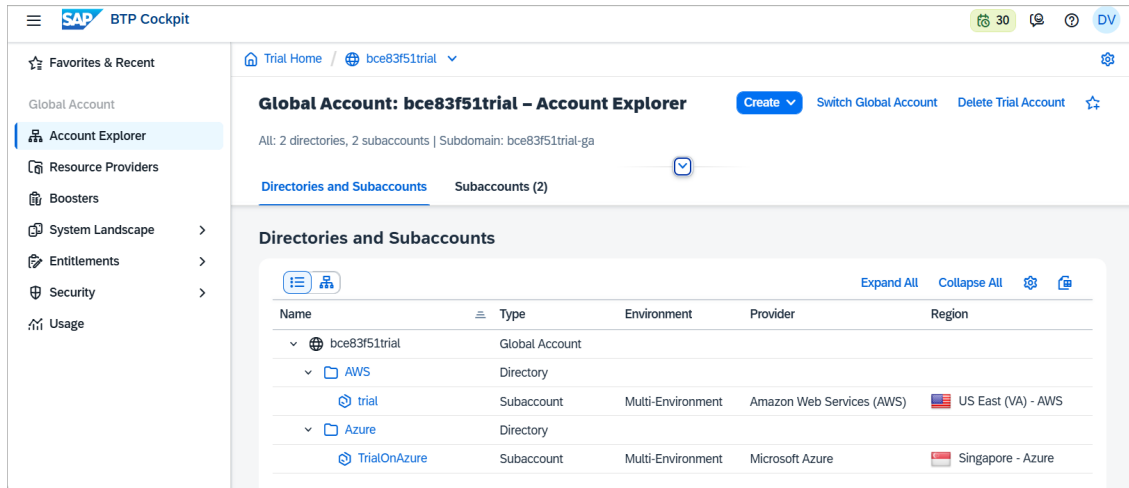


Figure A.9 — Global Account view. Navigate to Entitlements > Entity Assignments to manage service plan allocations.

Step 5 — Provision HANA Cloud

With the entitlement in place, you can now create the database instance.

Open the Service Marketplace

Navigate back to your **trial subaccount**. In the left navigation, click **Services** > **Service Marketplace**. The marketplace shows all services available to your subaccount. In the search bar, type “HANA Cloud.”

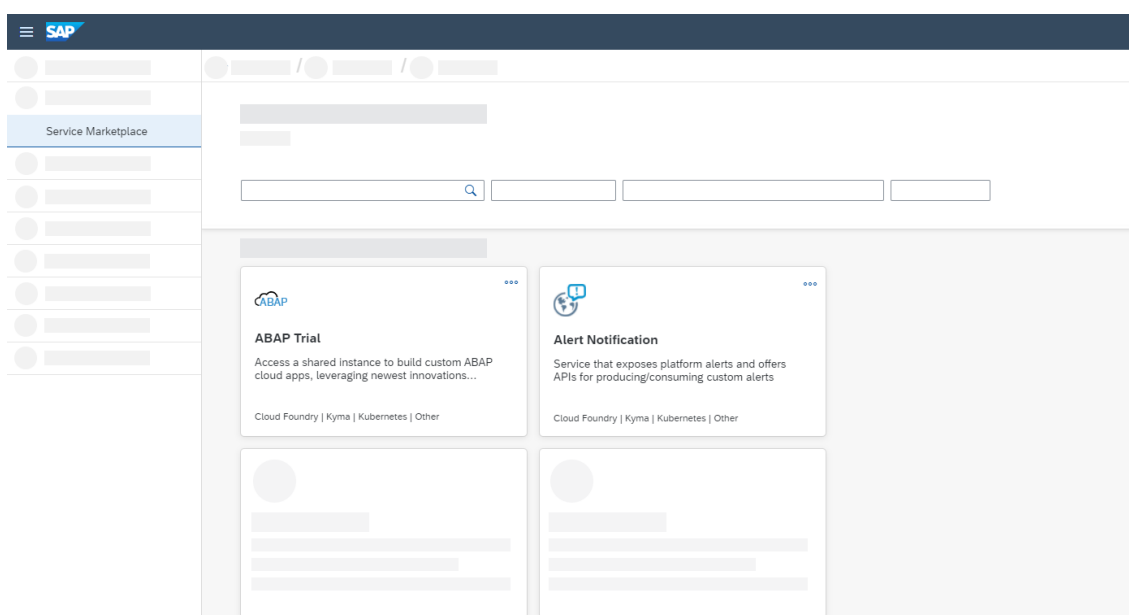


Figure A.10 — The BTP Service Marketplace. Search for “HANA Cloud” to find the tile.

Click the **SAP HANA Cloud** tile. On the service detail page, click **Create** in the upper right corner.

Fill in the Provisioning Form

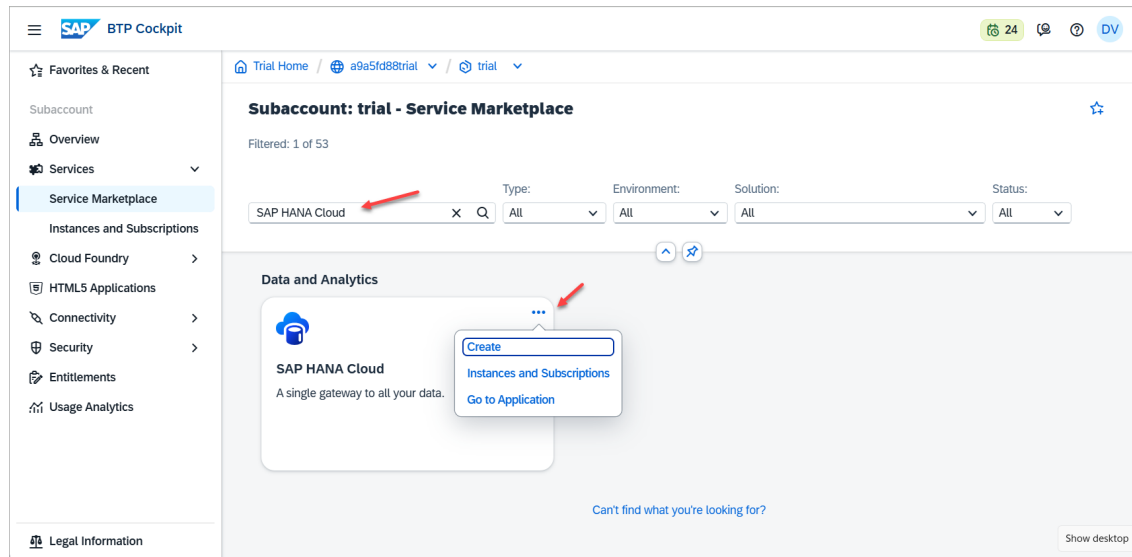


Figure A.11 — The HANA Cloud instance creation wizard. Fill in the instance name and administrator password.

Fill in the following fields:

Field	Recommended Value	Notes
Instance Name	msds-hana	Any name works; the code in this book uses <code>msds-hana</code> as a reference
Administrator Password	Your choice	Must be at least 8 characters with uppercase, lowercase, and a digit

Field	Recommended Value	Notes
Memory	30 GB	Trial default — do not reduce; the vector engine requires this headroom
Storage	120 GB	Trial default — leave as is

Critical: Write down the administrator password the moment you set it. You cannot retrieve it later. It will go into your `agents/.env` file as `HANA_PASSWORD`. If you lose it, you must delete the instance and reprovision.

Click **Create**. A confirmation dialog will appear. Click **Create** again to confirm.

Wait for Provisioning

BTP will redirect you to the **SAP HANA Cloud Central** tool, which shows all your HANA Cloud instances.

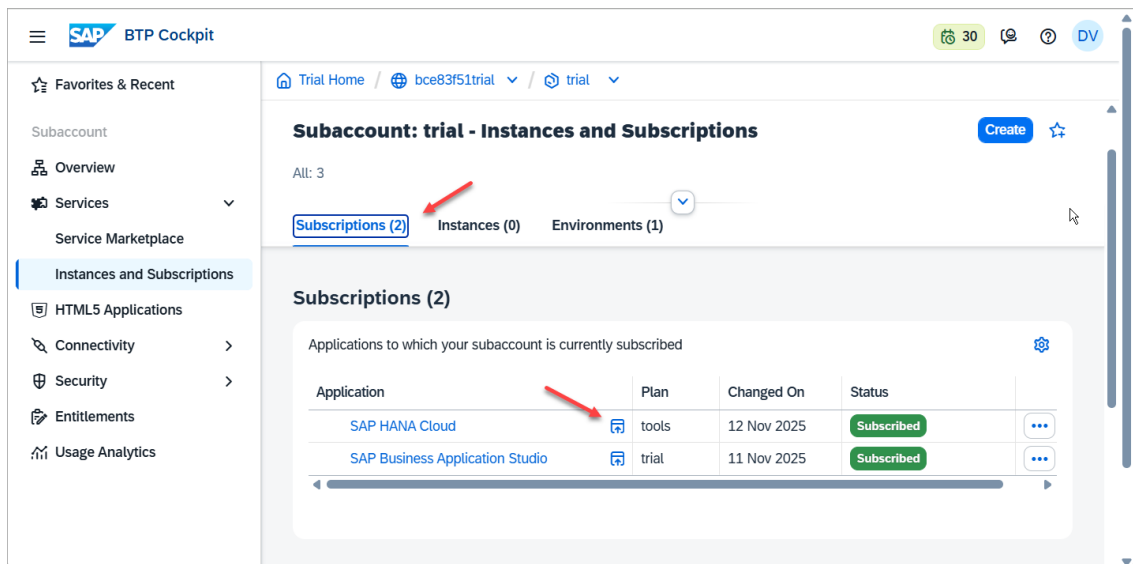


Figure A.12 — SAP HANA Cloud Central showing your new instance. The status will read “Starting” for 10–15 minutes.

The status will show **Starting** for approximately 10–15 minutes. This is normal. Do not close the browser tab. When the status changes to **Running**, the instance is ready.

If you see **Failed** status, the most common cause is that the entitlement was not properly

assigned. Go back to Step 4, verify the `hana` plan is assigned with quota ≥ 1 , and try creating the instance again.

Step 6 — Enable the Vector Engine

The `REAL_VECTOR` column type — which enables vector similarity search in HANA Cloud — is part of the HANA Cloud core engine. It is available by default in all HANA Cloud instances created from QRC2/2023 onwards, which includes all current trial instances. No separate toggle or purchase is required.

There are, however, two optional features that the code in Chapter 5 (document ingestion) benefits from: **Script Server** and **Document Store**. Enable these now to avoid re-editing the instance later.

Open HANA Cloud Central

If you are not already in HANA Cloud Central, navigate to it from your trial subaccount: click **Services** ▢ **Instances and Subscriptions**, find your HANA Cloud instance in the list, click the three-dot menu on the right side of its row, and select **Open in SAP HANA Cloud Central**.

Alternatively, if you are already in HANA Cloud Central, click on your instance name to open the instance detail view.

Edit the Instance Configuration

In HANA Cloud Central, click the three-dot menu (labeled **Actions**) next to your instance name. Select **Edit**.

The instance edit form opens. Scroll down until you see the **Additional Features** section. You will find checkboxes for:

- **Script Server** — enables server-side scripting and is required for certain built-in procedures
- **Document Store** — adds JSON document storage and is used by some ingestion patterns in Chapter 5

Check both boxes if they are not already checked. The vector engine (`REAL_VECTOR`) is

listed as a core capability in this section — confirm it shows as enabled.

Click **Save**. HANA Cloud will apply the configuration change, which takes 2–3 minutes. The instance status will briefly show **Updating** and return to **Running**.

Note on the vector engine: If you are reading the HANA Cloud documentation and see references to “Script Server” being required for vector search in older documentation, that applied to versions prior to QRC2/2023. Current HANA Cloud trial instances support `REAL_VECTOR` natively without Script Server. Script Server is still worth enabling for the scripting features used in Chapter 5.

Step 7 — Get the HANA Connection Details

You now need four pieces of information from your HANA Cloud instance. These go into the `agents/.env` file in Chapter 2.

Copy the SQL Endpoint

In SAP HANA Cloud Central, click the three-dot **Actions** menu next to your instance. Select **Copy SQL Endpoint**. This copies a string to your clipboard in the following format:

```
<instance-id>.hana.trial-us10.hanacloud.ondemand.com:443
```

The structure of your four connection parameters is:

Parameter	How to Derive It	Example
HANA_HOST	Everything in the SQL endpoint before the colon	abc123def456.hana.trial-us10.hanacloud.ondemand.com
HANA_PORT	The number after the colon	443
HANA_USER	Fixed value for trial instances	DBADMIN
HANA_PASSWORD	The password you set during provisioning in Step 5	(your chosen password)

Open `agents/.env` (or create it if it does not exist yet) and add these four lines:

```
1 HANA_HOST=<your-instance-id>.hana.trial-us10.hanacloud.ondemand.com
2 HANA_PORT=443
3 HANA_USER=DBADMIN
4 HANA_PASSWORD=<your-password>
```

Replace the placeholder values with the values specific to your instance. Do not add quotes around the values.

Security reminder: The `.env` file is listed in `.gitignore` in the project repository. Never commit this file to source control. If you are working in a shared environment, restrict read access to this file at the OS level.

Step 8 — Install the CF CLI

The CF CLI (Cloud Foundry Command Line Interface) is how you deploy applications to your BTP Cloud Foundry space from your local machine. You need it in Chapter 10 when you deploy the agents to production.

Install on macOS

If you have Homebrew installed, run:

```
1 brew install cloudfoundry/tap/cf-cli@8
```

If you do not have Homebrew, download the installer from <https://github.com/cloudfoundry/cli/releases> — choose the latest v8 release and the macOS binary.

Install on Windows

Download the Windows installer from cloudfoundry.org/cf-cli. Run the `.exe` installer and follow the prompts. The CF CLI will be added to your PATH automatically.

Verify the Installation

Open a terminal (or PowerShell on Windows) and run:

```
1 cf --version
```

You should see output similar to:

```
cf version 8.x.x+...
```

Authenticate to Your BTP Space

Use the Single Sign-On method to authenticate. This avoids storing your SAP password in the terminal and works reliably with multi-factor authentication.

```
1 cf login -a https://api.cf.us10.hana.ondemand.com --sso
```

Replace `us10` with your actual region code if different. The CLI will print a temporary passcode URL. Open that URL in your browser, copy the passcode it displays, and paste it back into the terminal.

After authentication, the CLI will show you a list of orgs and spaces. Select your trial org and then the `dev` space.

To verify you are in the right place, run:

```
1 cf target
```

The output should show your API endpoint, org name, and the `dev` space.

Verification Checklist

Before moving to Chapter 2, confirm each of the following items. Do not skip this — a missing item here causes non-obvious errors two or three chapters later.

- ☐ **BTP Cockpit login works.** You can open `cockpit.hanatrial.ondemand.com` and reach your Global Account home screen without being asked to register again.
- ☐ **Trial subaccount exists.** Clicking the **trial** tile takes you to a subaccount overview that shows a Cloud Foundry Environment section with an API Endpoint URL.
- ☐ **CF space named `dev` exists.** Under Cloud Foundry ▢ Spaces in the left navigation, you can see the `dev` space.
- ☐ **HANA Cloud instance is Running.** In SAP HANA Cloud Central (reachable via Services ▢ Instances and Subscriptions ▢ Actions menu ▢ Open in SAP HANA Cloud

Central), your instance shows green **Running** status.

- ❑ **HANA connection details are saved.** Your `agents/.env` file contains `HANA_HOST`, `HANA_PORT`, `HANA_USER`, and `HANA_PASSWORD` with real values. The host ends in `.hanacloud.ondemand.com`.
- ❑ **CF CLI is authenticated.** Running `cf target` in your terminal shows the correct API endpoint, org, and dev space — no authentication error.

All six checked? You are ready for Chapter 2.

Common Problems and Fixes

“I never received the verification email.” Check your spam folder. If the email is not there after 5 minutes, return to the trial signup page and request a new verification email. Gmail and Outlook accounts generally receive SAP trial emails without issue. Some corporate email servers block messages from `@sap.com` senders.

“My HANA Cloud instance status is stuck on Starting for more than 20 minutes.” This occasionally happens in trial environments under load. Refresh the HANA Cloud Central page. If the status has not changed after 30 minutes, click the three-dot Actions menu on the instance and select **Restart**. If the instance shows **Failed**, delete it and re-create it from Step 5.

“I cannot find the `hana` service plan in the marketplace.” The entitlement is not assigned. Go to your Global Account ▢ Entitlements ▢ Entity Assignments, select your trial subaccount, and confirm the SAP HANA Cloud `hana` plan is present with a quota of 1. If not, follow Step 4 to add it.

“`cf login` fails with ‘Invalid credentials’.” Use the `--sso` flag as shown in Step 8. Plain username/password login often fails if your SAP account requires multi-factor authentication or if the password contains special characters that the terminal interprets differently. The SSO passcode method avoids both issues.

“I set a HANA password but I am not sure I wrote it down correctly.” There is no password recovery for HANA Cloud trial instances. If you are unsure, the safest approach is to delete the instance and reprovision with a new password you can record with confidence. Deletion takes seconds; reprovisioning takes 10–15 minutes.

Appendix B: Google Cloud Setup — Vertex AI, Gemini, and Service Accounts

When to read this appendix. Chapter 2 walks through the full platform setup at a brisk pace. If you have worked with GCP before, Chapter 2 gives you everything you need. If GCP is new to you — or if you want the full step-by-step with security guidance — work through this appendix first, then return to Chapter 2. This appendix covers the same GCP steps in greater depth.

B.1 What You Will Have at the End

This appendix has a single, concrete goal: by the time you reach the verification checklist in section B.8, you will have:

1. A GCP project named `msds-hybrid-rag` (or any name you choose) with a Project ID recorded in your notes.
2. The Generative Language API and the Vertex AI API both enabled in that project.
3. Either an API key **or** a service account JSON key — the two authentication options are explained below, and you will choose one based on your situation.
4. A working Python test that calls `gemini-2.5-flash-preview-05-20` and receives a response.
5. A `.env` file in the `agents/` directory with the correct environment variable set.

Nothing here costs money. GCP provides \$300 USD in free credits when you activate a new account, and the total API usage for the MSDS demo in this book will cost under one dollar.

B.2 Two Authentication Options — Choose Before You Start

Before touching the GCP console, decide which authentication approach you will use. The book uses both at different stages: Option A locally, Option B for BTP Cloud Foundry deployment.

Option A: API Key

An API key is a string that begins with `AIza`. You pass it as the environment variable `GOOGLE_API_KEY`. The `google-generativeai` Python SDK reads this variable automatically when you call `genai.configure(api_key=os.getenv("GOOGLE_API_KEY"))`.

- Simpler to set up (two minutes, no JSON file).
- Suitable for local development and personal experimentation.
- Appropriate for anyone reading this book to learn the concepts.
- Not recommended for production BTP deployments — the key has no fine-grained scope binding and is easy to leak if you are not careful.

Option B: Service Account JSON Key

A service account is a non-human Google identity (e.g., `msds-rag-agent@your-project.iam.gserviceaccount.com`). You assign it the `Vertex AI User` role, then download a JSON key file. The file path is passed as `GOOGLE_APPLICATION_CREDENTIALS`. The Google SDK reads this path and authenticates all API calls using it.

- More complex to set up (about ten minutes).
- Strongly recommended for BTP Cloud Foundry deployment — the JSON key is stored as a BTP secret or as a Destination Service credential and never embedded in application code.
- Allows fine-grained IAM role binding, audit logging, and key rotation.

Which to use: If you are working through this book on a laptop for the first time, use Option A. When you reach Chapter 10 (BTP Deployment), the deployment guide will instruct you to create a service account at that point. If you want to complete all steps now, do Option A first to verify the connection, then follow section B.6 to create the service account.

B.3 Step 1 — Create a GCP Account

Open a browser and navigate to <https://cloud.google.com>.

If you have a Google account (Gmail, Google Workspace), click **Get started for free** and sign in with that account. If you do not have a Google account, create one first at <https://accounts.google.com/signup>.

GCP requires a credit card or debit card to activate the free trial. The card is used for identity verification — you will not be charged during the trial period. Google is explicit about this: you must manually upgrade to a paid account after the \$300 credit is exhausted. Until you do, billing is suspended and no charges are made.

Work through the activation form:

1. Agree to the Terms of Service.
2. Choose your country and confirm it matches your billing address.
3. Enter your payment details.
4. Click **Start my free trial**.

After activation, you are taken to the GCP Console at <https://console.cloud.google.com>. The banner at the top will confirm your free trial credit.



Figure 4: GCP free tier activation page

Note on existing GCP accounts. If you already have a GCP account and have used the \$300 credit, you can still follow this appendix. Create a new project under your existing account. Any Gemini API calls will be billed at

the standard rate, but the cost for the MSDS demo is under one dollar.

B.4 Step 2 — Create a New Project

Every GCP resource — APIs, credentials, service accounts, billing — lives inside a project. Create a dedicated project for this book's work.

In the GCP Console, look at the top navigation bar. You will see a project selector drop-down, which may say “My First Project” or the name of an existing project. Click it.

A dialog opens showing your existing projects. Click **New Project** in the upper-right corner of the dialog.

Fill in the form:

- **Project name:** `msds-hybrid-rag` (or any name — the name is for display only and can be changed later)
- **Organization:** Leave as “No organization” unless you are working inside a corporate Google Workspace account.
- **Location:** Leave as “No organization” for personal accounts.

Click **Create**. GCP will take about twenty seconds to provision the project.

Once created, use the project selector dropdown to switch to the new project. Confirm the project name appears in the top navigation bar.

Record the Project ID. The Project ID is different from the Project Name. It is a lowercase string, often `msds-hybrid-rag` followed by a number suffix (e.g., `msds-hybrid-rag-423901`). Find it on the project dashboard or in the project selector dialog. You will use this as `GCP_PROJECT_ID` in your `.env` file.

[Screenshot: GCP new project creation form]

B.5 Step 3 — Enable the APIs

GCP APIs are not active by default. You must explicitly enable each one you intend to use. This is a deliberate security design — it limits the attack surface of each project.

This book uses two APIs:

- **Generative Language API** — provides the `gemini-2.5-flash-preview-05-20` model and `text-embedding-004` via the `google-generativeai` Python SDK.
- **Vertex AI API** — provides the same models via the `langchain-google-genai` and `vertexai` SDKs, and is required for the BTP deployment configuration in Chapter 10.

Enable both now. It takes about two minutes each.

Enable the Generative Language API:

1. In the left navigation, click **APIs & Services > Library**.
2. In the search box, type `Generative Language API`.
3. Click the result titled “Generative Language API.”
4. Click **Enable**. The button will change to “Manage” once activation completes.

Enable the Vertex AI API:

1. Return to **APIs & Services > Library**.
2. Search for `Vertex AI API`.
3. Click **Vertex AI API** in the results.
4. Click **Enable**.

After both are enabled, go to **APIs & Services > Enabled APIs & Services** to confirm both appear in the list.

[SCREENSHOT: APIs & Services library showing search results for “Generative Language API” with the Enable button visible. Second screenshot: Enabled APIs list showing both “Generative Language API” and “Vertex AI API” with green status.]

B.6 Step 4A — Get an API Key (Option A — Recommended for Local Development)

If you are using Option A authentication, follow these steps to create an API key.

1. In the left navigation, click **APIs & Services > Credentials**.
2. Click **+ Create Credentials** at the top of the page.
3. Select **API Key** from the dropdown.

GCP creates the key immediately and shows it in a dialog. **Copy the key now**. The full

key value is shown only once in this dialog. After you close the dialog, you can still find the key in the Credentials list, but to see the full value again you must click into it. Copy it to a temporary notepad before closing.

Restrict the key (strongly recommended):

An unrestricted API key can be used to call any Google API in your project. Restrict it to the Generative Language API:

1. In the Credentials list, click the pencil icon next to your new key.
2. Under **API restrictions**, select **Restrict key**.
3. In the dropdown, select **Generative Language API**.
4. Click **Save**.

[SCREENSHOT: GCP Credentials page showing the API Key creation dialog with the key value visible. Second screenshot: API key restriction settings showing “Generative Language API” selected.]

Add the key to your `.env` file:

Open `agents/.env` (create it if it does not exist — use `agents/.env.example` as the template). Add the following line:

```
GOOGLE_API_KEY=AIzaSyXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Replace the value with your actual key. Save the file.

Verify `.gitignore` **covers** `.env`. The `agents/.gitignore` file in the book’s project already excludes `.env`. Confirm this by running:

```
1 cat agents/.gitignore | grep .env
```

If `.env` does not appear in the output, add the line `.env` to `agents/.gitignore` before doing anything else. An API key committed to a public repository will be detected by automated scanners within minutes and invalidated by Google — then you will need to create a new one.

B.7 Step 4B — Create a Service Account (Option B — Required for BTP Deployment)

Follow these steps if you are setting up the service account now. If you are using Option A for the moment, you can skip to section B.8 and return here before Chapter 10.

Create the service account:

1. In the left navigation, click **IAM & Admin > Service Accounts**.
2. Click **+ Create Service Account** at the top.
3. Fill in the form:
 - **Service account name:** `msds-rag-agent`
 - **Service account ID:** This auto-fills as `msds-rag-agent`. Leave it.
 - **Description:** Service account for MSDS hybrid RAG agent
4. Click **Create and Continue**.

Assign the role:

On the next screen, you will grant this service account access to your project:

1. Click the **Select a role** dropdown.
2. Search for `Vertex AI User`.
3. Select **Vertex AI User** (it appears under the Vertex AI category).
4. Click **Continue**, then click **Done**.

The `Vertex AI User` role grants permission to call the Vertex AI and Generative Language APIs. It does not grant access to GCP storage, databases, or administrative functions — this follows the principle of least privilege.

[SCREENSHOT: Service account creation form showing the name “msds-rag-agent” and description. Second screenshot: IAM role selection showing “Vertex AI User” selected.]

Download the JSON key:

1. In the Service Accounts list, click the email address of the `msds-rag-agent` account.
2. Click the **Keys** tab.
3. Click **Add Key > Create new key**.
4. Select **JSON** format.
5. Click **Create**.

The browser downloads a file named something like `msds-hybrid-rag-423901-a1b2c3d4e5f6.json`. This file contains the private key for the service account.

[SCREENSHOT: Service account Keys tab showing the “Add Key” dropdown and JSON format selection. Second screenshot: the key download confirmation dialog.]

Handle this file with care.

The JSON key file is a credential with the same power as a password. Anyone who has this file can call the Vertex AI API on your account. Several rules apply:

- Do not rename or move the file into the repository directory. Keep it in a directory outside the project, such as `~/.gcp/keys/`.
- Do not commit it to git under any circumstances. There is no recovery if you push it to a public repository — invalidate the key immediately and create a new one.
- Do not put it in Dropbox, Google Drive, or any sync service unless that service uses encryption at rest that you control.
- Rotate the key (delete and re-create) if you believe it has been exposed.

A safe location on macOS or Linux:

```
1 mkdir -p ~/.gcp/keys
2 mv ~/Downloads/msds-hybrid-rag-*.json ~/.gcp/keys/msds-rag-agent.json
3 chmod 600 ~/.gcp/keys/msds-rag-agent.json
```

The `chmod 600` command makes the file readable only by your user account.

Add the path to `.env`:

```
GOOGLE_APPLICATION_CREDENTIALS=/Users/yourname/.gcp/keys/msds-rag-agent.json
```

Use the absolute path. The `~` shorthand does not always expand correctly in all contexts — use the full path.

When `GOOGLE_APPLICATION_CREDENTIALS` is set, the Google SDK uses it automatically. You do not need to call `genai.configure()` — the credentials are picked up from the environment.

B.8 Step 5 — Test the Connection

Before closing the GCP console, verify that your credentials work. This test takes less than two minutes and eliminates an entire category of debugging later.

Create a temporary test file (do not put this inside the `agents/` directory — it is a throw-away):

```
1  # test_gemini.py — delete after testing
2  import os
3  from dotenv import load_dotenv
4  import google.generativeai as genai
5
6  # Load your .env file
7  load_dotenv("agents/.env")
8
9  api_key = os.getenv("GOOGLE_API_KEY")
10 if not api_key:
11     raise ValueError("GOOGLE_API_KEY not set in agents/.env")
12
13 genai.configure(api_key=api_key)
14
15 model = genai.GenerativeModel("gemini-2.5-flash-preview-05-20")
16 response = model.generate_content("Say hello in one sentence.")
17 print(response.text)
```

Run it from the book's project root:

```
1  cd /path/to/book-agentic-hybrid-rag-on-btp
2  python test_gemini.py
```

Expected output (the exact wording varies):

Hello! It's wonderful to meet you.

Any short, grammatical sentence confirms that the API call succeeded, the key is valid, and the model is accessible from your machine.

Common errors and their fixes:

Error message	Cause	Fix
API key not valid. Please pass a valid API key.	The key was not saved correctly in <code>.env</code> , or the <code>.env</code> path is wrong	Print <code>os.getenv("GOOGLE_API_KEY")</code> to confirm the value loads; check the path passed to <code>load_dotenv()</code>
403 Generative Language API has not been used in project	The API is not enabled	Return to section B.5 and enable it
404 models/gemini-2.5-flash-preview-05-20 is not found	The model name is wrong or not available in your region	Confirm the model name exactly — no extra spaces, correct hyphenation
ModuleNotFoundError: No module named 'google.generativeai'	Package not installed	Run <code>pip install google-generativeai python-dotenv</code>

Delete `test_gemini.py` after the test succeeds.

B.9 Step 6 — Understanding Free Tier and Costs

The GCP billing model for Generative AI is worth understanding before you run hundreds of API calls. Nothing in this book should surprise you with a large bill, but knowing the numbers builds confidence.

text-embedding-004

Embedding generation is priced per character of input. At the time of writing, `text-embedding-004` costs \$0.000 per character on the free tier up to 1,500 requests per minute. For the MSDS demo, each PDF page produces one embedding call, and the ten sample documents in the project total roughly 80 pages. The total embedding cost is effectively zero.

Even at paid rates, `text-embedding-004` is one of the least expensive API calls available — well under \$0.01 for the entire ingestion pipeline in this book.

gemini-2.5-flash-preview-05-20

Gemini 2.5 Flash is priced at approximately \$0.075 per million input tokens and \$0.30 per million output tokens for non-thinking responses. A typical MSDS query in this book sends about 2,000 input tokens (system prompt + retrieved context + question) and receives about 300 output tokens. That works out to roughly \$0.00015 per query — fifteen hundredths of a cent.

Running the full demo pipeline one hundred times — which is more than enough to complete all chapters — costs under \$0.02.

The \$300 credit

GCP's \$300 free credit expires after 90 days or when exhausted, whichever comes first. At the rates above, the credit is more than sufficient for the entire book plus several months of additional experimentation. The credit is not used for certain GCP services (compute quotas apply separately), but for the Vertex AI and Generative Language APIs, the credit applies in full.

Monitoring spend

To monitor your actual spend, go to **Billing > Reports** in the GCP Console. You can set a budget alert at **Billing > Budgets & alerts** to receive an email if you approach a threshold. Setting an alert at \$5 is a sensible precaution that takes two minutes to configure.

B.10 Verification Checklist

Before returning to Chapter 2, confirm each of the following. Check them off in order — do not skip ahead if one item is unresolved.

- ☐ **GCP project created.** You have a project named `msds-hybrid-rag` (or your chosen name) and you have recorded the Project ID. The project appears when you click the project selector in the GCP Console.
- ☐ **Both APIs enabled.** Navigate to **APIs & Services > Enabled APIs & Services** and confirm that both “Generative Language API” and “Vertex AI API” appear in the list with an active status.
- ☐ **Credentials in place.** Either `GOOGLE_API_KEY` is set in `agents/.env` (Option A) or `GOOGLE_APPLICATION_CREDENTIALS` points to a valid JSON key file outside the

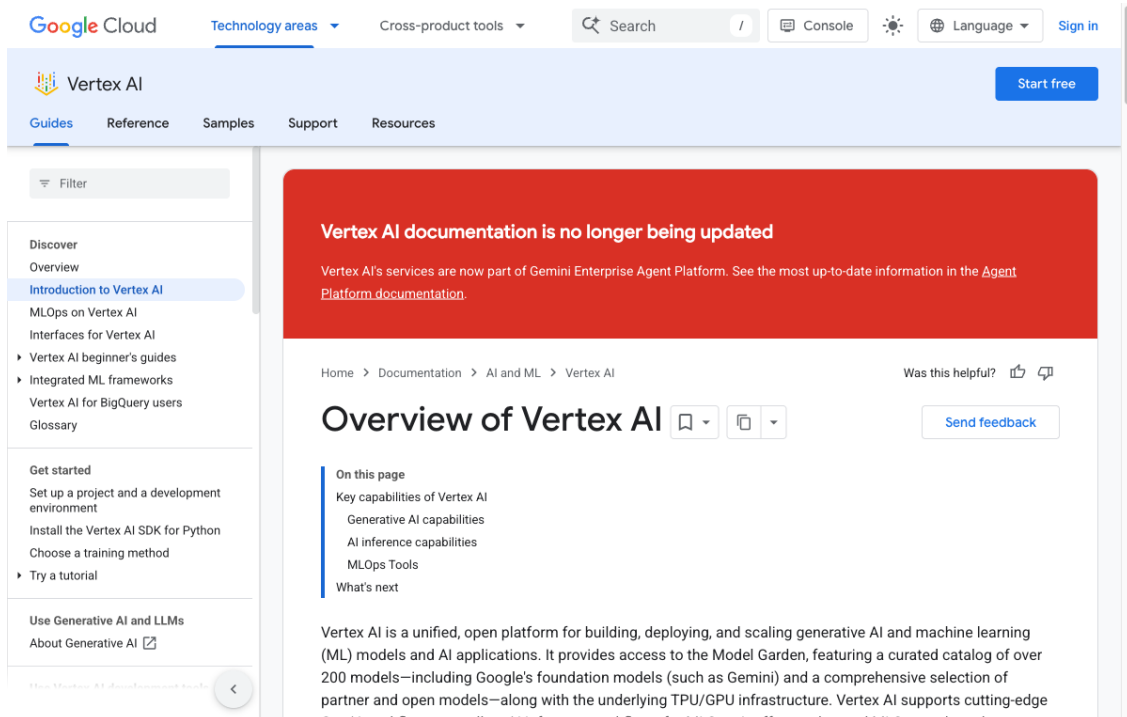


Figure 5: Vertex AI API overview and pricing page

repository directory (Option B). Not both — pick one for now.

- ☐ **.env excluded from git.** Run `git check-ignore -v agents/.env` from the project root. The output should confirm the file is ignored. If the command returns nothing (no output), the file is not ignored — add `.env` to `agents/.gitignore` immediately.
- ☐ **Test call succeeded.** You ran the Python test in section B.8 and received a valid sentence in response. The test file has been deleted.

If all five items are checked, your GCP environment is ready. Return to Chapter 2, section 3.4 to continue with the SAP BTP trial account setup.

Next step in Chapter 2. After completing this appendix, the GCP portion of Chapter 2 (section 3.4) will be familiar. That section adds one more step not covered here: creating a **BTP Destination** that stores the GCP service account credentials so that the deployed application on BTP Cloud Foundry can call Vertex AI without embedding credentials in the application bundle. That step requires the service account JSON key from section B.7 — if you skipped Option B, you will need to complete it before Chapter 10.

Appendix C — Full Code Reference

This appendix is a file-by-file reference for the complete repository. Each entry states what the file does, which chapter covers it in depth, and the key symbols it exports. Use this to locate any function or configuration value without re-reading the chapters.

Repository Layout

book-agentic-hybrid-rag-on-btp/

```
├─ agents/                                # Python FastAPI service
|   ├─ main.py
|   ├─ requirements.txt
|   ├─ manifest.yml
|   ├─ .env.example
|   └─ agents/                            # LangGraph chains and graphs
|       ├─ state.py
|       ├─ vector_chain.py
|       ├─ kg_chain.py
|       ├─ orchestrator.py
|       └─ supervisor.py
└─ srv/                                  # Service layer (HANA, Vertex AI)
    ├─ hdb_srv.py
    ├─ vector_srv.py
    ├─ kg_srv.py
    ├─ doc_srv.py
    └─ vertex_srv.py
├─ cap-srv/                             # Node.js CAP OData service
|   ├─ db/schema.cds
|   ├─ srv/service.cds
|   ├─ srv/service.js
|   ├─ package.json
|   ├─ mta.yaml
|   └─ .env.example
```

```
└─ MSDS_Ontology.ttl          # OWL ontology for the knowledge graph
└─ README.md
```

Python Agent Service (agents/)

agents/main.py

Chapters: 6, 8, 9

FastAPI application endpoint. Defines all HTTP routes, validates `material_number` against a strict regex, and delegates work to the service layer and LangGraph agents.

Routes

Method	Path	Chapter	Description
GET	/health	—	Liveness probe; returns <code>{"status": "ok"}</code>
POST	/query	8	Parallel hybrid RAG — runs vector and KG chains simultaneously
POST	/query-advanced	9	Multi-agent supervisor query; pass <code>use_supervisor: true</code> for complex questions
POST	/process-upload	6	Accepts PDF multipart upload; returns immediately with <code>{"status": "processing"}</code>
GET	/status/{material_number}	6	Polls dual-pipeline ingestion status from <code>MSDS_DOCUMENTS</code>
DELETE	/delete/{material_number}	6	Cascade-deletes vectors, named graph, and <code>MSDS_DOCUMENTS</code> row
POST	/admin/load-ontology	5	Loads <code>MSDS_Ontology.ttl</code> into HANA named graph

Key constants

Name	Value	Purpose
MATERIAL_RE	<code>^[A-Za-z0-9_-]+\$</code>	Guards all path parameters against SPARQL injection

Pydantic models

Model	Fields	Used by
QueryRequest	<code>question,</code> <code>material_number,</code> <code>history</code>	<code>/query</code>
QueryResponse	<code>answer,</code> <code>kg_sparql,</code> <code>kg_facts,</code> <code>vector_chunks,</code> <code>sources</code>	<code>/query,</code> <code>/query-advanced</code> responses
AdvancedQueryRequest	<code>question,</code> <code>material_number,</code> <code>history,</code> <code>use_supervisor</code>	<code>/query-advanced</code>

[agents/requirements.txt](#)

Chapter: 3 (environment setup)

Pins all Python dependencies. Key packages:

Package	Purpose
<code>fastapi,</code> <code>uvicorn</code>	HTTP server
<code>hdbcli</code>	SAP HANA Cloud driver
<code>google-cloud-aiplatform,</code> <code>vertexai</code>	Vertex AI SDK
<code>langgraph,</code> <code>langchain-google-genai,</code> <code>langchain-core</code>	LangGraph + Gemini integration
<code>PyMuPDF</code>	PDF text extraction
<code>python-dotenv</code>	<code>.env</code> file loading

`agents/manifest.yml`

Chapter: 11 (BTP deployment)

Cloud Foundry push descriptor for the Python service. Specifies the application name, memory allocation, Python buildpack, and environment variable bindings for the BTP CF environment.

`agents/agents/state.py`

Chapters: 8, 9

Defines the two `TypedDict` state schemas used by `LangGraph`. Every node in the graphs reads from and writes to these types.

Class	Chapter	Description
<code>HybridRAGState</code>	8	State for the parallel hybrid RAG orchestrator
<code>SupervisorState</code>	9	State for the multi-agent supervisor graph

`HybridRAGState` **fields**

Field	Type	Direction
<code>question</code>	<code>str</code>	Input
<code>material_number</code>	<code>str</code>	Input
<code>history</code>	<code>List[dict]</code>	Input
<code>vector_answer</code>	<code>str</code>	Vector chain output
<code>vector_chunks</code>	<code>List[dict]</code>	Vector chain output
<code>kg_answer</code>	<code>str</code>	KG chain output
<code>kg_sparql</code>	<code>str</code>	KG chain output (generated SPARQL)
<code>kg_facts</code>	<code>List[dict]</code>	KG chain output
<code>final_answer</code>	<code>str</code>	Merged output
<code>sources</code>	<code>List[str]</code>	Merged output
<code>error</code>	<code>Optional[str]</code>	Control

SupervisorState **fields** (additional fields beyond shared inputs/outputs)

Field	Type	Set by
sub_questions	Dict[str, str]	supervisor_node
specialists_needed	List[str]	supervisor_node
hazard_answer	str	hazard_agent
compliance_answer	str	compliance_agent
safety_answer	str	safety_agent

[agents/agents/vector_chain.py](#)

Chapter: 8

Implements the vector retrieval chain. Steps: embed question with `text-embedding-004`, run cosine similarity SQL against `MSDS_VECTORS`, summarise the top-5 chunks with Gemini.

Symbol	Description
<code>run_vector_chain(state)</code>	Main entry point; accepts <code>HybridRAGState</code> , returns dict of state updates (<code>vector_answer</code> , <code>vector_chunks</code>)
<code>COSINE_SEARCH_SQL</code>	Module-level SQL constant; uses <code>COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?))</code> ordered by score descending

[agents/agents/kg_chain.py](#)

Chapter: 8

Implements the knowledge graph retrieval chain. Steps: generate SPARQL from question using Gemini, execute via `SPARQL_EXECUTE`, retry with a broader fallback query if empty, summarise facts with Gemini.

Symbol	Description
<code>run_kg_chain(state)</code>	Main entry point; accepts <code>HybridRAGState</code> , returns dict of state updates (<code>kg_answer</code> , <code>kg_sparql</code> , <code>kg_facts</code>)
<code>SPARQL_GEN_PROMPT</code>	Prompt template instructing Gemini to generate a GRAPH-scoped SPARQL SELECT
<code>SPARQL_FALLBACK_PROMPT</code>	Broader prompt used when first SPARQL returns no rows
<code>GRAPH_URI_PREFIX</code>	"http://msds.knowledge-graph.org/MSDS_Graph/" — prefixed with <code>material_number</code> to form named-graph IRI
<code>_execute_sparql(sparql)</code>	Internal helper; calls <code>SPARQL_EXECUTE</code> stored procedure and returns rows

Important: every SPARQL triple pattern must be wrapped in `GRAPH <iri> { ... }` or HANA returns empty results silently.

[agents/agents/orchestrator.py](#)

Chapter: 8

Runs `run_vector_chain` and `run_kg_chain` in parallel using `ThreadPoolExecutor(max_workers=2)`. Wall-clock latency equals the slower of the two chains, not their sum.

Symbol	Description
<code>run_hybrid_rag(state)</code>	Dispatches both chains, merges state updates, calls <code>merge_results</code> , returns complete state dict
<code>merge_results(kg_answer, vector_answer, question, material_number)</code>	Synthesises both answers; if both are present, calls Gemini; if only one, returns it directly
<code>CHAIN_TIMEOUT_SECONDS</code>	30 — per-chain timeout before the future is abandoned

`agents/agents/supervisor.py`

Chapter: 9

Implements the multi-agent supervisor LangGraph. Three-node graph: `supervisor` decomposes the question into sub-questions, `specialists` runs domain agents in parallel, `summary` synthesises the final answer.

Symbol	Description
<code>build_supervisor_graph()</code>	Compiles and returns a <code>StateGraph(SupervisorState)</code> with nodes <code>supervisor</code> → <code>specialists</code> → <code>summary</code>
<code>supervisor_app</code>	Module-level pre-built instance; safe to import and call from <code>main.py</code>
<code>supervisor_node(state)</code>	Calls Gemini with <code>SUPERVISOR_PROMPT</code> to pick specialists and write sub-questions into state
<code>parallel_specialists_node(state)</code>	Runs only the specialists listed in <code>specialists_needed</code> via <code>ThreadPoolExecutor</code>
<code>hazard_agent(state)</code>	KG-focused; answers questions about GHS codes and hazard classifications
<code>compliance_agent(state)</code>	KG-focused; answers questions about exposure limits and regulatory thresholds
<code>safety_agent(state)</code>	Vector-focused; answers questions about precautions, PPE, first aid
<code>summary_agent(state)</code>	Synthesises answers from all invoked specialists into a single structured response

Service Layer (`agents/srv/`)

`agents/srv/hdb_srv.py`

Chapter: 4

Thread-local SAP HANA Cloud connection manager. Each thread in the FastAPI process gets its own `hdbcli` connection. Connections are never shared across threads.

Symbol	Description
<code>get_connection()</code>	Returns (or creates) the HANA connection for the current thread
<code>close_thread_connection()</code>	Closes and clears the current thread's connection
<code>_local</code>	<code>threading.local()</code> storage for per-thread connections

Connection parameters are read from environment variables `HANA_HOST`, `HANA_PORT`, `HANA_USER`, `HANA_PASSWORD`. TLS is always enabled (`encrypt=True`).

[agents/srv/vector_srv.py](#)

Chapter: 4

CRUD operations against the `MSDS_VECTORS` HANA table. The table is created lazily on first insert; the embedding dimension is inferred from the first embedding response.

Symbol	Description
<code>store_embedding(material_number, chunk_text, chunk_index, embedding)</code>	Inserts one chunk and its embedding vector
<code>upsert_vectors(conn, material_number, chunk_index, chunk_text, embedding)</code>	UPSERT by composite key; used by <code>doc_srv</code> during re-ingestion
<code>search_similar(question, material_number, top_k=5)</code>	Embeds question, runs <code>COSINE_SIMILARITY SQL</code> , returns list of <code>{chunk, chunk_index, score}</code>
<code>delete_vectors(material_number)</code>	Deletes all vectors for a material; returns row count
<code>count_vectors(material_number)</code>	Returns number of stored vectors for a material
<code>TABLE_NAME</code>	"MSDS_VECTORS"

[agents/srv/kg_srv.py](#)

Chapter: 5

SPARQL operations against HANA Cloud named graphs. Validates `material_number` before every SPARQL construction to prevent injection.

Symbol	Description
<code>store_triples(material_number, triples)</code>	Inserts a list of {subject, predicate, object} dicts as a named RDF graph; returns triples stored
<code>insert_triples(conn, material_number, triples)</code>	Explicit-connection variant of <code>store_triples</code> ; used by <code>doc_srv</code> for thread-local callers
<code>query_graph(material_number, question)</code>	Generates SPARQL with Gemini, executes it, returns {facts, sparql, count}
<code>delete_graph(material_number)</code>	Drops the named graph; returns <code>True</code> on success
<code>count_triples(material_number)</code>	Returns triple count for a material's named graph
<code>load_ontology(ttl_path)</code>	Loads a <code>.ttl</code> file into the ontology named graph; drops existing graph first
<code>extract_triples(text, material_number)</code>	Calls Gemini to extract structured triples from free text; returns list of dicts
<code>GRAPH_BASE</code>	"http://msds.knowledge-graph.org/MSDS_Graph"
<code>ONTOLOGY_GRAPH_IRI</code>	"http://msds.knowledge-graph.org/ontology"
<code>MATERIAL_RE</code>	<code>^[A-Za-z0-9_-]+\$</code>

Named-graph IRI pattern: `{GRAPH_BASE}/{material_number}`

[agents/srv/doc_srv.py](#)

Chapter: 6

Orchestrates the dual-pipeline PDF ingestion. Runs the vector pipeline and the KG pipeline in separate background threads. Status is persisted to `MSDS_DOCUMENTS` so polling survives process restarts.

Symbol	Description
<code>_run_dual_pipeline(tmp_path, material_number, material_name)</code>	Entry point for background execution; extracts PDF text, runs both pipelines in parallel, updates status
<code>_vector_pipeline(text, material_number, material_name)</code>	Chunks text, embeds each chunk, upserts into MSDS_VECTORS
<code>_kg_pipeline(text, material_number)</code>	Extracts triples with Gemini, stores them as a named graph
<code>_save_upload_sync(upload)</code>	Writes <code>UploadFile</code> to a temp file; returns path
<code>_mark_processing(material_number)</code>	UPSERTs MSDS_DOCUMENTS row with status <code>PROCESSING</code> before returning HTTP response
<code>chunk_text(text, material_name, chunk_tokens=500, overlap_tokens=50)</code>	Generator; yields overlapping chunks split on sentence boundaries
<code>extract_pdf_text(path)</code>	Opens PDF with PyMuPDF; returns <code>(full_text, page_count)</code> ; raises <code>ValueError</code> for image-only PDFs
<code>get_hana_connection()</code>	Thread-local HANA connection (mirrors <code>hdb_srv.get_connection()</code>)
<code>close_thread_connection()</code>	Releases thread-local connection

[agents/srv/vertex_srv.py](#)

Chapters: 4, 5

Vertex AI / Gemini client. Uses double-checked locking to initialise the SDK exactly once per process. All public functions are safe to call from multiple threads.

Symbol	Description
<code>get_llm()</code>	Returns a <code>GenerativeModel("gemini-2.5-flash-preview-05-20")</code> instance

Symbol	Description
<code>get_embedding_model()</code>	Returns a <code>TextEmbeddingModel</code> for <code>text-embedding-004</code>
<code>embed_text(text)</code>	Embeds a single string; returns a list of 768 float values
<code>generate_text(prompt)</code>	Calls Gemini with a plain-text prompt; returns response string
<code>_ensure_initialized()</code>	Internal; calls <code>vertexai.init(project, location)</code> behind a lock

Model identifiers: `LLM = gemini-2.5-flash-preview-05-20`, `embedding = text-embedding-004`.

CAP OData Service (`cap-srv/`)

`cap-srv/db/schema.cds`

Chapter: 10

CDS data model. Namespace `msds`.

Entity	Key	Description
Documents	<code>materialNumber</code>	Tracks each ingested MSDS document with dual-pipeline status columns
QueryLog	ID (UUID)	Optional audit trail for every query submitted via the UI

Documents columns

Column	Type	Notes
<code>materialNumber</code>	<code>String(100)</code>	Primary key; matches Python <code>material_number</code>
<code>materialName</code>	<code>String(500)</code>	Human-readable name

Column	Type	Notes
status	String(20)	KG pipeline: PENDING, PROCESSING, DONE, ERROR
kgError	String(1000)	KG error message if status = ERROR
vectorStatus	String(20)	Vector pipeline: same set of values
vectorError	String(1000)	Vector error message
createdAt, updatedAt	Timestamp	Lifecycle timestamps

[cap-srv/srv/service.cds](#)

Chapter: 10

OData V4 service definition. Service name `MSDSService`, path `/api`.

Artifact	Type	Description
Documents	Entity projection	Wraps <code>msds.Documents</code> with bound actions
QueryLog	Read-only projection	Wraps <code>msds.QueryLog</code>
<code>ingestDocument(materialName, fileContent, fileName)</code>	Bound action on Documents	Proxies to Python <code>/process-upload</code>
<code>deleteDocument()</code>	Bound action on Documents	Proxies to Python <code>DELETE /delete/{id}</code>
<code>query(question, materialNumber, history)</code>	Unbound action	Proxies to Python <code>POST /query</code>
<code>queryAdvanced(question, materialNumber, history, useSupervisor)</code>	Unbound action	Proxies to Python <code>POST /query-advanced</code>
<code>ingestionStatus(materialNumber)</code>	Unbound function	Proxies to Python <code>GET /status/{id}</code>

`cap-srv/srv/service.js`

Chapter: 10

CAP service handler. Implements all action handlers by forwarding requests to the Python FastAPI service. `AGENT_URL` is the single configuration point for the backend URL.

Symbol	Description
<code>agentPost(path, body)</code>	Internal helper; POST JSON to <code>AGENT_URL + path</code> , 60 s timeout
<code>agentDelete(path)</code>	Internal helper; DELETE to <code>AGENT_URL + path</code> , 30 s timeout
<code>srv.on("query", ...)</code>	Handles <code>query</code> action; serialises history to JSON before forwarding
<code>srv.on("queryAdvanced", ...)</code>	Handles <code>queryAdvanced</code> ; passes <code>use_supervisor</code> flag
<code>srv.on("ingestionStatus", ...)</code>	Handles <code>ingestionStatus</code> function
<code>srv.on("ingestDocument", "Documents", ...)</code>	Builds multipart form data and POSTs to <code>/process-upload</code>
<code>srv.on("deleteDocument", "Documents", ...)</code>	Calls <code>agentDelete</code> with the material number from URL params
<code>AGENT_URL</code>	<code>process.env.AGENT_URL</code> <code>\\ \\ </code> "http://localhost:8000"

Ontology (`MSDS_Ontology.ttl`)

Chapter: 5

OWL ontology in Turtle format. Namespace prefix: `msds: <http://msds.knowledge-graph.org/ontology#>`. Load via `POST /admin/load-ontology` after provisioning a fresh HANA instance.

Classes

IRI	Label	Description
msds:Material	Material	The MSDS document subject
msds:HazardCode	GHS Hazard Code	GHS classification codes (e.g. H225, H319)
msds:ExposureLimit	Occupational Exposure Limit	TWA / STEL values
msds:Precaution	Safety Precaution	Handling, storage, PPE instructions
msds:Supplier	Supplier	Manufacturer or distributor

Object properties (node to node)

Property	Domain	Range
msds:hasHazardCode	Material	HazardCode
msds:hasExposureLimit	Material	ExposureLimit
msds:requiresPrecaution	Material	Precaution
msds:hasSupplier	Material	Supplier

Datatype properties (node to literal)

Property	Domain	Description
msds:code	HazardCode	Hazard code string value
msds:description	Any	Free-text description
msds:limitValue	ExposureLimit	Numeric limit
msds:limitUnit	ExposureLimit	Unit (ppm, mg/m ³ , etc.)
msds:precautionText	Precaution	Precaution instruction text

Environment Variables Reference**Python agent service (agents/.env.example)**

Variable	Description	Where to find
HANA_HOST	HANA Cloud instance hostname	SAP BTP cockpit → HANA Cloud → Copy SQL Endpoint (hostname only)
HANA_PORT	HANA SQL/MDC port (default 443)	Same SQL endpoint — port after the colon
HANA_USER	HANA database user	Created in HANA Database Explorer or via SQL
HANA_PASSWORD	HANA user password	Set when creating the user
GCP_PROJECT_ID	Google Cloud project ID	GCP console → project selector
GCP_LOCATION	Vertex AI region (default us-central1)	Must match the region where Vertex AI APIs are enabled
GOOGLE_APPLICATION_CREDENTIALS	Path to GCP service account JSON key	GCP console → IAM → Service Accounts → Keys
LANGCHAIN_TRACING_V2	Enable LangSmith tracing (true/false)	LangSmith account settings
LANGCHAIN_ENDPOINT	LangSmith API endpoint	https://api.smith.langchain.com (fixed)
LANGCHAIN_API_KEY	LangSmith API key	LangSmith → Settings → API Keys

CAP service (cap-srv/.env.example)

Variable	Description	Where to find
AGENT_URL	Base URL of the running Python FastAPI service	Local: http://localhost:8000 ; CF: output of <code>cf app hybrid-rag-agent</code>

Material Number Format Constraint

All endpoints and service functions enforce the pattern `^[A-Za-z0-9_-]+$` on the `material_number` parameter.

Reason: material numbers are interpolated directly into SPARQL named-graph IRIs and

SPARQL query strings. Without this constraint, a crafted material number such as `ACET> . MALICIOUS_TRIPLE . <X` would break out of the IRI and inject arbitrary SPARQL — the triple-store equivalent of SQL injection. The regex allows only alphanumeric characters, hyphens, and underscores, none of which are SPARQL metacharacters.

The constraint is applied in three independent places:

1. `main.py` — `MATERIAL_RE` compiled regex validates all path parameters and request body fields via Pydantic validators before any function is called.
2. `kg_srv.py` — `_validate_material()` called at the top of every public function.
3. `service.js` — the CAP handler passes the material number through to the Python service unchanged; validation occurs server-side.

Quick curl Reference

Replace `localhost:8000` with the deployed CF application URL for production calls.

Liveness check

```
1 curl http://localhost:8000/health
```

Parallel hybrid RAG query

```
1 curl -X POST http://localhost:8000/query \
2   -H "Content-Type: application/json" \
3   -d '{
4     "question": "What are the GHS hazard codes?",
5     "material_number": "ACETONE-001",
6     "history": []
7   }'
```

Multi-agent supervisor query

```
1 curl -X POST http://localhost:8000/query-advanced \
2   -H "Content-Type: application/json" \
3   -d '{
4     "question": "What hazard codes apply and what PPE is required?",
5     "material_number": "ACETONE-001",
```



```

6     "use_supervisor": true,
7     "history": []
8 }'

```

Upload PDF for ingestion

```

1 curl -X POST http://localhost:8000/process-upload \
2     -F "file=@/path/to/ACETONE-001.pdf" \
3     -F "materialNumber=ACETONE-001" \
4     -F "materialName=Acetone"

```

Poll ingestion status

```

1 curl http://localhost:8000/status/ACETONE-001

```

Delete document (cascade)

```

1 curl -X DELETE http://localhost:8000/delete/ACETONE-001

```

Load ontology (run once after provisioning)

```

1 curl -X POST http://localhost:8000/admin/load-ontology

```

Cross-Reference: Chapter to File

Chapter	Primary files
3 — Platform Setup	requirements.txt, cap-srv/package.json, README.md
4 — Vector Search on HANA Cloud	srv/hdb_srv.py, srv/vector_srv.py, srv/vertex_srv.py
5 — Knowledge Graphs on HANA Cloud	srv/kg_srv.py, srv/vertex_srv.py, MSDS_Ontology.ttl
6 — PDF Ingestion Pipeline	srv/doc_srv.py, main.py (upload/status/delete endpoints)
7 — LangGraph Fundamentals	agents/state.py (TypedDict patterns)

Chapter	Primary files
8 — Parallel Hybrid RAG Agent	agents/state.py, agents/vector_chain.py, agents/kg_chain.py, agents/orchestrator.py, main.py (/query)
9 — Multi-Agent Supervisor	agents/supervisor.py, agents/state.py (SupervisorState), main.py (/query-advanced)
10 — CAP and Fiori UI	cap-srv/db/schema.cds, cap-srv/srv/service.cds, cap-srv/srv/service.js
11 — BTP Deployment	manifest.yml, cap-srv/mta.yaml

Appendix D: Joule A2A Integration

Joule 2.0 notice. At the time this book was written, SAP announced **Joule 2.0** with native Agent-to-Agent (A2A) protocol support as a first-class feature. The implementation in this appendix covers **Joule 1.x A2A** — the pattern that is available today on SAP S/4HANA Cloud Public Edition. The YAML structure, capability packaging, and deployment flow may change with Joule 2.0. The core concept — wrapping an external agent behind a Joule capability with a declarative YAML interface — will remain the same. We will update this appendix when Joule 2.0 is generally available.

Enterprise prerequisite. This appendix requires **SAP S/4HANA Cloud Public Edition** or **SAP SuccessFactors** with Joule enabled. It is NOT available on BTP free trial. If you do not have this subscription, read this appendix to understand the pattern and return to it when you have access.

Throughout this book we have built the intelligence layer: a Python FastAPI service that orchestrates a LangGraph hybrid RAG agent, stores knowledge in SAP HANA Cloud, and returns accurate answers about Material Safety Data Sheets. In Chapter 9 we gave that agent a Fiori Elements front-end. This appendix adds a third access channel: SAP Joule, the AI assistant embedded directly in S/4HANA and SuccessFactors.

The value is significant. A procurement specialist in S/4HANA who is looking at a purchase order for Acetone can ask Joule “what are the PPE requirements for this material?” without leaving their context. Joule routes that question to our hybrid RAG agent, which queries the knowledge graph and vector store, and returns the answer inside the Joule chat panel. The user never opens a separate application.

That routing is handled by the A2A protocol. This appendix explains how it works, walks through every YAML file in the `joule/` directory of the project, and shows you how to package and deploy the capability.

D.1 What Is Joule A2A?

Joule is SAP’s conversational AI assistant. It is embedded in S/4HANA Cloud, SuccessFactors, and other SAP products. When a user types a question, Joule matches it to a registered scenario, executes a function, and presents the result.

In early versions of Joule, functions were limited to SAP’s own built-in skills. The A2A (Agent-to-Agent) extension allows Joule to call an external HTTP endpoint and treat the response as a Joule answer. The external endpoint — in our case, the Python FastAPI service — is the “agent.” Joule is the “Joule agent” calling it.

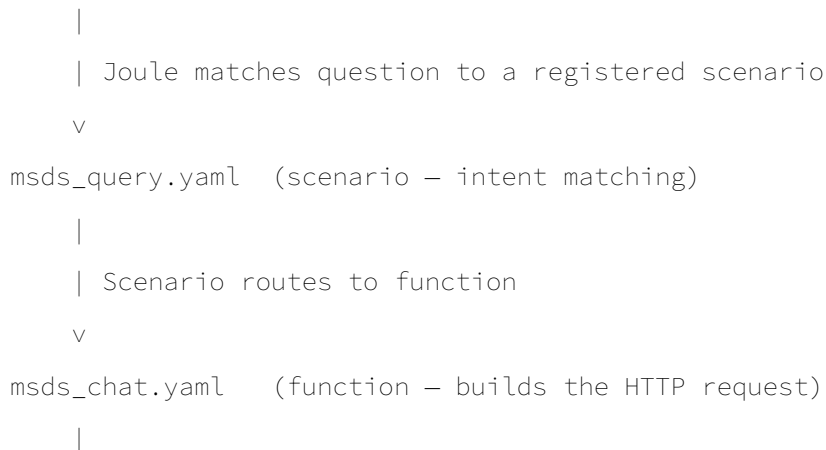
The A2A protocol used in Joule 1.x is a lightweight request-response protocol over HTTP. Joule sends a POST request to your agent endpoint with a message payload, and your agent returns a structured response. The YAML files in the `joule/` directory describe the capability: what scenarios trigger it, what parameters to pass, and how to map the result back to Joule’s response format.

This is deliberately simple. There is no persistent connection, no streaming, and no call-back mechanism. Joule sends a request and waits up to 15 seconds for a response. That constraint shapes everything about the endpoint design, as we will discuss in section D.6.

D.2 How the Pieces Fit Together

Before diving into individual files, it helps to see the full call chain.

User types question in Joule chat panel (S/4HANA or SuccessFactors)



```

    | system_alias resolves MSDS_KG_Agent -> BTP Destination
  v
BTP Destination: MSDS-KG-RAG-Agent
    |
    | HTTP POST /a2a
  v
Python FastAPI (agents/main.py – the /a2a endpoint)
    |
    |-- Knowledge Graph chain (SPARQL -> HANA RDF store)
    |-- Vector chain          (embedding -> HANA vector store)
    |-- LLM synthesis          (Vertex AI Gemini)
  v
JSON response {"type": "finalAnswer", "message": "..."}
    |
    | Function maps chat_result.body.message to answer
  v
Joule presents answer in chat panel

```

The BTP Destination is the key indirection point. It decouples the YAML files from the actual URL of your FastAPI service. When you redeploy the agent to a new URL, you update the destination once in BTP Cockpit; the YAML files are unchanged.

D.3 The File Structure

The complete `joule/` directory in this project contains the following files:

```

joule/
├── da.sapdas.yaml
├── msds_capability/
│   ├── capability.sapdas.yaml
│   ├── capability_context.yaml
│   ├── functions/
│   │   ├── msds_chat.yaml
│   │   └── msds_status.yaml
├── scenarios/

```

```
├─ msds_query.yaml
└─ msds_status.yaml
```

There are five distinct file types, each with a specific responsibility. We examine them in the order Joule processes them.

D.4 The Five Files Explained

D.4.1 `da.sapdas.yaml` — The Root Agent Declaration

```
1  schema_version: 1.0.0
2  name: msds_kg_agent
3  capabilities:
4    - type: local
5      folder: ./msds_capability
```

This is the entry point. It tells the Joule deployment tooling (`sapdas` CLI or Joule Admin UI) that there is an agent named `msds_kg_agent` and that its capability lives in the `./msds_capability` folder.

The `type: local` means the capability folder is included inside the same `.daar` package. An alternative is `type: remote`, which references a capability published to a shared registry — a pattern used when multiple agents share a common capability. For our purposes, `local` is simpler and self-contained.

The `schema_version: 1.0.0` refers to the outer agent declaration format. The capability itself uses a different, higher schema version, which is declared inside `capability.sapdas.yaml`.

D.4.2 `capability.sapdas.yaml` — The Capability Metadata

```
1  schema_version: 3.28.0
2  metadata:
3    display_name: MSDS Safety Knowledge Agent
4    namespace: com.sap.msds
5    name: msds_kg_agent
6    version: 2.0.0-SNAPSHOT
```

```

7   description: >--
8       Answer questions about Material Safety Data Sheets (MSDS): hazards,
9       PPE requirements, first aid, storage conditions, and regulatory
        compliance.
10      Uses hybrid RAG (Knowledge Graph + Vector search) for accurate answers.
11  system_aliases:
12      MSDS_KG_Agent:
13          destination: MSDS-KG-RAG-Agent

```

This file describes the capability to Joule’s capability registry. Several fields deserve explanation.

The `schema_version: 3.28.0` is the capability schema version, not the agent declaration version. The two schemas are independent. Use the version that matches your Joule environment; as of this writing, 3.28.0 is current for S/4HANA Cloud Public Edition 2024.

The `namespace: com.sap.msds` provides a unique prefix to avoid name collisions with other capabilities in the same Joule instance. Use a reverse-DNS style string based on your organization’s domain, not `com.sap` for your own capabilities in production; the `com.sap` namespace is used here because this is a reference implementation.

The `version: 2.0.0-SNAPSHOT` uses the Maven convention for pre-release versions. The `-SNAPSHOT` suffix signals that this version is not yet stable. The `.daar` package filename derives from these fields, as we will see in section D.8.

The `system_aliases` section is where the BTP Destination mapping lives. The alias `MSDS_KG_Agent` is the logical name used inside function YAML files. The `destination: MSDS-KG-RAG-Agent` is the name of the BTP Destination that Joule will look up at run-time to resolve the actual URL. This indirection is the reason the YAML files contain no hard-coded URLs.

D.4.3 capability_context.yaml — Persisted Variables

```

1  variables:
2      - name: thread_id
3      - name: material_number
4      - name: task_id

```

Joule maintains a capability context per conversation. Variables declared here persist across multiple turns of the same conversation. When a user asks “what are the haz-

ards?” and then follows up with “what about the storage conditions?”, Joule can carry the `material_number` forward without the user repeating it.

The scenario YAML files (section D.4.5) write to these variables at the end of each function call. Functions read from them via `$capability_context.material_number`. This is how the agent maintains conversational state without any server-side session — the state rides in the Joule context.

D.4.4 Function YAML Files — The HTTP Request Builders

Function files define how Joule constructs and sends an HTTP request to the agent. The project has two functions.

msds_chat.yaml handles conversational queries:

```

1 parameters:
2   - name: user_message
3     optional: true
4   - name: material_number
5     optional: true
6 action_groups:
7   - actions:
8     - type: status-update
9       message: Searching MSDS knowledge base...
10    - type: set-variables
11      variables:
12        - name: msg
13          value: "<? user_message != null ? user_message :
14$transient.input.text.raw ?>"
15    - type: set-variables
16      variables:
17        - name: tid
18          value: "default"
19    - type: set-variables
20      variables:
21        - name: mat
22          value: "<? material_number != null ? material_number : '' ?>"
23    - type: api-request
24      method: POST
25      system_alias: MSDS_KG_Agent

```



```

25     path: /a2a
26     headers:
27         Content-Type: application/json
28     body: >
29         {"Message": "<? msg ?>", "contextId": "<? tid ?>", "taskId": "<?
mat ?>"}
30     timeout: 15
31     result_variable: chat_result
32 result:
33     answer: "<? chat_result.body.message ?>"
34     thread_id: "<? tid ?>"
35     material_number: "<? chat_result.body.taskId != null ?
chat_result.body.taskId : mat ?>"

```

msds_status.yaml handles document processing status checks:

```

1 parameters:
2     - name: material_number
3       optional: true
4 action_groups:
5     - actions:
6         - type: status-update
7           message: Checking processing status...
8         - type: set-variables
9           variables:
10              - name: mat
11                value: "<? material_number != null ? material_number : '' ?>"
12         - type: api-request
13           method: POST
14           system_alias: MSDS_KG_Agent
15           path: /a2a
16           headers:
17               Content-Type: application/json
18           body: >
19               {"Message": "status", "contextId": "status-check", "taskId": "<?
mat ?>"}
20           timeout: 15
21           result_variable: chat_result
22 result:

```

```

23   status_message: "<? chat_result.body.message ?>"
24   material_number: "<? mat ?>"

```

The function file is the most complex part of the A2A configuration. The `action_groups` field contains a sequential list of actions that Joule executes before making the HTTP request.

The `status-update` action shows a loading message in the Joule chat panel while the request is in flight. This is important for user experience: a 10-second wait without feedback feels like a hang. Showing “Searching MSDS knowledge base...” communicates that work is in progress.

The `set-variables` actions build local variables that are used in the request body. Notice the ternary expression in the `msg` variable assignment — this is the SpEL-like expression language discussed in section D.5.

The `api-request` action sends the HTTP request. The `system_alias: MSDS_KG_Agent` references the alias defined in `capability.sapdas.yaml`, which resolves to the BTP Destination. The `timeout: 15` is Joule’s maximum wait time in seconds; any response that takes longer will be discarded. The `result_variable: chat_result` names the variable that receives the parsed response body.

The `result` section at the bottom maps response fields to named outputs. These output names are what scenario files reference via `$target_result.*`.

D.4.5 Scenario YAML Files — Intent Matching and Routing

Scenario files are what Joule uses to decide which function to call. They contain plain-language descriptions that Joule’s NLU engine matches against the user’s input.

msds_query.yaml:

```

1  description: >-
2    Answer questions about Material Safety Data Sheets (MSDS) for chemicals
   and materials.
3    This covers hazard information, GHS classification, PPE requirements,
   protective equipment,
4    first aid procedures, storage conditions, incompatible materials,
   regulatory compliance,

```

```

5   REACH, flammability, toxicity, exposure limits, and safety handling
   instructions.
6   Examples: hazards of Acetone, PPE for Methanol, how to store WD-40,
   material 200001001 safety data.
7   target:
8     type: function
9     name: msds_chat
10    parameters:
11      - name: material_number
12        value: $capability_context.material_number
13  response_context:
14    - value: $target_result.answer
15      description: MSDS safety answer from knowledge base
16  capability_context:
17    - name: thread_id
18      value: $target_result.thread_id
19    - name: material_number
20      value: $target_result.material_number

```

msds_status.yaml:

```

1  description: >-
2    Check the processing status of an MSDS material document including
   ingestion progress,
3    number of knowledge graph triples extracted, number of vectors stored,
4    and whether document processing is complete or still in progress.
5  target:
6    type: function
7    name: msds_status
8    parameters:
9      - name: material_number
10        value: $capability_context.material_number
11  response_context:
12    - value: $target_result.status_message
13      description: Processing status for the material
14  capability_context:
15    - name: material_number
16      value: $target_result.material_number

```

The **description** field is the most important part of a scenario file. Write it as a dense

enumeration of the topics the scenario covers, not as a sentence. Joule’s NLU engine scores the user’s input against this description using semantic similarity; the richer and more specific the description, the more accurately Joule routes questions to the right scenario. The examples at the end (“hazards of Acetone, PPE for Methanol”) act as few-shot hints.

The `target` section names the function to call and passes parameters from the capability context. Because `material_number` is stored in `$capability_context.material_number` from a previous turn, Joule can carry it forward without the user repeating it.

The `response_context` section tells Joule what text to present in the chat panel. The `$target_result.answer` is the output field named in the function’s `result` section.

The `capability_context` section writes values back to the persistent context after the function completes. This is the write path; the `$capability_context.*` reference in the function’s parameters is the read path.

D.5 The SpEL-like Expression Language

Joule’s YAML files use a small expression language enclosed in `<? ?>` delimiters. It is similar to Spring Expression Language (SpEL) but is not identical. Understanding the four reference patterns you will encounter is sufficient for most capability development.

`<? variable ?>` — evaluates a local variable set by a `set-variables` action. Used inside request body strings and result mappings.

`$transient.input.text.raw` — the raw text the user typed in the Joule chat panel, before any NLU processing. This is the fallback message when no explicit `user_message` parameter is passed by the scenario. The ternary expression `<? user_message != null ? user_message : $transient.input.text.raw ?>` covers both cases: an explicit parameter from the scenario or the raw user input.

`$capability_context.material_number` — reads a named variable from the persistent capability context. This is how multi-turn conversations carry state across requests without a server-side session.

`$target_result.answer` — reads a named field from the function’s `result` section. Used in scenario files to extract the value that Joule should present to the user, and

to extract values that should be written back to capability context.

Null-safe expressions. Joule's expression language does not throw on null; a null reference evaluates to an empty string in string contexts and to `null` in conditional expressions. The pattern `<? chat_result.body.taskId != null ? chat_result.body.taskId : mat ?>` handles the case where the agent response does not include a `taskId` field.

Accessing nested response fields. `chat_result.body.message` navigates the HTTP response: `chat_result` is the variable named in `result_variable`, `.body` is the parsed JSON response body, and `.message` is a field in that body. If the response body were `{"message": "...", "status": "ok"}`, then `chat_result.body.status` would access the status field.

D.6 The /a2a Endpoint

The Python FastAPI service exposes a `POST /a2a` endpoint that Joule calls. This endpoint is not in the main `agents/main.py` file in the repository skeleton — you will add it as part of this appendix. Here is what it must do and why.

D.6.1 Dual Format Support

The A2A protocol in Joule 1.x uses a flat JSON format:

```
1 {
2   "Message": "What are the hazards of Acetone?",
3   "contextId": "default",
4   "taskId": "200001001"
5 }
```

The emerging standard A2A protocol (as defined by the open A2A specification that other AI frameworks are adopting) uses a JSON-RPC envelope:

```
1 {
2   "jsonrpc": "2.0",
3   "method": "message/send",
4   "params": {
```

```

5     "message": {
6         "parts": [{"text": "What are the hazards of Acetone?"}]
7     },
8     "contextId": "default",
9     "taskId": "200001001"
10 }
11 }

```

Supporting both formats in the `/a2a` endpoint means the same service can be called by Joule 1.x today and by Joule 2.0 or other A2A-compatible clients in the future. The detection logic is straightforward: if the request body contains a `jsonrpc` field, extract the message from `params.message.parts[0].text`; otherwise read from `Message` directly.

The response uses the flat format in both cases, since Joule 1.x is the primary consumer:

```

1 {
2     "type": "finalAnswer",
3     "message": "Acetone (UN1090) is a highly flammable liquid...",
4     "contextId": "default",
5     "taskId": "200001001"
6 }

```

D.6.2 The 15-Second Timeout Constraint

Joule imposes a hard 15-second timeout on A2A requests, as declared in the function YAML (`timeout: 15`). Any response that arrives after 15 seconds is silently discarded. The user sees nothing, which is a poor experience.

The hybrid RAG agent in Chapter 7 runs the knowledge graph chain and the vector chain in parallel using `asyncio.gather`. On a cold start — first request after the service has been idle — this typically takes 4 to 6 seconds including the Gemini LLM call. On a warm service it runs in 2 to 4 seconds. This is comfortably within 15 seconds under normal conditions.

The cases that can exceed 15 seconds are:

- A cold-start combined with a complex SPARQL query that traverses many graph hops
- A vector search over an unusually large embedding corpus
- A Vertex AI API call that encounters rate limiting or a degraded region

Defensive patterns to stay within the timeout:

- Set an explicit timeout on the LLM call (`request_options={"timeout": 10}` for Vertex AI clients) so the agent fails fast rather than waiting indefinitely
- Limit SPARQL query depth to two hops maximum in the knowledge graph chain
- Cap the number of vector chunks returned to 5 before sending to the LLM

The `/a2a` endpoint should also wrap its internal call in a Python `asyncio.wait_for` with a 12-second limit, which gives Joule a 3-second margin to receive and process the response before its own timeout fires.

D.6.3 The Status Path

When the `Message` field is the literal string `"status"`, the endpoint checks the ingestion status of the material identified by `taskId`. It queries HANA for the triple count in the knowledge graph and the vector count in the vector store, and returns a natural-language summary:

Material 2000001001: processing complete.

Knowledge graph: 847 triples. Vector store: 23 chunks.

This is the path used by the `msds_status.yaml` function. Because it queries HANA rather than calling the LLM, it reliably completes in under 2 seconds.

D.7 The BTP Destination

The `system_aliases` section of `capability.sapdas.yaml` maps the logical name `MSDS_KG_Agent` to a BTP Destination named `MSDS-KG-RAG-Agent`. That destination must exist in the same BTP subaccount where Joule is configured.

To create the destination in BTP Cockpit:

1. Navigate to your BTP subaccount and open **Connectivity > Destinations**.
2. Click **New Destination**.
3. Fill in the fields:

Field	Value
Name	MSDS-KG-RAG-Agent
Type	HTTP

Field	Value
URL	The URL of your deployed FastAPI service (e.g., <code>https://msds-rag-agent.cfapps.eu10.hana.ondemand.com</code>)
Proxy Type	Internet
Authentication	NoAuthentication (or OAuth2ClientCredentials if your service requires a token)

4. Add the additional property:

Property	Value
HTML5.DynamicDestination	true

5. Save and test the connection using the **Check Connection** button.

The destination name in BTP Cockpit must match the `destination` field in `capability.sapdas.yaml` exactly, including case. `MSDS-KG-RAG-Agent` and `msds-kg-rag-agent` are different destinations.

For production, use `OAuth2ClientCredentials` authentication. Configure an XSUAA service instance on BTP, bind the FastAPI service to it, and set the destination's authentication type to `OAuth2ClientCredentials` with the client ID and secret from the service key. This ensures that only Joule — via the destination — can call the `/a2a` endpoint.

D.8 Building the .daar Package

The `.daar` file is simply a ZIP archive of the capability folder, with a naming convention that encodes the namespace, capability name, and version. For this project:

```
com.sap.msds_msds_kg_agent_2.0.0-SNAPSHOT.daar
```

The name is constructed as: `{namespace}_{name}_{version}.daar`, with all dots in the namespace replaced by dots (kept as-is) and spaces replaced with underscores.

To create the package from the `joule/` directory:


```

1 cd joule/
2 zip -r com.sap.msds_msds_kg_agent_2.0.0-SNAPSHOT.daar msds_capability/

```

The `da.sapdas.yaml` file at the root of the `joule/` directory is not included in the `.daar` package — it is used only by the `sapdas` CLI tool for local testing. The `.daar` package contains only the `msds_capability/` folder.

Verify the package structure after creating it:

```

1 unzip -l com.sap.msds_msds_kg_agent_2.0.0-SNAPSHOT.daar

```

You should see:

Archive: com.sap.msds_msds_kg_agent_2.0.0-SNAPSHOT.daar

Length	Date	Time	Name
...	2024-01-01	00:00	msds_capability/capability.sapdas.yaml
...	2024-01-01	00:00	msds_capability/capability_context.yaml
...	2024-01-01	00:00	msds_capability/functions/msds_chat.yaml
...	2024-01-01	00:00	msds_capability/functions/msds_status.yaml
...	2024-01-01	00:00	msds_capability/scenarios/msds_query.yaml
...	2024-01-01	00:00	msds_capability/scenarios/msds_status.yaml

If `da.sapdas.yaml` appears in the listing, the archive was created from the wrong directory. Delete it and repeat from inside `joule/`.

D.9 Deploying to Joule

Deployment requires administrator access to the Joule configuration in your SAP S/4HANA Cloud or SuccessFactors tenant.

Via the Joule Admin UI:

1. Open the SAP AI Launchpad or the S/4HANA Cloud administration tile for Joule.
2. Navigate to **Agent Configuration** or **Capability Management** (the exact label varies by release).
3. Click **Upload Capability** or **Import .daar**.

4. Select the `com.sap.msds_msds_kg_agent_2.0.0-SNAPSHOT.daar` file.
5. Joule validates the package structure and the schema version. If validation passes, the capability appears in the capability list with status **Inactive**.
6. Click **Activate**. The capability is now live for users in the configured scope (all users, or a specific group depending on your Joule configuration).

Via the sapdas CLI (if available in your environment):

```
1 sapdas deploy --file com.sap.msds_msds_kg_agent_2.0.0-SNAPSHOT.daar \
2 --tenant <your-tenant-id>
```

The CLI requires the `sapdas` tooling, which is distributed as part of the Joule Developer Experience package — available through SAP One Support Launchpad for tenants with the appropriate entitlement.

After deployment, verify the BTP Destination is reachable from Joule's runtime environment. The **Check Connection** result in BTP Cockpit confirms reachability from BTP, but Joule's runtime may sit in a different network segment. If Joule returns an error after deployment, use the Joule capability debug logs (in AI Launchpad under the capability details) to see the raw HTTP error from the destination call.

D.10 Testing in Joule

Open the Joule chat panel in your S/4HANA Cloud or SuccessFactors session. The following conversation illustrates the expected behavior.

Cold start — establishing material context:

User: What are the hazards of Acetone?

Joule: [Searching MSDS knowledge base...]

Acetone (UN1090, CAS 67-64-1) is classified as a highly flammable liquid and vapour (GHS02) and causes serious eye irritation (GHS07). The flash point is -20 degrees Celsius. The substance is not classified as carcinogenic or reproductively toxic under REACH.

Source: hybrid RAG – knowledge graph (GHS classification, UN number)
+ vector search (handling instructions).

Joule matched the question to the `msds_query` scenario because the description includes “hazard information” and “GHS classification.” The `material_number` capability context was empty on this first turn, so the agent resolved the material by name from the knowledge graph.

Follow-up — context preserved:

User: What PPE do I need?

Joule: [Searching MSDS knowledge base...]

For Acetone, the required PPE is: safety glasses or goggles to protect against eye irritation, nitrile gloves (minimum 0.4 mm thickness), and adequate ventilation. A respirator is required if ventilation is insufficient to maintain exposure below the TWA of 500 ppm.

The user did not mention Acetone in the second question. Joule passed the `material_number` from `$capability_context.material_number` to the function, which sent it as `taskId` in the `/a2a` request. The agent used it to scope the knowledge graph SPARQL query to Acetone’s document.

Checking processing status:

User: What is the processing status for material 200001001?

Joule: [Checking processing status...]

Material 200001001: processing complete.
Knowledge graph: 847 triples. Vector store: 23 chunks.
Last processed: 2024-03-15 14:22 UTC.

Joule matched this to the `msds_status` scenario. The `material_number` was extracted from the user’s message by Joule’s NLU and passed as a parameter to the `msds_status` function.

D.11 What Will Change with Joule 2.0

SAP's Joule 2.0 announcement describes several improvements to A2A support. Based on the public announcements available at the time of writing, the following changes are expected.

What will likely change:

- The YAML schema version will increment. The `schema_version: 3.28.0` in `capability.sapdas.yaml` will need to be updated to the Joule 2.0 schema version. The structure of the file is expected to remain largely compatible, with new optional fields for richer task management.
- A2A protocol alignment. Joule 2.0 is expected to align more closely with the open A2A specification, which means the JSON-RPC envelope format (`{"jsonrpc": "2.0", "method": "message/send", ...}`) will become the primary format rather than the flat format. The dual-format support in the `/a2a` endpoint described in section D.6.1 is designed for this transition.
- Richer multi-turn support. Joule 2.0 is expected to provide explicit task state management — a persistent task object with status, history, and artifacts — rather than the lightweight capability context variables used in Joule 1.x. The `thread_id` and `task_id` variables in `capability_context.yaml` will map to fields in the task object.
- Streaming support. Joule 2.0 is expected to support streaming responses for longer-running operations, which would relax the 15-second constraint.

What will stay the same:

- The core concept: wrapping an external HTTP agent behind a declarative YAML interface. The BTP Destination remains the routing mechanism.
- The scenario-based intent matching. Joule will continue to use semantic similarity against scenario descriptions to route questions.
- The capability packaging and deployment model. The `.daar` format or an equivalent ZIP-based package format will remain the delivery mechanism.
- The `system_aliases` pattern for destination indirection. Hard-coding URLs in YAML files was never the right approach; this pattern will carry forward.

When Joule 2.0 reaches general availability, the changes to this appendix will be limited to the schema version number, a revised `capability.sapdas.yaml` for the new schema,

and updates to the `/a2a` endpoint to prefer the JSON-RPC format. The five-file structure and the overall architecture will remain the same.

D.12 Files in the Repository

The `joule/` directory in the project repository contains the following files. Readers without an enterprise SAP subscription can study these files to understand the pattern and adapt them when access becomes available.

```
joule/
├── README.md
├── da.sapdas.yaml
├── msds_capability/
│   ├── capability.sapdas.yaml
│   ├── capability_context.yaml
│   ├── functions/
│   │   ├── msds_chat.yaml
│   │   └── msds_status.yaml
│   └── scenarios/
│       ├── msds_query.yaml
│       └── msds_status.yaml
```

The `.daar` package file (`com.sap.msds_msds_kg_agent_2.0.0-SNAPSHOT.daar`) is not committed to the repository because it is a build artifact. Generate it from the source files using the `zip` command in section D.8 whenever you need to deploy.

The `/a2a` endpoint implementation belongs in `agents/main.py` alongside the other FastAPI endpoints. It follows the same pattern as the `/query` endpoint from Chapter 7, with the dual-format request parsing and the flat-format response described in section D.6.

D.13 Summary

Joule A2A bridges the gap between a custom Python agent and SAP's enterprise AI assistant. The pattern is straightforward once you understand the five-file structure: a root declaration, capability metadata with a BTP Destination alias, a context variable store, function files that build HTTP requests, and scenario files that match user intent to functions.

The constraints are real — 15 seconds is a short window for a multi-hop RAG pipeline — but manageable with parallel execution and conservative query limits. The SpEL-like expression language is minimal; the four reference patterns in section D.5 cover nearly every use case you will encounter.

The most important design decision is the BTP Destination. It keeps URLs out of YAML, which means you can redeploy the Python agent to a new host, update one field in BTP Cockpit, and every Joule capability that uses the destination automatically routes to the new endpoint. In a system where the AI layer evolves faster than the configuration layer, that decoupling is worth the extra setup step.

When Joule 2.0 arrives with native A2A support, the knowledge in this appendix transfers directly. The schema version will change; the architecture will not.